Reinforcement Learning: An Introduction

Second edition, in progress

Richard S. Sutton and Andrew G. Barto
© 2012

A Bradford Book

The MIT Press
Cambridge, Massachusetts
London, England

In memory of A. Harry Klopf

Contents

Preface viii
Series Forward xii
Summary of Notation xiii

I The Problem **1**

1 Introduction **3**
1.1 Reinforcement Learning 4
1.2 Examples 6
1.3 Elements of Reinforcement Learning 7
1.4 An Extended Example: Tic-Tac-Toe 10
1.5 Summary 15
1.6 History of Reinforcement Learning 16
1.7 Bibliographical Remarks 23

2 Bandit Problems **25**
2.1 An n-Armed Bandit Problem 26
2.2 Action-Value Methods 27
2.3 Softmax Action Selection 30
2.4 Incremental Implementation 32
2.5 Tracking a Nonstationary Problem 33
2.6 Optimistic Initial Values 35
2.7 Associative Search (Contextual Bandits) 37

2.8 Conclusions 38
2.9 Bibliographical and Historical Remarks 40

3 The Reinforcement Learning Problem 43
3.1 The Agent–Environment Interface 43
3.2 Goals and Rewards 48
3.3 Returns 49
3.4 Unified Notation for Episodic and Continuing Tasks 52
*3.5 The Markov Property 53
3.6 Markov Decision Processes 58
3.7 Value Functions 60
3.8 Optimal Value Functions 66
3.9 Optimality and Approximation 71
3.10 Summary 72
3.11 Bibliographical and Historical Remarks 74

II Tabular Action-Value Methods 79

4 Dynamic Programming 83
4.1 Policy Evaluation 84
4.2 Policy Improvement 87
4.3 Policy Iteration 91
4.4 Value Iteration 95
4.5 Asynchronous Dynamic Programming 98
4.6 Generalized Policy Iteration 99
4.7 Efficiency of Dynamic Programming 101
4.8 Summary 102
4.9 Bibliographical and Historical Remarks 103

5 Monte Carlo Methods 107
5.1 Monte Carlo Policy Evaluation 108

5.2 Monte Carlo Estimation of Action Values 112
5.3 Monte Carlo Control 114
5.4 On-Policy Monte Carlo Control 118
*5.5 Evaluating One Policy While Following Another (Off-policy Policy Evaluation) 121
5.6 Off-Policy Monte Carlo Control 122
5.7 Incremental Implementation 124
5.8 Summary 126
5.9 Bibliographical and Historical Remarks 127

6 Temporal-Difference Learning 129
6.1 TD Prediction 129
6.2 Advantages of TD Prediction Methods 134
6.3 Optimality of TD(0) 137
6.4 Sarsa: On-Policy TD Control 141
6.5 Q-Learning: Off-Policy TD Control 144
6.6 Games, Afterstates, and Other Special Cases 147
6.7 Summary 148
6.8 Bibliographical and Historical Remarks 149

7 Eligibility Traces 153
7.1 n-Step TD Prediction 154
7.2 The Forward View of TD(λ) 159
7.3 The Backward View of TD(λ) 163
7.4 Equivalence of Forward and Backward Views 166
7.5 Sarsa(λ) 169
7.6 Q(λ) 172
7.7 Replacing Traces 175
7.8 Implementation Issues 178
*7.9 Variable λ 178
7.10 Conclusions 179

7.11 Bibliographical and Historical Remarks 180

8 Planning and Learning with Tabular Methods 183

8.1 Models and Planning 183
8.2 Integrating Planning, Acting, and Learning 186
8.3 When the Model Is Wrong 191
8.4 Prioritized Sweeping 194
8.5 Full vs. Sample Backups 198
8.6 Trajectory Sampling 202
8.7 Heuristic Search 205
8.8 Summary 208
8.9 Bibliographical and Historical Remarks 209

III Approximate Solution Methods 211

9 On-policy Approximation of Action Values 213

9.1 Value Prediction with Function Approximation 214
9.2 Gradient-Descent Methods 217
9.3 Linear Methods 221
9.4 Control with Function Approximation 230
9.5 Should We Bootstrap? 236
9.6 Summary 238
9.7 Bibliographical and Historical Remarks 239

10 Off-policy Approximation of Action Values 243

11 Policy Approximation 245

11.1 Actor–Critic Methods 245
11.2 R-Learning and the Average-Reward Setting 248

12 State Estimation 251

13 Temporal Abstraction **253**

IV Frontiers 255

14 Biological Reinforcement Learning **257**

15 Applications and Case Studies **259**

15.1 TD-Gammon 259

15.2 Samuel's Checkers Player 265

15.3 The Acrobot 268

15.4 Elevator Dispatching 272

15.5 Dynamic Channel Allocation 277

15.6 Job-Shop Scheduling 281

16 Prospects **289**

16.1 The Unified View 289

16.2 Other Frontier Dimensions 292

References **295**

Index **320**

Preface

We first came to focus on what is now known as reinforcement learning in late 1979. We were both at the University of Massachusetts, working on one of the earliest projects to revive the idea that networks of neuronlike adaptive elements might prove to be a promising approach to artificial adaptive intelligence. The project explored the "heterostatic theory of adaptive systems" developed by A. Harry Klopf. Harry's work was a rich source of ideas, and we were permitted to explore them critically and compare them with the long history of prior work in adaptive systems. Our task became one of teasing the ideas apart and understanding their relationships and relative importance. This continues today, but in 1979 we came to realize that perhaps the simplest of the ideas, which had long been taken for granted, had received surprisingly little attention from a computational perspective. This was simply the idea of a learning system that *wants* something, that adapts its behavior in order to maximize a special signal from its environment. This was the idea of a "hedonistic" learning system, or, as we would say now, the idea of reinforcement learning.

Like others, we had a sense that reinforcement learning had been thoroughly explored in the early days of cybernetics and artificial intelligence. On closer inspection, though, we found that it had been explored only slightly. While reinforcement learning had clearly motivated some of the earliest computational studies of learning, most of these researchers had gone on to other things, such as pattern classification, supervised learning, and adaptive control, or they had abandoned the study of learning altogether. As a result, the special issues involved in learning how to get something from the environment received relatively little attention. In retrospect, focusing on this idea was the critical step that set this branch of research in motion. Little progress could be made in the computational study of reinforcement learning until it was recognized that such a fundamental idea had not yet been thoroughly explored.

The field has come a long way since then, evolving and maturing in several directions. Reinforcement learning has gradually become one of the most active research areas in machine learning, artificial intelligence, and neural network research. The field has developed strong mathematical foundations and impressive applications. The computational study of reinforcement learning is now a large field, with hundreds of active researchers around the world in diverse disciplines such as psychology, control theory, artificial intelligence, and neuroscience. Particularly important have been the contributions establishing and developing the relationships to the theory of optimal control and dynamic programming. The overall problem of learning from interaction to achieve

goals is still far from being solved, but our understanding of it has improved significantly. We can now place component ideas, such as temporal-difference learning, dynamic programming, and function approximation, within a coherent perspective with respect to the overall problem.

Our goal in writing this book was to provide a clear and simple account of the key ideas and algorithms of reinforcement learning. We wanted our treatment to be accessible to readers in all of the related disciplines, but we could not cover all of these perspectives in detail. Our treatment takes almost exclusively the point of view of artificial intelligence and engineering, leaving coverage of connections to psychology, neuroscience, and other fields to others or to another time. We also chose not to produce a rigorous formal treatment of reinforcement learning. We did not reach for the highest possible level of mathematical abstraction and did not rely on a theorem–proof format. We tried to choose a level of mathematical detail that points the mathematically inclined in the right directions without distracting from the simplicity and potential generality of the underlying ideas.

The book consists of three parts. Part I is introductory and problem oriented. We focus on the simplest aspects of reinforcement learning and on its main distinguishing features. One full chapter is devoted to introducing the reinforcement learning problem whose solution we explore in the rest of the book. Part II presents tabular versions (assuming a small finite state space) of all the basic solution methods based on estimating action values. We introduce dynamic programming, Monte Carlo methods, and temporal-difference learning. There is a chapter on eligibility traces which unifies the latter two methods, and a chapter that unifies planning methods (such as dynamic programming and state-space search) and learning methods (such as Monte Carlo and temporal-difference learning). Part III is concerned with extending the tabular methods to include various forms of approximation including function approximation, policy-gradient methods, and methods designed for solving off-policy learning problems. Part IV surveys some of the frontiers of reinforcement learning in biology and applications.

This book was designed to be used as a text in a one-semester course, perhaps supplemented by readings from the literature or by a more mathematical text such as Bertsekas and Tsitsiklis (1996) or Szepesvari. This book can also be used as part of a broader course on machine learning, artificial intelligence, or neural networks. In this case, it may be desirable to cover only a subset of the material. We recommend covering Chapter 1 for a brief overview, Chapter 2 through Section 2.2, Chapter 3 except Sections 3.4, 3.5 and 3.9, and then selecting sections from the remaining chapters according to time and interests. The five chapters of Part II build on each other and are best covered in sequence; of these, Chapter 6 is the most important for the subject and for the

rest of the book. A course focusing on machine learning or neural networks should cover Chapter 9, and a course focusing on artificial intelligence or planning should cover Chapter 8. Throughout the book, sections that are more difficult and not essential to the rest of the book are marked with a $*$. These can be omitted on first reading without creating problems later on. Some exercises are marked with a $*$ to indicate that they are more advanced and not essential to understanding the basic material of the chapter.

The book is largely self-contained. The only mathematical background assumed is familiarity with elementary concepts of probability, such as expectations of random variables. Chapter 9 is substantially easier to digest if the reader has some knowledge of artificial neural networks or some other kind of supervised learning method, but it can be read without prior background. We strongly recommend working the exercises provided throughout the book. Solution manuals are available to instructors. This and other related and timely material is available via the Internet.

At the end of most chapters is a section entitled "Bibliographical and Historical Remarks," wherein we credit the sources of the ideas presented in that chapter, provide pointers to further reading and ongoing research, and describe relevant historical background. Despite our attempts to make these sections authoritative and complete, we have undoubtedly left out some important prior work. For that we apologize, and welcome corrections and extensions for incorporation into a subsequent edition.

In some sense we have been working toward this book for thirty years, and we have lots of people to thank. First, we thank those who have personally helped us develop the overall view presented in this book: Harry Klopf, for helping us recognize that reinforcement learning needed to be revived; Chris Watkins, Dimitri Bertsekas, John Tsitsiklis, and Paul Werbos, for helping us see the value of the relationships to dynamic programming; John Moore and Jim Kehoe, for insights and inspirations from animal learning theory; Oliver Selfridge, for emphasizing the breadth and importance of adaptation; and, more generally, our colleagues and students who have contributed in countless ways: Ron Williams, Charles Anderson, Satinder Singh, Sridhar Mahadevan, Steve Bradtke, Bob Crites, Peter Dayan, and Leemon Baird. Our view of reinforcement learning has been significantly enriched by discussions with Paul Cohen, Paul Utgoff, Martha Steenstrup, Gerry Tesauro, Mike Jordan, Leslie Kaelbling, Andrew Moore, Chris Atkeson, Tom Mitchell, Nils Nilsson, Stuart Russell, Tom Dietterich, Tom Dean, and Bob Narendra. We thank Michael Littman, Gerry Tesauro, Bob Crites, Satinder Singh, and Wei Zhang for providing specifics of Sections 4.7, 15.1, 15.4, 15.5, and 15.6 respectively. We thank the the Air Force Office of Scientific Research, the National Science Foundation, and GTE Laboratories for their long and farsighted support.

We also wish to thank the many people who have read drafts of this book and provided valuable comments, including Tom Kalt, John Tsitsiklis, Pawel Cichosz, Olle Gällmo, Chuck Anderson, Stuart Russell, Ben Van Roy, Paul Steenstrup, Paul Cohen, Sridhar Mahadevan, Jette Randlov, Brian Sheppard, Thomas O'Connell, Richard Coggins, Cristina Versino, John H. Hiett, Andreas Badelt, Jay Ponte, Joe Beck, Justus Piater, Martha Steenstrup, Satinder Singh, Tommi Jaakkola, Dimitri Bertsekas, Torbjörn Ekman, Christina Björkman, Jakob Carlström, and Olle Palmgren. Finally, we thank Gwyn Mitchell for helping in many ways, and Harry Stanton and Bob Prior for being our champions at MIT Press.

Series Forward

Summary of Notation

Capital letters are used for random variables and major algorithm variables. Lower case letters are used for the values of random variables and for scalar functions. Quantities that are required to be real-valued vectors are written in bold and in lower case (even if random variables).

s	state
a	action
$\mathcal{S}$	set of all nonterminal states
$\mathcal{S}^+$	set of all states, including the terminal state
$\mathcal{A}(s)$	set of actions possible in state s
t	discrete time step
T	final time step of an episode
S_t	state at t
A_t	action at t
R_t	reward at t, dependent, like S_t, on A_{t-1} and S_{t-1}
G_t	return (cumulative discounted reward) following t
$G_t^{(n)}$	n-step return (Section 7.1)
G_t^λ	λ-return (Section 7.2)
π	policy, decision-making rule
$\pi(s)$	action taken in state s under *deterministic* policy π
$\pi(a\|s)$	probability of taking action a in state s under *stochastic* policy π
$p(s'\|s,a)$	probability of transition from state s to state s' under action a
$r(s,a,s')$	expected immediate reward on transition from s to s' under action a
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
$q_\pi(s,a)$	value of taking action a in state s under policy π
$q_*(s,a)$	value of taking action a in state s under the optimal policy
V_t	estimate (a random variable) of v_π or v_*
Q_t	estimate (a random variable) of q_π or q_*
$\hat{v}(s,\mathbf{w})$	approximate value of state s given a vector of weights $\mathbf{w}$
$\hat{q}(s,a,\mathbf{w})$	approximate value of state–action pair s,a given weights $\mathbf{w}$
$\mathbf{w}, \mathbf{w}_t$	vector of (possibly learned) *weights* underlying an approximate value function
$\mathbf{x}(s)$	vector of features visible when in state s
$\mathbf{w}^\top\mathbf{x}$	inner product of vectors, $\mathbf{w}^\top\mathbf{x} = \sum_i w_i x_i$; e.g., $\hat{v}(s,\mathbf{w}) = \mathbf{w}^\top\mathbf{x}(s)$

δ_t	temporal-difference error at t (a random variable, even though not upper case)
$Z_t(s)$	eligibility trace for state s at t
$Z_t(s,a)$	eligibility trace for a state–action pair
$\mathbf{z}_t$	eligibility trace vector at t
γ	discount-rate parameter
ε	probability of random action in ε-greedy policy
α, β	step-size parameters
λ	decay-rate parameter for eligibility traces

Part I

The Problem

Chapter 1

Introduction

The idea that we learn by interacting with our environment is probably the first to occur to us when we think about the nature of learning. When an infant plays, waves its arms, or looks about, it has no explicit teacher, but it does have a direct sensorimotor connection to its environment. Exercising this connection produces a wealth of information about cause and effect, about the consequences of actions, and about what to do in order to achieve goals. Throughout our lives, such interactions are undoubtedly a major source of knowledge about our environment and ourselves. Whether we are learning to drive a car or to hold a conversation, we are acutely aware of how our environment responds to what we do, and we seek to influence what happens through our behavior. Learning from interaction is a foundational idea underlying nearly all theories of learning and intelligence.

In this book we explore a *computational* approach to learning from interaction. Rather than directly theorizing about how people or animals learn, we explore idealized learning situations and evaluate the effectiveness of various learning methods. That is, we adopt the perspective of an artificial intelligence researcher or engineer. We explore designs for machines that are effective in solving learning problems of scientific or economic interest, evaluating the designs through mathematical analysis or computational experiments. The approach we explore, called *reinforcement learning*, is much more focused on goal-directed learning from interaction than are other approaches to machine learning.

1.1 Reinforcement Learning

Reinforcement learning is learning what to do—how to map situations to actions—so as to maximize a numerical reward signal. The learner is not told which actions to take, as in most forms of machine learning, but instead must discover which actions yield the most reward by trying them. In the most interesting and challenging cases, actions may affect not only the immediate reward but also the next situation and, through that, all subsequent rewards. These two characteristics—trial-and-error search and delayed reward—are the two most important distinguishing features of reinforcement learning.

Reinforcement learning is defined not by characterizing learning methods, but by characterizing a learning *problem*. Any method that is well suited to solving that problem, we consider to be a reinforcement learning method. A full specification of the reinforcement learning problem in terms of optimal control of Markov decision processes must wait until Chapter 3, but the basic idea is simply to capture the most important aspects of the real problem facing a learning agent interacting with its environment to achieve a goal. Clearly, such an agent must be able to sense the state of the environment to some extent and must be able to take actions that affect the state. The agent also must have a goal or goals relating to the state of the environment. The formulation is intended to include just these three aspects—sensation, action, and goal—in their simplest possible forms without trivializing any of them.

Reinforcement learning is different from *supervised learning*, the kind of learning studied in most current research in machine learning, statistical pattern recognition, and artificial neural networks. Supervised learning is learning from examples provided by a knowledgable external supervisor. This is an important kind of learning, but alone it is not adequate for learning from interaction. In interactive problems it is often impractical to obtain examples of desired behavior that are both correct and representative of all the situations in which the agent has to act. In uncharted territory—where one would expect learning to be most beneficial—an agent must be able to learn from its own experience.

One of the challenges that arise in reinforcement learning and not in other kinds of learning is the trade-off between exploration and exploitation. To obtain a lot of reward, a reinforcement learning agent must prefer actions that it has tried in the past and found to be effective in producing reward. But to discover such actions, it has to try actions that it has not selected before. The agent has to *exploit* what it already knows in order to obtain reward, but it also has to *explore* in order to make better action selections in the future. The dilemma is that neither exploration nor exploitation can be pursued exclusively without failing at the task. The agent must try a variety of

actions *and* progressively favor those that appear to be best. On a stochastic task, each action must be tried many times to gain a reliable estimate its expected reward. The exploration–exploitation dilemma has been intensively studied by mathematicians for many decades (see Chapter 2). For now, we simply note that the entire issue of balancing exploration and exploitation does not even arise in supervised learning as it is usually defined.

Another key feature of reinforcement learning is that it explicitly considers the *whole* problem of a goal-directed agent interacting with an uncertain environment. This is in contrast with many approaches that consider subproblems without addressing how they might fit into a larger picture. For example, we have mentioned that much of machine learning research is concerned with supervised learning without explicitly specifying how such an ability would finally be useful. Other researchers have developed theories of planning with general goals, but without considering planning's role in real-time decision-making, or the question of where the predictive models necessary for planning would come from. Although these approaches have yielded many useful results, their focus on isolated subproblems is a significant limitation.

Reinforcement learning takes the opposite tack, starting with a complete, interactive, goal-seeking agent. All reinforcement learning agents have explicit goals, can sense aspects of their environments, and can choose actions to influence their environments. Moreover, it is usually assumed from the beginning that the agent has to operate despite significant uncertainty about the environment it faces. When reinforcement learning involves planning, it has to address the interplay between planning and real-time action selection, as well as the question of how environmental models are acquired and improved. When reinforcement learning involves supervised learning, it does so for specific reasons that determine which capabilities are critical and which are not. For learning research to make progress, important subproblems have to be isolated and studied, but they should be subproblems that play clear roles in complete, interactive, goal-seeking agents, even if all the details of the complete agent cannot yet be filled in.

One of the larger trends of which reinforcement learning is a part is that toward greater contact between artificial intelligence and other engineering disciplines. Not all that long ago, artificial intelligence was viewed as almost entirely separate from control theory and statistics. It had to do with logic and symbols, not numbers. Artificial intelligence was large LISP programs, not linear algebra, differential equations, or statistics. Over the last decades this view has gradually eroded. Modern artificial intelligence researchers accept statistical and control algorithms, for example, as relevant competing methods or simply as tools of their trade. The previously ignored areas lying between artificial intelligence and conventional engineering are now among the most

active, including new fields such as neural networks, intelligent control, and our topic, reinforcement learning. In reinforcement learning we extend ideas from optimal control theory and stochastic approximation to address the broader and more ambitious goals of artificial intelligence.

1.2 Examples

A good way to understand reinforcement learning is to consider some of the examples and possible applications that have guided its development.

- A master chess player makes a move. The choice is informed both by planning—anticipating possible replies and counterreplies—and by immediate, intuitive judgments of the desirability of particular positions and moves.

- An adaptive controller adjusts parameters of a petroleum refinery's operation in real time. The controller optimizes the yield/cost/quality trade-off on the basis of specified marginal costs without sticking strictly to the set points originally suggested by engineers.

- A gazelle calf struggles to its feet minutes after being born. Half an hour later it is running at 20 miles per hour.

- A mobile robot decides whether it should enter a new room in search of more trash to collect or start trying to find its way back to its battery recharging station. It makes its decision based on how quickly and easily it has been able to find the recharger in the past.

- Phil prepares his breakfast. Closely examined, even this apparently mundane activity reveals a complex web of conditional behavior and interlocking goal–subgoal relationships: walking to the cupboard, opening it, selecting a cereal box, then reaching for, grasping, and retrieving the box. Other complex, tuned, interactive sequences of behavior are required to obtain a bowl, spoon, and milk jug. Each step involves a series of eye movements to obtain information and to guide reaching and locomotion. Rapid judgments are continually made about how to carry the objects or whether it is better to ferry some of them to the dining table before obtaining others. Each step is guided by goals, such as grasping a spoon or getting to the refrigerator, and is in service of other goals, such as having the spoon to eat with once the cereal is prepared and ultimately obtaining nourishment.

These examples share features that are so basic that they are easy to overlook. All involve *interaction* between an active decision-making agent and its environment, within which the agent seeks to achieve a *goal* despite *uncertainty* about its environment. The agent's actions are permitted to affect the future state of the environment (e.g., the next chess position, the level of reservoirs of the refinery, the next location of the robot), thereby affecting the options and opportunities available to the agent at later times. Correct choice requires taking into account indirect, delayed consequences of actions, and thus may require foresight or planning.

At the same time, in all these examples the effects of actions cannot be fully predicted; thus the agent must monitor its environment frequently and react appropriately. For example, Phil must watch the milk he pours into his cereal bowl to keep it from overflowing. All these examples involve goals that are explicit in the sense that the agent can judge progress toward its goal based on what it can sense directly. The chess player knows whether or not he wins, the refinery controller knows how much petroleum is being produced, the mobile robot knows when its batteries run down, and Phil knows whether or not he is enjoying his breakfast.

In all of these examples the agent can use its experience to improve its performance over time. The chess player refines the intuition he uses to evaluate positions, thereby improving his play; the gazelle calf improves the efficiency with which it can run; Phil learns to streamline making his breakfast. The knowledge the agent brings to the task at the start—either from previous experience with related tasks or built into it by design or evolution—influences what is useful or easy to learn, but interaction with the environment is essential for adjusting behavior to exploit specific features of the task.

1.3 Elements of Reinforcement Learning

Beyond the agent and the environment, one can identify four main subelements of a reinforcement learning system: a *policy*, a *reward function*, a *value function*, and, optionally, a *model* of the environment.

A *policy* defines the learning agent's way of behaving at a given time. Roughly speaking, a policy is a mapping from perceived states of the environment to actions to be taken when in those states. It corresponds to what in psychology would be called a set of stimulus–response rules or associations. In some cases the policy may be a simple function or lookup table, whereas in others it may involve extensive computation such as a search process. The policy is the core of a reinforcement learning agent in the sense that it alone

is sufficient to determine behavior. In general, policies may be stochastic.

A *reward function* defines the goal in a reinforcement learning problem. Roughly speaking, it maps each perceived state (or state–action pair) of the environment to a single number, a *reward*, indicating the intrinsic desirability of that state. A reinforcement learning agent's sole objective is to maximize the total reward it receives in the long run. The reward function defines what are the good and bad events for the agent. In a biological system, it would not be inappropriate to identify rewards with pleasure and pain. They are the immediate and defining features of the problem faced by the agent. As such, the reward function must necessarily be unalterable by the agent. It may, however, serve as a basis for altering the policy. For example, if an action selected by the policy is followed by low reward, then the policy may be changed to select some other action in that situation in the future. In general, reward functions may be stochastic.

Whereas a reward function indicates what is good in an immediate sense, a *value function* specifies what is good in the long run. Roughly speaking, the *value* of a state is the total amount of reward an agent can expect to accumulate over the future, starting from that state. Whereas rewards determine the immediate, intrinsic desirability of environmental states, values indicate the *long-term* desirability of states after taking into account the states that are likely to follow, and the rewards available in those states. For example, a state might always yield a low immediate reward but still have a high value because it is regularly followed by other states that yield high rewards. Or the reverse could be true. To make a human analogy, rewards are like pleasure (if high) and pain (if low), whereas values correspond to a more refined and farsighted judgment of how pleased or displeased we are that our environment is in a particular state. Expressed this way, we hope it is clear that value functions formalize a basic and familiar idea.

Rewards are in a sense primary, whereas values, as predictions of rewards, are secondary. Without rewards there could be no values, and the only purpose of estimating values is to achieve more reward. Nevertheless, it is values with which we are most concerned when making and evaluating decisions. Action choices are made based on value judgments. We seek actions that bring about states of highest value, not highest reward, because these actions obtain the greatest amount of reward for us over the long run. In decision-making and planning, the derived quantity called value is the one with which we are most concerned. Unfortunately, it is much harder to determine values than it is to determine rewards. Rewards are basically given directly by the environment, but values must be estimated and reestimated from the sequences of observations an agent makes over its entire lifetime. In fact, the most important component of almost all reinforcement learning algorithms is a method for

efficiently estimating values. The central role of value estimation is arguably the most important thing we have learned about reinforcement learning over the last few decades.

Although all the reinforcement learning methods we consider in this book are structured around estimating value functions, it is not strictly necessary to do this to solve reinforcement learning problems. For example, search methods such as genetic algorithms, genetic programming, simulated annealing, and other function optimization methods have been used to solve reinforcement learning problems. These methods search directly in the space of policies without ever appealing to value functions. We call these *evolutionary* methods because their operation is analogous to the way biological evolution produces organisms with skilled behavior even when they do not learn during their individual lifetimes. If the space of policies is sufficiently small, or can be structured so that good policies are common or easy to find, then evolutionary methods can be effective. In addition, evolutionary methods have advantages on problems in which the learning agent cannot accurately sense the state of its environment.

Nevertheless, what we mean by reinforcement learning involves learning while interacting with the environment, which evolutionary methods do not do. It is our belief that methods able to take advantage of the details of individual behavioral interactions can be much more efficient than evolutionary methods in many cases. Evolutionary methods ignore much of the useful structure of the reinforcement learning problem: they do not use the fact that the policy they are searching for is a function from states to actions; they do not notice which states an individual passes through during its lifetime, or which actions it selects. In some cases this information can be misleading (e.g., when states are misperceived), but more often it should enable more efficient search. Although evolution and learning share many features and can naturally work together, as they do in nature, we do not consider evolutionary methods by themselves to be especially well suited to reinforcement learning problems. For simplicity, in this book when we use the term "reinforcement learning" we do not include evolutionary methods.

The fourth and final element of some reinforcement learning systems is a *model* of the environment. This is something that mimics the behavior of the environment. For example, given a state and action, the model might predict the resultant next state and next reward. Models are used for *planning*, by which we mean any way of deciding on a course of action by considering possible future situations before they are actually experienced. The incorporation of models and planning into reinforcement learning systems is a relatively new development. Early reinforcement learning systems were explicitly trial-and-error learners; what they did was viewed as almost the *opposite* of planning.

Nevertheless, it gradually became clear that reinforcement learning methods are closely related to dynamic programming methods, which do use models, and that they in turn are closely related to state–space planning methods. In Chapter 9 we explore reinforcement learning systems that simultaneously learn by trial and error, learn a model of the environment, and use the model for planning. Modern reinforcement learning spans the spectrum from low-level, trial-and-error learning to high-level, deliberative planning.

1.4 An Extended Example: Tic-Tac-Toe

To illustrate the general idea of reinforcement learning and contrast it with other approaches, we next consider a single example in more detail.

Consider the familiar child's game of tic-tac-toe. Two players take turns playing on a three-by-three board. One player plays Xs and the other Os until one player wins by placing three marks in a row, horizontally, vertically, or diagonally, as the X player has in this game:

X	O	O
O	X	X
		X

If the board fills up with neither player getting three in a row, the game is a draw. Because a skilled player can play so as never to lose, let us assume that we are playing against an imperfect player, one whose play is sometimes incorrect and allows us to win. For the moment, in fact, let us consider draws and losses to be equally bad for us. How might we construct a player that will find the imperfections in its opponent's play and learn to maximize its chances of winning?

Although this is a simple problem, it cannot readily be solved in a satisfactory way through classical techniques. For example, the classical "minimax" solution from game theory is not correct here because it assumes a particular way of playing by the opponent. For example, a minimax player would never reach a game state from which it could lose, even if in fact it always won from that state because of incorrect play by the opponent. Classical optimization methods for sequential decision problems, such as dynamic programming, can *compute* an optimal solution for any opponent, but require as input a complete specification of that opponent, including the probabilities with which the opponent makes each move in each board state. Let us assume that this information is not available a priori for this problem, as it is not for the vast

majority of problems of practical interest. On the other hand, such information can be estimated from experience, in this case by playing many games against the opponent. About the best one can do on this problem is first to learn a model of the opponent's behavior, up to some level of confidence, and then apply dynamic programming to compute an optimal solution given the approximate opponent model. In the end, this is not that different from some of the reinforcement learning methods we examine later in this book.

An evolutionary approach to this problem would directly search the space of possible policies for one with a high probability of winning against the opponent. Here, a policy is a rule that tells the player what move to make for every state of the game—every possible configuration of Xs and Os on the three-by-three board. For each policy considered, an estimate of its winning probability would be obtained by playing some number of games against the opponent. This evaluation would then direct which policy or policies were considered next. A typical evolutionary method would hill-climb in policy space, successively generating and evaluating policies in an attempt to obtain incremental improvements. Or, perhaps, a genetic-style algorithm could be used that would maintain and evaluate a population of policies. Literally hundreds of different optimization methods could be applied. By *directly* searching the policy space we mean that *entire policies* are proposed and compared on the basis of scalar evaluations.

Here is how the tic-tac-toe problem would be approached using reinforcement learning and approximate value functions. First we set up a table of numbers, one for each possible state of the game. Each number will be the latest estimate of the probability of our winning from that state. We treat this estimate as the state's *value*, and the whole table is the learned value function. State A has higher value than state B, or is considered "better" than state B, if the current estimate of the probability of our winning from A is higher than it is from B. Assuming we always play Xs, then for all states with three Xs in a row the probability of winning is 1, because we have already won. Similarly, for all states with three Os in a row, or that are "filled up," the correct probability is 0, as we cannot win from them. We set the initial values of all the other states to 0.5, representing a guess that we have a 50% chance of winning.

We play many games against the opponent. To select our moves we examine the states that would result from each of our possible moves (one for each blank space on the board) and look up their current values in the table. Most of the time we move *greedily*, selecting the move that leads to the state with greatest value, that is, with the highest estimated probability of winning. Occasionally, however, we select randomly from among the other moves instead. These are called *exploratory* moves because they cause us to experience states that we

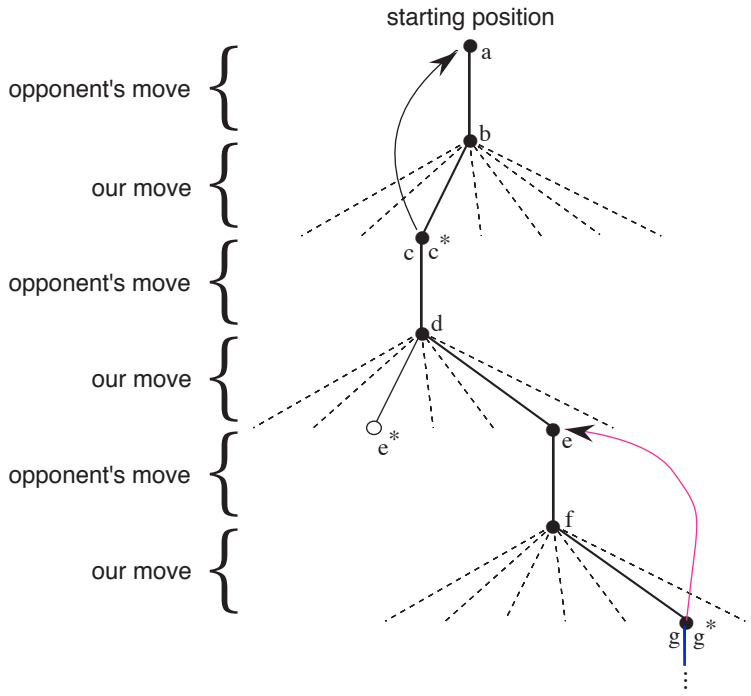


Figure 1.1: A sequence of tic-tac-toe moves. The solid lines represent the moves taken during a game; the dashed lines represent moves that we (our reinforcement learning player) considered but did not make. Our second move was an exploratory move, meaning that it was taken even though another sibling move, the one leading to e*, was ranked higher. Exploratory moves do not result in any learning, but each of our other moves does, causing *backups* as suggested by the curved arrows and detailed in the text.

might otherwise never see. A sequence of moves made and considered during a game can be diagrammed as in Figure 1.1.

While we are playing, we change the values of the states in which we find ourselves during the game. We attempt to make them more accurate estimates of the probabilities of winning. To do this, we "back up" the value of the state after each greedy move to the state before the move, as suggested by the arrows in Figure 1.1. More precisely, the current value of the earlier state is adjusted to be closer to the value of the later state. This can be done by moving the earlier state's value a fraction of the way toward the value of the later state. If we let s denote the state before the greedy move, and s' the state after the move, then the update to the estimated value of s, denoted $V(s)$, can be written as

$$V(s) \leftarrow V(s) + \alpha\Big[V(s') - V(s)\Big],$$

where α is a small positive fraction called the *step-size parameter*, which influences the rate of learning. This update rule is an example of a *temporal-difference* learning method, so called because its changes are based on a difference, $V(s') - V(s)$, between estimates at two different times.

The method described above performs quite well on this task. For example, if the step-size parameter is reduced properly over time, this method converges, for any fixed opponent, to the true probabilities of winning from each state given optimal play by our player. Furthermore, the moves then taken (except on exploratory moves) are in fact the optimal moves against the opponent. In other words, the method converges to an optimal policy for playing the game. If the step-size parameter is not reduced all the way to zero over time, then this player also plays well against opponents that slowly change their way of playing.

This example illustrates the differences between evolutionary methods and methods that learn value functions. To evaluate a policy, an evolutionary method must hold it fixed and play many games against the opponent, or simulate many games using a model of the opponent. The frequency of wins gives an unbiased estimate of the probability of winning with that policy, and can be used to direct the next policy selection. But each policy change is made only after many games, and only the final outcome of each game is used: what happens *during* the games is ignored. For example, if the player wins, then *all* of its behavior in the game is given credit, independently of how specific moves might have been critical to the win. Credit is even given to moves that never occurred! Value function methods, in contrast, allow individual states to be evaluated. In the end, both evolutionary and value function methods search the space of policies, but learning a value function takes advantage of information available during the course of play.

This simple example illustrates some of the key features of reinforcement learning methods. First, there is the emphasis on learning while interacting with an environment, in this case with an opponent player. Second, there is a clear goal, and correct behavior requires planning or foresight that takes into account delayed effects of one's choices. For example, the simple reinforcement learning player would learn to set up multimove traps for a shortsighted opponent. It is a striking feature of the reinforcement learning solution that it can achieve the effects of planning and lookahead without using a model of the opponent and without conducting an explicit search over possible sequences of future states and actions.

While this example illustrates some of the key features of reinforcement learning, it is so simple that it might give the impression that reinforcement learning is more limited than it really is. Although tic-tac-toe is a two-person

game, reinforcement learning also applies in the case in which there is no external adversary, that is, in the case of a "game against nature." Reinforcement learning also is not restricted to problems in which behavior breaks down into separate episodes, like the separate games of tic-tac-toe, with reward only at the end of each episode. It is just as applicable when behavior continues indefinitely and when rewards of various magnitudes can be received at any time.

Tic-tac-toe has a relatively small, finite state set, whereas reinforcement learning can be used when the state set is very large, or even infinite. For example, Gerry Tesauro (1992, 1995) combined the algorithm described above with an artificial neural network to learn to play backgammon, which has approximately 10^{20} states. With this many states it is impossible ever to experience more than a small fraction of them. Tesauro's program learned to play far better than any previous program, and now plays at the level of the world's best human players (see Chapter 15). The neural network provides the program with the ability to generalize from its experience, so that in new states it selects moves based on information saved from similar states faced in the past, as determined by its network. How well a reinforcement learning system can work in problems with such large state sets is intimately tied to how appropriately it can generalize from past experience. It is in this role that we have the greatest need for supervised learning methods with reinforcement learning. Neural networks are not the only, or necessarily the best, way to do this.

In this tic-tac-toe example, learning started with no prior knowledge beyond the rules of the game, but reinforcement learning by no means entails a tabula rasa view of learning and intelligence. On the contrary, prior information can be incorporated into reinforcement learning in a variety of ways that can be critical for efficient learning. We also had access to the true state in the tic-tac-toe example, whereas reinforcement learning can also be applied when part of the state is hidden, or when different states appear to the learner to be the same. That case, however, is substantially more difficult, and we do not cover it significantly in this book.

Finally, the tic-tac-toe player was able to look ahead and know the states that would result from each of its possible moves. To do this, it had to have a model of the game that allowed it to "think about" how its environment would change in response to moves that it might never make. Many problems are like this, but in others even a short-term model of the effects of actions is lacking. Reinforcement learning can be applied in either case. No model is required, but models can easily be used if they are available or can be learned.

Exercise 1.1: Self-Play Suppose, instead of playing against a random

opponent, the reinforcement learning algorithm described above played against itself. What do you think would happen in this case? Would it learn a different way of playing?

Exercise 1.2: *Symmetries* Many tic-tac-toe positions appear different but are really the same because of symmetries. How might we amend the reinforcement learning algorithm described above to take advantage of this? In what ways would this improve it? Now think again. Suppose the opponent did not take advantage of symmetries. In that case, should we? Is it true, then, that symmetrically equivalent positions should necessarily have the same value?

Exercise 1.3: *Greedy Play* Suppose the reinforcement learning player was *greedy*, that is, it always played the move that brought it to the position that it rated the best. Would it learn to play better, or worse, than a nongreedy player? What problems might occur?

Exercise 1.4: *Learning from Exploration* Suppose learning updates occurred after *all* moves, including exploratory moves. If the step-size parameter is appropriately reduced over time, then the state values would converge to a set of probabilities. What are the two sets of probabilities computed when we do, and when we do not, learn from exploratory moves? Assuming that we do continue to make exploratory moves, which set of probabilities might be better to learn? Which would result in more wins?

Exercise 1.5: *Other Improvements* Can you think of other ways to improve the reinforcement learning player? Can you think of any better way to solve the tic-tac-toe problem as posed?

1.5 Summary

Reinforcement learning is a computational approach to understanding and automating goal-directed learning and decision-making. It is distinguished from other computational approaches by its emphasis on learning by the individual from direct interaction with its environment, without relying on exemplary supervision or complete models of the environment. In our opinion, reinforcement learning is the first field to seriously address the computational issues that arise when learning from interaction with an environment in order to achieve long-term goals.

Reinforcement learning uses a formal framework defining the interaction between a learning agent and its environment in terms of states, actions, and rewards. This framework is intended to be a simple way of representing essential features of the artificial intelligence problem. These features include a

sense of cause and effect, a sense of uncertainty and nondeterminism, and the existence of explicit goals.

The concepts of value and value functions are the key features of the reinforcement learning methods that we consider in this book. We take the position that value functions are essential for efficient search in the space of policies. Their use of value functions distinguishes reinforcement learning methods from evolutionary methods that search directly in policy space guided by scalar evaluations of entire policies.

1.6 History of Reinforcement Learning

The history of reinforcement learning has two main threads, both long and rich, that were pursued independently before intertwining in modern reinforcement learning. One thread concerns learning by trial and error and started in the psychology of animal learning. This thread runs through some of the earliest work in artificial intelligence and led to the revival of reinforcement learning in the early 1980s. The other thread concerns the problem of optimal control and its solution using value functions and dynamic programming. For the most part, this thread did not involve learning. Although the two threads have been largely independent, the exceptions revolve around a third, less distinct thread concerning temporal-difference methods such as used in the tic-tac-toe example in this chapter. All three threads came together in the late 1980s to produce the modern field of reinforcement learning as we present it in this book.

The thread focusing on trial-and-error learning is the one with which we are most familiar and about which we have the most to say in this brief history. Before doing that, however, we briefly discuss the optimal control thread.

The term "optimal control" came into use in the late 1950s to describe the problem of designing a controller to minimize a measure of a dynamical system's behavior over time. One of the approaches to this problem was developed in the mid-1950s by Richard Bellman and others through extending a nineteenth century theory of Hamilton and Jacobi. This approach uses the concepts of a dynamical system's state and of a value function, or "optimal return function," to define a functional equation, now often called the Bellman equation. The class of methods for solving optimal control problems by solving this equation came to be known as dynamic programming (Bellman, 1957a). Bellman (1957b) also introduced the discrete stochastic version of the optimal control problem known as Markovian decision processes (MDPs), and Ron Howard (1960) devised the policy iteration method for MDPs. All of

these are essential elements underlying the theory and algorithms of modern reinforcement learning.

Dynamic programming is widely considered the only feasible way of solving general stochastic optimal control problems. It suffers from what Bellman called "the curse of dimensionality," meaning that its computational requirements grow exponentially with the number of state variables, but it is still far more efficient and more widely applicable than any other general method. Dynamic programming has been extensively developed since the late 1950s, including extensions to partially observable MDPs (surveyed by Lovejoy, 1991), many applications (surveyed by White, 1985, 1988, 1993), approximation methods (surveyed by Rust, 1996), and asynchronous methods (Bertsekas, 1982, 1983). Many excellent modern treatments of dynamic programming are available (e.g., Bertsekas, 1995; Puterman, 1994; Ross, 1983; and Whittle, 1982, 1983). Bryson (1996) provides an authoritative history of optimal control.

In this book, we consider all of the work in optimal control also to be, in a sense, work in reinforcement learning. We define reinforcement learning as any effective way of solving reinforcement learning problems, and it is now clear that these problems are closely related to optimal control problems, particularly those formulated as MDPs. Accordingly, we must consider the solution methods of optimal control, such as dynamic programming, also to be reinforcement learning methods. Of course, almost all of these methods require complete knowledge of the system to be controlled, and for this reason it feels a little unnatural to say that they are part of reinforcement *learning*. On the other hand, many dynamic programming methods are incremental and iterative. Like learning methods, they gradually reach the correct answer through successive approximations. As we show in the rest of this book, these similarities are far more than superficial. The theories and solution methods for the cases of complete and incomplete knowledge are so closely related that we feel they must be considered together as part of the same subject matter.

Let us return now to the other major thread leading to the modern field of reinforcement learning, that centered on the idea of trial-and-error learning. This thread began in psychology, where "reinforcement" theories of learning are common. Perhaps the first to succinctly express the essence of trial-and-error learning was Edward Thorndike. We take this essence to be the idea that actions followed by good or bad outcomes have their tendency to be reselected altered accordingly. In Thorndike's words:

> Of several responses made to the same situation, those which are accompanied or closely followed by satisfaction to the animal will, other things being equal, be more firmly connected with the sit-

> uation, so that, when it recurs, they will be more likely to recur; those which are accompanied or closely followed by discomfort to the animal will, other things being equal, have their connections with that situation weakened, so that, when it recurs, they will be less likely to occur. The greater the satisfaction or discomfort, the greater the strengthening or weakening of the bond. (Thorndike, 1911, p. 244)

Thorndike called this the "Law of Effect" because it describes the effect of reinforcing events on the tendency to select actions. Although sometimes controversial (e.g., see Kimble, 1961, 1967; Mazur, 1994), the Law of Effect is widely regarded as an obvious basic principle underlying much behavior (e.g., Hilgard and Bower, 1975; Dennett, 1978; Campbell, 1960; Cziko, 1995).

The Law of Effect includes the two most important aspects of what we mean by trial-and-error learning. First, it is *selectional*, meaning that it involves trying alternatives and selecting among them by comparing their consequences. Second, it is *associative*, meaning that the alternatives found by selection are associated with particular situations. Natural selection in evolution is a prime example of a selectional process, but it is not associative. Supervised learning is associative, but not selectional. It is the combination of these two that is essential to the Law of Effect and to trial-and-error learning. Another way of saying this is that the Law of Effect is an elementary way of combining *search* and *memory*: search in the form of trying and selecting among many actions in each situation, and memory in the form of remembering what actions worked best, associating them with the situations in which they were best. Combining search and memory in this way is essential to reinforcement learning.

In early artificial intelligence, before it was distinct from other branches of engineering, several researchers began to explore trial-and-error learning as an engineering principle. The earliest computational investigations of trial-and-error learning were perhaps by Minsky and by Farley and Clark, both in 1954. In his Ph.D. dissertation, Minsky discussed computational models of reinforcement learning and described his construction of an analog machine composed of components he called SNARCs (Stochastic Neural-Analog Reinforcement Calculators). Farley and Clark described another neural-network learning machine designed to learn by trial and error. In the 1960s the terms "reinforcement" and "reinforcement learning" were used in the engineering literature for the first time (e.g., Waltz and Fu, 1965; Mendel, 1966; Fu, 1970; Mendel and McClaren, 1970). Particularly influential was Minsky's paper "Steps Toward Artificial Intelligence" (Minsky, 1961), which discussed several issues relevant to reinforcement learning, including what he called the *credit assignment problem*: How do you distribute credit for success among the many decisions that may have been involved in producing it? All of the methods we

discuss in this book are, in a sense, directed toward solving this problem.

The interests of Farley and Clark (1954; Clark and Farley, 1955) shifted from trial-and-error learning to generalization and pattern recognition, that is, from reinforcement learning to supervised learning. This began a pattern of confusion about the relationship between these types of learning. Many researchers seemed to believe that they were studying reinforcement learning when they were actually studying supervised learning. For example, neural network pioneers such as Rosenblatt (1962) and Widrow and Hoff (1960) were clearly motivated by reinforcement learning—they used the language of rewards and punishments—but the systems they studied were supervised learning systems suitable for pattern recognition and perceptual learning. Even today, researchers and textbooks often minimize or blur the distinction between these types of learning. Some modern neural-network textbooks use the term "trial-and-error" to describe networks that learn from training examples because they use error information to update connection weights. This is an understandable confusion, but it substantially misses the essential selectional character of trial-and-error learning.

Partly as a result of these confusions, research into genuine trial-and-error learning became rare in the the 1960s and 1970s. In the next few paragraphs we discuss some of the exceptions and partial exceptions to this trend.

One of these was the work by a New Zealand researcher named John Andreae. Andreae (1963) developed a system called STeLLA that learned by trial and error in interaction with its environment. This system included an internal model of the world and, later, an "internal monologue" to deal with problems of hidden state (Andreae, 1969a). Andreae's later work (1977) placed more emphasis on learning from a teacher, but still included trial and error. Unfortunately, his pioneering research was not well known, and did not greatly impact subsequent reinforcement learning research.

More influential was the work of Donald Michie. In 1961 and 1963 he described a simple trial-and-error learning system for learning how to play tic-tac-toe (or naughts and crosses) called MENACE (for Matchbox Educable Naughts and Crosses Engine). It consisted of a matchbox for each possible game position, each matchbox containing a number of colored beads, a different color for each possible move from that position. By drawing a bead at random from the matchbox corresponding to the current game position, one could determine MENACE's move. When a game was over, beads were added to or removed from the boxes used during play to reinforce or punish MENACE's decisions. Michie and Chambers (1968) described another tic-tac-toe reinforcement learner called GLEE (Game Learning Expectimaxing Engine) and a reinforcement learning controller called BOXES. They applied BOXES

to the task of learning to balance a pole hinged to a movable cart on the basis of a failure signal occurring only when the pole fell or the cart reached the end of a track. This task was adapted from the earlier work of Widrow and Smith (1964), who used supervised learning methods, assuming instruction from a teacher already able to balance the pole. Michie and Chambers's version of pole-balancing is one of the best early examples of a reinforcement learning task under conditions of incomplete knowledge. It influenced much later work in reinforcement learning, beginning with some of our own studies (Barto, Sutton, and Anderson, 1983; Sutton, 1984). Michie has consistently emphasized the role of trial and error and learning as essential aspects of artificial intelligence (Michie, 1974).

Widrow, Gupta, and Maitra (1973) modified the LMS algorithm of Widrow and Hoff (1960) to produce a reinforcement learning rule that could learn from success and failure signals instead of from training examples. They called this form of learning "selective bootstrap adaptation" and described it as "learning with a critic" instead of "learning with a teacher." They analyzed this rule and showed how it could learn to play blackjack. This was an isolated foray into reinforcement learning by Widrow, whose contributions to supervised learning were much more influential.

Research on *learning automata* had a more direct influence on the trial-and-error thread leading to modern reinforcement learning research. These are methods for solving a nonassociative, purely selectional learning problem known as the *n-armed bandit* by analogy to a slot machine, or "one-armed bandit," except with n levers (see Chapter 2). Learning automata are simple, low-memory machines for solving this problem. Learning automata originated in Russia with the work of Tsetlin (1973) and has been extensively developed since then within engineering (see Narendra and Thathachar, 1974, 1989). Barto and Anandan (1985) extended these methods to the associative case.

John Holland (1975) outlined a general theory of adaptive systems based on selectional principles. His early work concerned trial and error primarily in its nonassociative form, as in evolutionary methods and the n-armed bandit. In 1986 he introduced *classifier systems*, true reinforcement learning systems including association and value functions. A key component of Holland's classifier systems was always a *genetic algorithm*, an evolutionary method whose role was to evolve useful representations. Classifier systems have been extensively developed by many researchers to form a major branch of reinforcement learning research (e.g., see Goldberg, 1989; Wilson, 1994), but genetic algorithms—which by themselves are not reinforcement learning systems—have received much more attention.

The individual most responsible for reviving the trial-and-error thread to

reinforcement learning within artificial intelligence was Harry Klopf (1972, 1975, 1982). Klopf recognized that essential aspects of adaptive behavior were being lost as learning researchers came to focus almost exclusively on supervised learning. What was missing, according to Klopf, were the hedonic aspects of behavior, the drive to achieve some result from the environment, to control the environment toward desired ends and away from undesired ends. This is the essential idea of trial-and-error learning. Klopf's ideas were especially influential on the authors because our assessment of them (Barto and Sutton, 1981a) led to our appreciation of the distinction between supervised and reinforcement learning, and to our eventual focus on reinforcement learning. Much of the early work that we and colleagues accomplished was directed toward showing that reinforcement learning and supervised learning were indeed different (Barto, Sutton, and Brouwer, 1981; Barto and Sutton, 1981b; Barto and Anandan, 1985). Other studies showed how reinforcement learning could address important problems in neural network learning, in particular, how it could produce learning algorithms for multilayer networks (Barto, Anderson, and Sutton, 1982; Barto and Anderson, 1985; Barto and Anandan, 1985; Barto, 1985, 1986; Barto and Jordan, 1987).

We turn now to the third thread to the history of reinforcement learning, that concerning temporal-difference learning. Temporal-difference learning methods are distinctive in being driven by the difference between temporally successive estimates of the same quantity—for example, of the probability of winning in the tic-tac-toe example. This thread is smaller and less distinct than the other two, but it has played a particularly important role in the field, in part because temporal-difference methods seem to be new and unique to reinforcement learning.

The origins of temporal-difference learning are in part in animal learning psychology, in particular, in the notion of *secondary reinforcers*. A secondary reinforcer is a stimulus that has been paired with a primary reinforcer such as food or pain and, as a result, has come to take on similar reinforcing properties. Minsky (1954) may have been the first to realize that this psychological principle could be important for artificial learning systems. Arthur Samuel (1959) was the first to propose and implement a learning method that included temporal-difference ideas, as part of his celebrated checkers-playing program. Samuel made no reference to Minsky's work or to possible connections to animal learning. His inspiration apparently came from Claude Shannon's (1950) suggestion that a computer could be programmed to use an evaluation function to play chess, and that it might be able to improve its play by modifying this function on-line. (It is possible that these ideas of Shannon's also influenced Bellman, but we know of no evidence for this.) Minsky (1961) extensively discussed Samuel's work in his "Steps" paper, suggesting the connection to

secondary reinforcement theories, both natural and artificial.

As we have discussed, in the decade following the work of Minsky and Samuel, little computational work was done on trial-and-error learning, and apparently no computational work at all was done on temporal-difference learning. In 1972, Klopf brought trial-and-error learning together with an important component of temporal-difference learning. Klopf was interested in principles that would scale to learning in large systems, and thus was intrigued by notions of local reinforcement, whereby subcomponents of an overall learning system could reinforce one another. He developed the idea of "generalized reinforcement," whereby every component (nominally, every neuron) views all of its inputs in reinforcement terms: excitatory inputs as rewards and inhibitory inputs as punishments. This is not the same idea as what we now know as temporal-difference learning, and in retrospect it is farther from it than was Samuel's work. On the other hand, Klopf linked the idea with trial-and-error learning and related it to the massive empirical database of animal learning psychology.

Sutton (1978a, 1978b, 1978c) developed Klopf's ideas further, particularly the links to animal learning theories, describing learning rules driven by changes in temporally successive predictions. He and Barto refined these ideas and developed a psychological model of classical conditioning based on temporal-difference learning (Sutton and Barto, 1981a; Barto and Sutton, 1982). There followed several other influential psychological models of classical conditioning based on temporal-difference learning (e.g., Klopf, 1988; Moore et al., 1986; Sutton and Barto, 1987, 1990). Some neuroscience models developed at this time are well interpreted in terms of temporal-difference learning (Hawkins and Kandel, 1984; Byrne, Gingrich, and Baxter, 1990; Gelperin, Hopfield, and Tank, 1985; Tesauro, 1986; Friston et al., 1994), although in most cases there was no historical connection. A recent summary of links between temporal-difference learning and neuroscience ideas is provided by Schultz, Dayan, and Montague (1997).

Our early work on temporal-difference learning was strongly influenced by animal learning theories and by Klopf's work. Relationships to Minsky's "Steps" paper and to Samuel's checkers players appear to have been recognized only afterward. By 1981, however, we were fully aware of all the prior work mentioned above as part of the temporal-difference and trial-and-error threads. At this time we developed a method for using temporal-difference learning in trial-and-error learning, known as the *actor–critic architecture*, and applied this method to Michie and Chambers's pole-balancing problem (Barto, Sutton, and Anderson, 1983). This method was extensively studied in Sutton's (1984) Ph.D. dissertation and extended to use backpropagation neural networks in Anderson's (1986) Ph.D. dissertation. Around this time, Holland (1986) incor-

porated temporal-difference ideas explicitly into his classifier systems. A key step was taken by Sutton in 1988 by separating temporal-difference learning from control, treating it as a general prediction method. That paper also introduced the TD(λ) algorithm and proved some of its convergence properties.

As we were finalizing our work on the actor–critic architecture in 1981, we discovered a paper by Ian Witten (1977) that contains the earliest known publication of a temporal-difference learning rule. He proposed the method that we now call tabular TD(0) for use as part of an adaptive controller for solving MDPs. Witten's work was a descendant of Andreae's early experiments with STeLLA and other trial-and-error learning systems. Thus, Witten's 1977 paper spanned both major threads of reinforcement learning research—trial-and-error learning and optimal control—while making a distinct early contribution to temporal-difference learning.

Finally, the temporal-difference and optimal control threads were fully brought together in 1989 with Chris Watkins's development of Q-learning. This work extended and integrated prior work in all three threads of reinforcement learning research. Paul Werbos (1987) contributed to this integration by arguing for the convergence of trial-and-error learning and dynamic programming since 1977. By the time of Watkins's work there had been tremendous growth in reinforcement learning research, primarily in the machine learning subfield of artificial intelligence, but also in neural networks and artificial intelligence more broadly. In 1992, the remarkable success of Gerry Tesauro's backgammon playing program, TD-Gammon, brought additional attention to the field. Other important contributions made in the recent history of reinforcement learning are too numerous to mention in this brief account; we cite these at the end of the individual chapters in which they arise.

1.7 Bibliographical Remarks

For additional general coverage of reinforcement learning, we refer the reader to the books by Bertsekas and Tsitsiklis (1996) and Kaelbling (1993a). Two special issues of the journal *Machine Learning* focus on reinforcement learning: Sutton (1992) and Kaelbling (1996). Useful surveys are provided by Barto (1995b); Kaelbling, Littman, and Moore (1996); and Keerthi and Ravindran (1997).

The example of Phil's breakfast in this chapter was inspired by Agre (1988). We direct the reader to Chapter 6 for references to the kind of temporal-difference method we used in the tic-tac-toe example.

Modern attempts to relate the kinds of algorithms used in reinforcement

learning to the nervous system are made by Hampson (1989), Friston et al. (1994), Barto (1995a), Houk, Adams, and Barto (1995), Montague, Dayan, and Sejnowski (1996), and Schultz, Dayan, and Montague (1997).

Chapter 2

Bandit Problems

The most important feature distinguishing reinforcement learning from other types of learning is that it uses training information that *evaluates* the actions taken rather than *instructs* by giving correct actions. This is what creates the need for active exploration, for an explicit trial-and-error search for good behavior. Purely evaluative feedback indicates how good the action taken is, but not whether it is the best or the worst action possible. Evaluative feedback is the basis of methods for function optimization, including evolutionary methods. Purely instructive feedback, on the other hand, indicates the correct action to take, independently of the action actually taken. This kind of feedback is the basis of supervised learning, which includes large parts of pattern classification, artificial neural networks, and system identification. In their pure forms, these two kinds of feedback are quite distinct: evaluative feedback depends entirely on the action taken, whereas instructive feedback is independent of the action taken. There are also interesting intermediate cases in which evaluation and instruction blend together.

In this chapter we study the evaluative aspect of reinforcement learning in a simplified setting, one that does not involve learning to act in more than one situation. This *nonassociative* setting is the one in which most prior work involving evaluative feedback has been done, and it avoids much of the complexity of the full reinforcement learning problem. Studying this case will enable us to see most clearly how evaluative feedback differs from, and yet can be combined with, instructive feedback.

The particular nonassociative, evaluative feedback problem that we explore is a simple version of the n-armed bandit problem. We use this problem to introduce a number of basic learning methods which we extend in later chapters to apply to the full reinforcement learning problem. At the end of this chapter, we take a step closer to the full reinforcement learning problem by discussing

what happens when the bandit problem becomes associative, that is, when actions are taken in more than one situation.

2.1 An n-Armed Bandit Problem

Consider the following learning problem. You are faced repeatedly with a choice among n different options, or actions. After each choice you receive a numerical reward chosen from a stationary probability distribution that depends on the action you selected. Your objective is to maximize the expected total reward over some time period, for example, over 1000 action selections, or *time steps*.

This is the original form of the *n-armed bandit problem*, so named by analogy to a slot machine, or "one-armed bandit," except that it has n levers instead of one. Each action selection is like a play of one of the slot machine's levers, and the rewards are the payoffs for hitting the jackpot. Through repeated action selections you are to maximize your winnings by concentrating your actions on the best levers. Another analogy is that of a doctor choosing between experimental treatments for a series of seriously ill patients. Each action selection is a treatment selection, and each reward is the survival or well-being of the patient. Today the term "n-armed bandit problem" is sometimes used for a generalization of the problem described above, but in this book we use it to refer just to this simple case.

In our n-armed bandit problem, each action has an expected or mean reward given that that action is selected; let us call this the *value* of that action. If you knew the value of each action, then it would be trivial to solve the n-armed bandit problem: you would always select the action with highest value. We assume that you do not know the action values with certainty, although you may have estimates.

If you maintain estimates of the action values, then at any time step there is at least one action whose estimated value is greatest. We call this a *greedy* action. If you select a greedy action, we say that you are *exploiting* your current knowledge of the values of the actions. If instead you select one of the nongreedy actions, then we say you are *exploring*, because this enables you to improve your estimate of the nongreedy action's value. Exploitation is the right thing to do to maximize the expected reward on the one step, but exploration may produce the greater total reward in the long run. For example, suppose the greedy action's value is known with certainty, while several other actions are estimated to be nearly as good but with substantial uncertainty. The uncertainty is such that at least one of these other actions probably is

actually better than the greedy action, but you don't know which one. If you have many time steps ahead on which to make action selections, then it may be better to explore the nongreedy actions and discover which of them are better than the greedy action. Reward is lower in the short run, during exploration, but higher in the long run because after you have discovered the better actions, you can exploit *them* many times. Because it is not possible both to explore and to exploit with any single action selection, one often refers to the "conflict" between exploration and exploitation.

In any specific case, whether it is better to explore or exploit depends in a complex way on the precise values of the estimates, uncertainties, and the number of remaining steps. There are many sophisticated methods for balancing exploration and exploitation for particular mathematical formulations of the n-armed bandit and related problems. However, most of these methods make strong assumptions about stationarity and prior knowledge that are either violated or impossible to verify in applications and in the full reinforcement learning problem that we consider in subsequent chapters. The guarantees of optimality or bounded loss for these methods are of little comfort when the assumptions of their theory do not apply.

In this book we do not worry about balancing exploration and exploitation in a sophisticated way; we worry only about balancing them at all. In this chapter we present several simple balancing methods for the n-armed bandit problem and show that they work much better than methods that always exploit. The need to balance exploration and exploitation is a distinctive challenge that arises in reinforcement learning; the simplicity of the n-armed bandit problem enables us to show this in a particularly clear form.

2.2 Action-Value Methods

We begin by looking more closely at some simple methods for estimating the values of actions and for using the estimates to make action selection decisions. In this chapter, we denote the true (actual) value of action a as $q_*(a)$, and the estimated value on the tth time step as $Q_t(a)$. Recall that the true value of an action is the mean reward received when that action is selected. One natural way to estimate this is by averaging the rewards actually received when the action was selected. In other words, if by the tth time step action a has been chosen K_a times prior to t, yielding rewards $R_1, R_2, \ldots, R_{K_a}$, then its value is estimated to be

$$Q_t(a) = \frac{R_1 + R_2 + \cdots + R_{K_a}}{K_a}. \tag{2.1}$$

If $K_a = 0$, then we define $Q_t(a)$ instead as some default value, such as $Q_1(a) = 0$. As $K_a \to \infty$, by the law of large numbers, $Q_t(a)$ converges to $q_*(a)$. We call this the *sample-average* method for estimating action values because each estimate is a simple average of the sample of relevant rewards. Of course this is just one way to estimate action values, and not necessarily the best one. Nevertheless, for now let us stay with this simple estimation method and turn to the question of how the estimates might be used to select actions.

The simplest action selection rule is to select the action (or one of the actions) with highest estimated action value, that is, to select at step t one of the greedy actions, A_t^*, for which $Q_t(A_t^*) = \max_a Q_t(a)$. This method always exploits current knowledge to maximize immediate reward; it spends no time at all sampling apparently inferior actions to see if they might really be better. A simple alternative is to behave greedily most of the time, but every once in a while, say with small probability ε, instead to select randomly from amongst all the actions with equal probability independently of the action-value estimates. We call methods using this near-greedy action selection rule *ε-greedy* methods. An advantage of these methods is that, in the limit as the number of plays increases, every action will be sampled an infinite number of times, guaranteeing that $K_a \to \infty$ for all a, and thus ensuring that all the $Q_t(a)$ converge to $q_*(a)$. This of course implies that the probability of selecting the optimal action converges to greater than $1 - \varepsilon$, that is, to near certainty. These are just asymptotic guarantees, however, and say little about the practical effectiveness of the methods.

To roughly assess the relative effectiveness of the greedy and ε-greedy methods, we compared them numerically on a suite of test problems. This was a set of 2000 randomly generated n-armed bandit tasks with $n = 10$. For each bandit, the action values, $q_*(a)$, $a = 1, \ldots, 10$, were selected according to a normal (Gaussian) distribution with mean 0 and variance 1. On tth time step with a given bandit, the actual reward R_t was the $q_*(A_t)$ for the bandit (where A_t was the action selected) plus a normally distributed noise term that was mean 0 and variance 1. Averaging over bandits, we can plot the performance and behavior of various methods as they improve with experience over 1000 steps, as in Figure 2.1. We call this suite of test tasks the *10-armed testbed*.

Figure 2.1 compares a greedy method with two ε-greedy methods ($\varepsilon = 0.01$ and $\varepsilon = 0.1$), as described above, on the 10-armed testbed. Both methods formed their action-value estimates using the sample-average technique. The upper graph shows the increase in expected reward with experience. The greedy method improved slightly faster than the other methods at the very beginning, but then leveled off at a lower level. It achieved a reward per step of only about 1, compared with the best possible of about 1.55 on this testbed. The greedy method performs significantly worse in the long run because it

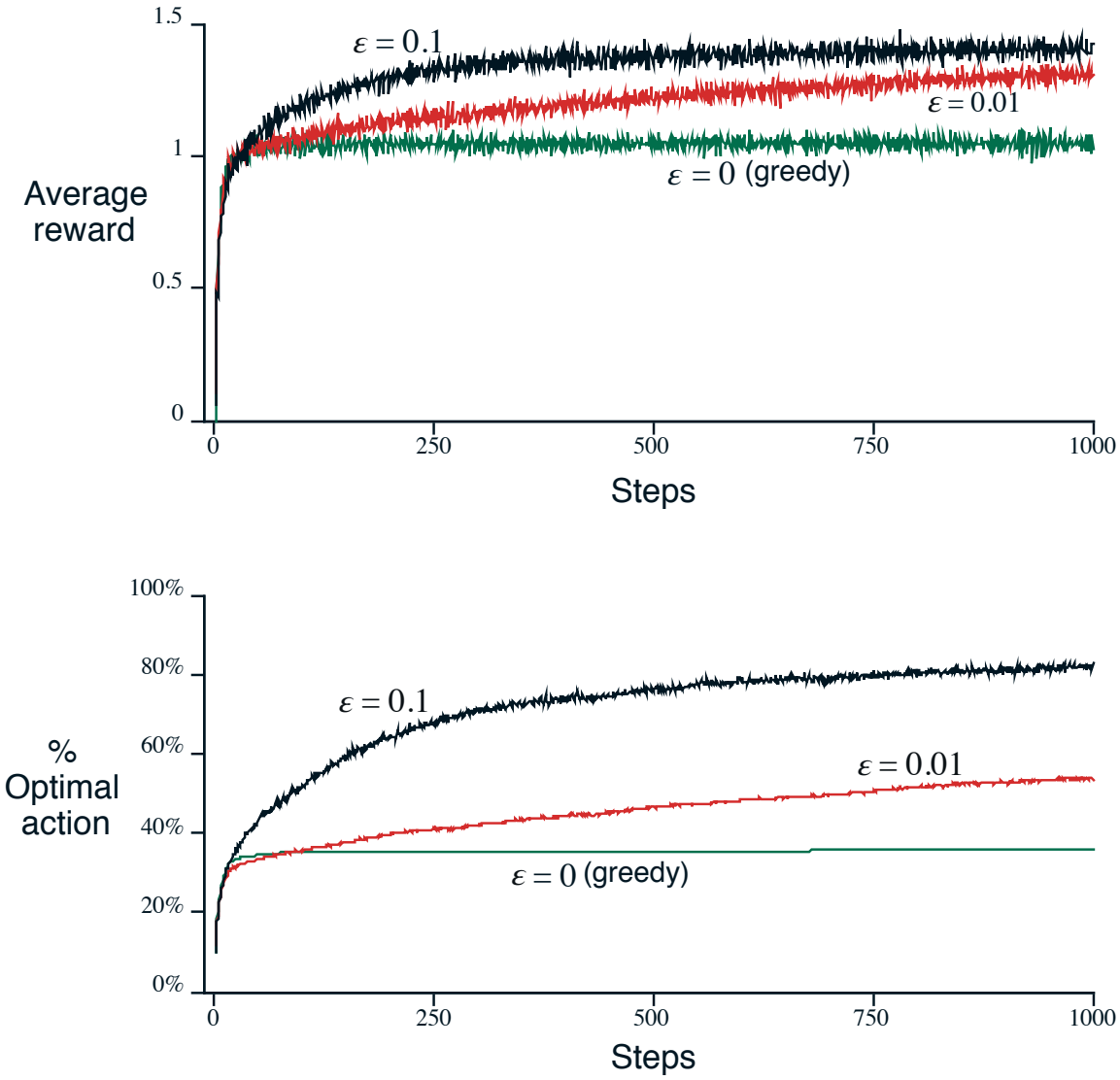


Figure 2.1: Average performance of ε-greedy action-value methods on the 10-armed testbed. These data are averages over 2000 tasks. All methods used sample averages as their action-value estimates. The detailed structure at the beginning of these curves depends on how actions are selected when multiple actions have the same maximal action value. Here such ties were broken randomly. An alternative that has a similar effect is to add a very small amount of randomness to each of the initial action values, so that ties effectively never happen.

often gets stuck performing suboptimal actions. The lower graph shows that the greedy method found the optimal action in only approximately one-third of the tasks. In the other two-thirds, its initial samples of the optimal action were disappointing, and it never returned to it. The ε-greedy methods eventually perform better because they continue to explore, and to improve their chances of recognizing the optimal action. The $\varepsilon = 0.1$ method explores more, and usually finds the optimal action earlier, but never selects it more than 91% of the time. The $\varepsilon = 0.01$ method improves more slowly, but eventually performs better than the $\varepsilon = 0.1$ method on both performance measures. It is also possible to reduce ε over time to try to get the best of both high and low values.

The advantage of ε-greedy over greedy methods depends on the task. For example, suppose the reward variance had been larger, say 10 instead of 1. With noisier rewards it takes more exploration to find the optimal action, and ε-greedy methods should fare even better relative to the greedy method. On the other hand, if the reward variances were zero, then the greedy method would know the true value of each action after trying it once. In this case the greedy method might actually perform best because it would soon find the optimal action and then never explore. But even in the deterministic case, there is a large advantage to exploring if we weaken some of the other assumptions. For example, suppose the bandit task were nonstationary, that is, that the true values of the actions changed over time. In this case exploration is needed even in the deterministic case to make sure one of the nongreedy actions has not changed to become better than the greedy one. As we will see in the next few chapters, effective nonstationarity is the case most commonly encountered in reinforcement learning. Even if the underlying task is stationary and deterministic, the learner faces a set of banditlike decision tasks each of which changes over time due to the learning process itself. Reinforcement learning requires a balance between exploration and exploitation.

Exercise 2.1 In the comparison shown in Figure 2.1, which method will perform best in the long run in terms of cumulative reward and cumulative probability of selecting the best action? How much better will it be? Express your answer quantitatively.

2.3 Softmax Action Selection

Although ε-greedy action selection is an effective and popular means of balancing exploration and exploitation in reinforcement learning, one drawback is that when it explores it chooses equally among all actions. This means that it is as likely to choose the worst-appearing action as it is to choose the

next-to-best action. In tasks where the worst actions are very bad, this may be unsatisfactory. The obvious solution is to vary the action probabilities as a graded function of estimated value. The greedy action is still given the highest selection probability, but all the others are ranked and weighted according to their value estimates. These are called *softmax* action selection rules. The most common softmax method uses a Gibbs, or Boltzmann, distribution. It chooses action a on the tth time step with probability

$$\frac{e^{Q_t(a)/\tau}}{\sum_{i=1}^{n} e^{Q_t(i)/\tau}}, \tag{2.2}$$

where e is the exponent of the natural logarithm (approximately 2.718), and τ is a positive parameter called the *temperature*. High temperatures cause the actions to be all (nearly) equiprobable. Low temperatures cause a greater difference in selection probability for actions that differ in their value estimates. In the limit as $\tau \to 0$, softmax action selection becomes the same as greedy action selection. Of course, the softmax effect can be produced in a large number of ways other than by a Gibbs distribution. For example, one could simply add a random number from a long-tailed distribution to each $Q_t(a)$ and then pick the action whose sum was largest.

Whether softmax action selection or ε-greedy action selection is better is unclear and may depend on the task and on human factors. Both methods have only one parameter that must be set. Most people find it easier to set the ε parameter with confidence; setting τ requires knowledge of the likely action values and of powers of e. We know of no careful comparative studies of these two simple action-selection rules.

In recent years, the use of softmax action selection as described here has tended to be disparaged. This is because of the difficulty of selecting the temperature parameter without knowing a lot about the action values. One way to think about it is that the units are wrong. Action values are in units of reward, and these are converted through the exponential function into action probabilities. There is no natural way to relate them without knowing what the likely range of true action values are. However, we will return to the idea of softmax action selection in a future chapter where we consider policy-based methods for reinforcement learning.

Exercise 2.2 (programming) How does the softmax action selection method using the Gibbs distribution fare on the 10-armed testbed? Implement the method and run it at several temperatures to produce graphs similar to those in Figure 2.1. To verify your code, first implement the ε-greedy methods and reproduce some specific aspect of the results in Figure 2.1.

*Exercise 2.3 Show that in the case of two actions, the softmax operation using the Gibbs distribution becomes the logistic, or sigmoid, function commonly used in artificial neural networks. What effect does the temperature parameter have on the function?

2.4 Incremental Implementation

The action-value methods we have discussed so far all estimate action values as sample averages of observed rewards. The obvious implementation is to maintain, for each action a, a record of all the rewards that have followed the selection of that action. Then, when the estimate of the value of action a is needed at time t, it can be computed according to (2.1), which we repeat here:

$$Q_t(a) = \frac{R_1 + R_2 + \cdots + R_{K_a}}{K_a},$$

where here $R_1, \ldots, R_{K_a}$ are all the rewards received following all selections of action a prior to play t. A problem with this straightforward implementation is that its memory and computational requirements grow over time without bound. That is, each additional reward following a selection of action a requires more memory to store it and results in more computation being required to determine $Q_t(a)$.

As you might suspect, this is not really necessary. It is easy to devise incremental update formulas for computing averages with small, constant computation required to process each new reward. For some action, let Q_k denote the estimate for its kth reward, that is, the average of its first $k-1$ rewards. Given this average and a kth reward for the action, R_k, then the average of all k rewards can be computed by

$$\begin{aligned} Q_{k+1} &= \frac{1}{k}\sum_{i=1}^{k} R_i \\ &= \frac{1}{k}\left(R_k + \sum_{i=1}^{k-1} R_i\right) \\ &= \frac{1}{k}\Big(R_k + (k-1)Q_k + Q_k - Q_k\Big) \\ &= \frac{1}{k}\Big(R_k + kQ_k - Q_k\Big) \\ &= Q_k + \frac{1}{k}\Big[R_k - Q_k\Big], \end{aligned} \tag{2.3}$$

which holds even for $k = 1$, obtaining $Q_2 = R_1$ for arbitrary Q_1. This implementation requires memory only for Q_k and k, and only the small computation (2.3) for each new reward.

The update rule (2.3) is of a form that occurs frequently throughout this book. The general form is

$$NewEstimate \leftarrow OldEstimate + StepSize \Big[Target - OldEstimate\Big]. \quad (2.4)$$

The expression $[Target - OldEstimate]$ is an *error* in the estimate. It is reduced by taking a step toward the "Target." The target is presumed to indicate a desirable direction in which to move, though it may be noisy. In the case above, for example, the target is the kth reward.

Note that the step-size parameter ($StepSize$) used in the incremental method described above changes from time step to time step. In processing the kth reward for action a, that method uses a step-size parameter of $\frac{1}{k}$. In this book we denote the step-size parameter by the symbol α or, more generally, by $\alpha_t(a)$. We sometimes use the informal shorthand $\alpha = \frac{1}{k}$ to refer to this case, leaving the dependence of k on the action implicit.

Exercise 2.4 Give pseudocode for a complete algorithm for the n-armed bandit problem. Use greedy action selection and incremental computation of action values with $\alpha = \frac{1}{k}$ step-size parameter. Assume a function $bandit(a)$ that takes an action and returns a reward. Use arrays and variables; do not subscript anything by the time index t (for examples of this style of pseudocode, see Figures 4.1 and 4.3). Indicate how the action values are initialized and updated after each reward. Indicate how the step-size parameters are set for each action as a function of how many times it has been tried.

2.5 Tracking a Nonstationary Problem

The averaging methods discussed so far are appropriate in a stationary environment, but not if the bandit is changing over time. As noted earlier, we often encounter reinforcement learning problems that are effectively nonstationary. In such cases it makes sense to weight recent rewards more heavily than long-past ones. One of the most popular ways of doing this is to use a constant step-size parameter. For example, the incremental update rule (2.3) for updating an average Q_k of the $k-1$ past rewards is modified to be

$$Q_{k+1} = Q_k + \alpha\Big[R_k - Q_k\Big], \quad (2.5)$$

where the step-size parameter, α, $0 < \alpha \leq 1$, is constant. This results in Q_{k+1} being a weighted average of past rewards and the initial estimate Q_1:

$$\begin{aligned} Q_{k+1} &= Q_k + \alpha\Big[R_k - Q_k\Big] \\ &= \alpha R_k + (1-\alpha)Q_k \\ &= \alpha R_k + (1-\alpha)\left[\alpha R_{k-1} + (1-\alpha)Q_{k-1}\right] \\ &= \alpha R_k + (1-\alpha)\alpha R_{k-1} + (1-\alpha)^2 Q_{k-1} \\ &= \alpha R_k + (1-\alpha)\alpha R_{k-1} + (1-\alpha)^2\alpha R_{k-2} + \\ &\qquad\qquad \cdots + (1-\alpha)^{k-1}\alpha R_1 + (1-\alpha)^k Q_1 \\ &= (1-\alpha)^k Q_1 + \sum_{i=1}^{k} \alpha(1-\alpha)^{k-i} R_i. \end{aligned} \tag{2.6}$$

We call this a weighted average because the sum of the weights is $(1-\alpha)^k + \sum_{i=1}^{k} \alpha(1-\alpha)^{k-i} = 1$, as you can check yourself. Note that the weight, $\alpha(1-\alpha)^{k-i}$, given to the reward R_i depends on how many rewards ago, $k-i$, it was observed. The quantity $1-\alpha$ is less than 1, and thus the weight given to R_i decreases as the number of intervening rewards increases. In fact, the weight decays exponentially according to the exponent on $1-\alpha$. (If $1-\alpha = 0$, then all the weight goes on the very last reward, R_k, because of the convention that $0^0 = 1$.) Accordingly, this is sometimes called an *exponential, recency-weighted average*.

Sometimes it is convenient to vary the step-size parameter from step to step. Let $\alpha_k(a)$ denote the step-size parameter used to process the reward received after the kth selection of action a. As we have noted, the choice $\alpha_k(a) = \frac{1}{k}$ results in the sample-average method, which is guaranteed to converge to the true action values by the law of large numbers. But of course convergence is not guaranteed for all choices of the sequence $\{\alpha_k(a)\}$. A well-known result in stochastic approximation theory gives us the conditions required to assure convergence with probability 1:

$$\sum_{k=1}^{\infty} \alpha_k(a) = \infty \qquad \text{and} \qquad \sum_{k=1}^{\infty} \alpha_k^2(a) < \infty. \tag{2.7}$$

The first condition is required to guarantee that the steps are large enough to eventually overcome any initial conditions or random fluctuations. The second condition guarantees that eventually the steps become small enough to assure convergence.

Note that both convergence conditions are met for the sample-average case, $\alpha_k(a) = \frac{1}{k}$, but not for the case of constant step-size parameter, $\alpha_k(a) = \alpha$. In

the latter case, the second condition is not met, indicating that the estimates never completely converge but continue to vary in response to the most recently received rewards. As we mentioned above, this is actually desirable in a nonstationary environment, and problems that are effectively nonstationary are the norm in reinforcement learning. In addition, sequences of step-size parameters that meet the conditions (2.7) often converge very slowly or need considerable tuning in order to obtain a satisfactory convergence rate. Although sequences of step-size parameters that meet these convergence conditions are often used in theoretical work, they are seldom used in applications and empirical research.

Exercise 2.5 If the step-size parameters, α_k, are not constant, then the estimate Q_k is a weighted average of previously received rewards with a weighting different from that given by (2.6). What is the weighting on each prior reward for the general case, analogous to (2.6), in terms of α_k?

Exercise 2.6 (programming) Design and conduct an experiment to demonstrate the difficulties that sample-average methods have for nonstationary problems. Use a modified version of the 10-armed testbed in which all the $q_*(a)$ start out equal and then take independent random walks. Prepare plots like Figure 2.1 for an action-value method using sample averages, incrementally computed by $\alpha = \frac{1}{k}$, and another action-value method using a constant step-size parameter, $\alpha = 0.1$. Use $\varepsilon = 0.1$ and, if necessary, runs longer than 1000 plays.

2.6 Optimistic Initial Values

All the methods we have discussed so far are dependent to some extent on the initial action-value estimates, $Q_1(a)$. In the language of statistics, these methods are *biased* by their initial estimates. For the sample-average methods, the bias disappears once all actions have been selected at least once, but for methods with constant α, the bias is permanent, though decreasing over time as given by (2.6). In practice, this kind of bias is usually not a problem, and can sometimes be very helpful. The downside is that the initial estimates become, in effect, a set of parameters that must be picked by the user, if only to set them all to zero. The upside is that they provide an easy way to supply some prior knowledge about what level of rewards can be expected.

Initial action values can also be used as a simple way of encouraging exploration. Suppose that instead of setting the initial action values to zero, as we did in the 10-armed testbed, we set them all to +5. Recall that the $q_*(a)$ in this problem are selected from a normal distribution with mean 0 and vari-

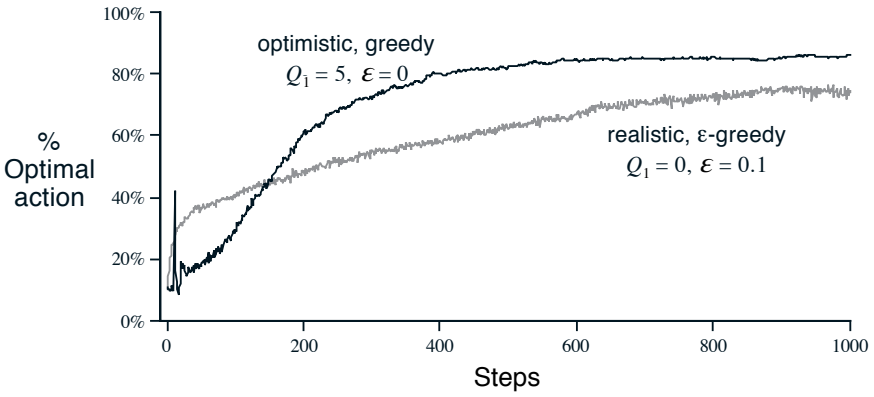


Figure 2.2: The effect of optimistic initial action-value estimates on the 10-armed testbed.

ance 1. An initial estimate of +5 is thus wildly optimistic. But this optimism encourages action-value methods to explore. Whichever actions are initially selected, the reward is less than the starting estimates; the learner switches to other actions, being "disappointed" with the rewards it is receiving. The result is that all actions are tried several times before the value estimates converge. The system does a fair amount of exploration even if greedy actions are selected all the time.

Figure 2.2 shows the performance on the 10-armed bandit testbed of a greedy method using $Q_1(a) = +5$, for all a. For comparison, also shown is an ε-greedy method with $Q_1(a) = 0$. Both methods used a constant step-size parameter, $\alpha = 0.1$. Initially, the optimistic method performs worse because it explores more, but eventually it performs better because its exploration decreases with time. We call this technique for encouraging exploration *optimistic initial values*. We regard it as a simple trick that can be quite effective on stationary problems, but it is far from being a generally useful approach to encouraging exploration. For example, it is not well suited to nonstationary problems because its drive for exploration is inherently temporary. If the task changes, creating a renewed need for exploration, this method cannot help. Indeed, any method that focuses on the initial state in any special way is unlikely to help with the general nonstationary case. The beginning of time occurs only once, and thus we should not focus on it too much. This criticism applies as well to the sample-average methods, which also treat the beginning of time as a special event, averaging all subsequent rewards with equal weights. Nevertheless, all of these methods are very simple, and one of them or some simple combination of them is often adequate in practice. In the rest of this book we make frequent use of several of these simple exploration techniques.

Exercise 2.7 The results shown in Figure 2.2 should be quite reliable because they are averages over 2000 individual, randomly chosen 10-armed bandit tasks. Why, then, are there oscillations and spikes in the early part of the curve for the optimistic method? What might make this method perform particularly better or worse, on average, on particular early plays?

2.7 Associative Search (Contextual Bandits)

So far in this chapter we have considered only nonassociative tasks, in which there is no need to associate different actions with different situations. In these tasks the learner either tries to find a single best action when the task is stationary, or tries to track the best action as it changes over time when the task is nonstationary. However, in a general reinforcement learning task there is more than one situation, and the goal is to learn a policy: a mapping from situations to the actions that are best in those situations. To set the stage for the full problem, we briefly discuss the simplest way in which nonassociative tasks extend to the associative setting.

As an example, suppose there are several different n-armed bandit tasks, and that on each play you confront one of these chosen at random. Thus, the bandit task changes randomly from play to play. This would appear to you as a single, nonstationary n-armed bandit task whose true action values change randomly from play to play. You could try using one of the methods described in this chapter that can handle nonstationarity, but unless the true action values change slowly, these methods will not work very well. Now suppose, however, that when a bandit task is selected for you, you are given some distinctive clue about its identity (but not its action values). Maybe you are facing an actual slot machine that changes the color of its display as it changes its action values. Now you can learn a policy associating each task, signaled by the color you see, with the best action to take when facing that task—for instance, if red, play arm 1; if green, play arm 2. With the right policy you can usually do much better than you could in the absence of any information distinguishing one bandit task from another.

This is an example of an *associative search* task, so called because it involves both trial-and-error learning in the form of *search* for the best actions and *association* of these actions with the situations in which they are best.[1] Associative search tasks are intermediate between the n-armed bandit problem and the full reinforcement learning problem. They are like the full reinforcement learning problem in that they involve learning a policy, but like our

[1] Associative search tasks are often now termed *contextual bandits* in the literature.

version of the n-armed bandit problem in that each action affects only the immediate reward. If actions are allowed to affect the *next situation* as well as the reward, then we have the full reinforcement learning problem. We present this problem in the next chapter and consider its ramifications throughout the rest of the book.

Exercise 2.8 Suppose you face a binary bandit task whose true action values change randomly from play to play. Specifically, suppose that for any play the true values of actions 1 and 2 are respectively 0.1 and 0.2 with probability 0.5 (case A), and 0.9 and 0.8 with probability 0.5 (case B). If you are not able to tell which case you face at any play, what is the best expectation of success you can achieve and how should you behave to achieve it? Now suppose that on each play you are told if you are facing case A or case B (although you still don't know the true action values). This is an associative search task. What is the best expectation of success you can achieve in this task, and how should you behave to achieve it?

2.8 Conclusions

We have presented in this chapter some simple ways of balancing exploration and exploitation. The ε-greedy methods choose randomly a small fraction of the time, and the softmax methods grade their action probabilities according to the current action-value estimates. Are these simple methods really the best we can do in terms of practically useful algorithms? So far, the answer appears to be "yes." Despite their simplicity, in our opinion the methods presented in this chapter can fairly be considered the state of the art. There are more sophisticated methods, but their complexity and assumptions make them impractical for the full reinforcement learning problem that is our real focus. Starting in Chapter 5 we present learning methods for solving the full reinforcement learning problem that use in part the simple methods explored in this chapter. Also, in a later (future) chapter we will consider policy-based methods for action selection with some distinct advantages over those examined here.

Although the simple methods explored in this chapter may be the best we can do at present, they are far from a fully satisfactory solution to the problem of balancing exploration and exploitation. We conclude this chapter with a brief look at some of the current ideas that, while not yet practically useful, may point the way toward better solutions.

One promising idea is to use estimates of the uncertainty of the action-value estimates to direct and encourage exploration. For example, suppose there

are two actions estimated to have values slightly less than that of the greedy action, but that differ greatly in their degree of uncertainty. One estimate is nearly certain; perhaps that action has been tried many times and many rewards have been observed. The uncertainty for this action's estimated value is so low that its true value is very unlikely to be higher than the value of the greedy action. The other action is known less well, and the estimate of its value is very uncertain. The true value of this action could easily be better than that of the greedy action. Obviously, it makes more sense to explore the second action than the first.

This line of thought leads to *interval estimation* methods. These methods estimate for each action a confidence interval of the action's value. That is, rather than learning that the action's value is approximately 10, they learn that it is between 9 and 11 with, say, 95% confidence. The action selected is then the action whose confidence interval has the highest upper limit. This encourages exploration of actions that are uncertain *and* have a chance of ultimately being the best action. In some cases one can obtain guarantees that the optimal action has been found with confidence equal to the confidence factor (e.g., the 95%). Unfortunately, interval estimation methods are problematic in practice because of the complexity of the statistical methods used to estimate the confidence intervals. Moreover, the underlying statistical assumptions required by these methods are often not satisfied.

Nevertheless, the idea of using confidence intervals, or some other measure of uncertainty, to encourage exploration of particular actions is sound and appealing. More recently, it has been extensively and very successfully developed as the *Upper Confidence Bound* (UCB) method and its many derivatives (see Auer, 2002). This topic deserves a section of its own.

There is also a well-known algorithm for computing the Bayes optimal way to balance exploration and exploitation. This method is computationally intractable when done exactly, but there may be efficient ways to approximate it. In this method we assume that we know the distribution of problem instances, that is, the probability of each possible set of true action values. Given any action selection, we can then compute the probability of each possible immediate reward and the resultant posterior probability distribution over action values. This evolving distribution becomes the *information state* of the problem. Given a horizon, say 1000 plays, one can consider all possible actions, all possible resulting rewards, all possible next actions, all next rewards, and so on for all 1000 plays. Given the assumptions, the rewards and probabilities of each possible chain of events can be determined, and one need only pick the best. But the tree of possibilities grows extremely rapidly; even if there are only two actions and two rewards, the tree will have 2^{2000} leaves. This approach effectively turns the bandit problem into an instance of the full rein-

forcement learning problem. In the end, we may be able to use reinforcement learning methods to approximate this optimal solution. But that is a topic for current research and beyond the scope of this introductory book.

The classical solution to balancing exploration and exploitation in n-armed bandit problems is to compute special functions called *Gittins indices*. These provide an optimal solution to a certain kind of bandit problem more general than that considered here but that assumes the prior distribution of possible problems is known. Unfortunately, neither the theory nor the computational tractability of this method appear to generalize to the full reinforcement learning problem that we consider in the rest of the book.

2.9 Bibliographical and Historical Remarks

2.1 Bandit problems have been studied in statistics, engineering, and psychology. In statistics, bandit problems fall under the heading "sequential design of experiments," introduced by Thompson (1933, 1934) and Robbins (1952), and studied by Bellman (1956). Berry and Fristedt (1985) provide an extensive treatment of bandit problems from the perspective of statistics. Narendra and Thathachar (1989) treat bandit problems from the engineering perspective, providing a good discussion of the various theoretical traditions that have focused on them. In psychology, bandit problems have played roles in statistical learning theory (e.g., Bush and Mosteller, 1955; Estes, 1950).

The term *greedy* is often used in the heuristic search literature (e.g., Pearl, 1984). The conflict between exploration and exploitation is known in control engineering as the conflict between identification (or estimation) and control (e.g., Witten, 1976). Feldbaum (1965) called it the *dual control* problem, referring to the need to solve the two problems of identification and control simultaneously when trying to control a system under uncertainty. In discussing aspects of genetic algorithms, Holland (1975) emphasized the importance of this conflict, referring to it as the conflict between the need to exploit and the need for new information.

2.2 Action-value methods for our n-armed bandit problem were first proposed by Thathachar and Sastry (1985). These are often called *estimator algorithms* in the learning automata literature. The term *action value* is due to Watkins (1989). The first to use ε-greedy methods may also have been Watkins (1989, p. 187), but the idea is so simple that some earlier use seems likely.

2.3 The term *softmax* for the action selection rule (2.2) is due to Bridle (1990). This rule appears to have been first proposed by Luce (1959). The parameter τ is called temperature in simulated annealing algorithms (Kirkpatrick, Gelatt, and Vecchi, 1983).

2.4–5 This material falls under the general heading of stochastic iterative algorithms, which is well covered by Bertsekas and Tsitsiklis (1996).

2.7 The term *associative search* and the corresponding problem were introduced by Barto, Sutton, and Brouwer (1981). The term *associative reinforcement learning* has also been used for associative search (Barto and Anandan, 1985), but we prefer to reserve that term as a synonym for the full reinforcement learning problem (as in Sutton, 1984). (And, as we noted, the modern literature also uses the term "contextual bandits" for this problem.) We note that Thorndike's Law of Effect (quoted in Chapter 1) describes associative search by referring to the formation of associative links between situations (states) and actions. According to the terminology of operant, or instrumental, conditioning (e.g., Skinner, 1938), a discriminative stimulus is a stimulus that signals the presence of a particular reinforcement contingency. In our terms, different discriminative stimuli correspond to different states.

2.8 Interval estimation methods are due to Lai (1987) and Kaelbling (1993a). Bellman (1956) was the first to show how dynamic programming could be used to compute the optimal balance between exploration and exploitation within a Bayesian formulation of the problem. The survey by Kumar (1985) provides a good discussion of Bayesian and non-Bayesian approaches to these problems. The term *information state* comes from the literature on partially observable MDPs; see, e.g., Lovejoy (1991). The Gittins index approach is due to Gittins and Jones (1974). Duff (1995) showed how it is possible to learn Gittins indices for bandit problems through reinforcement learning.

Use slides for MDP definition.

Chapter 3

The Reinforcement Learning Problem

In this chapter we introduce the problem that we try to solve in the rest of the book. For us, this problem defines the field of reinforcement learning: any method that is suited to solving this problem we consider to be a reinforcement learning method.

Our objective in this chapter is to describe the reinforcement learning problem in a broad sense. We try to convey the wide range of possible applications that can be framed as reinforcement learning tasks. We also describe mathematically idealized forms of the reinforcement learning problem for which precise theoretical statements can be made. We introduce key elements of the problem's mathematical structure, such as value functions and Bellman equations. As in all of artificial intelligence, there is a tension between breadth of applicability and mathematical tractability. In this chapter we introduce this tension and discuss some of the trade-offs and challenges that it implies.

3.1 The Agent–Environment Interface

The reinforcement learning problem is meant to be a straightforward framing of the problem of learning from interaction to achieve a goal. The learner and decision-maker is called the *agent*. The thing it interacts with, comprising everything outside the agent, is called the *environment*. These interact continually, the agent selecting actions and the environment responding to those actions and presenting new situations to the agent.[1] The environment also

[1]We use the terms *agent*, *environment*, and *action* instead of the engineers' terms *controller*, *controlled system* (or *plant*), and *control signal* because they are meaningful to a

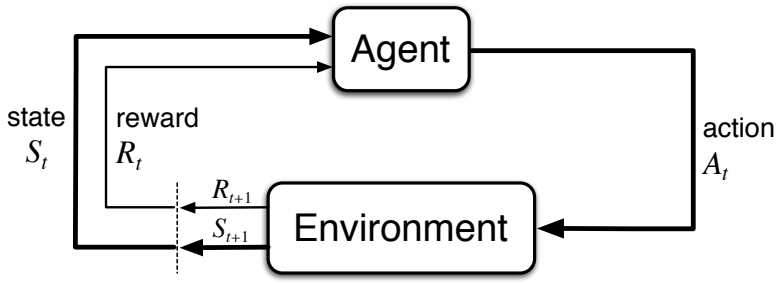


Figure 3.1: The agent–environment interaction in reinforcement learning.

gives rise to rewards, special numerical values that the agent tries to maximize over time. A complete specification of an environment defines a *task*, one instance of the reinforcement learning problem.

More specifically, the agent and environment interact at each of a sequence of discrete time steps, $t = 0, 1, 2, 3, \ldots$.[2] At each time step t, the agent receives some representation of the environment's *state*, $S_t \in \mathcal{S}$, where $\mathcal{S}$ is the set of possible states, and on that basis selects an *action*, $A_t \in \mathcal{A}(S_t)$, where $\mathcal{A}(S_t)$ is the set of actions available in state S_t. One time step later, in part as a consequence of its action, the agent receives a numerical *reward*, $R_{t+1} \in \mathbb{R}$, and finds itself in a new state, S_{t+1}.[3] Figure 3.1 diagrams the agent–environment interaction.

At each time step, the agent implements a mapping from states to probabilities of selecting each possible action. This mapping is called the agent's *policy* and is denoted π_t, where $\pi_t(a|s)$ is the probability that $A_t = a$ if $S_t = s$. Reinforcement learning methods specify how the agent changes its policy as a result of its experience. The agent's goal, roughly speaking, is to maximize the total amount of reward it receives over the long run.

This framework is abstract and flexible and can be applied to many different problems in many different ways. For example, the time steps need not refer to fixed intervals of real time; they can refer to arbitrary successive stages of decision-making and acting. The actions can be low-level controls, such as the voltages applied to the motors of a robot arm, or high-level decisions, such as whether or not to have lunch or to go to graduate school. Similarly, the states can take a wide variety of forms. They can be completely determined by

---

wider audience.

[2]We restrict attention to discrete time to keep things as simple as possible, even though many of the ideas can be extended to the continuous-time case (e.g., see Bertsekas and Tsitsiklis, 1996; Werbos, 1992; Doya, 1996).

[3]We use R_{t+1} instead of R_t to denote the immediate reward due to the action taken at time t because it emphasizes that the next reward and the next state, S_{t+1}, are jointly determined.

low-level sensations, such as direct sensor readings, or they can be more high-level and abstract, such as symbolic descriptions of objects in a room. Some of what makes up a state could be based on memory of past sensations or even be entirely mental or subjective. For example, an agent could be in "the state" of not being sure where an object is, or of having just been "surprised" in some clearly defined sense. Similarly, some actions might be totally mental or computational. For example, some actions might control what an agent chooses to think about, or where it focuses its attention. In general, actions can be any decisions we want to learn how to make, and the states can be anything we can know that might be useful in making them.

In particular, the boundary between agent and environment is not often the same as the physical boundary of a robot's or animal's body. Usually, the boundary is drawn closer to the agent than that. For example, the motors and mechanical linkages of a robot and its sensing hardware should usually be considered parts of the environment rather than parts of the agent. Similarly, if we apply the framework to a person or animal, the muscles, skeleton, and sensory organs should be considered part of the environment. Rewards, too, presumably are computed inside the physical bodies of natural and artificial learning systems, but are considered external to the agent.

The general rule we follow is that anything that cannot be changed arbitrarily by the agent is considered to be outside of it and thus part of its environment. We do not assume that everything in the environment is unknown to the agent. For example, the agent often knows quite a bit about how its rewards are computed as a function of its actions and the states in which they are taken. But we always consider the reward computation to be external to the agent because it defines the task facing the agent and thus must be beyond its ability to change arbitrarily. In fact, in some cases the agent may know *everything* about how its environment works and still face a difficult reinforcement learning task, just as we may know exactly how a puzzle like Rubik's cube works, but still be unable to solve it. The agent–environment boundary represents the limit of the agent's *absolute control*, not of its knowledge.

The agent–environment boundary can be located at different places for different purposes. In a complicated robot, many different agents may be operating at once, each with its own boundary. For example, one agent may make high-level decisions which form part of the states faced by a lower-level agent that implements the high-level decisions. In practice, the agent–environment boundary is determined once one has selected particular states, actions, and rewards, and thus has identified a specific decision-making task of interest.

The reinforcement learning framework is a considerable abstraction of the

problem of goal-directed learning from interaction. It proposes that whatever the details of the sensory, memory, and control apparatus, and whatever objective one is trying to achieve, any problem of learning goal-directed behavior can be reduced to three signals passing back and forth between an agent and its environment: one signal to represent the choices made by the agent (the actions), one signal to represent the basis on which the choices are made (the states), and one signal to define the agent's goal (the rewards). This framework may not be sufficient to represent all decision-learning problems usefully, but it has proved to be widely useful and applicable.

Of course, the particular states and actions vary greatly from application to application, and how they are represented can strongly affect performance. In reinforcement learning, as in other kinds of learning, such representational choices are at present more art than science. In this book we offer some advice and examples regarding good ways of representing states and actions, but our primary focus is on general principles for learning how to behave once the representations have been selected.

Example 3.1: Bioreactor Suppose reinforcement learning is being applied to determine moment-by-moment temperatures and stirring rates for a bioreactor (a large vat of nutrients and bacteria used to produce useful chemicals). The actions in such an application might be target temperatures and target stirring rates that are passed to lower-level control systems that, in turn, directly activate heating elements and motors to attain the targets. The states are likely to be thermocouple and other sensory readings, perhaps filtered and delayed, plus symbolic inputs representing the ingredients in the vat and the target chemical. The rewards might be moment-by-moment measures of the rate at which the useful chemical is produced by the bioreactor. Notice that here each state is a list, or vector, of sensor readings and symbolic inputs, and each action is a vector consisting of a target temperature and a stirring rate. It is typical of reinforcement learning tasks to have states and actions with such structured representations. Rewards, on the other hand, are always single numbers. ■

Example 3.2: Pick-and-Place Robot Consider using reinforcement learning to control the motion of a robot arm in a repetitive pick-and-place task. If we want to learn movements that are fast and smooth, the learning agent will have to control the motors directly and have low-latency information about the current positions and velocities of the mechanical linkages. The actions in this case might be the voltages applied to each motor at each joint, and the states might be the latest readings of joint angles and velocities. The reward might be +1 for each object successfully picked up and placed. To encourage smooth movements, on each time step a small, negative reward can be given as a function of the moment-to-moment "jerkiness" of the motion.

■

Example 3.3: Recycling Robot A mobile robot has the job of collecting empty soda cans in an office environment. It has sensors for detecting cans, and an arm and gripper that can pick them up and place them in an onboard bin; it runs on a rechargeable battery. The robot's control system has components for interpreting sensory information, for navigating, and for controlling the arm and gripper. High-level decisions about how to search for cans are made by a reinforcement learning agent based on the current charge level of the battery. This agent has to decide whether the robot should (1) actively search for a can for a certain period of time, (2) remain stationary and wait for someone to bring it a can, or (3) head back to its home base to recharge its battery. This decision has to be made either periodically or whenever certain events occur, such as finding an empty can. The agent therefore has three actions, and its state is determined by the state of the battery. The rewards might be zero most of the time, but then become positive when the robot secures an empty can, or large and negative if the battery runs all the way down. In this example, the reinforcement learning agent is not the entire robot. The states it monitors describe conditions within the robot itself, not conditions of the robot's external environment. The agent's environment therefore includes the rest of the robot, which might contain other complex decision-making systems, as well as the robot's external environment. ■

Exercise 3.1 Devise three example tasks of your own that fit into the reinforcement learning framework, identifying for each its states, actions, and rewards. Make the three examples as *different* from each other as possible. The framework is abstract and flexible and can be applied in many different ways. Stretch its limits in some way in at least one of your examples.

Exercise 3.2 Is the reinforcement learning framework adequate to usefully represent *all* goal-directed learning tasks? Can you think of any clear exceptions?

Exercise 3.3 Consider the problem of driving. You could define the actions in terms of the accelerator, steering wheel, and brake, that is, where your body meets the machine. Or you could define them farther out—say, where the rubber meets the road, considering your actions to be tire torques. Or you could define them farther in—say, where your brain meets your body, the actions being muscle twitches to control your limbs. Or you could go to a really high level and say that your actions are your choices of *where* to drive. What is the right level, the right place to draw the line between agent and environment? On what basis is one location of the line to be preferred over another? Is there any fundamental reason for preferring one location over another, or is it a free choice?

3.2 Goals and Rewards

In reinforcement learning, the purpose or goal of the agent is formalized in terms of a special reward signal passing from the environment to the agent. At each time step, the reward is a simple number, $R_t \in \mathbb{R}$. Informally, the agent's goal is to maximize the total amount of reward it receives. This means maximizing not immediate reward, but cumulative reward in the long run.

The use of a reward signal to formalize the idea of a goal is one of the most distinctive features of reinforcement learning. Although this way of formulating goals might at first appear limiting, in practice it has proved to be flexible and widely applicable. The best way to see this is to consider examples of how it has been, or could be, used. For example, to make a robot learn to walk, researchers have provided reward on each time step proportional to the robot's forward motion. In making a robot learn how to escape from a maze, the reward is often zero until it escapes, when it becomes $+1$. Another common approach in maze learning is to give a reward of -1 for every time step that passes prior to escape; this encourages the agent to escape as quickly as possible. To make a robot learn to find and collect empty soda cans for recycling, one might give it a reward of zero most of the time, and then a reward of $+1$ for each can collected (and confirmed as empty). One might also want to give the robot negative rewards when it bumps into things or when somebody yells at it. For an agent to learn to play checkers or chess, the natural rewards are $+1$ for winning, -1 for losing, and 0 for drawing and for all nonterminal positions.

You can see what is happening in all of these examples. The agent always learns to maximize its reward. If we want it to do something for us, we must provide rewards to it in such a way that in maximizing them the agent will also achieve our goals. It is thus critical that the rewards we set up truly indicate what we want accomplished. In particular, the reward signal is not the place to impart to the agent prior knowledge about *how* to achieve what we want it to do.[4] For example, a chess-playing agent should be rewarded only for actually winning, not for achieving subgoals such taking its opponent's pieces or gaining control of the center of the board. If achieving these sorts of subgoals were rewarded, then the agent might find a way to achieve them without achieving the real goal. For example, it might find a way to take the opponent's pieces even at the cost of losing the game. The reward signal is your way of communicating to the robot *what* you want it to achieve, not *how* you want it achieved.

[4]Better places for imparting this kind of prior knowledge are the initial policy or value function, or in influences on these. See Lin (1992), Maclin and Shavlik (1994), and Clouse (1996).

Newcomers to reinforcement learning are sometimes surprised that the rewards—which define of the goal of learning—are computed in the environment rather than in the agent. Certainly most ultimate goals for animals are recognized by computations occurring inside their bodies, for example, by sensors for recognizing food, hunger, pain, and pleasure. Nevertheless, as we discussed in the previous section, one can redraw the agent–environment interface in such a way that these parts of the body are considered to be outside of the agent (and thus part of the agent's environment). For example, if the goal concerns a robot's internal energy reservoirs, then these are considered to be part of the environment; if the goal concerns the positions of the robot's limbs, then these too are considered to be part of the environment—that is, the agent's boundary is drawn at the interface between the limbs and their control systems. These things are considered internal to the robot but external to the learning agent. For our purposes, it is convenient to place the boundary of the learning agent not at the limit of its physical body, but at the limit of its control.

The reason we do this is that the agent's ultimate goal should be something over which it has imperfect control: it should not be able, for example, to simply *decree* that the reward has been received in the same way that it might arbitrarily change its actions. Therefore, we place the reward source outside of the agent. This does not preclude the agent from defining for itself a kind of internal reward, or a sequence of internal rewards. Indeed, this is exactly what many reinforcement learning methods do.

3.3 Returns

So far we have been imprecise regarding the objective of learning. We have said that the agent's goal is to maximize the reward it receives in the long run. How might this be formally defined? If the sequence of rewards received after time step t is denoted $R_{t+1}, R_{t+2}, R_{t+3}, \ldots$, then what precise aspect of this sequence do we wish to maximize? In general, we seek to maximize the *expected return*, where the return, G_t, is defined as some specific function of the reward sequence. In the simplest case the return is the sum of the rewards:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T, \tag{3.1}$$

where T is a final time step. This approach makes sense in applications in which there is a natural notion of final time step, that is, when the agent–environment interaction breaks naturally into subsequences, which we call

episodes,[5] such as plays of a game, trips through a maze, or any sort of repeated interactions. Each episode ends in a special state called the *terminal state*, followed by a reset to a standard starting state or to a sample from a standard distribution of starting states. Tasks with episodes of this kind are called *episodic tasks*. In episodic tasks we sometimes need to distinguish the set of all nonterminal states, denoted $\mathcal{S}$, from the set of all states plus the terminal state, denoted $\mathcal{S}^+$.

On the other hand, in many cases the agent–environment interaction does not break naturally into identifiable episodes, but goes on continually without limit. For example, this would be the natural way to formulate a continual process-control task, or an application to a robot with a long life span. We call these *continuing tasks*. The return formulation (3.1) is problematic for continuing tasks because the final time step would be $T = \infty$, and the return, which is what we are trying to maximize, could itself easily be infinite. (For example, suppose the agent receives a reward of +1 at each time step.) Thus, in this book we usually use a definition of return that is slightly more complex conceptually but much simpler mathematically.

The additional concept that we need is that of *discounting*. According to this approach, the agent tries to select actions so that the sum of the discounted rewards it receives over the future is maximized. In particular, it chooses A_t to maximize the expected *discounted return*:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \quad = \quad \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \tag{3.2}$$

where γ is a parameter, $0 \leq \gamma \leq 1$, called the *discount rate*.

The discount rate determines the present value of future rewards: a reward received k time steps in the future is worth only γ^{k-1} times what it would be worth if it were received immediately. If $\gamma < 1$, the infinite sum has a finite value as long as the reward sequence $\{R_k\}$ is bounded. If $\gamma = 0$, the agent is "myopic" in being concerned only with maximizing immediate rewards: its objective in this case is to learn how to choose A_t so as to maximize only R_{t+1}. If each of the agent's actions happened to influence only the immediate reward, not future rewards as well, then a myopic agent could maximize (3.2) by separately maximizing each immediate reward. But in general, acting to maximize immediate reward can reduce access to future rewards so that the return may actually be reduced. As γ approaches 1, the objective takes future rewards into account more strongly: the agent becomes more farsighted.

[5] Episodes are sometimes called "trials" in the literature.

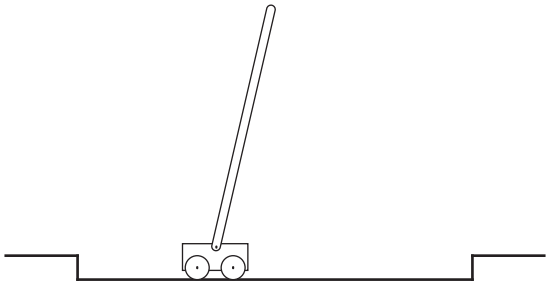


Figure 3.2: The pole-balancing task.

Example 3.4: *Pole-Balancing* Figure 3.2 shows a task that served as an early illustration of reinforcement learning. The objective here is to apply forces to a cart moving along a track so as to keep a pole hinged to the cart from falling over. A failure is said to occur if the pole falls past a given angle from vertical or if the cart runs off the track. The pole is reset to vertical after each failure. This task could be treated as episodic, where the natural episodes are the repeated attempts to balance the pole. The reward in this case could be +1 for every time step on which failure did not occur, so that the return at each time would be the number of steps until failure. Alternatively, we could treat pole-balancing as a continuing task, using discounting. In this case the reward would be -1 on each failure and zero at all other times. The return at each time would then be related to $-\gamma^K$, where K is the number of time steps before failure. In either case, the return is maximized by keeping the pole balanced for as long as possible. ■

Exercise 3.4 Suppose you treated pole-balancing as an episodic task but also used discounting, with all rewards zero except for -1 upon failure. What then would the return be at each time? How does this return differ from that in the discounted, continuing formulation of this task?

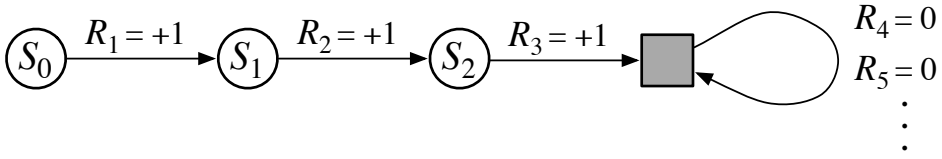
Exercise 3.5 Imagine that you are designing a robot to run a maze. You decide to give it a reward of +1 for escaping from the maze and a reward of zero at all other times. The task seems to break down naturally into episodes—the successive runs through the maze—so you decide to treat it as an episodic task, where the goal is to maximize expected total reward (3.1). After running the learning agent for a while, you find that it is showing no improvement in escaping from the maze. What is going wrong? Have you effectively communicated to the agent what you want it to achieve?

3.4 Unified Notation for Episodic and Continuing Tasks

In the preceding section we described two kinds of reinforcement learning tasks, one in which the agent–environment interaction naturally breaks down into a sequence of separate episodes (episodic tasks), and one in which it does not (continuing tasks). The former case is mathematically easier because each action affects only the finite number of rewards subsequently received during the episode. In this book we consider sometimes one kind of problem and sometimes the other, but often both. It is therefore useful to establish one notation that enables us to talk precisely about both cases simultaneously.

To be precise about episodic tasks requires some additional notation. Rather than one long sequence of time steps, we need to consider a series of episodes, each of which consists of a finite sequence of time steps. We number the time steps of each episode starting anew from zero. Therefore, we have to refer not just to S_t, the state representation at time t, but to $S_{t,i}$, the state representation at time t of episode i (and similarly for $A_{t,i}$, $R_{t,i}$, $\pi_{t,i}$, T_i, etc.). However, it turns out that, when we discuss episodic tasks we will almost never have to distinguish between different episodes. We will almost always be considering a particular single episode, or stating something that is true for all episodes. Accordingly, in practice we will almost always abuse notation slightly by dropping the explicit reference to episode number. That is, we will write S_t to refer to $S_{t,i}$, and so on.

We need one other convention to obtain a single notation that covers both episodic and continuing tasks. We have defined the return as a sum over a finite number of terms in one case (3.1) and as a sum over an infinite number of terms in the other (3.2). These can be unified by considering episode termination to be the entering of a special *absorbing state* that transitions only to itself and that generates only rewards of zero. For example, consider the state transition diagram



Here the solid square represents the special absorbing state corresponding to the end of an episode. Starting from S_0, we get the reward sequence $+1, +1, +1, 0, 0, 0, \ldots$. Summing these, we get the same return whether we sum over the first T rewards (here $T = 3$) or over the full infinite sequence.

This remains true even if we introduce discounting. Thus, we can define the return, in general, according to (3.2), using the convention of omitting episode numbers when they are not needed, and including the possibility that $\gamma = 1$ if the sum remains defined (e.g., because all episodes terminate). Alternatively, we can also write the return as

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}, \tag{3.3}$$

including the possibility that $T = \infty$ or $\gamma = 1$ (but not both[6]). We use these conventions throughout the rest of the book to simplify notation and to express the close parallels between episodic and continuing tasks.

*3.5 The Markov Property

In the reinforcement learning framework, the agent makes its decisions as a function of a signal from the environment called the environment's *state*. In this section we discuss what is required of the state signal, and what kind of information we should and should not expect it to provide. In particular, we formally define a property of environments and their state signals that is of particular interest, called the Markov property.

In this book, by "the state" we mean whatever information is available to the agent. We assume that the state is given by some preprocessing system that is nominally part of the environment. We do not address the issues of constructing, changing, or learning the state signal in this book. We take this approach not because we consider state representation to be unimportant, but in order to focus fully on the decision-making issues. In other words, our main concern is not with designing the state signal, but with deciding what action to take as a function of whatever state signal is available.

Certainly the state signal should include immediate sensations such as sensory measurements, but it can contain much more than that. State representations can be highly processed versions of original sensations, or they can be complex structures built up over time from the sequence of sensations. For example, we can move our eyes over a scene, with only a tiny spot corresponding to the fovea visible in detail at any one time, yet build up a rich and detailed representation of a scene. Or, more obviously, we can look at an object, then

[6] Ways to formulate tasks that are both continuing and undiscounted are the subject of current research (e.g., Mahadevan, 1996; Schwartz, 1993; Tadepalli and Ok, 1994). Some of the ideas are discussed in Section 6.7.

look away, and know that it is still there. We can hear the word "yes" and consider ourselves to be in totally different states depending on the question that came before and which is no longer audible. At a more mundane level, a control system can measure position at two different times to produce a state representation including information about velocity. In all of these cases the state is constructed and maintained on the basis of immediate sensations together with the previous state or some other memory of past sensations. In this book, we do not explore how that is done, but certainly it can be and has been done. There is no reason to restrict the state representation to immediate sensations; in typical applications we should expect the state representation to be able to inform the agent of more than that.

On the other hand, the state signal should not be expected to inform the agent of everything about the environment, or even everything that would be useful to it in making decisions. If the agent is playing blackjack, we should not expect it to know what the next card in the deck is. If the agent is answering the phone, we should not expect it to know in advance who the caller is. If the agent is a paramedic called to a road accident, we should not expect it to know immediately the internal injuries of an unconscious victim. In all of these cases there is hidden state information in the environment, and that information would be useful if the agent knew it, but the agent cannot know it because it has never received any relevant sensations. In short, we don't fault an agent for not knowing something that matters, but only for having known something and then forgotten it!

What we would like, ideally, is a state signal that summarizes past sensations compactly, yet in such a way that all relevant information is retained. This normally requires more than the immediate sensations, but never more than the complete history of all past sensations. A state signal that succeeds in retaining all relevant information is said to be *Markov*, or to have *the Markov property* (we define this formally below). For example, a checkers position—the current configuration of all the pieces on the board—would serve as a Markov state because it summarizes everything important about the complete sequence of positions that led to it. Much of the information about the sequence is lost, but all that really matters for the future of the game is retained. Similarly, the current position and velocity of a cannonball is all that matters for its future flight. It doesn't matter how that position and velocity came about. This is sometimes also referred to as an "independence of path" property because all that matters is in the current state signal; its meaning is independent of the "path," or history, of signals that have led up to it.

We now formally define the Markov property for the reinforcement learning problem. To keep the mathematics simple, we assume here that there are a finite number of states and reward values. This enables us to work in terms

of sums and probabilities rather than integrals and probability densities, but the argument can easily be extended to include continuous states and rewards. Consider how a general environment might respond at time $t+1$ to the action taken at time t. In the most general, causal case this response may depend on everything that has happened earlier. In this case the dynamics can be defined only by specifying the complete probability distribution:

$$\mathbf{Pr}\{R_{t+1} = r, S_{t+1} = s' \mid S_0, A_0, R_1, \ldots, S_{t-1}, A_{t-1}, R_t, S_t, A_t\}, \tag{3.4}$$

for all r, s', and all possible values of the past events: S_0, A_0, R_1, ..., S_{t-1}, A_{t-1}, R_t, S_t, A_t. If the state signal has the *Markov property*, on the other hand, then the environment's response at $t+1$ depends only on the state and action representations at t, in which case the environment's dynamics can be defined by specifying only

$$\mathbf{Pr}\{R_{t+1} = r, S_{t+1} = s' \mid S_t, A_t\}, \tag{3.5}$$

for all r, s', S_t, and A_t. In other words, a state signal has the Markov property, and is a Markov state, if and only if (3.5) is equal to (3.4) for all s', r, and histories, S_0, A_0, R_1, ..., S_{t-1}, A_{t-1}, R_t, S_t, A_t. In this case, the environment and task as a whole are also said to have the Markov property.

If an environment has the Markov property, then its one-step dynamics (3.5) enable us to predict the next state and expected next reward given the current state and action. One can show that, by iterating this equation, one can predict all future states and expected rewards from knowledge only of the current state as well as would be possible given the complete history up to the current time. It also follows that Markov states provide the best possible basis for choosing actions. That is, the best policy for choosing actions as a function of a Markov state is just as good as the best policy for choosing actions as a function of complete histories.

Even when the state signal is non-Markov, it is still appropriate to think of the state in reinforcement learning as an approximation to a Markov state. In particular, we always want the state to be a good basis for predicting future rewards and for selecting actions. In cases in which a model of the environment is learned (see Chapter 8), we also want the state to be a good basis for predicting subsequent states. Markov states provide an unsurpassed basis for doing all of these things. To the extent that the state approaches the ability of Markov states in these ways, one will obtain better performance from reinforcement learning systems. For all of these reasons, it is useful to think of the state at each time step as an approximation to a Markov state, although one should remember that it may not fully satisfy the Markov property.

The Markov property is important in reinforcement learning because decisions and values are assumed to be a function only of the current state. In order for these to be effective and informative, the state representation must be informative. All of the theory presented in this book assumes Markov state signals. This means that not all the theory strictly applies to cases in which the Markov property does not strictly apply. However, the theory developed for the Markov case still helps us to understand the behavior of the algorithms, and the algorithms can be successfully applied to many tasks with states that are not strictly Markov. A full understanding of the theory of the Markov case is an essential foundation for extending it to the more complex and realistic non-Markov case. Finally, we note that the assumption of Markov state representations is not unique to reinforcement learning but is also present in most if not all other approaches to artificial intelligence.

Example 3.5: Pole-Balancing State In the pole-balancing task introduced earlier, a state signal would be Markov if it specified exactly, or made it possible to reconstruct exactly, the position and velocity of the cart along the track, the angle between the cart and the pole, and the rate at which this angle is changing (the angular velocity). In an idealized cart–pole system, this information would be sufficient to exactly predict the future behavior of the cart and pole, given the actions taken by the controller. In practice, however, it is never possible to know this information exactly because any real sensor would introduce some distortion and delay in its measurements. Furthermore, in any real cart–pole system there are always other effects, such as the bending of the pole, the temperatures of the wheel and pole bearings, and various forms of backlash, that slightly affect the behavior of the system. These factors would cause violations of the Markov property if the state signal were only the positions and velocities of the cart and the pole.

However, often the positions and velocities serve quite well as states. Some early studies of learning to solve the pole-balancing task used a coarse state signal that divided cart positions into three regions: right, left, and middle (and similar rough quantizations of the other three intrinsic state variables). This distinctly non-Markov state was sufficient to allow the task to be solved easily by reinforcement learning methods. In fact, this coarse representation may have facilitated rapid learning by forcing the learning agent to ignore fine distinctions that would not have been useful in solving the task. ■

Example 3.6: Draw Poker In draw poker, each player is dealt a hand of five cards. There is a round of betting, in which each player exchanges some of his cards for new ones, and then there is a final round of betting. At each round, each player must match or exceed the highest bets of the other players, or else drop out (fold). After the second round of betting, the player with the best hand who has not folded is the winner and collects all the bets.

The state signal in draw poker is different for each player. Each player knows the cards in his own hand, but can only guess at those in the other players' hands. A common mistake is to think that a Markov state signal should include the contents of all the players' hands and the cards remaining in the deck. In a fair game, however, we assume that the players are in principle unable to determine these things from their past observations. If a player did know them, then she could predict some future events (such as the cards one could exchange for) *better* than by remembering all past observations.

In addition to knowledge of one's own cards, the state in draw poker should include the bets and the numbers of cards drawn by the other players. For example, if one of the other players drew three new cards, you may suspect he retained a pair and adjust your guess of the strength of his hand accordingly. The players' bets also influence your assessment of their hands. In fact, much of your past history with these particular players is part of the Markov state. Does Ellen like to bluff, or does she play conservatively? Does her face or demeanor provide clues to the strength of her hand? How does Joe's play change when it is late at night, or when he has already won a lot of money?

Although everything ever observed about the other players may have an effect on the probabilities that they are holding various kinds of hands, in practice this is far too much to remember and analyze, and most of it will have no clear effect on one's predictions and decisions. Very good poker players are adept at remembering just the key clues, and at sizing up new players quickly, but no one remembers everything that is relevant. As a result, the state representations people use to make their poker decisions are undoubtedly non-Markov, and the decisions themselves are presumably imperfect. Nevertheless, people still make very good decisions in such tasks. We conclude that the inability to have access to a *perfect* Markov state representation is probably not a severe problem for a reinforcement learning agent. ■

Exercise 3.6: Broken Vision System Imagine that you are a vision system. When you are first turned on for the day, an image floods into your camera. You can see lots of things, but not all things. You can't see objects that are occluded, and of course you can't see objects that are behind you. After seeing that first scene, do you have access to the Markov state of the environment? Suppose your camera was broken that day and you received no images at all, all day. Would you have access to the Markov state then?

3.6 Markov Decision Processes

A reinforcement learning task that satisfies the Markov property is called a *Markov decision process*, or *MDP*. If the state and action spaces are finite, then it is called a *finite Markov decision process (finite MDP)*. Finite MDPs are particularly important to the theory of reinforcement learning. We treat them extensively throughout this book; they are all you need to understand 90% of modern reinforcement learning.

A particular finite MDP is defined by its state and action sets and by the one-step dynamics of the environment. Given any state and action, s and a, the probability of each possible next state, s', is

$$p(s'|s,a) = \mathbf{Pr}\{S_{t+1}=s' \mid S_t=s, A_t=a\}. \tag{3.6}$$

These quantities are called *transition probabilities*. Similarly, given any current state and action, s and a, together with any next state, s', the expected value of the next reward is

$$r(s,a,s') = \mathbb{E}[R_{t+1} \mid S_t=s, A_t=a, S_{t+1}=s']\,. \tag{3.7}$$

These quantities, $p(s'|s,a)$ and $r(s,a,s')$, completely specify the most important aspects of the dynamics of a finite MDP (only information about the distribution of rewards around the expected value is lost). Most of the theory we present in the rest of this book implicitly assumes the environment is a finite MDP.

Example 3.7: Recycling Robot MDP The recycling robot (Example 3.3) can be turned into a simple example of an MDP by simplifying it and providing some more details. (Our aim is to produce a simple example, not a particularly realistic one.) Recall that the agent makes a decision at times determined by external events (or by other parts of the robot's control system). At each such time the robot decides whether it should (1) actively search for a can, (2) remain stationary and wait for someone to bring it a can, or (3) go back to home base to recharge its battery. Suppose the environment works as follows. The best way to find cans is to actively search for them, but this runs down the robot's battery, whereas waiting does not. Whenever the robot is searching, the possibility exists that its battery will become depleted. In this case the robot must shut down and wait to be rescued (producing a low reward).

The agent makes its decisions solely as a function of the energy level of the battery. It can distinguish two levels, `high` and `low`, so that the state set

s	s'	a	$p(s'\|s,a)$	$r(s,a,s')$
high	high	search	α	$r_{\texttt{search}}$
high	low	search	$1-\alpha$	$r_{\texttt{search}}$
low	high	search	$1-\beta$	-3
low	low	search	β	$r_{\texttt{search}}$
high	high	wait	1	$r_{\texttt{wait}}$
high	low	wait	0	$r_{\texttt{wait}}$
low	high	wait	0	$r_{\texttt{wait}}$
low	low	wait	1	$r_{\texttt{wait}}$
low	high	recharge	1	0
low	low	recharge	0	0.

Table 3.1: Transition probabilities and expected rewards for the finite MDP of the recycling robot example. There is a row for each possible combination of current state, s, next state, s', and action possible in the current state, $a \in \mathcal{A}(s)$.

is $\mathcal{S} = \{\texttt{high}, \texttt{low}\}$. Let us call the possible decisions—the agent's actions—`wait`, `search`, and `recharge`. When the energy level is `high`, recharging would always be foolish, so we do not include it in the action set for this state. The agent's action sets are

$$\begin{aligned} \mathcal{A}(\texttt{high}) &= \{\texttt{search}, \texttt{wait}\} \\ \mathcal{A}(\texttt{low}) &= \{\texttt{search}, \texttt{wait}, \texttt{recharge}\}. \end{aligned}$$

If the energy level is `high`, then a period of active search can always be completed without risk of depleting the battery. A period of searching that begins with a `high` energy level leaves the energy level `high` with probability α and reduces it to `low` with probability $1-\alpha$. On the other hand, a period of searching undertaken when the energy level is `low` leaves it `low` with probability β and depletes the battery with probability $1-\beta$. In the latter case, the robot must be rescued, and the battery is then recharged back to `high`. Each can collected by the robot counts as a unit reward, whereas a reward of -3 results whenever the robot has to be rescued. Let $r_{\texttt{search}}$ and $r_{\texttt{wait}}$, with $r_{\texttt{search}} > r_{\texttt{wait}}$, respectively denote the expected number of cans the robot will collect (and hence the expected reward) while searching and while waiting. Finally, to keep things simple, suppose that no cans can be collected during a run home for recharging, and that no cans can be collected on a step in which the battery is depleted. This system is then a finite MDP, and we can write down the transition probabilities and the expected rewards, as in Table 3.1.

A *transition graph* is a useful way to summarize the dynamics of a finite

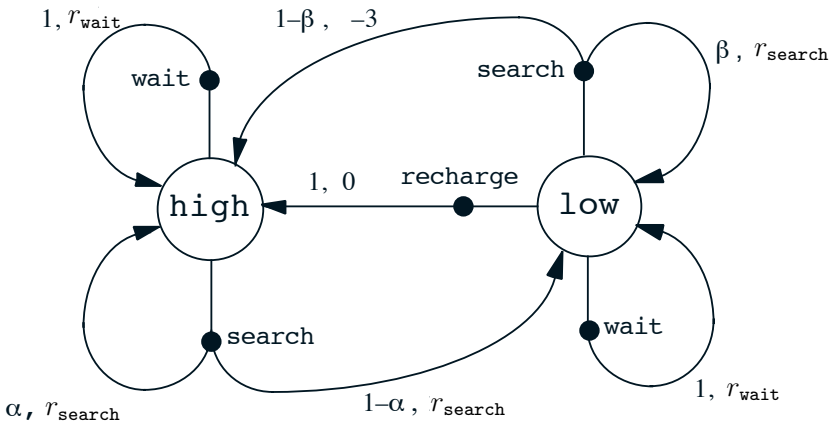


Figure 3.3: Transition graph for the recycling robot example.

MDP. Figure 3.3 shows the transition graph for the recycling robot example. There are two kinds of nodes: *state nodes* and *action nodes*. There is a state node for each possible state (a large open circle labeled by the name of the state), and an action node for each state–action pair (a small solid circle labeled by the name of the action and connected by a line to the state node). Starting in state s and taking action a moves you along the line from state node s to action node (s, a). Then the environment responds with a transition to the next state's node via one of the arrows leaving action node (s, a). Each arrow corresponds to a triple (s, s', a), where s' is the next state, and we label the arrow with the transition probability, $p(s'|s, a)$, and the expected reward for that transition, $r(s, a, s')$. Note that the transition probabilities labeling the arrows leaving an action node always sum to 1. ■

Exercise 3.7 Assuming a finite MDP with a finite number of reward values, write an equation for the transition probabilities and the expected rewards in terms of the joint conditional distribution in (3.5).

3.7 Value Functions

Almost all reinforcement learning algorithms involve estimating *value functions*—functions of states (or of state–action pairs) that estimate *how good* it is for the agent to be in a given state (or how good it is to perform a given action in a given state). The notion of "how good" here is defined in terms of future rewards that can be expected, or, to be precise, in terms of expected return. Of course the rewards the agent can expect to receive in the future depend on what actions it will take. Accordingly, value functions are defined

with respect to particular policies.

Recall that a policy, π, is a mapping from each state, $s \in \mathcal{S}$, and action, $a \in \mathcal{A}(s)$, to the probability $\pi(a|s)$ of taking action a when in state s. Informally, the *value* of a state s under a policy π, denoted $v_\pi(s)$, is the expected return when starting in s and following π thereafter. For MDPs, we can define $v_\pi(s)$ formally as

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\middle|\; S_t = s\right], \tag{3.8}$$

where $\mathbb{E}_\pi[\cdot]$ denotes the expected value given that the agent follows policy π, and t is any time step. Note that the value of the terminal state, if any, is always zero. We call the function v_π the *state-value function for policy* π.

Similarly, we define the value of taking action a in state s under a policy π, denoted $q_\pi(s, a)$, as the expected return starting from s, taking the action a, and thereafter following policy π:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\middle|\; S_t = s, A_t = a\right]. \tag{3.9}$$

We call q_π the *action-value function for policy* π.

The value functions v_π and q_π can be estimated from experience. For example, if an agent follows policy π and maintains an average, for each state encountered, of the actual returns that have followed that state, then the average will converge to the state's value, $v_\pi(s)$, as the number of times that state is encountered approaches infinity. If separate averages are kept for each action taken in a state, then these averages will similarly converge to the action values, $q_\pi(s, a)$. We call estimation methods of this kind *Monte Carlo methods* because they involve averaging over many random samples of actual returns. These kinds of methods are presented in Chapter 5. Of course, if there are very many states, then it may not be practical to keep separate averages for each state individually. Instead, the agent would have to maintain v_π and q_π as parameterized functions and adjust the parameters to better match the observed returns. This can also produce accurate estimates, although much depends on the nature of the parameterized function approximator (Chapter 9).

A fundamental property of value functions used throughout reinforcement learning and dynamic programming is that they satisfy particular recursive relationships. For any policy π and any state s, the following consistency condition holds between the value of s and the value of its possible successor

states:

$$\begin{aligned}
v_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\
&= \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\middle|\; S_t = s\right] \\
&= \mathbb{E}_\pi\left[R_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k R_{t+k+2} \;\middle|\; S_t = s\right] \\
&= \sum_a \pi(a|s) \sum_{s'} p(s'|s,a) \left[r(s,a,s') + \gamma \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+2} \;\middle|\; S_{t+1} = s'\right]\right] \\
&= \sum_a \pi(a|s) \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma v_\pi(s')\Big], \qquad (3.10)
\end{aligned}$$

where it is implicit that the actions, a, are taken from the set $\mathcal{A}(s)$, and the next states, s', are taken from the set $\mathcal{S}$, or from $\mathcal{S}^+$ in the case of an episodic problem. Equation (3.10) is the *Bellman equation for* v_π. It expresses a relationship between the value of a state and the values of its successor states. Think of looking ahead from one state to its possible successor states, as suggested by Figure 3.4a. Each open circle represents a state and each solid circle represents a state–action pair. Starting from state s, the root node at the top, the agent could take any of some set of actions—three are shown in Figure 3.4a. From each of these, the environment could respond with one of several next states, s', along with a reward, r. The Bellman equation (3.10) averages over all the possibilities, weighting each by its probability of occurring. It states that the value of the start state must equal the (discounted) value of the expected next state, plus the reward expected along the way.

The value function v_π is the unique solution to its Bellman equation. We show in subsequent chapters how this Bellman equation forms the basis of a number of ways to compute, approximate, and learn v_π. We call diagrams like those shown in Figure 3.4 *backup diagrams* because they diagram relationships that form the basis of the update or *backup* operations that are at the heart of reinforcement learning methods. These operations transfer value information *back* to a state (or a state–action pair) from its successor states (or state–action pairs). We use backup diagrams throughout the book to provide graphical summaries of the algorithms we discuss. (Note that unlike transition graphs, the state nodes of backup diagrams do not necessarily represent distinct states; for example, a state might be its own successor. We also omit explicit arrowheads because time always flows downward in a backup diagram.)

Example 3.8: Gridworld Figure 3.5a uses a rectangular grid to illustrate value functions for a simple finite MDP. The cells of the grid correspond to

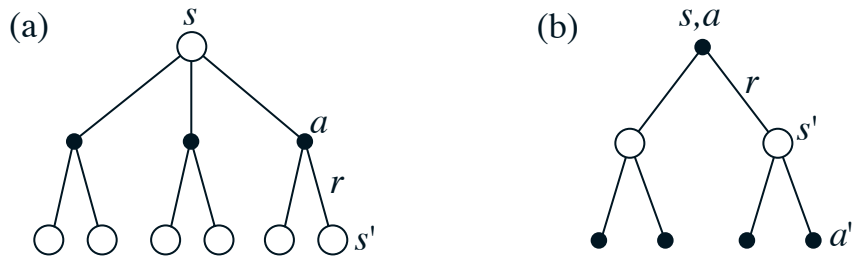


Figure 3.4: Backup diagrams for (a) v_π and (b) q_π.

the states of the environment. At each cell, four actions are possible: `north`, `south`, `east`, and `west`, which deterministically cause the agent to move one cell in the respective direction on the grid. Actions that would take the agent off the grid leave its location unchanged, but also result in a reward of -1. Other actions result in a reward of 0, except those that move the agent out of the special states A and B. From state A, all four actions yield a reward of $+10$ and take the agent to A$'$. From state B, all actions yield a reward of $+5$ and take the agent to B$'$.

Suppose the agent selects all four actions with equal probability in all states. Figure 3.5b shows the value function, v_π, for this policy, for the discounted reward case with $\gamma = 0.9$. This value function was computed by solving the system of equations (3.10). Notice the negative values near the lower edge; these are the result of the high probability of hitting the edge of the grid there under the random policy. State A is the best state to be in under this policy, but its expected return is less than 10, its immediate reward, because from A the agent is taken to A$'$, from which it is likely to run into the edge of the grid. State B, on the other hand, is valued more than 5, its immediate reward, because from B the agent is taken to B$'$, which has a positive value. From B$'$ the expected penalty (negative reward) for possibly running into an edge is more

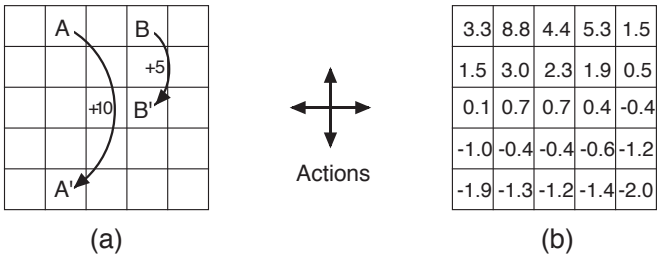


Figure 3.5: Grid example: (a) exceptional reward dynamics; (b) state-value function for the equiprobable random policy.

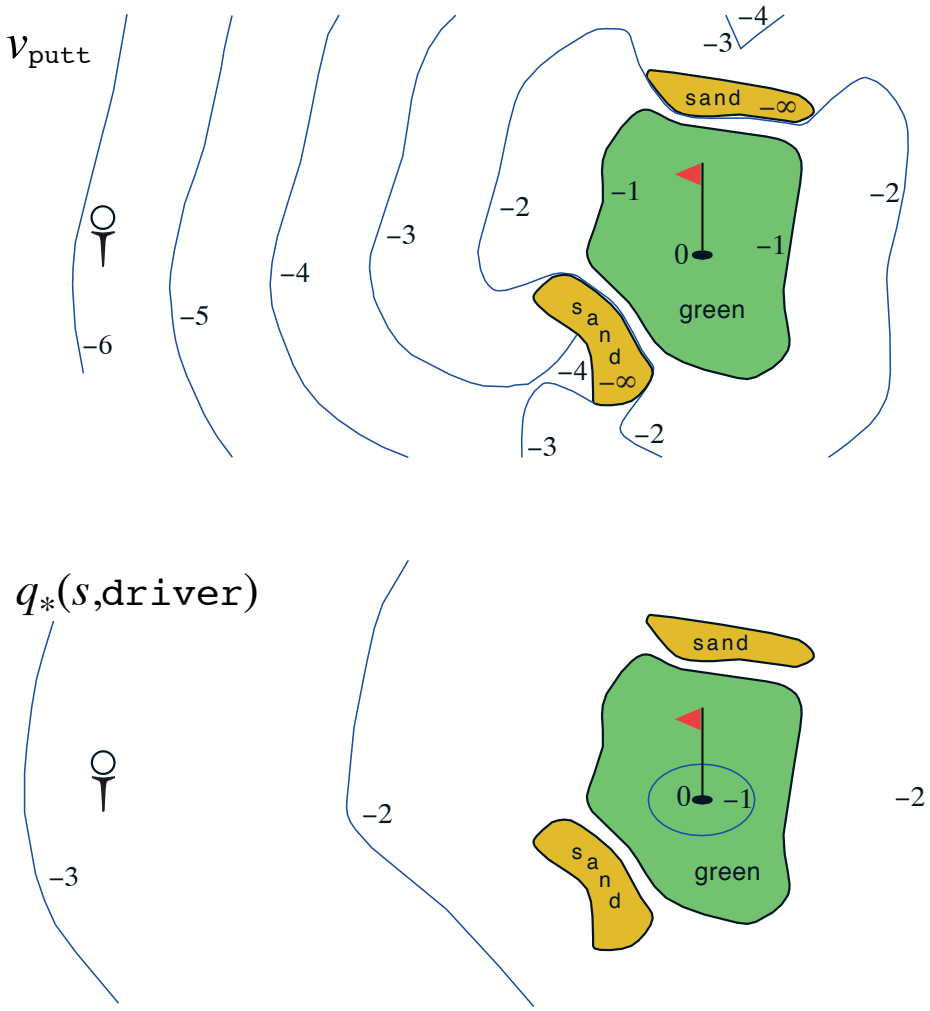


Figure 3.6: A golf example: the state-value function for putting (above) and the optimal action-value function for using the driver (below).

than compensated for by the expected gain for possibly stumbling onto A or B. ■

Example 3.9: Golf To formulate playing a hole of golf as a reinforcement learning task, we count a penalty (negative reward) of -1 for each stroke until we hit the ball into the hole. The state is the location of the ball. The value of a state is the negative of the number of strokes to the hole from that location. Our actions are how we aim and swing at the ball, of course, and which club we select. Let us take the former as given and consider just the choice of club, which we assume is either a putter or a driver. The upper part of Figure 3.6 shows a possible state-value function, $v_{\texttt{putt}}(s)$, for the policy that always uses the putter. The terminal state *in-the-hole* has a value of 0. From anywhere

on the green we assume we can make a putt; these states have value -1. Off the green we cannot reach the hole by putting, and the value is greater. If we can reach the green from a state by putting, then that state must have value one less than the green's value, that is, -2. For simplicity, let us assume we can putt very precisely and deterministically, but with a limited range. This gives us the sharp contour line labeled -2 in the figure; all locations between that line and the green require exactly two strokes to complete the hole. Similarly, any location within putting range of the -2 contour line must have a value of -3, and so on to get all the contour lines shown in the figure. Putting doesn't get us out of sand traps, so they have a value of $-\infty$. Overall, it takes us six strokes to get from the tee to the hole by putting. ■

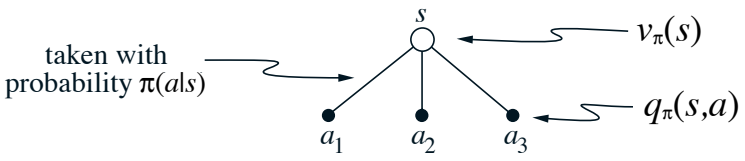
Exercise 3.8 What is the Bellman equation for action values, that is, for q_π? It must give the action value $q_\pi(s, a)$ in terms of the action values, $q_\pi(s', a')$, of possible successors to the state–action pair (s, a). As a hint, the backup diagram corresponding to this equation is given in Figure 3.4b. Show the sequence of equations analogous to (3.10), but for action values.

Exercise 3.9 The Bellman equation (3.10) must hold for each state for the value function v_π shown in Figure 3.5b. As an example, show numerically that this equation holds for the center state, valued at $+0.7$, with respect to its four neighboring states, valued at $+2.3$, $+0.4$, -0.4, and $+0.7$. (These numbers are accurate only to one decimal place.)

Exercise 3.10 In the gridworld example, rewards are positive for goals, negative for running into the edge of the world, and zero the rest of the time. Are the signs of these rewards important, or only the intervals between them? Prove, using (3.2), that adding a constant C to all the rewards adds a constant, K, to the values of all states, and thus does not affect the relative values of any states under any policies. What is K in terms of C and γ?

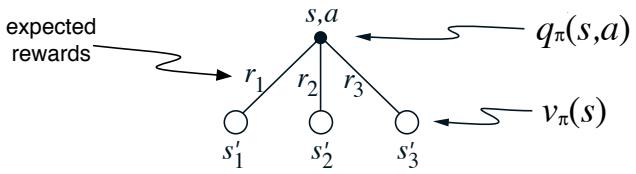
Exercise 3.11 Now consider adding a constant C to all the rewards in an episodic task, such as maze running. Would this have any effect, or would it leave the task unchanged as in the continuing task above? Why or why not? Give an example.

Exercise 3.12 The value of a state depends on the the values of the actions possible in that state and on how likely each action is to be taken under the current policy. We can think of this in terms of a small backup diagram rooted at the state and considering each possible action:



Give the equation corresponding to this intuition and diagram for the value at the root node, $v_\pi(s)$, in terms of the value at the expected leaf node, $q_\pi(s,a)$, given $S_t = s$. This expectation depends on the policy, π. Then give a second equation in which the expected value is written out explicitly in terms of $\pi(a|s)$ such that no expected value notation appears in the equation.

Exercise 3.13 The value of an action, $q_\pi(s,a)$, can be divided into two parts, the expected next reward, which does not depend on the policy π, and the expected sum of the remaining rewards, which depends on the next state and the policy. Again we can think of this in terms of a small backup diagram, this one rooted at an action (state–action pair) and branching to the possible next states:



Give the equation corresponding to this intuition and diagram for the action value, $q_\pi(s,a)$, in terms of the expected next reward, R_{t+1}, and the expected next state value, $v_\pi(S_{t+1})$, given that $S_t = s$ and $A_t = a$. Then give a second equation, writing out the expected value explicitly in terms of $p(s'|s,a)$ and $r(s,a,s')$, defined respectively by (3.6) and (3.7), such that no expected value notation appears in the equation.

3.8 Optimal Value Functions

Solving a reinforcement learning task means, roughly, finding a policy that achieves a lot of reward over the long run. For finite MDPs, we can precisely define an optimal policy in the following way. Value functions define a partial ordering over policies. A policy π is defined to be better than or equal to a policy π' if its expected return is greater than or equal to that of π' for all states. In other words, $\pi \geq \pi'$ if and only if $v_\pi(s) \geq v_{\pi'}(s)$ for all $s \in \mathcal{S}$. There is always at least one policy that is better than or equal to all other policies. This is an *optimal policy.* Although there may be more than one, we denote all the optimal policies by π_*. They share the same state-value function, called

the *optimal state-value function*, denoted v_*, and defined as

$$v_*(s) = \max_\pi v_\pi(s), \tag{3.11}$$

for all $s \in \mathcal{S}$.

Optimal policies also share the same *optimal action-value function*, denoted q_*, and defined as

$$q_*(s,a) = \max_\pi q_\pi(s,a), \tag{3.12}$$

for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$. For the state–action pair (s,a), this function gives the expected return for taking action a in state s and thereafter following an optimal policy. Thus, we can write q_* in terms of v_* as follows:

$$q_*(s,a) = \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]. \tag{3.13}$$

Example 3.10: Optimal Value Functions for Golf The lower part of Figure 3.6 shows the contours of a possible optimal action-value function $q_*(s, \texttt{driver})$. These are the values of each state if we first play a stroke with the driver and afterward select either the driver or the putter, whichever is better. The driver enables us to hit the ball farther, but with less accuracy. We can reach the hole in one shot using the driver only if we are already very close; thus the -1 contour for $q_*(s, \texttt{driver})$ covers only a small portion of the green. If we have two strokes, however, then we can reach the hole from much farther away, as shown by the -2 contour. In this case we don't have to drive all the way to within the small -1 contour, but only to anywhere on the green; from there we can use the putter. The optimal action-value function gives the values after committing to a particular *first* action, in this case, to the driver, but afterward using whichever actions are best. The -3 contour is still farther out and includes the starting tee. From the tee, the best sequence of actions is two drives and one putt, sinking the ball in three strokes. ■

Because v_* is the value function for a policy, it must satisfy the self-consistency condition given by the Bellman equation for state values (3.10). Because it is the optimal value function, however, v_*'s consistency condition can be written in a special form without reference to any specific policy. This is the Bellman equation for v_*, or the *Bellman optimality equation*. Intuitively, the Bellman optimality equation expresses the fact that the value of a state under an optimal policy must equal the expected return for the best action

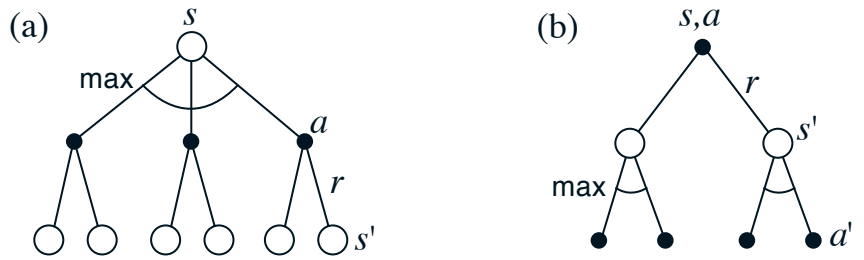


Figure 3.7: Backup diagrams for (a) v_* and (b) q_*

from that state:

$$\begin{aligned}
v_*(s) &= \max_{a\in\mathcal{A}(s)} q_{\pi_*}(s,a) \\
&= \max_a \mathbb{E}_{\pi^*}[G_t \mid S_t{=}s, A_t{=}a] \\
&= \max_a \mathbb{E}_{\pi^*}\left[\sum_{k=0}^{\infty}\gamma^k R_{t+k+1} \;\middle|\; S_t{=}s, A_t{=}a\right] \\
&= \max_a \mathbb{E}_{\pi^*}\left[R_{t+1} + \gamma\sum_{k=0}^{\infty}\gamma^k R_{t+k+2} \;\middle|\; S_t{=}s, A_t{=}a\right] \\
&= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t{=}s, A_t{=}a] \qquad (3.14) \\
&= \max_{a\in\mathcal{A}(s)} \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma v_*(s')\Big]. \qquad (3.15)
\end{aligned}$$

The last two equations are two forms of the Bellman optimality equation for v_*. The Bellman optimality equation for q_* is

$$\begin{aligned}
q_*(s,a) &= \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \;\Big|\; S_t = s, A_t = a\Big] \\
&= \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma \max_{a'} q_*(s',a')\Big].
\end{aligned}$$

The backup diagrams in Figure 3.7 show graphically the spans of future states and actions considered in the Bellman optimality equations for v_* and q_*. These are the same as the backup diagrams for v_π and q_π except that arcs have been added at the agent's choice points to represent that the maximum over that choice is taken rather than the expected value given some policy. Figure 3.7a graphically represents the Bellman optimality equation (3.15).

For finite MDPs, the Bellman optimality equation (3.15) has a unique solution independent of the policy. The Bellman optimality equation is actually

a system of equations, one for each state, so if there are N states, then there are N equations in N unknowns. If the dynamics of the environment are known ($r(s, a, s')$ and $p(s'|s, a)$), then in principle one can solve this system of equations for v_* using any one of a variety of methods for solving systems of nonlinear equations. One can solve a related set of equations for q_*.

Once one has v_*, it is relatively easy to determine an optimal policy. For each state s, there will be one or more actions at which the maximum is obtained in the Bellman optimality equation. Any policy that assigns nonzero probability only to these actions is an optimal policy. You can think of this as a one-step search. If you have the optimal value function, v_*, then the actions that appear best after a one-step search will be optimal actions. Another way of saying this is that any policy that is *greedy* with respect to the optimal evaluation function v_* is an optimal policy. The term greedy is used in computer science to describe any search or decision procedure that selects alternatives based only on local or immediate considerations, without considering the possibility that such a selection may prevent future access to even better alternatives. Consequently, it describes policies that select actions based only on their short-term consequences. The beauty of v_* is that if one uses it to evaluate the short-term consequences of actions—specifically, the one-step consequences—then a greedy policy is actually optimal in the long-term sense in which we are interested because v_* already takes into account the reward consequences of all possible future behavior. By means of v_*, the optimal expected long-term return is turned into a quantity that is locally and immediately available for each state. Hence, a one-step-ahead search yields the long-term optimal actions.

Having q_* makes choosing optimal actions still easier. With q_*, the agent does not even have to do a one-step-ahead search: for any state s, it can simply find any action that maximizes $q_*(s, a)$. The action-value function effectively caches the results of all one-step-ahead searches. It provides the optimal expected long-term return as a value that is locally and immediately available for each state–action pair. Hence, at the cost of representing a function of state–action pairs, instead of just of states, the optimal action-value function allows optimal actions to be selected without having to know anything about possible successor states and their values, that is, without having to know anything about the environment's dynamics.

Example 3.11: Bellman Optimality Equations for the Recycling Robot Using (3.15), we can explicitly give the Bellman optimality equation for the recycling robot example. To make things more compact, we abbreviate the states `high` and `low`, and the actions `search`, `wait`, and `recharge` respectively by `h`, `l`, `s`, `w`, and `re`. Since there are only two states, the Bellman optimality equation consists of two equations. The equation for $v_*(\texttt{h})$ can be

written as follows:

$$\begin{aligned} v_*(\mathtt{h}) &= \max \left\{ \begin{array}{l} p(\mathtt{h}|\mathtt{h},\mathtt{s})[r(\mathtt{h},\mathtt{s},\mathtt{h}) + \gamma v_*(\mathtt{h})] + p(\mathtt{l}|\mathtt{h},\mathtt{s})[r(\mathtt{h},\mathtt{s},\mathtt{l}) + \gamma v_*(\mathtt{l})], \\ p(\mathtt{h}|\mathtt{h},\mathtt{w})[r(\mathtt{h},\mathtt{w},\mathtt{h}) + \gamma v_*(\mathtt{h})] + p(\mathtt{l}|\mathtt{h},\mathtt{w})[r(\mathtt{h},\mathtt{w},\mathtt{l}) + \gamma v_*(\mathtt{l})] \end{array} \right\} \\ &= \max \left\{ \begin{array}{l} \alpha[r_\mathtt{s} + \gamma v_*(\mathtt{h})] + (1-\alpha)[r_\mathtt{s} + \gamma v_*(\mathtt{l})], \\ 1[r_\mathtt{w} + \gamma v_*(\mathtt{h})] + 0[r_\mathtt{w} + \gamma v_*(\mathtt{l})] \end{array} \right\} \\ &= \max \left\{ \begin{array}{l} r_\mathtt{s} + \gamma[\alpha v_*(\mathtt{h}) + (1-\alpha) v_*(\mathtt{l})], \\ r_\mathtt{w} + \gamma v_*(\mathtt{h}) \end{array} \right\}. \end{aligned}$$

Following the same procedure for $v_*(\mathtt{l})$ yields the equation

$$v_*(\mathtt{l}) = \max \left\{ \begin{array}{l} \beta r_\mathtt{s} - 3(1-\beta) + \gamma[(1-\beta) v_*(\mathtt{h}) + \beta v_*(\mathtt{l})] \\ r_\mathtt{w} + \gamma v_*(\mathtt{l}), \\ \gamma v_*(\mathtt{h}) \end{array} \right\}.$$

For any choice of $r_\mathtt{s}$, $r_\mathtt{w}$, α, β, and γ, with $0 \leq \gamma < 1$, $0 \leq \alpha, \beta \leq 1$, there is exactly one pair of numbers, $v_*(\mathtt{h})$ and $v_*(\mathtt{l})$, that simultaneously satisfy these two nonlinear equations.

Example 3.12: Solving the Gridworld Suppose we solve the Bellman equation for v_* for the simple grid task introduced in Example 3.8 and shown again in Figure 3.8a. Recall that state A is followed by a reward of +10 and transition to state A′, while state B is followed by a reward of +5 and transition to state B′. Figure 3.8b shows the optimal value function, and Figure 3.8c shows the corresponding optimal policies. Where there are multiple arrows in a cell, any of the corresponding actions is optimal.

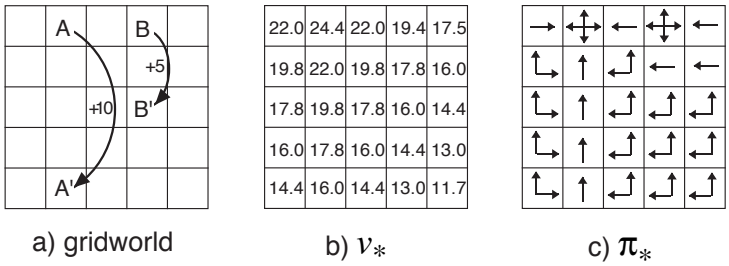


Figure 3.8: Optimal solutions to the gridworld example.

Explicitly solving the Bellman optimality equation provides one route to finding an optimal policy, and thus to solving the reinforcement learning problem. However, this solution is rarely directly useful. It is akin to an exhaustive search, looking ahead at all possibilities, computing their probabilities of occurrence and their desirabilities in terms of expected rewards. This solution

relies on at least three assumptions that are rarely true in practice: (1) we accurately know the dynamics of the environment; (2) we have enough computational resources to complete the computation of the solution; and (3) the Markov property. For the kinds of tasks in which we are interested, one is generally not able to implement this solution exactly because various combinations of these assumptions are violated. For example, although the first and third assumptions present no problems for the game of backgammon, the second is a major impediment. Since the game has about 10^{20} states, it would take thousands of years on today's fastest computers to solve the Bellman equation for v_*, and the same is true for finding q_*. In reinforcement learning one typically has to settle for approximate solutions.

Many different decision-making methods can be viewed as ways of approximately solving the Bellman optimality equation. For example, heuristic search methods can be viewed as expanding the right-hand side of (3.15) several times, up to some depth, forming a "tree" of possibilities, and then using a heuristic evaluation function to approximate v_* at the "leaf" nodes. (Heuristic search methods such as A* are almost always based on the episodic case.) The methods of dynamic programming can be related even more closely to the Bellman optimality equation. Many reinforcement learning methods can be clearly understood as approximately solving the Bellman optimality equation, using actual experienced transitions in place of knowledge of the expected transitions. We consider a variety of such methods in the following chapters.

Exercise 3.14 Draw or describe the optimal state-value function for the golf example.

Exercise 3.15 Draw or describe the contours of the optimal action-value function for putting, $q_*(s, \texttt{putter})$, for the golf example.

Exercise 3.16 Give the Bellman equation for q_* for the recycling robot.

Exercise 3.17 Figure 3.8 gives the optimal value of the best state of the gridworld as 24.4, to one decimal place. Use your knowledge of the optimal policy and (3.2) to express this value symbolically, and then to compute it to three decimal places.

3.9 Optimality and Approximation

We have defined optimal value functions and optimal policies. Clearly, an agent that learns an optimal policy has done very well, but in practice this rarely happens. For the kinds of tasks in which we are interested, optimal policies can be generated only with extreme computational cost. A well-defined

notion of optimality organizes the approach to learning we describe in this book and provides a way to understand the theoretical properties of various learning algorithms, but it is an ideal that agents can only approximate to varying degrees. As we discussed above, even if we have a complete and accurate model of the environment's dynamics, it is usually not possible to simply compute an optimal policy by solving the Bellman optimality equation. For example, board games such as chess are a tiny fraction of human experience, yet large, custom-designed computers still cannot compute the optimal moves. A critical aspect of the problem facing the agent is always the computational power available to it, in particular, the amount of computation it can perform in a single time step.

The memory available is also an important constraint. A large amount of memory is often required to build up approximations of value functions, policies, and models. In tasks with small, finite state sets, it is possible to form these approximations using arrays or tables with one entry for each state (or state–action pair). This we call the *tabular* case, and the corresponding methods we call tabular methods. In many cases of practical interest, however, there are far more states than could possibly be entries in a table. In these cases the functions must be approximated, using some sort of more compact parameterized function representation.

Our framing of the reinforcement learning problem forces us to settle for approximations. However, it also presents us with some unique opportunities for achieving useful approximations. For example, in approximating optimal behavior, there may be many states that the agent faces with such a low probability that selecting suboptimal actions for them has little impact on the amount of reward the agent receives. Tesauro's backgammon player, for example, plays with exceptional skill even though it might make very bad decisions on board configurations that never occur in games against experts. In fact, it is possible that TD-Gammon makes bad decisions for a large fraction of the game's state set. The on-line nature of reinforcement learning makes it possible to approximate optimal policies in ways that put more effort into learning to make good decisions for frequently encountered states, at the expense of less effort for infrequently encountered states. This is one key property that distinguishes reinforcement learning from other approaches to approximately solving MDPs.

3.10 Summary

Let us summarize the elements of the reinforcement learning problem that we have presented in this chapter. Reinforcement learning is about learning

from interaction how to behave in order to achieve a goal. The reinforcement learning *agent* and its *environment* interact over a sequence of discrete time steps. The specification of their interface defines a particular task: the *actions* are the choices made by the agent; the *states* are the basis for making the choices; and the *rewards* are the basis for evaluating the choices. Everything inside the agent is completely known and controllable by the agent; everything outside is incompletely controllable but may or may not be completely known. A *policy* is a stochastic rule by which the agent selects actions as a function of states. The agent's objective is to maximize the amount of reward it receives over time.

The *return* is the function of future rewards that the agent seeks to maximize. It has several different definitions depending upon the nature of the task and whether one wishes to *discount* delayed reward. The undiscounted formulation is appropriate for *episodic tasks*, in which the agent–environment interaction breaks naturally into *episodes*; the discounted formulation is appropriate for *continuing tasks*, in which the interaction does not naturally break into episodes but continues without limit.

An environment satisfies the *Markov property* if its state signal compactly summarizes the past without degrading the ability to predict the future. This is rarely exactly true, but often nearly so; the state signal should be chosen or constructed so that the Markov property holds as nearly as possible. In this book we assume that this has already been done and focus on the decision-making problem: how to decide what to do as a function of whatever state signal is available. If the Markov property does hold, then the environment is called a *Markov decision process* (MDP). A *finite MDP* is an MDP with finite state and action sets. Most of the current theory of reinforcement learning is restricted to finite MDPs, but the methods and ideas apply more generally.

A policy's *value functions* assign to each state, or state–action pair, the expected return from that state, or state–action pair, given that the agent uses the policy. The *optimal value functions* assign to each state, or state–action pair, the largest expected return achievable by any policy. A policy whose value functions are optimal is an *optimal policy*. Whereas the optimal value functions for states and state–action pairs are unique for a given MDP, there can be many optimal policies. Any policy that is *greedy* with respect to the optimal value functions must be an optimal policy. The *Bellman optimality equations* are special consistency condition that the optimal value functions must satisfy and that can, in principle, be solved for the optimal value functions, from which an optimal policy can be determined with relative ease.

A reinforcement learning problem can be posed in a variety of different ways depending on assumptions about the level of knowledge initially available to

the agent. In problems of *complete knowledge*, the agent has a complete and accurate model of the environment's dynamics. If the environment is an MDP, then such a model consists of the one-step *transition probabilities* and *expected rewards* for all states and their allowable actions. In problems of *incomplete knowledge*, a complete and perfect model of the environment is not available.

Even if the agent has a complete and accurate environment model, the agent is typically unable to perform enough computation per time step to fully use it. The memory available is also an important constraint. Memory may be required to build up accurate approximations of value functions, policies, and models. In most cases of practical interest there are far more states than could possibly be entries in a table, and approximations must be made.

A well-defined notion of optimality organizes the approach to learning we describe in this book and provides a way to understand the theoretical properties of various learning algorithms, but it is an ideal that reinforcement learning agents can only approximate to varying degrees. In reinforcement learning we are very much concerned with cases in which optimal solutions cannot be found but must be approximated in some way.

3.11 Bibliographical and Historical Remarks

The reinforcement learning problem is deeply indebted to the idea of Markov decision processes (MDPs) from the field of optimal control. These historical influences and other major influences from psychology are described in the brief history given in Chapter 1. Reinforcement learning adds to MDPs a focus on approximation and incomplete information for realistically large problems. MDPs and the reinforcement learning problem are only weakly linked to traditional learning and decision-making problems in artificial intelligence. However, artificial intelligence is now vigorously exploring MDP formulations for planning and decision-making from a variety of perspectives. MDPs are more general than previous formulations used in artificial intelligence in that they permit more general kinds of goals and uncertainty.

Our presentation of the reinforcement learning problem was influenced by Watkins (1989).

3.1 The bioreactor example is based on the work of Ungar (1990) and Miller and Williams (1992). The recycling robot example was inspired by the can-collecting robot built by Jonathan Connell (1989).

3.3–4 The terminology of *episodic* and *continuing* tasks is different from that usually used in the MDP literature. In that literature it is common

to distinguish three types of tasks: (1) finite-horizon tasks, in which interaction terminates after a particular *fixed* number of time steps; (2) indefinite-horizon tasks, in which interaction can last arbitrarily long but must eventually terminate; and (3) infinite-horizon tasks, in which interaction does not terminate. Our episodic and continuing tasks are similar to indefinite-horizon and infinite-horizon tasks, respectively, but we prefer to emphasize the difference in the nature of the interaction. This difference seems more fundamental than the difference in the objective functions emphasized by the usual terms. Often episodic tasks use an indefinite-horizon objective function and continuing tasks an infinite-horizon objective function, but we see this as a common coincidence rather than a fundamental difference.

The pole-balancing example is from Michie and Chambers (1968) and Barto, Sutton, and Anderson (1983).

3.5 For further discussion of the concept of state, see Minsky (1967).

3.6 The theory of MDPs is treated by, e.g., Bertsekas (1995), Ross (1983), White (1969), and Whittle (1982, 1983). This theory is also studied under the heading of stochastic optimal control, where *adaptive* optimal control methods are most closely related to reinforcement learning (e.g., Kumar, 1985; Kumar and Varaiya, 1986).

The theory of MDPs evolved from efforts to understand the problem of making sequences of decisions under uncertainty, where each decision can depend on the previous decisions and their outcomes. It is sometimes called the theory of multistage decision processes, or sequential decision processes, and has roots in the statistical literature on sequential sampling beginning with the papers by Thompson (1933, 1934) and Robbins (1952) that we cited in Chapter 2 in connection with bandit problems (which are prototypical MDPs if formulated as multiple-situation problems).

The earliest instance of which we are aware in which reinforcement learning was discussed using the MDP formalism is Andreae's (1969b) description of a unified view of learning machines. Witten and Corbin (1973) experimented with a reinforcement learning system later analyzed by Witten (1977) using the MDP formalism. Although he did not explicitly mention MDPs, Werbos (1977) suggested approximate solution methods for stochastic optimal control problems that are related to modern reinforcement learning methods (see also Werbos, 1982, 1987, 1988, 1989, 1992). Although Werbos's ideas were not widely recognized

at the time, they were prescient in emphasizing the importance of approximately solving optimal control problems in a variety of domains, including artificial intelligence. The most influential integration of reinforcement learning and MDPs is due to Watkins (1989). His treatment of reinforcement learning using the MDP formalism has been widely adopted.

Our characterization of the reward dynamics of an MDP in terms of $r(s, a, s')$ is slightly unusual. It is more common in the MDP literature to describe the reward dynamics in terms of the expected next reward given just the current state and action, that is, by $r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$. This quantity is related to our $r(s, a, s')$ as follows:

$$r(s, a) = \sum_{s'} p(s'|s, a) r(s, a, s').$$

In conventional MDP theory, $r(s, a, s')$ always appears in an expected value sum like this one, and therefore it is easier to use $r(s, a)$. In reinforcement learning, however, we more often have to refer to individual actual or sample outcomes. In teaching reinforcement learning, we have found the $r(s, a, s')$ notation to be more straightforward conceptually and easier to understand.

3.7–8 Assigning value on the basis of what is good or bad in the long run has ancient roots. In control theory, mapping states to numerical values representing the long-term consequences of control decisions is a key part of optimal control theory, which was developed in the 1950s by extending nineteenth century state-function theories of classical mechanics (see, e.g., Schultz and Melsa, 1967). In describing how a computer could be programmed to play chess, Shannon (1950) suggested using an evaluation function that took into account the long-term advantages and disadvantages of chess positions.

Watkins's (1989) Q-learning algorithm for estimating q_* (Chapter 6) made action-value functions an important part of reinforcement learning, and consequently these functions are often called *Q-functions*. But the idea of an action-value function is much older than this. Shannon (1950) suggested that a function $h(P, M)$ could be used by a chess-playing program to decide whether a move M in position P is worth exploring. Michie's (1961, 1963) MENACE system and Michie and Chambers's (1968) BOXES system can be understood as estimating action-value functions. In classical physics, Hamilton's principal function is an action-value function; Newtonian dynamics are greedy with

respect to this function (e.g., Goldstein, 1957). Action-value functions also played a central role in Denardo's (1967) theoretical treatment of DP in terms of contraction mappings.

What we call the Bellman equation for v_* was first introduced by Richard Bellman (1957a), who called it the "basic functional equation." The counterpart of the Bellman optimality equation for continuous time and state problems is known as the Hamilton–Jacobi–Bellman equation (or often just the Hamilton–Jacobi equation), indicating its roots in classical physics (e.g., Schultz and Melsa, 1967).

The golf example was suggested by Chris Watkins.

Part II

Tabular Action-Value Methods

In this part of the book we describe almost all the core ideas of reinforcement learning algorithms in their simplest forms, that in which the state and action spaces are small enough for the approximate action-value function to be represented as an array, or *table*. In this case, the methods can often find exact solutions, that is, they can often find exactly the optimal value function and the optimal policy. This contrasts with the approximate methods described in the next part of the book, which only find approximate solutions, but which in return can be applied effectively to much larger problems.

The first three chapters of this part of the book describe three fundamental classes of methods for solving finite Markov decision problems: dynamic programming, Monte Carlo methods, and temporal-difference learning. Each class of methods has its strengths and weaknesses. Dynamic programming methods are well developed mathematically, but require a complete and accurate model of the environment. Monte Carlo methods don't require a model and are conceptually simple, but are not suited for step-by-step incremental computation. Finally, temporal-difference methods require no model and are fully incremental, but are more complex to analyze. The methods also differ in several ways with respect to their efficiency and speed of convergence.

The remaining two chapters describe how these three classes of methods can be combined to obtain the best features of each of them. In one chapter we describe how the strengths of Monte Carlo methods can be combined with the strengths of temporal-difference methods via the use of eligibility traces. In the final chapter of this part of the book we show these two learning methods can be combined with model learning and planning methods (such as dynamic programming) for a complete and unified solution to the tabular reinforcement learning problem.

Chapter 4

Dynamic Programming

The term dynamic programming (DP) refers to a collection of algorithms that can be used to compute optimal policies given a perfect model of the environment as a Markov decision process (MDP). Classical DP algorithms are of limited utility in reinforcement learning both because of their assumption of a perfect model and because of their great computational expense, but they are still important theoretically. DP provides an essential foundation for the understanding of the methods presented in the rest of this book. In fact, all of these methods can be viewed as attempts to achieve much the same effect as DP, only with less computation and without assuming a perfect model of the environment.

Starting with this chapter, we usually assume that the environment is a finite MDP. That is, we assume that its state and action sets, $\mathcal{S}$ and $\mathcal{A}(s)$, for $s \in \mathcal{S}$, are finite, and that its dynamics are given by a set of transition probabilities, $p(s'|s,a) = \mathbf{Pr}\{S_{t+1}{=}s' \mid S_t{=}s, A_t{=}a\}$, and expected immediate rewards, $r(s,a,s') = \mathbb{E}[R_{t+1} \mid A_t{=}a, S_t{=}s, S_{t+1}{=}s']$, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, and $s' \in \mathcal{S}^+$ ($\mathcal{S}^+$ is $\mathcal{S}$ plus a terminal state if the problem is episodic). Although DP ideas can be applied to problems with continuous state and action spaces, exact solutions are possible only in special cases. A common way of obtaining approximate solutions for tasks with continuous states and actions is to quantize the state and action spaces and then apply finite-state DP methods. The methods we explore in Chapter 9 are applicable to continuous problems and are a significant extension of that approach.

The key idea of DP, and of reinforcement learning generally, is the use of value functions to organize and structure the search for good policies. In this chapter we show how DP can be used to compute the value functions defined in Chapter 3. As discussed there, we can easily obtain optimal policies once we have found the optimal value functions, v_* or q_*, which satisfy the Bellman

optimality equations:

$$\begin{aligned} v_*(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \max_a \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma v_*(s')\Big] \end{aligned} \tag{4.1}$$

or

$$\begin{aligned} q_*(s,a) &= \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \;\Big|\; S_t = s, A_t = a\Big] \\ &= \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma \max_{a'} q_*(s',a')\Big], \end{aligned} \tag{4.2}$$

for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, and $s' \in \mathcal{S}^+$. As we shall see, DP algorithms are obtained by turning Bellman equations such as these into assignments, that is, into update rules for improving approximations of the desired value functions.

4.1 Policy Evaluation

First we consider how to compute the state-value function v_π for an arbitrary policy π. This is called *policy evaluation* in the DP literature. We also refer to it as the *prediction problem*. Recall from Chapter 3 that, for all $s \in \mathcal{S}$,

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi\big[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \;\big|\; S_t = s\big] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \qquad (4.3) \\ &= \sum_a \pi(a|s) \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma v_\pi(s')\Big], \qquad (4.4) \end{aligned}$$

where $\pi(a|s)$ is the probability of taking action a in state s under policy π, and the expectations are subscripted by π to indicate that they are conditional on π being followed. The existence and uniqueness of v_π are guaranteed as long as either $\gamma < 1$ or eventual termination is guaranteed from all states under the policy π.

If the environment's dynamics are completely known, then (4.4) is a system of $|\mathcal{S}|$ simultaneous linear equations in $|\mathcal{S}|$ unknowns (the $v_\pi(s)$, $s \in \mathcal{S}$). In principle, its solution is a straightforward, if tedious, computation. For our purposes, iterative solution methods are most suitable. Consider a sequence of approximate value functions $v_0, v_1, v_2, \ldots$, each mapping $\mathcal{S}^+$ to $\mathbb{R}$. The initial approximation, v_0, is chosen arbitrarily (except that the terminal state, if any,

must be given value 0), and each successive approximation is obtained by using the Bellman equation for v_π (3.10) as an update rule:

$$\begin{aligned} v_{k+1}(s) &= \mathbb{E}_\pi[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma v_k(s')\Big], \end{aligned} \tag{4.5}$$

for all $s \in \mathcal{S}$. Clearly, $v_k = v_\pi$ is a fixed point for this update rule because the Bellman equation for v_π assures us of equality in this case. Indeed, the sequence $\{v_k\}$ can be shown in general to converge to v_π as $k \to \infty$ under the same conditions that guarantee the existence of v_π. This algorithm is called *iterative policy evaluation*.

To produce each successive approximation, v_{k+1} from v_k, iterative policy evaluation applies the same operation to each state s: it replaces the old value of s with a new value obtained from the old values of the successor states of s, and the expected immediate rewards, along all the one-step transitions possible under the policy being evaluated. We call this kind of operation a *full backup*. Each iteration of iterative policy evaluation *backs up* the value of every state once to produce the new approximate value function v_{k+1}. There are several different kinds of full backups, depending on whether a state (as here) or a state–action pair is being backed up, and depending on the precise way the estimated values of the successor states are combined. All the backups done in DP algorithms are called *full* backups because they are based on all possible next states rather than on a sample next state. The nature of a backup can be expressed in an equation, as above, or in a backup diagram like those introduced in Chapter 3. For example, Figure 3.4a is the backup diagram corresponding to the full backup used in iterative policy evaluation.

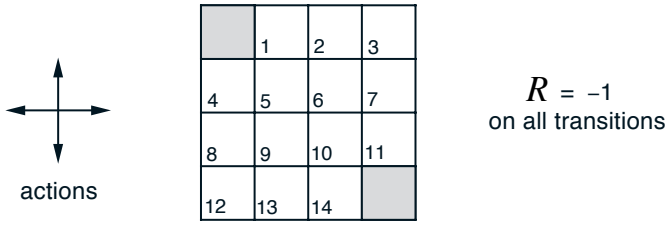
To write a sequential computer program to implement iterative policy evaluation, as given by (4.5), you would have to use two arrays, one for the old values, $v_k(s)$, and one for the new values, $v_{k+1}(s)$. This way, the new values can be computed one by one from the old values without the old values being changed. Of course it is easier to use one array and update the values "in place," that is, with each new backed-up value immediately overwriting the old one. Then, depending on the order in which the states are backed up, sometimes new values are used instead of old ones on the right-hand side of (4.5). This slightly different algorithm also converges to v_π; in fact, it usually converges faster than the two-array version, as you might expect, since it uses new data as soon as they are available. We think of the backups as being done in a *sweep* through the state space. For the in-place algorithm, the order in which states are backed up during the sweep has a significant influence on the rate of convergence. We usually have the in-place version in mind when we think of DP algorithms.

Input π, the policy to be evaluated
Initialize an array $v(s) = 0$, for all $s \in \mathcal{S}^+$
Repeat
  $\Delta \leftarrow 0$
  For each $s \in \mathcal{S}$:
    $temp \leftarrow v(s)$
    $v(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} p(s'|s,a)[r(s,a,s') + \gamma v(s')]$
    $\Delta \leftarrow \max(\Delta, |temp - v(s)|)$
until $\Delta < \theta$ (a small positive number)
Output $v \approx v_\pi$

Figure 4.1: Iterative policy evaluation.

Another implementation point concerns the termination of the algorithm. Formally, iterative policy evaluation converges only in the limit, but in practice it must be halted short of this. A typical stopping condition for iterative policy evaluation is to test the quantity $\max_{s \in \mathcal{S}} |v_{k+1}(s) - v_k(s)|$ after each sweep and stop when it is sufficiently small. Figure 4.1 gives a complete algorithm for iterative policy evaluation with this stopping criterion.

Example 4.1 Consider the 4×4 gridworld shown below.



The nonterminal states are $\mathcal{S} = \{1, 2, \ldots, 14\}$. There are four actions possible in each state, $\mathcal{A} = \{\texttt{up}, \texttt{down}, \texttt{right}, \texttt{left}\}$, which deterministically cause the corresponding state transitions, except that actions that would take the agent off the grid in fact leave the state unchanged. Thus, for instance, $p(6|5, \texttt{right}) = 1$, $p(10|5, \texttt{right}) = 0$, and $p(7|7, \texttt{right}) = 1$. This is an undiscounted, episodic task. The reward is -1 on all transitions until the terminal state is reached. The terminal state is shaded in the figure (although it is shown in two places, it is formally one state). The expected reward function is thus $r(s, a, s') = -1$ for all states s, s' and actions a. Suppose the agent follows the equiprobable random policy (all actions equally likely). The left side of Figure 4.2 shows the sequence of value functions $\{v_k\}$ computed by iterative policy evaluation. The final estimate is in fact v_π, which in this case gives for each state the negation of the expected number of steps from that state until

termination. ■

Exercise 4.1 If π is the equiprobable random policy, what is $q_\pi(11, \texttt{down})$? What is $q_\pi(7, \texttt{down})$?

Exercise 4.2 Suppose a new state 15 is added to the gridworld just below state 13, and its actions, `left`, `up`, `right`, and `down`, take the agent to states 12, 13, 14, and 15, respectively. Assume that the transitions *from* the original states are unchanged. What, then, is $v_\pi(15)$ for the equiprobable random policy? Now suppose the dynamics of state 13 are also changed, such that action `down` from state 13 takes the agent to the new state 15. What is $v_\pi(15)$ for the equiprobable random policy in this case?

Exercise 4.3 What are the equations analogous to (4.3), (4.4), and (4.5) for the action-value function q_π and its successive approximation by a sequence of functions $q_0, q_1, q_2, \ldots$?

Exercise 4.4 In some undiscounted episodic tasks there may be policies for which eventual termination is not guaranteed. For example, in the grid problem above it is possible to go back and forth between two states forever. In a task that is otherwise perfectly sensible, $v_\pi(s)$ may be negative infinity for some policies and states, in which case the algorithm for iterative policy evaluation given in Figure 4.1 will not terminate. As a purely practical matter, how might we amend this algorithm to assure termination even in this case? Assume that eventual termination *is* guaranteed under the optimal policy.

4.2 Policy Improvement

Our reason for computing the value function for a policy is to help find better policies. Suppose we have determined the value function v_π for an arbitrary deterministic policy π. For some state s we would like to know whether or not we should change the policy to deterministically choose an action $a \neq \pi(s)$. We know how good it is to follow the current policy from s—that is $v_\pi(s)$—but would it be better or worse to change to the new policy? One way to answer this question is to consider selecting a in s and thereafter following the existing policy, π. The value of this way of behaving is

$$\begin{aligned} q_\pi(s,a) &= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t{=}s, A_t{=}a] \\ &= \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma v_\pi(s')\Big]. \end{aligned} \tag{4.6}$$

The key criterion is whether this is greater than or less than $v_\pi(s)$. If it is greater—that is, if it is better to select a once in s and thereafter follow π

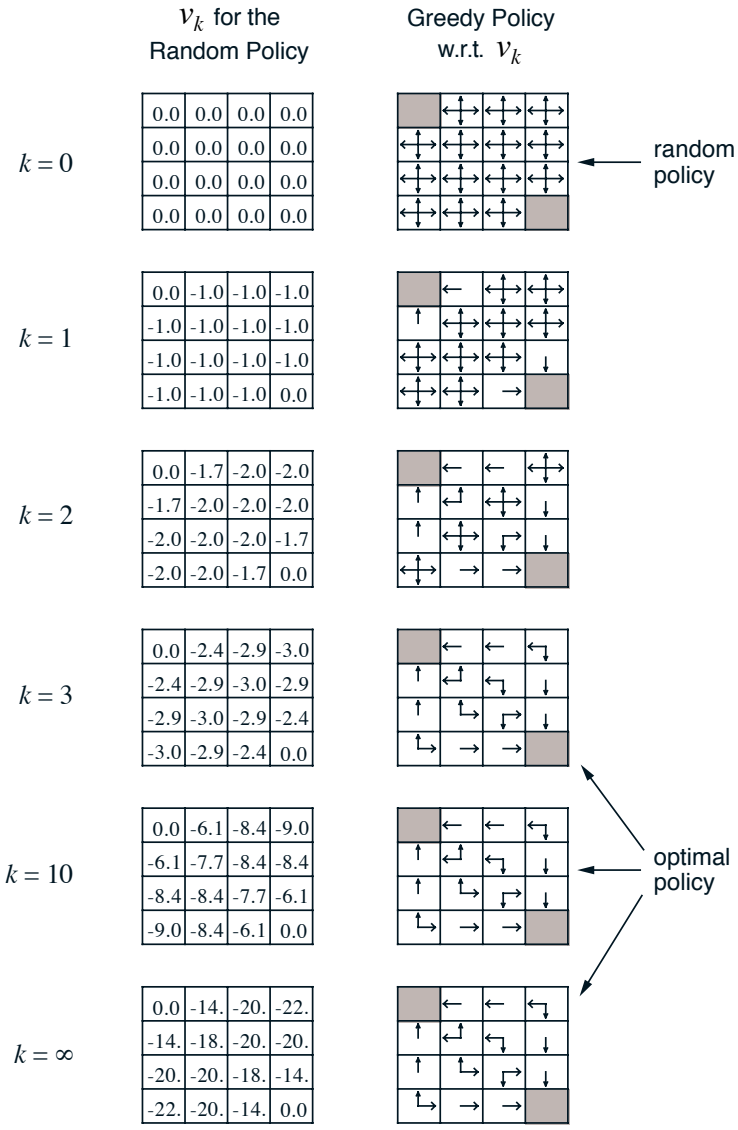


Figure 4.2: Convergence of iterative policy evaluation on a small gridworld. The left column is the sequence of approximations of the state-value function for the random policy (all actions equal). The right column is the sequence of greedy policies corresponding to the value function estimates (arrows are shown for all actions achieving the maximum). The last policy is guaranteed only to be an improvement over the random policy, but in this case it, and all policies after the third iteration, are optimal.

than it would be to follow π all the time—then one would expect it to be better still to select a every time s is encountered, and that the new policy would in fact be a better one overall.

That this is true is a special case of a general result called the *policy improvement theorem*. Let π and π' be any pair of deterministic policies such that, for all $s \in \mathcal{S}$,

$$q_\pi(s, \pi'(s)) \geq v_\pi(s). \tag{4.7}$$

Then the policy π' must be as good as, or better than, π. That is, it must obtain greater or equal expected return from all states $s \in \mathcal{S}$:

$$v_{\pi'}(s) \geq v_\pi(s). \tag{4.8}$$

Moreover, if there is strict inequality of (4.7) at any state, then there must be strict inequality of (4.8) at at least one state. This result applies in particular to the two policies that we considered in the previous paragraph, an original deterministic policy, π, and a changed policy, π', that is identical to π except that $\pi'(s) = a \neq \pi(s)$. Obviously, (4.7) holds at all states other than s. Thus, if $q_\pi(s, a) > v_\pi(s)$, then the changed policy is indeed better than π.

The idea behind the proof of the policy improvement theorem is easy to understand. Starting from (4.7), we keep expanding the q_π side and reapplying (4.7) until we get $v_{\pi'}(s)$:

$$\begin{aligned}
v_\pi(s) &\leq q_\pi(s, \pi'(s)) \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \\
&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma \mathbb{E}_{\pi'}[R_{t+2} + \gamma v_\pi(S_{t+2})] \mid S_t = s] \\
&= \mathbb{E}_{\pi'}\big[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \;\big|\; S_t = s\big] \\
&\leq \mathbb{E}_{\pi'}\big[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 v_\pi(S_{t+3}) \;\big|\; S_t = s\big] \\
&\;\;\vdots \\
&\leq \mathbb{E}_{\pi'}\big[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \;\big|\; S_t = s\big] \\
&= v_{\pi'}(s).
\end{aligned}$$

So far we have seen how, given a policy and its value function, we can easily evaluate a change in the policy at a single state to a particular action. It is a natural extension to consider changes at *all* states and to *all* possible actions, selecting at each state the action that appears best according to $q_\pi(s, a)$. In

other words, to consider the new *greedy* policy, π', given by

$$\begin{aligned}\pi'(s) &= \arg\max_a q_\pi(s,a) \\ &= \arg\max_a \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t{=}s, A_t{=}a] \\ &= \arg\max_a \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma v_\pi(s')\Big],\end{aligned} \tag{4.9}$$

where $\arg\max_a$ denotes the value of a at which the expression that follows is maximized (with ties broken arbitrarily). The greedy policy takes the action that looks best in the short term—after one step of lookahead—according to v_π. By construction, the greedy policy meets the conditions of the policy improvement theorem (4.7), so we know that it is as good as, or better than, the original policy. The process of making a new policy that improves on an original policy, by making it greedy with respect to the value function of the original policy, is called *policy improvement*.

Suppose the new greedy policy, π', is as good as, but not better than, the old policy π. Then $v_\pi = v_{\pi'}$, and from (4.9) it follows that for all $s \in \mathcal{S}$:

$$\begin{aligned}v_{\pi'}(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_{\pi'}(S_{t+1}) \mid S_t{=}s, A_t{=}a] \\ &= \max_a \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma v_{\pi'}(s')\Big].\end{aligned}$$

But this is the same as the Bellman optimality equation (4.1), and therefore, $v_{\pi'}$ must be v_*, and both π and π' must be optimal policies. Policy improvement thus must give us a strictly better policy except when the original policy is already optimal.

So far in this section we have considered the special case of deterministic policies. In the general case, a stochastic policy π specifies probabilities, $\pi(a|s)$, for taking each action, a, in each state, s. We will not go through the details, but in fact all the ideas of this section extend easily to stochastic policies. In particular, the policy improvement theorem carries through as stated for the stochastic case, under the natural definition:

$$q_\pi(s, \pi'(s)) = \sum_a \pi'(a|s) q_\pi(s,a).$$

In addition, if there are ties in policy improvement steps such as (4.9)—that is, if there are several actions at which the maximum is achieved—then in the stochastic case we need not select a single action from among them. Instead, each maximizing action can be given a portion of the probability of being

selected in the new greedy policy. Any apportioning scheme is allowed as long as all submaximal actions are given zero probability.

The last row of Figure 4.2 shows an example of policy improvement for stochastic policies. Here the original policy, π, is the equiprobable random policy, and the new policy, π', is greedy with respect to v_π. The value function v_π is shown in the bottom-left diagram and the set of possible π' is shown in the bottom-right diagram. The states with multiple arrows in the π' diagram are those in which several actions achieve the maximum in (4.9); any apportionment of probability among these actions is permitted. The value function of any such policy, $v_{\pi'}(s)$, can be seen by inspection to be either -1, -2, or -3 at all states, $s \in \mathcal{S}$, whereas $v_\pi(s)$ is at most -14. Thus, $v_{\pi'}(s) \geq v_\pi(s)$, for all $s \in \mathcal{S}$, illustrating policy improvement. Although in this case the new policy π' happens to be optimal, in general only an improvement is guaranteed.

4.3 Policy Iteration

Once a policy, π, has been improved using v_π to yield a better policy, π', we can then compute $v_{\pi'}$ and improve it again to yield an even better π''. We can thus obtain a sequence of monotonically improving policies and value functions:

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*,$$

where $\xrightarrow{\text{E}}$ denotes a policy *evaluation* and $\xrightarrow{\text{I}}$ denotes a policy *improvement*. Each policy is guaranteed to be a strict improvement over the previous one (unless it is already optimal). Because a finite MDP has only a finite number of policies, this process must converge to an optimal policy and optimal value function in a finite number of iterations.

This way of finding an optimal policy is called *policy iteration*. A complete algorithm is given in Figure 4.3. Note that each policy evaluation, itself an iterative computation, is started with the value function for the previous policy. This typically results in a great increase in the speed of convergence of policy evaluation (presumably because the value function changes little from one policy to the next).

Policy iteration often converges in surprisingly few iterations. This is illustrated by the example in Figure 4.2. The bottom-left diagram shows the value function for the equiprobable random policy, and the bottom-right diagram shows a greedy policy for this value function. The policy improvement theorem assures us that these policies are better than the original random policy. In this case, however, these policies are not just better, but optimal, proceed-

1. Initialization
   $v(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Repeat
      $\Delta \leftarrow 0$
      For each $s \in \mathcal{S}$:
         $temp \leftarrow v(s)$
         $v(s) \leftarrow \sum_{s'} p(s'|s, \pi(s)) \Big[r(s, \pi(s), s') + \gamma v(s') \Big]$
         $\Delta \leftarrow \max(\Delta, |temp - v(s)|)$
   until $\Delta < \theta$ (a small positive number)

3. Policy Improvement
   policy-stable $\leftarrow true$
   For each $s \in \mathcal{S}$:
      $temp \leftarrow \pi(s)$
      $\pi(s) \leftarrow \arg\max_a \sum_{s'} p(s'|s, a) \Big[r(s, a, s') + \gamma v(s') \Big]$
      If $temp \neq \pi(s)$, then *policy-stable* $\leftarrow false$
   If *policy-stable*, then stop and return v and π; else go to 2

Figure 4.3: Policy iteration (using iterative policy evaluation) for v_*. This algorithm has a subtle bug, in that it may never terminate if the policy continually switches between two or more policies that are equally good. The bug can be fixed by adding additional flags, but it makes the pseudocode so ugly that it is not worth it. :-)

ing to the terminal states in the minimum number of steps. In this example, policy iteration would find the optimal policy after just one iteration.

Example 4.2: Jack's Car Rental Jack manages two locations for a nationwide car rental company. Each day, some number of customers arrive at each location to rent cars. If Jack has a car available, he rents it out and is credited \$10 by the national company. If he is out of cars at that location, then the business is lost. Cars become available for renting the day after they are returned. To help ensure that cars are available where they are needed, Jack can move them between the two locations overnight, at a cost of \$2 per car moved. We assume that the number of cars requested and returned at each location are Poisson random variables, meaning that the probability that the number is n is $\frac{\lambda^n}{n!}e^{-\lambda}$, where λ is the expected number. Suppose λ is 3 and 4 for rental requests at the first and second locations and 3 and 2 for returns. To simplify the problem slightly, we assume that there can be no more than 20 cars at each location (any additional cars are returned to the nationwide company, and thus disappear from the problem) and a maximum of five cars can be moved from one location to the other in one night. We take the discount rate to be $\gamma = 0.9$ and formulate this as a continuing finite MDP, where the time steps are days, the state is the number of cars at each location at the end of the day, and the actions are the net numbers of cars moved between the two locations overnight. Figure 4.4 shows the sequence of policies found by policy iteration starting from the policy that never moves any cars. ■

Exercise 4.5 (programming) Write a program for policy iteration and re-solve Jack's car rental problem with the following changes. One of Jack's employees at the first location rides a bus home each night and lives near the second location. She is happy to shuttle one car to the second location for free. Each additional car still costs \$2, as do all cars moved in the other direction. In addition, Jack has limited parking space at each location. If more than 10 cars are kept overnight at a location (after any moving of cars), then an additional cost of \$4 must be incurred to use a second parking lot (independent of how many cars are kept there). These sorts of nonlinearities and arbitrary dynamics often occur in real problems and cannot easily be handled by optimization methods other than dynamic programming. To check your program, first replicate the results given for the original problem. If your computer is too slow for the full problem, cut all the numbers of cars in half.

Exercise 4.6 How would policy iteration be defined for action values? Give a complete algorithm for computing q_*, analogous to Figure 4.3 for computing v_*. Please pay special attention to this exercise, because the ideas involved will be used throughout the rest of the book.

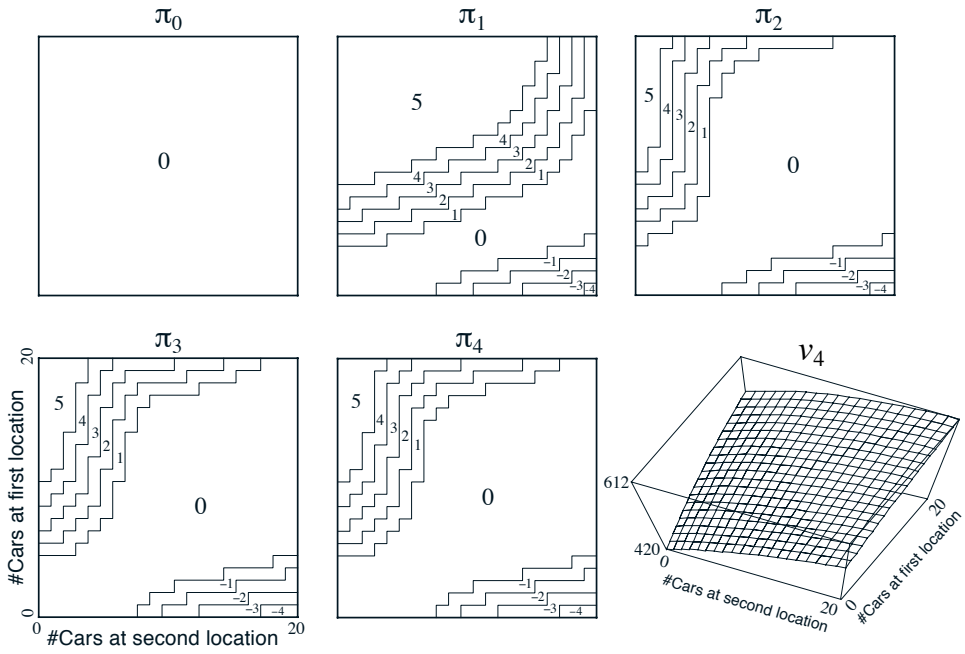


Figure 4.4: The sequence of policies found by policy iteration on Jack's car rental problem, and the final state-value function. The first five diagrams show, for each number of cars at each location at the end of the day, the number of cars to be moved from the first location to the second (negative numbers indicate transfers from the second location to the first). Each successive policy is a strict improvement over the previous policy, and the last policy is optimal.

Exercise 4.7 Suppose you are restricted to considering only policies that are *ε-soft*, meaning that the probability of selecting each action in each state, s, is at least $\varepsilon/|\mathcal{A}(s)|$. Describe qualitatively the changes that would be required in each of the steps 3, 2, and 1, in that order, of the policy iteration algorithm for v_* (Figure 4.3).

4.4 Value Iteration

One drawback to policy iteration is that each of its iterations involves policy evaluation, which may itself be a protracted iterative computation requiring multiple sweeps through the state set. If policy evaluation is done iteratively, then convergence exactly to v_π occurs only in the limit. Must we wait for exact convergence, or can we stop short of that? The example in Figure 4.2 certainly suggests that it may be possible to truncate policy evaluation. In that example, policy evaluation iterations beyond the first three have no effect on the corresponding greedy policy.

In fact, the policy evaluation step of policy iteration can be truncated in several ways without losing the convergence guarantees of policy iteration. One important special case is when policy evaluation is stopped after just one sweep (one backup of each state). This algorithm is called *value iteration*. It can be written as a particularly simple backup operation that combines the policy improvement and truncated policy evaluation steps:

$$\begin{aligned} v_{k+1}(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \max_a \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma v_k(s')\Big], \end{aligned} \tag{4.10}$$

for all $s \in \mathcal{S}$. For arbitrary v_0, the sequence $\{v_k\}$ can be shown to converge to v_* under the same conditions that guarantee the existence of v_*.

Another way of understanding value iteration is by reference to the Bellman optimality equation (4.1). Note that value iteration is obtained simply by turning the Bellman optimality equation into an update rule. Also note how the value iteration backup is identical to the policy evaluation backup (4.5) except that it requires the maximum to be taken over all actions. Another way of seeing this close relationship is to compare the backup diagrams for these algorithms: Figure 3.4a shows the backup diagram for policy evaluation and Figure 3.7a shows the backup diagram for value iteration. These two are the natural backup operations for computing v_π and v_*.

Finally, let us consider how value iteration terminates. Like policy evaluation, value iteration formally requires an infinite number of iterations to

Initialize array v arbitrarily (e.g., $v(s) = 0$ for all $s \in \mathcal{S}^+$)

Repeat
  $\Delta \leftarrow 0$
  For each $s \in \mathcal{S}$:
    $temp \leftarrow v(s)$
    $v(s) \leftarrow \max_a \sum_{s'} p(s'|s,a)[r(s,a,s') + \gamma v(s')]$
    $\Delta \leftarrow \max(\Delta, |temp - v(s)|)$
until $\Delta < \theta$ (a small positive number)

Output a deterministic policy, π, such that
  $\pi(s) = \arg\max_a \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma v(s')\Big]$

Figure 4.5: Value iteration.

converge exactly to v_*. In practice, we stop once the value function changes by only a small amount in a sweep. Figure 4.5 gives a complete value iteration algorithm with this kind of termination condition.

Value iteration effectively combines, in each of its sweeps, one sweep of policy evaluation and one sweep of policy improvement. Faster convergence is often achieved by interposing multiple policy evaluation sweeps between each policy improvement sweep. In general, the entire class of truncated policy iteration algorithms can be thought of as sequences of sweeps, some of which use policy evaluation backups and some of which use value iteration backups. Since the max operation in (4.10) is the only difference between these backups, this just means that the max operation is added to some sweeps of policy evaluation. All of these algorithms converge to an optimal policy for discounted finite MDPs.

Example 4.3: Gambler's Problem A gambler has the opportunity to make bets on the outcomes of a sequence of coin flips. If the coin comes up heads, he wins as many dollars as he has staked on that flip; if it is tails, he loses his stake. The game ends when the gambler wins by reaching his goal of \$100, or loses by running out of money. On each flip, the gambler must decide what portion of his capital to stake, in integer numbers of dollars. This problem can be formulated as an undiscounted, episodic, finite MDP. The state is the gambler's capital, $s \in \{1, 2, \ldots, 99\}$ and the actions are stakes, $a \in \{0, 1, \ldots, \min(s, 100 - s)\}$. The reward is zero on all transitions except those on which the gambler reaches his goal, when it is $+1$. The state-value function then gives the probability of winning from each state. A policy is a mapping from levels of capital to stakes. The optimal policy maximizes the

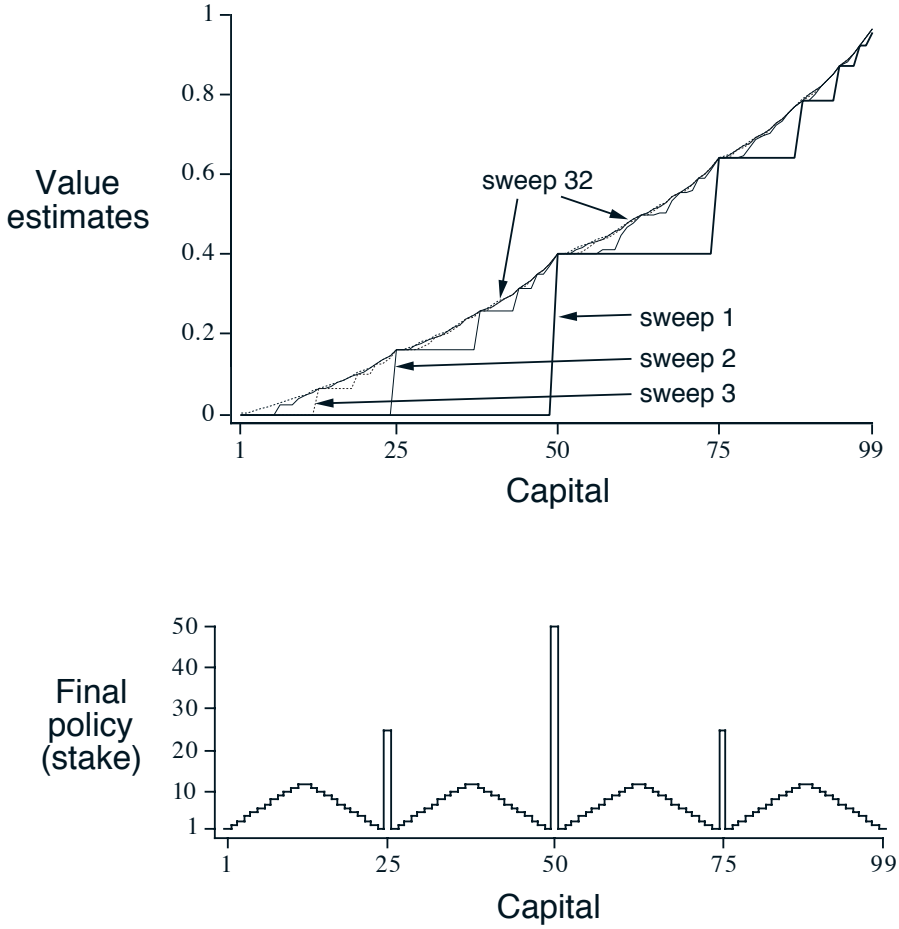


Figure 4.6: The solution to the gambler's problem for $p_h = 0.4$. The upper graph shows the value function found by successive sweeps of value iteration. The lower graph shows the final policy.

probability of reaching the goal. Let p_h denote the probability of the coin coming up heads. If p_h is known, then the entire problem is known and it can be solved, for instance, by value iteration. Figure 4.6 shows the change in the value function over successive sweeps of value iteration, and the final policy found, for the case of $p_h = 0.4$. This policy is optimal, but not unique. In fact, there is a whole family of optimal policies, all corresponding to ties for the argmax action selection with respect to the optimal value function. Can you guess what the entire family looks like? ■

Exercise 4.8 Why does the optimal policy for the gambler's problem have such a curious form? In particular, for capital of 50 it bets it all on one flip, but for capital of 51 it does not. Why is this a good policy?

Exercise 4.9 (programming) Implement value iteration for the gambler's problem and solve it for $p_h = 0.25$ and $p_h = 0.55$. In programming, you may find it convenient to introduce two dummy states corresponding to termination with capital of 0 and 100, giving them values of 0 and 1 respectively. Show your results graphically, as in Figure 4.6. Are your results stable as $\theta \to 0$?

Exercise 4.10 What is the analog of the value iteration backup (4.10) for action values, $q_{k+1}(s, a)$?

4.5 Asynchronous Dynamic Programming

A major drawback to the DP methods that we have discussed so far is that they involve operations over the entire state set of the MDP, that is, they require sweeps of the state set. If the state set is very large, then even a single sweep can be prohibitively expensive. For example, the game of backgammon has over 10^{20} states. Even if we could perform the value iteration backup on a million states per second, it would take over a thousand years to complete a single sweep.

Asynchronous DP algorithms are in-place iterative DP algorithms that are not organized in terms of systematic sweeps of the state set. These algorithms back up the values of states in any order whatsoever, using whatever values of other states happen to be available. The values of some states may be backed up several times before the values of others are backed up once. To converge correctly, however, an asynchronous algorithm must continue to backup the values of all the states: it can't ignore any state after some point in the computation. Asynchronous DP algorithms allow great flexibility in selecting states to which backup operations are applied.

For example, one version of asynchronous value iteration backs up the value, in place, of only one state, s_k, on each step, k, using the value iteration backup (4.10). If $0 \leq \gamma < 1$, asymptotic convergence to v_* is guaranteed given only that all states occur in the sequence $\{s_k\}$ an infinite number of times (the sequence could even be stochastic). (In the undiscounted episodic case, it is possible that there are some orderings of backups that do not result in convergence, but it is relatively easy to avoid these.) Similarly, it is possible to intermix policy evaluation and value iteration backups to produce a kind of asynchronous truncated policy iteration. Although the details of this and other more unusual DP algorithms are beyond the scope of this book, it is clear that a few different backups form building blocks that can be used flexibly in a wide variety of sweepless DP algorithms.

Of course, avoiding sweeps does not necessarily mean that we can get away

with less computation. It just means that an algorithm does not need to get locked into any hopelessly long sweep before it can make progress improving a policy. We can try to take advantage of this flexibility by selecting the states to which we apply backups so as to improve the algorithm's rate of progress. We can try to order the backups to let value information propagate from state to state in an efficient way. Some states may not need their values backed up as often as others. We might even try to skip backing up some states entirely if they are not relevant to optimal behavior. Some ideas for doing this are discussed in Chapter 8.

Asynchronous algorithms also make it easier to intermix computation with real-time interaction. To solve a given MDP, we can run an iterative DP algorithm *at the same time that an agent is actually experiencing the MDP*. The agent's experience can be used to determine the states to which the DP algorithm applies its backups. At the same time, the latest value and policy information from the DP algorithm can guide the agent's decision-making. For example, we can apply backups to states as the agent visits them. This makes it possible to *focus* the DP algorithm's backups onto parts of the state set that are most relevant to the agent. This kind of focusing is a repeated theme in reinforcement learning.

4.6 Generalized Policy Iteration

Policy iteration consists of two simultaneous, interacting processes, one making the value function consistent with the current policy (policy evaluation), and the other making the policy greedy with respect to the current value function (policy improvement). In policy iteration, these two processes alternate, each completing before the other begins, but this is not really necessary. In value iteration, for example, only a single iteration of policy evaluation is performed in between each policy improvement. In asynchronous DP methods, the evaluation and improvement processes are interleaved at an even finer grain. In some cases a single state is updated in one process before returning to the other. As long as both processes continue to update all states, the ultimate result is typically the same—convergence to the optimal value function and an optimal policy.

We use the term *generalized policy iteration* (GPI) to refer to the general idea of letting policy evaluation and policy improvement processes interact, independent of the granularity and other details of the two processes. Almost all reinforcement learning methods are well described as GPI. That is, all have identifiable policies and value functions, with the policy always being improved with respect to the value function and the value function always

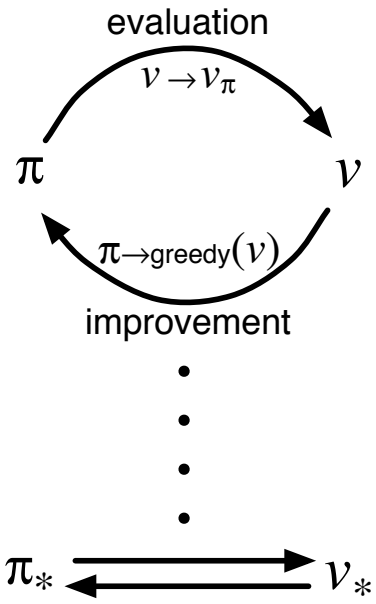


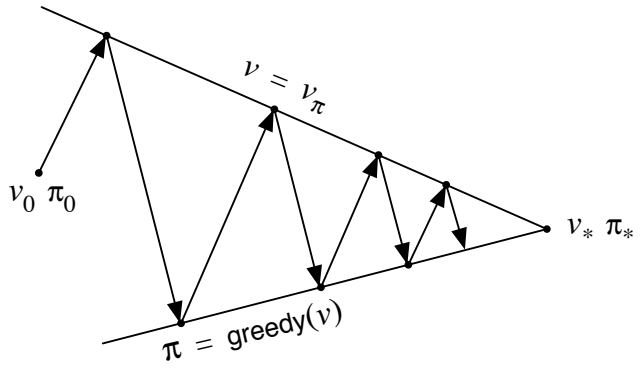
Figure 4.7: Generalized policy iteration: Value and policy functions interact until they are optimal and thus consistent with each other.

being driven toward the value function for the policy. This overall schema for GPI is illustrated in Figure 4.7.

It is easy to see that if both the evaluation process and the improvement process stabilize, that is, no longer produce changes, then the value function and policy must be optimal. The value function stabilizes only when it is consistent with the current policy, and the policy stabilizes only when it is greedy with respect to the current value function. Thus, both processes stabilize only when a policy has been found that is greedy with respect to its own evaluation function. This implies that the Bellman optimality equation (4.1) holds, and thus that the policy and the value function are optimal.

The evaluation and improvement processes in GPI can be viewed as both competing and cooperating. They compete in the sense that they pull in opposing directions. Making the policy greedy with respect to the value function typically makes the value function incorrect for the changed policy, and making the value function consistent with the policy typically causes that policy no longer to be greedy. In the long run, however, these two processes interact to find a single joint solution: the optimal value function and an optimal policy.

One might also think of the interaction between the evaluation and improvement processes in GPI in terms of two constraints or goals—for example, as two lines in two-dimensional space:



Although the real geometry is much more complicated than this, the diagram suggests what happens in the real case. Each process drives the value function or policy toward one of the lines representing a solution to one of the two goals. The goals interact because the two lines are not orthogonal. Driving directly toward one goal causes some movement away from the other goal. Inevitably, however, the joint process is brought closer to the overall goal of optimality. The arrows in this diagram correspond to the behavior of policy iteration in that each takes the system all the way to achieving one of the two goals completely. In GPI one could also take smaller, incomplete steps toward each goal. In either case, the two processes together achieve the overall goal of optimality even though neither is attempting to achieve it directly.

4.7 Efficiency of Dynamic Programming

DP may not be practical for very large problems, but compared with other methods for solving MDPs, DP methods are actually quite efficient. If we ignore a few technical details, then the (worst case) time DP methods take to find an optimal policy is polynomial in the number of states and actions. If n and m denote the number of states and actions, this means that a DP method takes a number of computational operations that is less than some polynomial function of n and m. A DP method is guaranteed to find an optimal policy in polynomial time even though the total number of (deterministic) policies is m^n. In this sense, DP is exponentially faster than any direct search in policy space could be, because direct search would have to exhaustively examine each policy to provide the same guarantee. Linear programming methods can also be used to solve MDPs, and in some cases their worst-case convergence guarantees are better than those of DP methods. But linear programming methods become impractical at a much smaller number of states than do DP methods (by a factor of about 100). For the largest problems, only DP methods are feasible.

DP is sometimes thought to be of limited applicability because of the *curse*

of dimensionality (Bellman, 1957a), the fact that the number of states often grows exponentially with the number of state variables. Large state sets do create difficulties, but these are inherent difficulties of the problem, not of DP as a solution method. In fact, DP is comparatively better suited to handling large state spaces than competing methods such as direct search and linear programming.

In practice, DP methods can be used with today's computers to solve MDPs with millions of states. Both policy iteration and value iteration are widely used, and it is not clear which, if either, is better in general. In practice, these methods usually converge much faster than their theoretical worst-case run times, particularly if they are started with good initial value functions or policies.

On problems with large state spaces, *asynchronous* DP methods are often preferred. To complete even one sweep of a synchronous method requires computation and memory for every state. For some problems, even this much memory and computation is impractical, yet the problem is still potentially solvable because only a relatively few states occur along optimal solution trajectories. Asynchronous methods and other variations of GPI can be applied in such cases and may find good or optimal policies much faster than synchronous methods can.

4.8 Summary

In this chapter we have become familiar with the basic ideas and algorithms of dynamic programming as they relate to solving finite MDPs. *Policy evaluation* refers to the (typically) iterative computation of the value functions for a given policy. *Policy improvement* refers to the computation of an improved policy given the value function for that policy. Putting these two computations together, we obtain *policy iteration* and *value iteration*, the two most popular DP methods. Either of these can be used to reliably compute optimal policies and value functions for finite MDPs given complete knowledge of the MDP.

Classical DP methods operate in sweeps through the state set, performing a *full backup* operation on each state. Each backup updates the value of one state based on the values of all possible successor states and their probabilities of occurring. Full backups are closely related to Bellman equations: they are little more than these equations turned into assignment statements. When the backups no longer result in any changes in value, convergence has occurred to values that satisfy the corresponding Bellman equation. Just as there are four primary value functions (v_π, v_*, q_π, and q_*), there are four corresponding

Bellman equations and four corresponding full backups. An intuitive view of the operation of backups is given by *backup diagrams*.

Insight into DP methods and, in fact, into almost all reinforcement learning methods, can be gained by viewing them as *generalized policy iteration* (GPI). GPI is the general idea of two interacting processes revolving around an approximate policy and an approximate value function. One process takes the policy as given and performs some form of policy evaluation, changing the value function to be more like the true value function for the policy. The other process takes the value function as given and performs some form of policy improvement, changing the policy to make it better, assuming that the value function is its value function. Although each process changes the basis for the other, overall they work together to find a joint solution: a policy and value function that are unchanged by either process and, consequently, are optimal. In some cases, GPI can be proved to converge, most notably for the classical DP methods that we have presented in this chapter. In other cases convergence has not been proved, but still the idea of GPI improves our understanding of the methods.

It is not necessary to perform DP methods in complete sweeps through the state set. *Asynchronous DP* methods are in-place iterative methods that back up states in an arbitrary order, perhaps stochastically determined and using out-of-date information. Many of these methods can be viewed as fine-grained forms of GPI.

Finally, we note one last special property of DP methods. All of them update estimates of the values of states based on estimates of the values of successor states. That is, they update estimates on the basis of other estimates. We call this general idea *bootstrapping*. Many reinforcement learning methods perform bootstrapping, even those that do not require, as DP requires, a complete and accurate model of the environment. In the next chapter we explore reinforcement learning methods that do not require a model and do not bootstrap. In the chapter after that we explore methods that do not require a model but do bootstrap. These key features and properties are separable, yet can be mixed in interesting combinations.

4.9 Bibliographical and Historical Remarks

The term "dynamic programming" is due to Bellman (1957a), who showed how these methods could be applied to a wide range of problems. Extensive treatments of DP can be found in many texts, including Bertsekas (1995), Bertsekas and Tsitsiklis (1996), Dreyfus and Law (1977), Ross (1983), White

(1969), and Whittle (1982, 1983). Our interest in DP is restricted to its use in solving MDPs, but DP also applies to other types of problems. Kumar and Kanal (1988) provide a more general look at DP.

To the best of our knowledge, the first connection between DP and reinforcement learning was made by Minsky (1961) in commenting on Samuel's checkers player. In a footnote, Minsky mentioned that it is possible to apply DP to problems in which Samuel's backing-up process can be handled in closed analytic form. This remark may have misled artificial intelligence researchers into believing that DP was restricted to analytically tractable problems and therefore largely irrelevant to artificial intelligence. Andreae (1969b) mentioned DP in the context of reinforcement learning, specifically policy iteration, although he did not make specific connections between DP and learning algorithms. Werbos (1977) suggested an approach to approximating DP called "heuristic dynamic programming" that emphasizes gradient-descent methods for continuous-state problems (Werbos, 1982, 1987, 1988, 1989, 1992). These methods are closely related to the reinforcement learning algorithms that we discuss in this book. Watkins (1989) was explicit in connecting reinforcement learning to DP, characterizing a class of reinforcement learning methods as "incremental dynamic programming."

4.1–4 These sections describe well-established DP algorithms that are covered in any of the general DP references cited above. The policy improvement theorem and the policy iteration algorithm are due to Bellman (1957a) and Howard (1960). Our presentation was influenced by the local view of policy improvement taken by Watkins (1989). Our discussion of value iteration as a form of truncated policy iteration is based on the approach of Puterman and Shin (1978), who presented a class of algorithms called *modified policy iteration*, which includes policy iteration and value iteration as special cases. An analysis showing how value iteration can be made to find an optimal policy in finite time is given by Bertsekas (1987).

Iterative policy evaluation is an example of a classical successive approximation algorithm for solving a system of linear equations. The version of the algorithm that uses two arrays, one holding the old values while the other is updated, is often called a *Jacobi-style* algorithm, after Jacobi's classical use of this method. It is also sometimes called a *synchronous* algorithm because it can be performed in parallel, with separate processors simultaneously updating the values of individual states using input from other processors. The second array is needed to simulate this parallel computation sequentially. The in-place version of the algorithm is often called a *Gauss–Seidel-style* algorithm after

the classical Gauss–Seidel algorithm for solving systems of linear equations. In addition to iterative policy evaluation, other DP algorithms can be implemented in these different versions. Bertsekas and Tsitsiklis (1989) provide excellent coverage of these variations and their performance differences.

4.5 Asynchronous DP algorithms are due to Bertsekas (1982, 1983), who also called them distributed DP algorithms. The original motivation for asynchronous DP was its implementation on a multiprocessor system with communication delays between processors and no global synchronizing clock. These algorithms are extensively discussed by Bertsekas and Tsitsiklis (1989). Jacobi-style and Gauss–Seidel-style DP algorithms are special cases of the asynchronous version. Williams and Baird (1990) presented DP algorithms that are asynchronous at a finer grain than the ones we have discussed: the backup operations themselves are broken into steps that can be performed asynchronously.

4.7 This section, written with the help of Michael Littman, is based on Littman, Dean, and Kaelbling (1995).

Chapter 5

Monte Carlo Methods

In this chapter we consider our first learning methods for estimating value functions and discovering optimal policies. Unlike the previous chapter, here we do not assume complete knowledge of the environment. Monte Carlo methods require only *experience*—sample sequences of states, actions, and rewards from on-line or simulated interaction with an environment. Learning from *on-line* experience is striking because it requires no prior knowledge of the environment's dynamics, yet can still attain optimal behavior. Learning from *simulated* experience is also powerful. Although a model is required, the model need only generate sample transitions, not the complete probability distributions of all possible transitions that is required by dynamic programming (DP) methods. In surprisingly many cases it is easy to generate experience sampled according to the desired probability distributions, but infeasible to obtain the distributions in explicit form.

Monte Carlo methods are ways of solving the reinforcement learning problem based on averaging sample returns. To ensure that well-defined returns are available, we define Monte Carlo methods only for episodic tasks. That is, we assume experience is divided into episodes, and that all episodes eventually terminate no matter what actions are selected. It is only upon the completion of an episode that value estimates and policies are changed. Monte Carlo methods are thus incremental in an episode-by-episode sense, but not in a step-by-step sense. The term "Monte Carlo" is often used more broadly for any estimation method whose operation involves a significant random component. Here we use it specifically for methods based on averaging complete returns (as opposed to methods that learn from partial returns, considered in the next chapter).

Despite the differences between Monte Carlo and DP methods, the most important ideas carry over from DP to the Monte Carlo case. Not only do

we compute the same value functions, but they interact to attain optimality in essentially the same way. As in the DP chapter, we consider first policy evaluation, the computation of v_π and q_π for a fixed arbitrary policy π, then policy improvement, and, finally, generalized policy iteration. Each of these ideas taken from DP is extended to the Monte Carlo case in which only sample experience is available.

5.1 Monte Carlo Policy Evaluation

We begin by considering Monte Carlo methods for learning the state-value function for a given policy. Recall that the value of a state is the expected return—expected cumulative future discounted reward—starting from that state. An obvious way to estimate it from experience, then, is simply to average the returns observed after visits to that state. As more returns are observed, the average should converge to the expected value. This idea underlies all Monte Carlo methods.

In particular, suppose we wish to estimate $v_\pi(s)$, the value of a state s under policy π, given a set of episodes obtained by following π and passing through s. Each occurrence of state s in an episode is called a *visit* to s. The *every-visit MC method* estimates $v_\pi(s)$ as the average of the returns following all the visits to s in a set of episodes. Within a given episode, the first time s is visited is called the *first visit* to s. The *first-visit MC method* averages just the returns following first visits to s. These two Monte Carlo methods are very similar but have slightly different theoretical properties. First-visit MC has been most widely studied, dating back to the 1940s, and is the one we focus on in this chapter. We reconsider every-visit MC in Chapter 7. First-visit MC is shown in procedural form in Figure 5.1.

Both first-visit MC and every-visit MC converge to $v_\pi(s)$ as the number of visits (or first visits) to s goes to infinity. This is easy to see for the case of first-visit MC. In this case each return is an independent, identically distributed estimate of $v_\pi(s)$. By the law of large numbers the sequence of averages of these estimates converges to their expected value. Each average is itself an unbiased estimate, and the standard deviation of its error falls as $1/\sqrt{n}$, where n is the number of returns averaged. Every-visit MC is less straightforward, but its estimates also converge asymptotically to $v_\pi(s)$ (Singh and Sutton, 1996).

The use of Monte Carlo methods is best illustrated through an example.

Example 5.1 *Blackjack* is a popular casino card game. The object is to obtain cards the sum of whose numerical values is as great as possible without exceeding 21. All face cards count as 10, and the ace can count either as 1 or

Initialize:
$\pi \leftarrow$ policy to be evaluated
$V \leftarrow$ an arbitrary state-value function
$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Repeat forever:
(a) Generate an episode using π
(b) For each state s appearing in the episode:
$G \leftarrow$ return following the first occurrence of s
Append G to $Returns(s)$
$V(s) \leftarrow$ average($Returns(s)$)

Figure 5.1: First-visit MC method for estimating v_π. Note that we use a capital letter V for the approximate value function because, after initialization, it quickly becomes a random variable.

as 11. We consider the version in which each player competes independently against the dealer. The game begins with two cards dealt to both dealer and player. One of the dealer's cards is faceup and the other is facedown. If the player has 21 immediately (an ace and a 10-card), it is called a *natural*. He then wins unless the dealer also has a natural, in which case the game is a draw. If the player does not have a natural, then he can request additional cards, one by one (*hits*), until he either stops (*sticks*) or exceeds 21 (*goes bust*). If he goes bust, he loses; if he sticks, then it becomes the dealer's turn. The dealer hits or sticks according to a fixed strategy without choice: he sticks on any sum of 17 or greater, and hits otherwise. If the dealer goes bust, then the player wins; otherwise, the outcome—win, lose, or draw—is determined by whose final sum is closer to 21.

Playing blackjack is naturally formulated as an episodic finite MDP. Each game of blackjack is an episode. Rewards of $+1$, -1, and 0 are given for winning, losing, and drawing, respectively. All rewards within a game are zero, and we do not discount ($\gamma = 1$); therefore these terminal rewards are also the returns. The player's actions are to hit or to stick. The states depend on the player's cards and the dealer's showing card. We assume that cards are dealt from an infinite deck (i.e., with replacement) so that there is no advantage to keeping track of the cards already dealt. If the player holds an ace that he could count as 11 without going bust, then the ace is said to be *usable*. In this case it is always counted as 11 because counting it as 1 would make the sum 11 or less, in which case there is no decision to be made because, obviously, the player should always hit. Thus, the player makes decisions on the basis of three variables: his current sum (12–21), the dealer's one showing card (ace–10), and whether or not he holds a usable ace. This makes for a

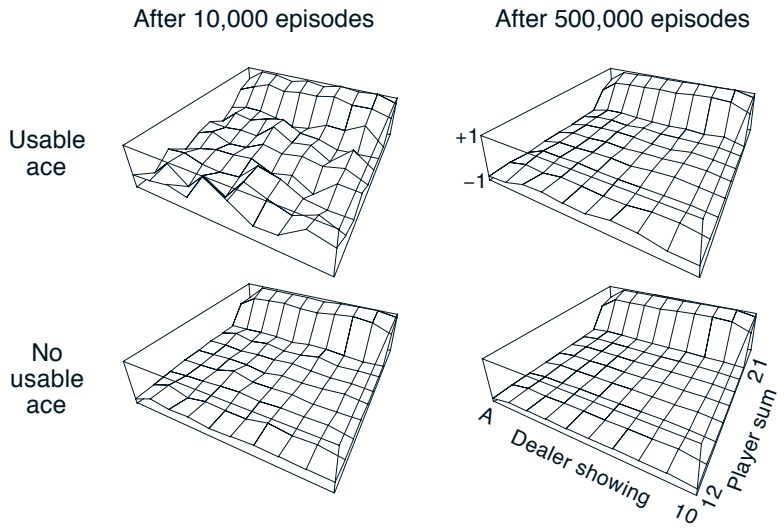


Figure 5.2: Approximate state-value functions for the blackjack policy that sticks only on 20 or 21, computed by Monte Carlo policy evaluation.

total of 200 states.

Consider the policy that sticks if the player's sum is 20 or 21, and otherwise hits. To find the state-value function for this policy by a Monte Carlo approach, one simulates many blackjack games using the policy and averages the returns following each state. Note that in this task the same state never recurs within one episode, so there is no difference between first-visit and every-visit MC methods. In this way, we obtained the estimates of the state-value function shown in Figure 5.2. The estimates for states with a usable ace are less certain and less regular because these states are less common. In any event, after 500,000 games the value function is very well approximated.

Although we have complete knowledge of the environment in this task, it would not be easy to apply DP policy evaluation to compute the value function. DP methods require the distribution of next events—in particular, they require the quantities $p(s'|s,a)$ and $r(s,a,s')$—and it is not easy to determine these for blackjack. For example, suppose the player's sum is 14 and he chooses to stick. What is his expected reward as a function of the dealer's showing card? All of these expected rewards and transition probabilities must be computed *before* DP can be applied, and such computations are often complex and error-prone. In contrast, generating the sample games required by Monte Carlo methods is easy. This is the case surprisingly often; the ability of Monte Carlo methods to work with sample episodes alone can be a significant advantage even when one has complete knowledge of the environment's dynamics. ■

Can we generalize the idea of backup diagrams to Monte Carlo algorithms?

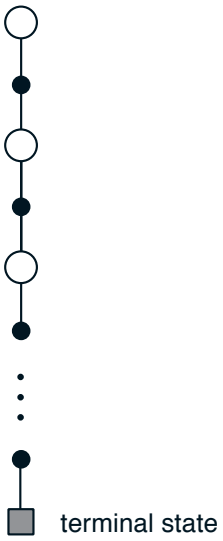


Figure 5.3: The backup diagram for Monte Carlo estimation of v_π.

The general idea of a backup diagram is to show at the top the root node to be updated and to show below all the transitions and leaf nodes whose rewards and estimated values contribute to the update. For Monte Carlo estimation of v_π, the root is a state node, and below is the entire sequence of transitions along a particular episode, ending at the terminal state, as in Figure 5.3. Whereas the DP diagram (Figure 3.4a) shows all possible transitions, the Monte Carlo diagram shows only those sampled on the one episode. Whereas the DP diagram includes only one-step transitions, the Monte Carlo diagram goes all the way to the end of the episode. These differences in the diagrams accurately reflect the fundamental differences between the algorithms.

An important fact about Monte Carlo methods is that the estimates for each state are independent. The estimate for one state does not build upon the estimate of any other state, as is the case in DP. In other words, Monte Carlo methods do not "bootstrap" as we described it in the previous chapter.

In particular, note that the computational expense of estimating the value of a single state is independent of the number of states. This can make Monte Carlo methods particularly attractive when one requires the value of only a subset of the states. One can generate many sample episodes starting from these states, averaging returns only from of these states ignoring all others. This is a third advantage Monte Carlo methods can have over DP methods (after the ability to learn from actual experience and from simulated experience).

Example 5.2: Soap Bubble Suppose a wire frame forming a closed loop

is dunked in soapy water to form a soap surface or bubble conforming at its edges to the wire frame. If the geometry of the wire frame is irregular but known, how can you compute the shape of the surface? The shape has the property that the total force on each point exerted by neighboring points is zero (or else the shape would change). This means that the surface's height at any point is the average of its heights at points in a small circle around that point. In addition, the surface must meet at its boundaries with the wire frame. The usual approach to problems of this kind is to put a grid over the area covered by the surface and solve for its height at the grid points by an iterative computation. Grid points at the boundary are forced to the wire frame, and all others are adjusted toward the average of the heights of their four nearest neighbors. This process then iterates, much like DP's iterative policy evaluation, and ultimately converges to a close approximation to the desired surface.

This is similar to the kind of problem for which Monte Carlo methods were originally designed. Instead of the iterative computation described above, imagine standing on the surface and taking a random walk, stepping randomly from grid point to neighboring grid point, with equal probability, until you reach the boundary. It turns out that the expected value of the height at the boundary is a close approximation to the height of the desired surface at the starting point (in fact, it is exactly the value computed by the iterative method described above). Thus, one can closely approximate the height of the surface at a point by simply averaging the boundary heights of many walks started at the point. If one is interested in only the value at one point, or any fixed small set of points, then this Monte Carlo method can be far more efficient than the iterative method based on local consistency. ■

Exercise 5.1 Consider the diagrams on the right in Figure 5.2. Why does the estimated value function jump up for the last two rows in the rear? Why does it drop off for the whole last row on the left? Why are the frontmost values higher in the upper diagrams than in the lower?

5.2 Monte Carlo Estimation of Action Values

If a model is not available, then it is particularly useful to estimate *action* values rather than *state* values. With a model, state values alone are sufficient to determine a policy; one simply looks ahead one step and chooses whichever action leads to the best combination of reward and next state, as we did in the chapter on DP. Without a model, however, state values alone are not sufficient. One must explicitly estimate the value of each action in order for the values to be useful in suggesting a policy. Thus, one of our primary goals for Monte

Carlo methods is to estimate q_*. To achieve this, we first consider another policy evaluation problem.

The policy evaluation problem for action values is to estimate $q_\pi(s, a)$, the expected return when starting in state s, taking action a, and thereafter following policy π. The Monte Carlo methods here are essentially the same as just presented for state values. The every-visit MC method estimates the value of a state–action pair as the average of the returns that have followed visits to the state in which the action was selected. The first-visit MC method averages the returns following the first time in each episode that the state was visited and the action was selected. These methods converge quadratically, as before, to the true expected values as the number of visits to each state–action pair approaches infinity.

The only complication is that many relevant state–action pairs may never be visited. If π is a deterministic policy, then in following π one will observe returns only for one of the actions from each state. With no returns to average, the Monte Carlo estimates of the other actions will not improve with experience. This is a serious problem because the purpose of learning action values is to help in choosing among the actions available in each state. To compare alternatives we need to estimate the value of *all* the actions from each state, not just the one we currently favor.

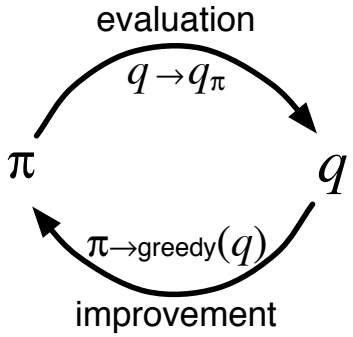
This is the general problem of *maintaining exploration*, as discussed in the context of the n-armed bandit problem in Chapter 2. For policy evaluation to work for action values, we must assure continual exploration. One way to do this is by specifying that the first step of each episode starts at a state–action *pair*, and that every such pair has a nonzero probability of being selected as the start. This guarantees that all state–action pairs will be visited an infinite number of times in the limit of an infinite number of episodes. We call this the assumption of *exploring starts*.

The assumption of exploring starts is sometimes useful, but of course it cannot be relied upon in general, particularly when learning directly from real interactions with an environment. In that case the starting conditions are unlikely to be so helpful. The most common alternative approach to assuring that all state–action pairs are encountered is to consider only policies that are stochastic with a nonzero probability of selecting all actions. We discuss two important variants of this approach in later sections. For now, we retain the assumption of exploring starts and complete the presentation of a full Monte Carlo control method.

Exercise 5.2 What is the backup diagram for Monte Carlo estimation of q_π?

5.3 Monte Carlo Control

We are now ready to consider how Monte Carlo estimation can be used in control, that is, to approximate optimal policies. The overall idea is to proceed according to the same pattern as in the DP chapter, that is, according to the idea of generalized policy iteration (GPI). In GPI one maintains both an approximate policy and an approximate value function. The value function is repeatedly altered to more closely approximate the value function for the current policy, and the policy is repeatedly improved with respect to the current value function:



These two kinds of changes work against each other to some extent, as each creates a moving target for the other, but together they cause both policy and value function to approach optimality.

To begin, let us consider a Monte Carlo version of classical policy iteration. In this method, we perform alternating complete steps of policy evaluation and policy improvement, beginning with an arbitrary policy π_0 and ending with the optimal policy and optimal action-value function:

$$\pi_0 \xrightarrow{\text{E}} q_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} q_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} q_*,$$

where $\xrightarrow{\text{E}}$ denotes a complete policy evaluation and $\xrightarrow{\text{I}}$ denotes a complete policy improvement. Policy evaluation is done exactly as described in the preceding section. Many episodes are experienced, with the approximate action-value function approaching the true function asymptotically. For the moment, let us assume that we do indeed observe an infinite number of episodes and that, in addition, the episodes are generated with exploring starts. Under these assumptions, the Monte Carlo methods will compute each q_{π_k} exactly, for arbitrary π_k.

Policy improvement is done by making the policy greedy with respect to the current value function. In this case we have an *action*-value function, and therefore no model is needed to construct the greedy policy. For any action-

value function q, the corresponding greedy policy is the one that, for each $s \in \mathcal{S}$, deterministically chooses an action with maximal action-value:

$$\pi(s) = \arg\max_a q(s,a). \tag{5.1}$$

Policy improvement then can be done by constructing each π_{k+1} as the greedy policy with respect to q_{π_k}. The policy improvement theorem (Section 4.2) then applies to π_k and π_{k+1} because, for all $s \in \mathcal{S}$,

$$\begin{aligned} q_{\pi_k}(s, \pi_{k+1}(s)) &= q_{\pi_k}(s, \arg\max_a q_{\pi_k}(s,a)) \\ &= \max_a q_{\pi_k}(s,a) \\ &\geq q_{\pi_k}(s, \pi_k(s)) \\ &= v_{\pi_k}(s). \end{aligned}$$

As we discussed in the previous chapter, the theorem assures us that each π_{k+1} is uniformly better than π_k, unless it is equal to π_k, in which case they are both optimal policies. This in turn assures us that the overall process converges to the optimal policy and optimal value function. In this way Monte Carlo methods can be used to find optimal policies given only sample episodes and no other knowledge of the environment's dynamics.

We made two unlikely assumptions above in order to easily obtain this guarantee of convergence for the Monte Carlo method. One was that the episodes have exploring starts, and the other was that policy evaluation could be done with an infinite number of episodes. To obtain a practical algorithm we will have to remove both assumptions. We postpone consideration of the first assumption until later in this chapter.

For now we focus on the assumption that policy evaluation operates on an infinite number of episodes. This assumption is relatively easy to remove. In fact, the same issue arises even in classical DP methods such as iterative policy evaluation, which also converge only asymptotically to the true value function. In both DP and Monte Carlo cases there are two ways to solve the problem. One is to hold firm to the idea of approximating q_{π_k} in each policy evaluation. Measurements and assumptions are made to obtain bounds on the magnitude and probability of error in the estimates, and then sufficient steps are taken during each policy evaluation to assure that these bounds are sufficiently small. This approach can probably be made completely satisfactory in the sense of guaranteeing correct convergence up to some level of approximation. However, it is also likely to require far too many episodes to be useful in practice on any but the smallest problems.

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
$Q(s,a) \leftarrow$ arbitrary
$\pi(s) \leftarrow$ arbitrary
$Returns(s,a) \leftarrow$ empty list

Repeat forever:
(a) Choose $S_0 \in \mathcal{S}$ and $A_0 \in \mathcal{A}(S_0)$ s.t. all pairs have probability > 0
Generate an episode starting from S_0, A_0, following π
(b) For each pair s, a appearing in the episode:
$G \leftarrow$ return following the first occurrence of s, a
Append G to $Returns(s,a)$
$Q(s,a) \leftarrow$ average$(Returns(s,a))$
(c) For each s in the episode:
$\pi(s) \leftarrow \arg\max_a Q(s,a)$

Figure 5.4: Monte Carlo ES: A Monte Carlo control algorithm assuming exploring starts and that episodes always terminate for all policies.

The second approach to avoiding the infinite number of episodes nominally required for policy evaluation is to forgo trying to complete policy evaluation before returning to policy improvement. On each evaluation step we move the value function *toward* q_{π_k}, but we do not expect to actually get close except over many steps. We used this idea when we first introduced the idea of GPI in Section 4.6. One extreme form of the idea is value iteration, in which only one iteration of iterative policy evaluation is performed between each step of policy improvement. The in-place version of value iteration is even more extreme; there we alternate between improvement and evaluation steps for single states.

For Monte Carlo policy evaluation it is natural to alternate between evaluation and improvement on an episode-by-episode basis. After each episode, the observed returns are used for policy evaluation, and then the policy is improved at all the states visited in the episode. A complete simple algorithm along these lines is given in Figure 5.4. We call this algorithm *Monte Carlo ES*, for Monte Carlo with Exploring Starts.

In Monte Carlo ES, all the returns for each state–action pair are accumulated and averaged, irrespective of what policy was in force when they were observed. It is easy to see that Monte Carlo ES cannot converge to any suboptimal policy. If it did, then the value function would eventually converge to the value function for that policy, and that in turn would cause the policy to change. Stability is achieved only when both the policy and the value function are optimal. Convergence to this optimal fixed point seems inevitable as the changes to the action-value function decrease over time, but has not yet been formally proved. In our opinion, this is one of the most fundamental

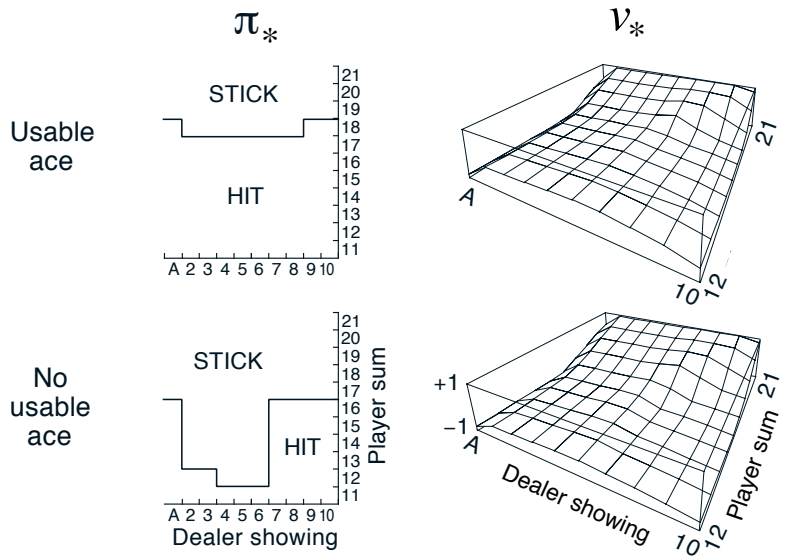


Figure 5.5: The optimal policy and state-value function for blackjack, found by Monte Carlo ES (Figure 5.4). The state-value function shown was computed from the action-value function found by Monte Carlo ES.

open theoretical questions in reinforcement learning (for a partial solution, see Tsitsiklis, 2002).

Example 5.3: Solving Blackjack It is straightforward to apply Monte Carlo ES to blackjack. Since the episodes are all simulated games, it is easy to arrange for exploring starts that include all possibilities. In this case one simply picks the dealer's cards, the player's sum, and whether or not the player has a usable ace, all at random with equal probability. As the initial policy we use the policy evaluated in the previous blackjack example, that which sticks only on 20 or 21. The initial action-value function can be zero for all state–action pairs. Figure 5.5 shows the optimal policy for blackjack found by Monte Carlo ES. This policy is the same as the "basic" strategy of Thorp (1966) with the sole exception of the leftmost notch in the policy for a usable ace, which is not present in Thorp's strategy. We are uncertain of the reason for this discrepancy, but confident that what is shown here is indeed the optimal policy for the version of blackjack we have described.

■

5.4 On-Policy Monte Carlo Control

How can we avoid the unlikely assumption of exploring starts? The only general way to ensure that all actions are selected infinitely often is for the agent to continue to select them. There are two approaches to ensuring this, resulting in what we call *on-policy* methods and *off-policy* methods. On-policy methods attempt to evaluate or improve the policy that is used to make decisions. In this section we present an on-policy Monte Carlo control method in order to illustrate the idea. Off-policy methods are of great interest but the issues in designing them are considerably more complicated; we put off consideration of them until a later (future) chapter.

In on-policy control methods the policy is generally *soft*, meaning that $\pi(a|s) > 0$ for all $s \in \mathcal{S}$ and all $a \in \mathcal{A}(s)$. There are many possible variations on on-policy methods. One possibility is to gradually shift the policy toward a deterministic optimal policy. Many of the methods discussed in Chapter 2 provide mechanisms for this. The on-policy method we present in this section uses ε-*greedy* policies, meaning that most of the time they choose an action that has maximal estimated action value, but with probability ε they instead select an action at random. That is, all nongreedy actions are given the minimal probability of selection, $\frac{\varepsilon}{|\mathcal{A}(s)|}$, and the remaining bulk of the probability, $1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|}$, is given to the greedy action. The ε-greedy policies are examples of ε-*soft* policies, defined as policies for which $\pi(a|s) \geq \frac{\varepsilon}{|\mathcal{A}(s)|}$ for all states and actions, for some $\varepsilon > 0$. Among ε-soft policies, ε-greedy policies are in some sense those that are closest to greedy.

The overall idea of on-policy Monte Carlo control is still that of GPI. As in Monte Carlo ES, we use first-visit MC methods to estimate the action-value function for the current policy. Without the assumption of exploring starts, however, we cannot simply improve the policy by making it greedy with respect to the current value function, because that would prevent further exploration of nongreedy actions. Fortunately, GPI does not require that the policy be taken all the way to a greedy policy, only that it be moved *toward* a greedy policy. In our on-policy method we will move it only to an ε-greedy policy. For any ε-soft policy, π, any ε-greedy policy with respect to q_π is guaranteed to be better than or equal to π.

That any ε-greedy policy with respect to q_π is an improvement over any ε-soft policy π is assured by the policy improvement theorem. Let π' be the ε-greedy policy. The conditions of the policy improvement theorem apply because for any $s \in \mathcal{S}$:

$$q_\pi(s, \pi'(s)) \;=\; \sum_a \pi'(a|s) q_\pi(s, a)$$

$$\begin{aligned} &= \frac{\varepsilon}{|\mathcal{A}(s)|}\sum_a q_\pi(s,a) \;+\; (1-\varepsilon)\max_a q_\pi(s,a) \qquad (5.2)\\ &\geq \frac{\varepsilon}{|\mathcal{A}(s)|}\sum_a q_\pi(s,a) \;+\; (1-\varepsilon)\sum_a \frac{\pi(a|s)-\frac{\varepsilon}{|\mathcal{A}(s)|}}{1-\varepsilon} q_\pi(s,a) \end{aligned}$$

(the sum is a weighted average with nonnegative weights summing to 1, and as such it must be less than or equal to the largest number averaged)

$$\begin{aligned} &= \frac{\varepsilon}{|\mathcal{A}(s)|}\sum_a q_\pi(s,a) \;-\; \frac{\varepsilon}{|\mathcal{A}(s)|}\sum_a q_\pi(s,a) \;+\; \sum_a \pi(a|s) q_\pi(s,a)\\ &= v_\pi(s). \end{aligned}$$

Thus, by the policy improvement theorem, $\pi' \geq \pi$ (i.e., $v_{\pi'}(s) \geq v_\pi(s)$, for all $s \in \mathcal{S}$). We now prove that equality can hold only when both π' and π are optimal among the ε-soft policies, that is, when they are better than or equal to all other ε-soft policies.

Consider a new environment that is just like the original environment, except with the requirement that policies be ε-soft "moved inside" the environment. The new environment has the same action and state set as the original and behaves as follows. If in state s and taking action a, then with probability $1-\varepsilon$ the new environment behaves exactly like the old environment. With probability ε it repicks the action at random, with equal probabilities, and then behaves like the old environment with the new, random action. The best one can do in this new environment with general policies is the same as the best one could do in the original environment with ε-soft policies. Let $\widetilde{v}_*$ and $\widetilde{q}_*$ denote the optimal value functions for the new environment. Then a policy π is optimal among ε-soft policies if and only if $v_\pi = \widetilde{v}_*$. From the definition of $\widetilde{v}_*$ we know that it is the unique solution to

$$\begin{aligned} \widetilde{v}_*(s) &= (1-\varepsilon)\max_a \widetilde{q}_*(s,a) + \frac{\varepsilon}{|\mathcal{A}(s)|}\sum_a \widetilde{q}_*(s,a)\\ &= (1-\varepsilon)\max_a \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma\widetilde{v}_*(s')\Big]\\ &\quad + \frac{\varepsilon}{|\mathcal{A}(s)|}\sum_a\sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma\widetilde{v}_*(s')\Big]. \end{aligned}$$

When equality holds and the ε-soft policy π is no longer improved, then we also know, from (5.2), that

$$\begin{aligned} v_\pi(s) &= (1-\varepsilon)\max_a q_\pi(s,a) + \frac{\varepsilon}{|\mathcal{A}(s)|}\sum_a q_\pi(s,a)\\ &= (1-\varepsilon)\max_a \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma v_\pi(s')\Big] \end{aligned}$$

$$+ \quad \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a \sum_{s'} p(s'|s,a)\Big[r(s,a,s') + \gamma v_\pi(s')\Big].$$

However, this equation is the same as the previous one, except for the substitution of v_π for $\widetilde{v}_*$. Since $\widetilde{v}_*$ is the unique solution, it must be that $v_\pi = \widetilde{v}_*$.

In essence, we have shown in the last few pages that policy iteration works for ε-soft policies. Using the natural notion of greedy policy for ε-soft policies, one is assured of improvement on every step, except when the best policy has been found among the ε-soft policies. This analysis is independent of how the action-value functions are determined at each stage, but it does assume that they are computed exactly. This brings us to roughly the same point as in the previous section. Now we only achieve the best policy among the ε-soft policies, but on the other hand, we have eliminated the assumption of exploring starts. The complete algorithm is given in Figure 5.6.

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
  $Q(s,a) \leftarrow$ arbitrary
  $Returns(s,a) \leftarrow$ empty list
  $\pi \leftarrow$ an arbitrary ε-soft policy

Repeat forever:
  (a) Generate an episode using π
  (b) For each pair s, a appearing in the episode:
    $G \leftarrow$ return following the first occurrence of s, a
    Append G to $Returns(s,a)$
    $Q(s,a) \leftarrow$ average($Returns(s,a)$)
  (c) For each s in the episode:
    $a^* \leftarrow \arg\max_a Q(s,a)$
    For all $a \in \mathcal{A}(s)$:

$$\pi(a|s) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(s)| & \text{if } a = a^* \\ \varepsilon/|\mathcal{A}(s)| & \text{if } a \neq a^* \end{cases}$$

Figure 5.6: An ε-soft on-policy Monte Carlo control algorithm.

*5.5 Evaluating One Policy While Following Another (Off-policy Policy Evaluation)

So far we have considered methods for estimating the value functions for a policy given an infinite supply of episodes generated using that policy. Suppose now that all we have are episodes generated from a *different* policy. That is, suppose we wish to estimate v_π or q_π, but all we have are episodes following another policy μ, where $\mu \neq \pi$. We call π the *target policy* because learning its value function is the target of the learning process, and we can μ the *behavior policy* because it is the policy controlling the agent and generating behavior. The overall problem is called *off-policy learning* because it is learning about a policy given only experience "off" (not following) that policy.

Of course, in order to use episodes from μ to estimate values for π, we require that every action taken under π is also taken, at least occasionally, under μ. That is, we require that $\pi(a|s) > 0$ implies $\mu(a|s) > 0$. Thus, μ must be stochastic. The target policy π, on the other hand, may be deterministic, and in fact, this is the case of greatest interest to us. Typically the target policy is be the deterministic greedy policy with respect to the current action-value function estimate. This policy we hope becomes a deterministic optimal policy while the behavior policy remains stochastic and more exploratory, for example, an ε-greedy policy.

Consider the ith episode (following μ) in which state s is visited and, in that episode, the complete sequence of states and actions following the first visit to s. Let $p_i(s)$ and $p'_i(s)$ denote the probabilities of that complete sequence happening given policies π and μ and starting from s. That is,

$$p_i(S_t) = \prod_{k=t}^{T_i(S_t)-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k),$$

where $T_i(s)$ is the time of termination of the ith episode involving state s, and

$$p'_i(s_t) = \prod_{k=t}^{T_i(s)-1} \mu(a_k|s_k)p(s_{k+1}|s_k, a_k).$$

Let $G_i(s)$ denote the corresponding observed return from state s. To average these to obtain an unbiased estimate of $v_\pi(s)$, we need only weight each return by its relative probability of occurring under π and μ, that is, by $p_i(s)/p'_i(s)$. The desired Monte Carlo estimate after observing n_s returns from state s is

then

$$V(s) = \frac{\sum_{i=1}^{n_s} \frac{p_i(s)}{p'_i(s)} G_i(s)}{\sum_{i=1}^{n_s} \frac{p_i(s)}{p'_i(s)}}. \tag{5.3}$$

This equation involves the probabilities $p_i(s)$ and $p'_i(s)$, and that both of these probabilities involve the world's transition probabilities, which are normally considered unknown in applications of Monte Carlo methods. Fortunately, here we need only their ratio, $p_i(s)/p'_i(s)$, which *can* be determined with no knowledge of the environment's dynamics:

$$\frac{p_i(S_t)}{p'_i(S_t)} = \frac{\prod_{k=t}^{T_i(S_t)-1} \pi(A_k|S_k) p(S_{k+1}|S_k, A_k)}{\prod_{k=t}^{T_i(S_t)-1} \mu(A_k|S_k) p(S_{k+1}|S_k, A_k)} = \prod_{k=t}^{T_i(S_t)-1} \frac{\pi(A_k|S_k)}{\mu(A_k|S_k)}.$$

Thus the weight needed in (5.3), $p_i(s)/p'_i(s)$, depends only on the two policies and not at all on the environment's dynamics.

Exercise 5.3 What is the Monte Carlo estimate analogous to (5.3) for *action* values, given returns generated using μ?

5.6 Off-Policy Monte Carlo Control

We are now ready to present an example of the second class of learning control methods we consider in this book: off-policy methods. Recall that the distinguishing feature of on-policy methods is that they estimate the value of a policy while using it for control. In off-policy methods these two functions are separated. The policy used to generate behavior, called the *behavior* policy, may in fact be unrelated to the policy that is evaluated and improved, called the *estimation* policy. An advantage of this separation is that the estimation policy may be deterministic (e.g., greedy), while the behavior policy can continue to sample all possible actions.

Off-policy Monte Carlo control methods use the technique presented in the preceding section for estimating the value function for one policy while following another. They follow the behavior policy while learning about and improving the estimation policy. This technique requires that the behavior policy have a nonzero probability of selecting all actions that might be selected by the estimation policy. To explore all possibilities, we require that the behavior policy be soft.

Figure 5.7 shows an off-policy Monte Carlo method, based on GPI, for estimating q_*. The behavior policy μ is maintained as an arbitrary soft policy.

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
  $Q(s,a) \leftarrow$ arbitrary
  $N(s,a) \leftarrow 0$; Numerator and
  $D(s,a) \leftarrow 0$; Denominator of $Q(s,a)$
  $\pi \leftarrow$ an arbitrary deterministic policy

Repeat forever:
  (a) Select a policy μ and use it to generate an episode:
    $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T, S_T$
  (b) $\tau \leftarrow$ latest time at which $A_\tau \neq \pi(S_\tau)$
  (c) For each pair s, a appearing in the episode at time τ or later:
    $t \leftarrow$ the time of first occurrence of s, a such that $t \geq \tau$
    $W \leftarrow \prod_{k=t+1}^{T-1} \frac{1}{\mu(A_k|S_k)}$
    $N(s,a) \leftarrow N(s,a) + WG_t$
    $D(s,a) \leftarrow D(s,a) + W$
    $Q(s,a) \leftarrow \frac{N(s,a)}{D(s,a)}$
  (d) For each $s \in \mathcal{S}$:
    $\pi(s) \leftarrow \arg\max_a Q(s,a)$

Figure 5.7: An off-policy Monte Carlo control algorithm.

The estimation policy π is the greedy policy with respect to Q, an estimate of q_π. The behavior policy chosen in (a) can be anything, but in order to assure convergence of π to the optimal policy, an infinite number of returns suitable for use in (c) must be obtained for each pair of state and action. This can be assured by careful choice of the behavior policy. For example, any ε-soft behavior policy will suffice.

A potential problem is that this method learns only from the *tails* of episodes, after the last nongreedy action. If nongreedy actions are frequent, then learning will be slow, particularly for states appearing in the early portions of long episodes. Potentially, this could greatly slow learning. There has been insufficient experience with off-policy Monte Carlo methods to assess how serious this problem is.

Exercise 5.4: Racetrack (programming) Consider driving a race car around a turn like those shown in Figure 5.8. You want to go as fast as possible, but not so fast as to run off the track. In our simplified racetrack, the car is at one of a discrete set of grid positions, the cells in the diagram. The velocity is also discrete, a number of grid cells moved horizontally and vertically per time step. The actions are increments to the velocity components. Each may be changed by $+1$, -1, or 0 in one step, for a total of nine actions. Both velocity components are restricted to be nonnegative and less than 5, and they cannot

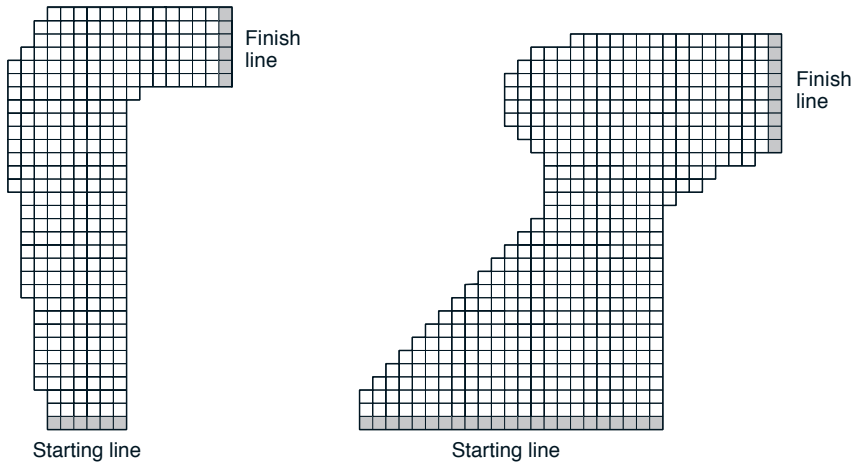


Figure 5.8: A couple of right turns for the racetrack task.

both be zero. Each episode begins in one of the randomly selected start states and ends when the car crosses the finish line. The rewards are -1 for each step that stays on the track, and -5 if the agent tries to drive off the track. Actually leaving the track is not allowed, but the position is always advanced by at least one cell along either the horizontal or vertical axes. With these restrictions and considering only right turns, such as shown in the figure, all episodes are guaranteed to terminate, yet the optimal policy is unlikely to be excluded. To make the task more challenging, we assume that on half of the time steps the position is displaced forward or to the right by one additional cell beyond that specified by the velocity. Apply the on-policy Monte Carlo control method to this task to compute the optimal policy from each starting state. Exhibit several trajectories following the optimal policy.

Exercise 5.5 The algorithm in Figure 5.7 is only valid if the environment is such that all policies are *proper*, meaning that they produce episodes that always eventually terminate (this assumption was made on the first page of this chapter). This restriction can be lifted if the algorithm is modified to use ε-soft policies, which are proper for all environments. What modifications are needed to the algorithm to restrict it to ε-soft policies?

5.7 Incremental Implementation

Monte Carlo methods can be implemented incrementally, on an episode-by-episode basis, using extensions of techniques described in Chapter 2. They use averages of *returns* just as some of the methods for solving n-armed bandit tasks described in Chapter 2 use averages of *rewards*. The techniques in

Sections 2.5 and 2.6 extend immediately to the Monte Carlo case. They enable Monte Carlo methods to process each new return incrementally with no increase in computation or memory as the number of episodes increases.

There are two differences between the Monte Carlo and bandit cases. One is that the Monte Carlo case typically involves multiple situations, that is, a different averaging process for each state, whereas bandit problems involve just one state (at least in the simple form treated in Chapter 2). The other difference is that the reward distributions in bandit problems are typically stationary, whereas in Monte Carlo methods the return distributions are typically nonstationary. This is because the returns depend on the policy, and the policy is typically changing and improving over time.

The incremental implementation described in Section 2.5 handles the case of sample or arithmetic averages, in which each return is weighted equally. Suppose we instead want to implement a *weighted* average, in which each return G_n is weighted by a random variable W_n, and we want to compute

$$V_n = \frac{\sum_{k=1}^{n-1} W_k G_k}{\sum_{k=1}^{n-1} W_k}. \tag{5.4}$$

For example, the method described for estimating one policy while following another in Section 5.5 uses weights of $W_n(s) = p_n(s)/p'_n(s)$. Weighted averages also have a simple incremental update rule. In addition to keeping track of V_n, we must maintain for each state the cumulative sum C_n of the weights given to the first n returns. The update rule for V_n is

$$V_{n+1} = V_n + \frac{W_n}{C_n}\Big[G_n - V_n\Big] \tag{5.5}$$

and

$$C_{n+1} = C_n + W_{n+1}$$

where $C_0 = 0$.

Exercise 5.6 Modify the algorithm for first-visit MC policy evaluation (Figure 5.1) to use the incremental implementation for sample averages described in Section 2.5.

Exercise 5.7 Derive the weighted-average update rule (5.5) from (5.4). Follow the pattern of the derivation of the unweighted rule (2.3).

Exercise 5.8 Modify the algorithm for the off-policy Monte Carlo control algorithm (Figure 5.7) to use the method described above for incrementally computing weighted averages.

5.8 Summary

The Monte Carlo methods presented in this chapter learn value functions and optimal policies from experience in the form of *sample episodes*. This gives them at least three kinds of advantages over DP methods. First, they can be used to learn optimal behavior directly from interaction with the environment, with no model of the environment's dynamics. Second, they can be used with simulation or *sample models*. For surprisingly many applications it is easy to simulate sample episodes even though it is difficult to construct the kind of explicit model of transition probabilities required by DP methods. Third, it is easy and efficient to *focus* Monte Carlo methods on a small subset of the states. A region of special interest can be accurately evaluated without going to the expense of accurately evaluating the rest of the state set (we explore this further in Chapter 8).

A fourth advantage of Monte Carlo methods, which we discuss later in the book, is that they may be less harmed by violations of the Markov property. This is because they do not update their value estimates on the basis of the value estimates of successor states. In other words, it is because they do not bootstrap.

In designing Monte Carlo control methods we have followed the overall schema of *generalized policy iteration* (GPI) introduced in Chapter 4. GPI involves interacting processes of policy evaluation and policy improvement. Monte Carlo methods provide an alternative policy evaluation process. Rather than use a model to compute the value of each state, they simply average many returns that start in the state. Because a state's value is the expected return, this average can become a good approximation to the value. In control methods we are particularly interested in approximating action-value functions, because these can be used to improve the policy without requiring a model of the environment's transition dynamics. Monte Carlo methods intermix policy evaluation and policy improvement steps on an episode-by-episode basis, and can be incrementally implemented on an episode-by-episode basis.

Maintaining *sufficient exploration* is an issue in Monte Carlo control methods. It is not enough just to select the actions currently estimated to be best, because then no returns will be obtained for alternative actions, and it may never be learned that they are actually better. One approach is to ignore this problem by assuming that episodes begin with state–action pairs randomly selected to cover all possibilities. Such *exploring starts* can sometimes be arranged in applications with simulated episodes, but are unlikely in learning from real experience. In *on-policy* methods, the agent commits to always exploring and tries to find the best policy that still explores. In *off-policy* methods, the agent also explores, but learns a deterministic optimal policy

that may be unrelated to the policy followed. More instances of both kinds of methods are presented in the next chapter.

Monte Carlo methods based on directly updating a policy are also put off until a later (future) chapter.

Monte Carlo methods differ from DP methods in two ways. First, they operate on sample experience, and thus can be used for direct learning without a model. Second, they do not bootstrap. That is, they do not update their value estimates on the basis of other value estimates. These two differences are not tightly linked and can be separated. In the next chapter we consider methods that learn from experience, like Monte Carlo methods, but also bootstrap, like DP methods.

5.9 Bibliographical and Historical Remarks

The term "Monte Carlo" dates from the 1940s, when physicists at Los Alamos devised games of chance that they could study to help understand complex physical phenomena relating to the atom bomb. Coverage of Monte Carlo methods in this sense can be found in several textbooks (e.g., Kalos and Whitlock, 1986; Rubinstein, 1981).

An early use of Monte Carlo methods to estimate action values in a reinforcement learning context was by Michie and Chambers (1968). In pole balancing (Example 3.4), they used averages of episode durations to assess the worth (expected balancing "life") of each possible action in each state, and then used these assessments to control action selections. Their method is similar in spirit to Monte Carlo ES. In our terms, they used a form of every-visit MC method. Narendra and Wheeler (1986) studied a Monte Carlo method for ergodic finite Markov chains that used the return accumulated from one visit to a state to the next as a reward for adjusting a learning automaton's action probabilities.

Barto and Duff (1994) discussed policy evaluation in the context of classical Monte Carlo algorithms for solving systems of linear equations. They used the analysis of Curtiss (1954) to point out the computational advantages of Monte Carlo policy evaluation for large problems. Singh and Sutton (1996) distinguished between every-visit and first-visit MC methods and proved results relating these methods to reinforcement learning algorithms.

The blackjack example is based on an example used by Widrow, Gupta, and Maitra (1973). The soap bubble example is a classical Dirichlet problem whose Monte Carlo solution was first proposed by Kakutani (1945; see Hersh and Griego, 1969; Doyle and Snell, 1984). The racetrack exercise is adapted

from Barto, Bradtke, and Singh (1995), and from Gardner (1973).

Chapter 6

Temporal-Difference Learning

If one had to identify one idea as central and novel to reinforcement learning, it would undoubtedly be *temporal-difference* (TD) learning. TD learning is a combination of Monte Carlo ideas and dynamic programming (DP) ideas. Like Monte Carlo methods, TD methods can learn directly from raw experience without a model of the environment's dynamics. Like DP, TD methods update estimates based in part on other learned estimates, without waiting for a final outcome (they bootstrap). The relationship between TD, DP, and Monte Carlo methods is a recurring theme in the theory of reinforcement learning. This chapter is the beginning of our exploration of it. Before we are done, we will see that these ideas and methods blend into each other and can be combined in many ways. In particular, in Chapter 7 we introduce the TD(λ) algorithm, which seamlessly integrates TD and Monte Carlo methods.

As usual, we start by focusing on the policy evaluation or *prediction* problem, that of estimating the value function v_π for a given policy π. For the *control* problem (finding an optimal policy), DP, TD, and Monte Carlo methods all use some variation of generalized policy iteration (GPI). The differences in the methods are primarily differences in their approaches to the prediction problem.

6.1 TD Prediction

Both TD and Monte Carlo methods use experience to solve the prediction problem. Given some experience following a policy π, both methods update their estimate v of v_π for the nonterminal states S_t occurring in that experience. Roughly speaking, Monte Carlo methods wait until the return following the visit is known, then use that return as a target for $V(S_t)$. A simple every-visit

Monte Carlo method suitable for nonstationary environments is

$$V(S_t) \leftarrow V(S_t) + \alpha\Big[G_t - V(S_t)\Big], \tag{6.1}$$

where G_t is the actual return following time t, and α is a constant step-size parameter (c.f., Equation 2.4). Let us call this method *constant-α MC*. Whereas Monte Carlo methods must wait until the end of the episode to determine the increment to $V(S_t)$ (only then is G_t known), TD methods need wait only until the next time step. At time $t+1$ they immediately form a target and make a useful update using the observed reward R_{t+1} and the estimate $V(S_{t+1})$. The simplest TD method, known as *TD(0)*, is

$$V(S_t) \leftarrow V(S_t) + \alpha\Big[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\Big]. \tag{6.2}$$

In effect, the target for the Monte Carlo update is G_t, whereas the target for the TD update is $R_{t+1} + \gamma V_t(S_{t+1})$.

Because the TD method bases its update in part on an existing estimate, we say that it is a *bootstrapping* method, like DP. We know from Chapter 3 that

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] && (6.3)\\ &= \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\middle|\; S_t = s\right] \\ &= \mathbb{E}_\pi\left[R_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k R_{t+k+2} \;\middle|\; S_t = s\right] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]\,. && (6.4) \end{aligned}$$

Roughly speaking, Monte Carlo methods use an estimate of (6.3) as a target, whereas DP methods use an estimate of (6.4) as a target. The Monte Carlo target is an estimate because the expected value in (6.3) is not known; a sample return is used in place of the real expected return. The DP target is an estimate not because of the expected values, which are assumed to be completely provided by a model of the environment, but because $v_\pi(S_{t+1})$ is not known and the current estimate, $V(S_{t+1})$, is used instead. The TD target is an estimate for both reasons: it samples the expected values in (6.4) *and* it uses the current estimate V instead of the true v_π. Thus, TD methods combine the sampling of Monte Carlo with the bootstrapping of DP. As we shall see, with care and imagination this can take us a long way toward obtaining the advantages of both Monte Carlo and DP methods.

Input: the policy π to be evaluated
Initialize $V(s)$ arbitrarily (e.g., $V(s) = 0, \forall s \in \mathcal{S}^+$)
Repeat (for each episode):
  Initialize S
  Repeat (for each step of episode):
    $A \leftarrow$ action given by π for S
    Take action A; observe reward, R, and next state, S'
    $V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$
    $S \leftarrow S'$
  until S is terminal

Figure 6.1: Tabular TD(0) for estimating v_π.



Figure 6.2: The backup diagram for TD(0).

Figure 6.1 specifies TD(0) completely in procedural form, and Figure 6.2 shows its backup diagram. The value estimate for the state node at the top of the backup diagram is updated on the basis of the one sample transition from it to the immediately following state. We refer to TD and Monte Carlo updates as *sample backups* because they involve looking ahead to a sample successor state (or state–action pair), using the value of the successor and the reward along the way to compute a backed-up value, and then changing the value of the original state (or state–action pair) accordingly. *Sample* backups differ from the *full* backups of DP methods in that they are based on a single sample successor rather than on a complete distribution of all possible successors.

Example 6.1: Driving Home Each day as you drive home from work, you try to predict how long it will take to get home. When you leave your office, you note the time, the day of week, and anything else that might be relevant. Say on this Friday you are leaving at exactly 6 o'clock, and you estimate that it will take 30 minutes to get home. As you reach your car it is 6:05, and you notice it is starting to rain. Traffic is often slower in the rain, so you reestimate that it will take 35 minutes from then, or a total of 40 minutes. Fifteen minutes later you have completed the highway portion of your journey in good time. As you exit onto a secondary road you cut your estimate of total travel time to 35 minutes. Unfortunately, at this point you get stuck behind a slow truck, and the road is too narrow to pass. You end up having to follow

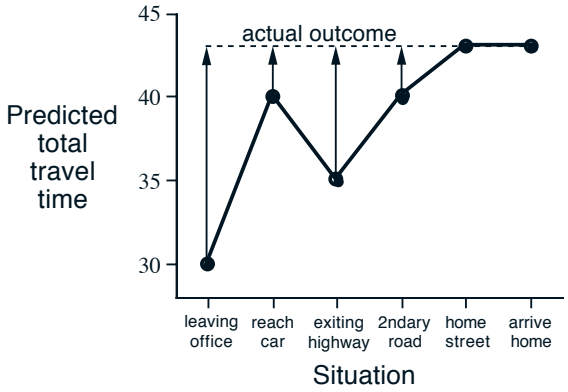


Figure 6.3: Changes recommended by Monte Carlo methods in the driving home example.

the truck until you turn onto the side street where you live at 6:40. Three minutes later you are home. The sequence of states, times, and predictions is thus as follows:

State	*Elapsed Time (minutes)*	*Predicted Time to Go*	*Predicted Total Time*
leaving office, friday at 6	0	30	30
reach car, raining	5	35	40
exiting highway	20	15	35
2ndary road, behind truck	30	10	40
entering home street	40	3	43
arrive home	43	0	43

The rewards in this example are the elapsed times on each leg of the journey.[1] We are not discounting ($\gamma = 1$), and thus the return for each state is the actual time to go from that state. The value of each state is the *expected* time to go. The second column of numbers gives the current estimated value for each state encountered.

A simple way to view the operation of Monte Carlo methods is to plot the predicted total time (the last column) over the sequence, as in Figure 6.3. The arrows show the changes in predictions recommended by the constant-α MC method (6.1), for $\alpha = 1$. These are exactly the errors between the estimated value (predicted time to go) in each state and the actual return (actual time to go). For example, when you exited the highway you thought it would take only 15 minutes more to get home, but in fact it took 23 minutes. Equation

[1] If this were a control problem with the objective of minimizing travel time, then we would of course make the rewards the *negative* of the elapsed time. But since we are concerned here only with prediction (policy evaluation), we can keep things simple by using positive numbers.

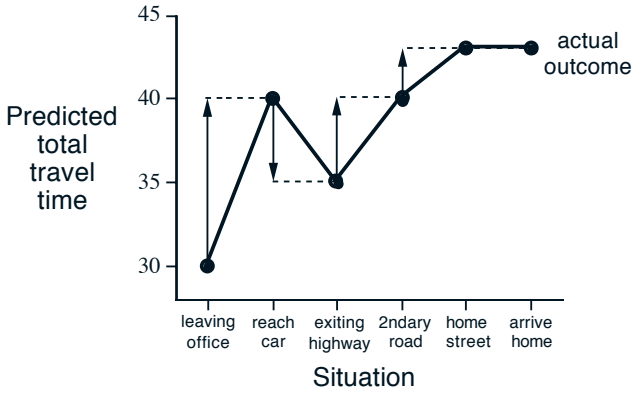


Figure 6.4: Changes recommended by TD methods in the driving home example.

6.1 applies at this point and determines an increment in the estimate of time to go after exiting the highway. The error, $G_t - V(S_t)$, at this time is eight minutes. Suppose the step-size parameter, α, is 1/2. Then the predicted time to go after exiting the highway would be revised upward by four minutes as a result of this experience. This is probably too large a change in this case; the truck was probably just an unlucky break. In any event, the change can only be made off-line, that is, after you have reached home. Only at this point do you know any of the actual returns.

Is it necessary to wait until the final outcome is known before learning can begin? Suppose on another day you again estimate when leaving your office that it will take 30 minutes to drive home, but then you become stuck in a massive traffic jam. Twenty-five minutes after leaving the office you are still bumper-to-bumper on the highway. You now estimate that it will take another 25 minutes to get home, for a total of 50 minutes. As you wait in traffic, you already know that your initial estimate of 30 minutes was too optimistic. Must you wait until you get home before increasing your estimate for the initial state? According to the Monte Carlo approach you must, because you don't yet know the true return.

According to a TD approach, on the other hand, you would learn immediately, shifting your initial estimate from 30 minutes toward 50. In fact, each estimate would be shifted toward the estimate that immediately follows it. Returning to our first day of driving, Figure 6.4 shows the same predictions as Figure 6.3, except with the changes recommended by the TD rule (6.2) (these are the changes made by the rule if $\alpha = 1$). Each error is proportional to the change over time of the prediction, that is, to the *temporal differences* in predictions.

Besides giving you something to do while waiting in traffic, there are several computational reasons why it is advantageous to learn based on your current predictions rather than waiting until termination when you know the actual return. We briefly discuss some of these next.

6.2 Advantages of TD Prediction Methods

TD methods learn their estimates in part on the basis of other estimates. They learn a guess from a guess—they *bootstrap*. Is this a good thing to do? What advantages do TD methods have over Monte Carlo and DP methods? Developing and answering such questions will take the rest of this book and more. In this section we briefly anticipate some of the answers.

Obviously, TD methods have an advantage over DP methods in that they do not require a model of the environment, of its reward and next-state probability distributions.

The next most obvious advantage of TD methods over Monte Carlo methods is that they are naturally implemented in an on-line, fully incremental fashion. With Monte Carlo methods one must wait until the end of an episode, because only then is the return known, whereas with TD methods one need wait only one time step. Surprisingly often this turns out to be a critical consideration. Some applications have very long episodes, so that delaying all learning until an episode's end is too slow. Other applications are continuing tasks and have no episodes at all. Finally, as we noted in the previous chapter, some Monte Carlo methods must ignore or discount episodes on which experimental actions are taken, which can greatly slow learning. TD methods are much less susceptible to these problems because they learn from each transition regardless of what subsequent actions are taken.

But are TD methods sound? Certainly it is convenient to learn one guess from the next, without waiting for an actual outcome, but can we still guarantee convergence to the correct answer? Happily, the answer is yes. For any fixed policy π, the TD algorithm described above has been proved to converge to v_π, in the mean for a constant step-size parameter if it is sufficiently small, and with probability 1 if the step-size parameter decreases according to the usual stochastic approximation conditions (2.7). Most convergence proofs apply only to the table-based case of the algorithm presented above (6.2), but some also apply to the case of general linear function approximation. These results are discussed in a more general setting in the next two chapters.

If both TD and Monte Carlo methods converge asymptotically to the correct predictions, then a natural next question is "Which gets there first?" In



Figure 6.5: A small Markov process for generating random walks.

other words, which method learns faster? Which makes the more efficient use of limited data? At the current time this is an open question in the sense that no one has been able to prove mathematically that one method converges faster than the other. In fact, it is not even clear what is the most appropriate formal way to phrase this question! In practice, however, TD methods have usually been found to converge faster than constant-α MC methods on stochastic tasks, as illustrated in the following example.

Example 6.2: Random Walk In this example we empirically compare the prediction abilities of TD(0) and constant-α MC applied to the small Markov process shown in Figure 6.5. All episodes start in the center state, C, and proceed either left or right by one state on each step, with equal probability. This behavior is presumably due to the combined effect of a fixed policy and an environment's state-transition probabilities, but we do not care which; we are concerned only with predicting returns however they are generated. Episodes terminate either on the extreme left or the extreme right. When an episode terminates on the right a reward of +1 occurs; all other rewards are zero. For example, a typical walk might consist of the following state-and-reward sequence: $\mathsf{C}, 0, \mathsf{B}, 0, \mathsf{C}, 0, \mathsf{D}, 0, \mathsf{E}, 1$. Because this task is undiscounted and episodic, the true value of each state is the probability of terminating on the right if starting from that state. Thus, the true value of the center state is $v_\pi(\mathsf{C}) = 0.5$. The true values of all the states, A through E, are $\frac{1}{6}, \frac{2}{6}, \frac{3}{6}, \frac{4}{6}$, and $\frac{5}{6}$. Figure 6.6 shows the values learned by TD(0) approaching the true values as more episodes are experienced. Averaging over many episode sequences, Figure 6.7 shows the average error in the predictions found by TD(0) and constant-α MC, for a variety of values of α, as a function of number of episodes. In all cases the approximate value function was initialized to the intermediate value $V(s) = 0.5$, for all s. The TD method is consistently better than the MC method on this task over this number of episodes. ■

Exercise 6.1 This is an exercise to help develop your intuition about why TD methods are often more efficient than Monte Carlo methods. Consider the driving home example and how it is addressed by TD and Monte Carlo methods. Can you imagine a scenario in which a TD update would be better on average than an Monte Carlo update? Give an example scenario—a description of past experience and a current state—in which you would expect the TD

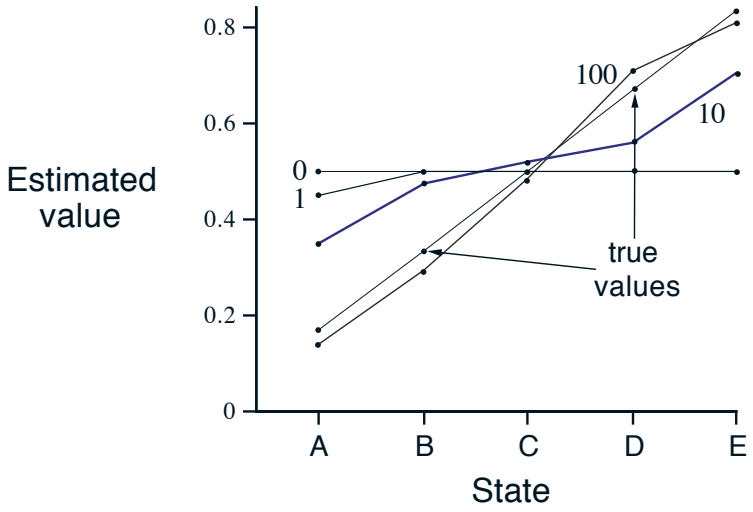


Figure 6.6: Values learned by TD(0) after various numbers of episodes. The final estimate is about as close as the estimates ever get to the true values. With a constant step-size parameter ($\alpha = 0.1$ in this example), the values fluctuate indefinitely in response to the outcomes of the most recent episodes.

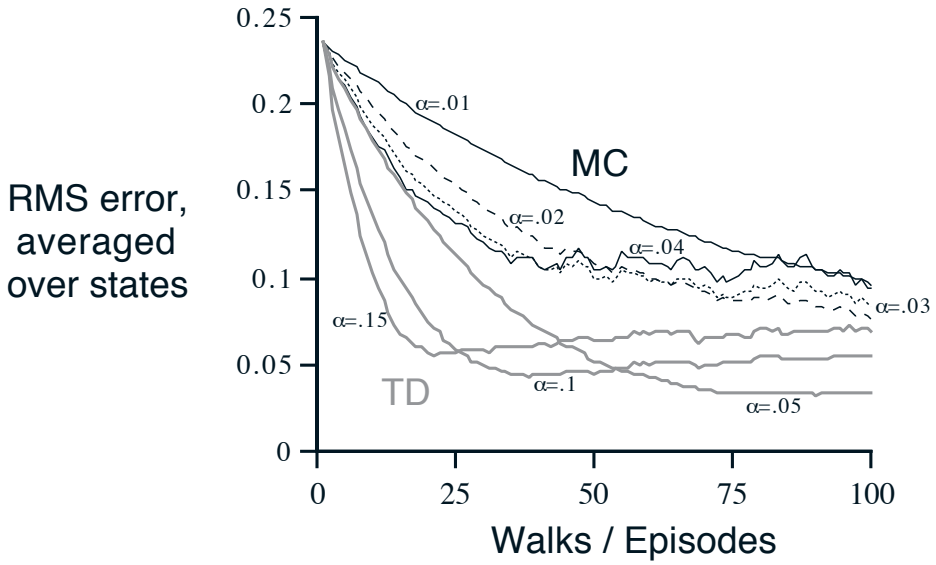


Figure 6.7: Learning curves for TD(0) and constant-α MC methods, for various values of α, on the prediction problem for the random walk. The performance measure shown is the root mean-squared (RMS) error between the value function learned and the true value function, averaged over the five states. These data are averages over 100 different sequences of episodes.

update to be better. Here's a hint: Suppose you have lots of experience driving home from work. Then you move to a new building and a new parking lot (but you still enter the highway at the same place). Now you are starting to learn predictions for the new building. Can you see why TD updates are likely to be much better, at least initially, in this case? Might the same sort of thing happen in the original task?

Exercise 6.2 From Figure 6.6, it appears that the first episode results in a change in only $V(\mathsf{A})$. What does this tell you about what happened on the first episode? Why was only the estimate for this one state changed? By exactly how much was it changed?

Exercise 6.3 The specific results shown in Figure 6.7 are dependent on the value of the step-size parameter, α. Do you think the conclusions about which algorithm is better would be affected if a wider range of α values were used? Is there a different, fixed value of α at which either algorithm would have performed significantly better than shown? Why or why not?

Exercise 6.4 In Figure 6.7, the RMS error of the TD method seems to go down and then up again, particularly at high α's. What could have caused this? Do you think this always occurs, or might it be a function of how the approximate value function was initialized?

Exercise 6.5 Above we stated that the true values for the random walk task are $\frac{1}{6}, \frac{2}{6}, \frac{3}{6}, \frac{4}{6}$, and $\frac{5}{6}$, for states A through E. Describe at least two different ways that these could have been computed. Which would you guess we actually used? Why?

6.3 Optimality of TD(0)

Suppose there is available only a finite amount of experience, say 10 episodes or 100 time steps. In this case, a common approach with incremental learning methods is to present the experience repeatedly until the method converges upon an answer. Given an approximate value function, V, the increments specified by (6.1) or (6.2) are computed for every time step t at which a nonterminal state is visited, but the value function is changed only once, by the sum of all the increments. Then all the available experience is processed again with the new value function to produce a new overall increment, and so on, until the value function converges. We call this *batch updating* because updates are made only after processing each complete *batch* of training data.

Under batch updating, TD(0) converges deterministically to a single answer independent of the step-size parameter, α, as long as α is chosen to be

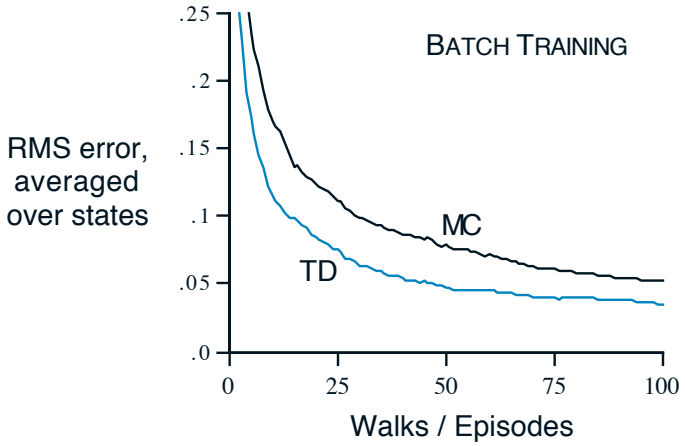


Figure 6.8: Performance of TD(0) and constant-α MC under batch training on the random walk task.

sufficiently small. The constant-α MC method also converges deterministically under the same conditions, but to a different answer. Understanding these two answers will help us understand the difference between the two methods. Under normal updating the methods do not move all the way to their respective batch answers, but in some sense they take steps in these directions. Before trying to understand the two answers in general, for all possible tasks, we first look at a few examples.

Example 6.3 *Random walk under batch updating.* Batch-updating versions of TD(0) and constant-α MC were applied as follows to the random walk prediction example (Example 6.2). After each new episode, all episodes seen so far were treated as a batch. They were repeatedly presented to the algorithm, either TD(0) or constant-α MC, with α sufficiently small that the value function converged. The resulting value function was then compared with v_π, and the average root mean-squared error across the five states (and across 100 independent repetitions of the whole experiment) was plotted to obtain the learning curves shown in Figure 6.8. Note that the batch TD method was consistently better than the batch Monte Carlo method. ■

Under batch training, constant-α MC converges to values, $V(s)$, that are sample averages of the actual returns experienced after visiting each state s. These are optimal estimates in the sense that they minimize the mean-squared error from the actual returns in the training set. In this sense it is surprising that the batch TD method was able to perform better according to the root mean-squared error measure shown in Figure 6.8. How is it that batch TD was able to perform better than this optimal method? The answer is that the Monte Carlo method is optimal only in a limited way, and that TD is optimal

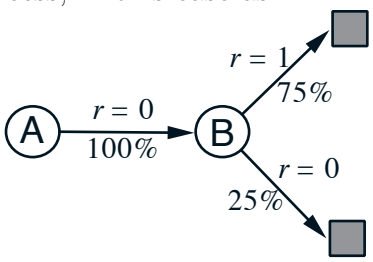
in a way that is more relevant to predicting returns. But first let's develop our intuitions about different kinds of optimality through another example.

Example 6.4: You are the Predictor Place yourself now in the role of the predictor of returns for an unknown Markov reward process. Suppose you observe the following eight episodes:

A, 0, B, 0	B, 1
B, 1	B, 1
B, 1	B, 1
B, 1	B, 0

This means that the first episode started in state A, transitioned to B with a reward of 0, and then terminated from B with a reward of 0. The other seven episodes were even shorter, starting from B and terminating immediately. Given this batch of data, what would you say are the optimal predictions, the best values for the estimates $V(\mathsf{A})$ and $V(\mathsf{B})$? Everyone would probably agree that the optimal value for $V(\mathsf{B})$ is $\frac{3}{4}$, because six out of the eight times in state B the process terminated immediately with a return of 1, and the other two times in B the process terminated immediately with a return of 0.

But what is the optimal value for the estimate $V(\mathsf{A})$ given this data? Here there are two reasonable answers. One is to observe that 100% of the times the process was in state A it traversed immediately to B (with a reward of 0); and since we have already decided that B has value $\frac{3}{4}$, therefore A must have value $\frac{3}{4}$ as well. One way of viewing this answer is that it is based on first modeling the Markov process, in this case as



and then computing the correct estimates given the model, which indeed in this case gives $V(\mathsf{A}) = \frac{3}{4}$. This is also the answer that batch TD(0) gives.

The other reasonable answer is simply to observe that we have seen A once and the return that followed it was 0; we therefore estimate $V(\mathsf{A})$ as 0. This is the answer that batch Monte Carlo methods give. Notice that it is also the answer that gives minimum squared error on the training data. In fact, it gives zero error on the data. But still we expect the first answer to be better. If the process is Markov, we expect that the first answer will produce lower error on *future* data, even though the Monte Carlo answer is better on the existing

data. ■

The above example illustrates a general difference between the estimates found by batch TD(0) and batch Monte Carlo methods. Batch Monte Carlo methods always find the estimates that minimize mean-squared error on the training set, whereas batch TD(0) always finds the estimates that would be exactly correct for the maximum-likelihood model of the Markov process. In general, the *maximum-likelihood estimate* of a parameter is the parameter value whose probability of generating the data is greatest. In this case, the maximum-likelihood estimate is the model of the Markov process formed in the obvious way from the observed episodes: the estimated transition probability from i to j is the fraction of observed transitions from i that went to j, and the associated expected reward is the average of the rewards observed on those transitions. Given this model, we can compute the estimate of the value function that would be exactly correct if the model were exactly correct. This is called the *certainty-equivalence estimate* because it is equivalent to assuming that the estimate of the underlying process was known with certainty rather than being approximated. In general, batch TD(0) converges to the certainty-equivalence estimate.

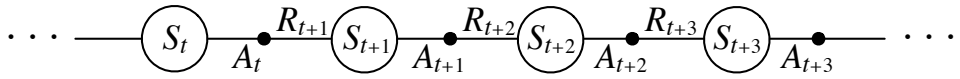
This helps explain why TD methods converge more quickly than Monte Carlo methods. In batch form, TD(0) is faster than Monte Carlo methods because it computes the true certainty-equivalence estimate. This explains the advantage of TD(0) shown in the batch results on the random walk task (Figure 6.8). The relationship to the certainty-equivalence estimate may also explain in part the speed advantage of nonbatch TD(0) (e.g., Figure 6.7). Although the nonbatch methods do not achieve either the certainty-equivalence or the minimum squared-error estimates, they can be understood as moving roughly in these directions. Nonbatch TD(0) may be faster than constant-α MC because it is moving toward a better estimate, even though it is not getting all the way there. At the current time nothing more definite can be said about the relative efficiency of on-line TD and Monte Carlo methods.

Finally, it is worth noting that although the certainty-equivalence estimate is in some sense an optimal solution, it is almost never feasible to compute it directly. If N is the number of states, then just forming the maximum-likelihood estimate of the process may require N^2 memory, and computing the corresponding value function requires on the order of N^3 computational steps if done conventionally. In these terms it is indeed striking that TD methods can approximate the same solution using memory no more than N and repeated computations over the training set. On tasks with large state spaces, TD methods may be the only feasible way of approximating the certainty-equivalence solution.

6.4 Sarsa: On-Policy TD Control

We turn now to the use of TD prediction methods for the control problem. As usual, we follow the pattern of generalized policy iteration (GPI), only this time using TD methods for the evaluation or prediction part. As with Monte Carlo methods, we face the need to trade off exploration and exploitation, and again approaches fall into two main classes: on-policy and off-policy. In this section we present an on-policy TD control method.

The first step is to learn an action-value function rather than a state-value function. In particular, for an on-policy method we must estimate $q_\pi(s, a)$ for the current behavior policy π and for all states s and actions a. This can be done using essentially the same TD method described above for learning v_π. Recall that an episode consists of an alternating sequence of states and state–action pairs:



In the previous section we considered transitions from state to state and learned the values of states. Now we consider transitions from state–action pair to state–action pair, and learn the value of state–action pairs. Formally these cases are identical: they are both Markov chains with a reward process. The theorems assuring the convergence of state values under TD(0) also apply to the corresponding algorithm for action values:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big]. \qquad (6.5)$$

This update is done after every transition from a nonterminal state S_t. If S_{t+1} is terminal, then $Q(S_{t+1}, A_{t+1})$ is defined as zero. This rule uses every element of the quintuple of events, $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, that make up a transition from one state–action pair to the next. This quintuple gives rise to the name *Sarsa* for the algorithm.

It is straightforward to design an on-policy control algorithm based on the Sarsa prediction method. As in all on-policy methods, we continually estimate q_π for the behavior policy π, and at the same time change π toward greediness with respect to q_π. The general form of the Sarsa control algorithm is given in Figure 6.9.

The convergence properties of the Sarsa algorithm depend on the nature of the policy's dependence on q. For example, one could use ε-greedy or ε-soft policies. According to Satinder Singh (personal communication), Sarsa converges with probability 1 to an optimal policy and action-value function as

Initialize $Q(s,a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\textit{terminal-state}, \cdot) = 0$
Repeat (for each episode):
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Repeat (for each step of episode):
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma Q(S',A') - Q(S,A)]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

Figure 6.9: Sarsa: An on-policy TD control algorithm.

long as all state–action pairs are visited an infinite number of times and the policy converges in the limit to the greedy policy (which can be arranged, for example, with ε-greedy policies by setting $\varepsilon = 1/t$), but this result has not yet been published in the literature.

Example 6.5: Windy Gridworld Figure 6.10 shows a standard gridworld, with start and goal states, but with one difference: there is a crosswind upward through the middle of the grid. The actions are the standard four—`up`, `down`, `right`, and `left`—but in the middle region the resultant next states are shifted upward by a "wind," the strength of which varies from column to column. The strength of the wind is given below each column, in number of cells shifted upward. For example, if you are one cell to the right of the goal, then the action `left` takes you to the cell just above the goal. Let us treat this as an undiscounted episodic task, with constant rewards of -1 until the goal state is reached. Figure 6.11 shows the result of applying ε-greedy Sarsa to this task, with $\varepsilon = 0.1$, $\alpha = 0.5$, and the initial values $Q(s,a) = 0$ for all s, a. The increasing slope of the graph shows that the goal is reached more and more quickly over time. By 8000 time steps, the greedy policy (shown inset) was long since optimal; continued ε-greedy exploration kept the average episode length at about 17 steps, two more than the minimum of 15. Note that Monte Carlo methods cannot easily be used on this task because termination is not guaranteed for all policies. If a policy was ever found that caused the agent to stay in the same state, then the next episode would never end. Step-by-step learning methods such as Sarsa do not have this problem because they quickly learn *during the episode* that such policies are poor, and switch to something else. ■

Exercise 6.6: Windy Gridworld with King's Moves Re-solve the windy gridworld task assuming eight possible actions, including the diagonal moves, rather than the usual four. How much better can you do with the extra

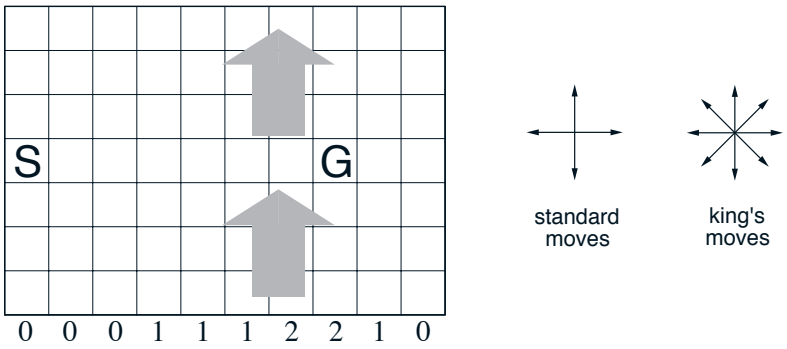


Figure 6.10: Gridworld in which movement is altered by a location-dependent, upward "wind."

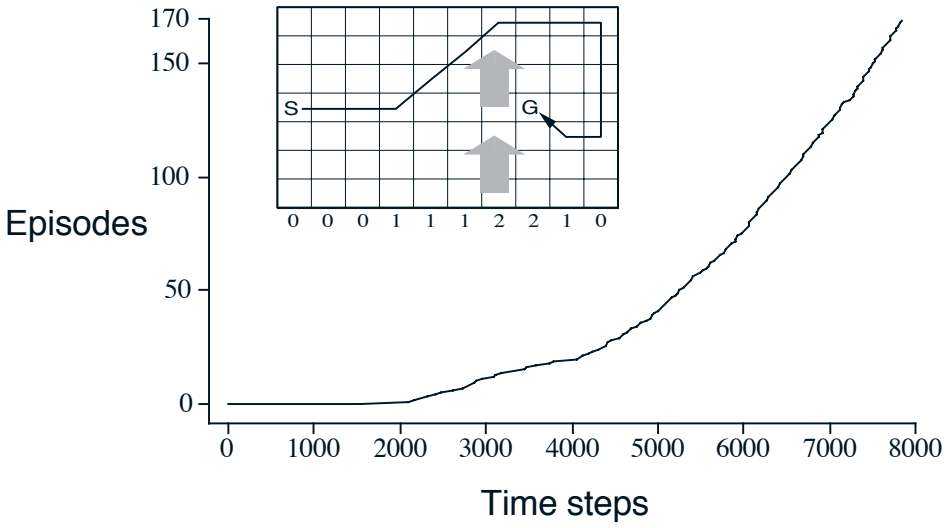


Figure 6.11: Results of Sarsa applied to the windy gridworld.

actions? Can you do even better by including a ninth action that causes no movement at all other than that caused by the wind?

Exercise 6.7: Stochastic Wind Resolve the windy gridworld task with King's moves, assuming that the effect of the wind, if there is any, is stochastic, sometimes varying by 1 from the mean values given for each column. That is, a third of the time you move exactly according to these values, as in the previous exercise, but also a third of the time you move one cell above that, and another third of the time you move one cell below that. For example, if you are one cell to the right of the goal and you move `left`, then one-third of the time you move one cell above the goal, one-third of the time you move two cells above the goal, and one-third of the time you move to the goal.

Exercise 6.8 What is the backup diagram for Sarsa?

6.5 Q-Learning: Off-Policy TD Control

One of the most important breakthroughs in reinforcement learning was the development of an off-policy TD control algorithm known as *Q-learning* (Watkins, 1989). Its simplest form, *one-step Q-learning*, is defined by

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big]. \quad (6.6)$$

In this case, the learned action-value function, Q, directly approximates q_*, the optimal action-value function, independent of the policy being followed. This dramatically simplifies the analysis of the algorithm and enabled early convergence proofs. The policy still has an effect in that it determines which state–action pairs are visited and updated. However, all that is required for correct convergence is that all pairs continue to be updated. As we observed in Chapter 5, this is a minimal requirement in the sense that any method guaranteed to find optimal behavior in the general case must require it. Under this assumption and a variant of the usual stochastic approximation conditions on the sequence of step-size parameters, Q has been shown to converge with probability 1 to q_*. The Q-learning algorithm is shown in procedural form in Figure 6.12.

What is the backup diagram for Q-learning? The rule (6.6) updates a state–action pair, so the top node, the root of the backup, must be a small, filled action node. The backup is also *from* action nodes, maximizing over all those actions possible in the next state. Thus the bottom nodes of the backup diagram should be all these action nodes. Finally, remember that we indicate taking the maximum of these "next action" nodes with an arc across them

Initialize $Q(s,a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\textit{terminal-state}, \cdot) = 0$
Repeat (for each episode):
  Initialize S
  Repeat (for each step of episode):
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$
    $S \leftarrow S'$;
  until S is terminal

Figure 6.12: Q-learning: An off-policy TD control algorithm.

(Figure 3.7). Can you guess now what the diagram is? If so, please do make a guess before turning to the answer in Figure 6.14.

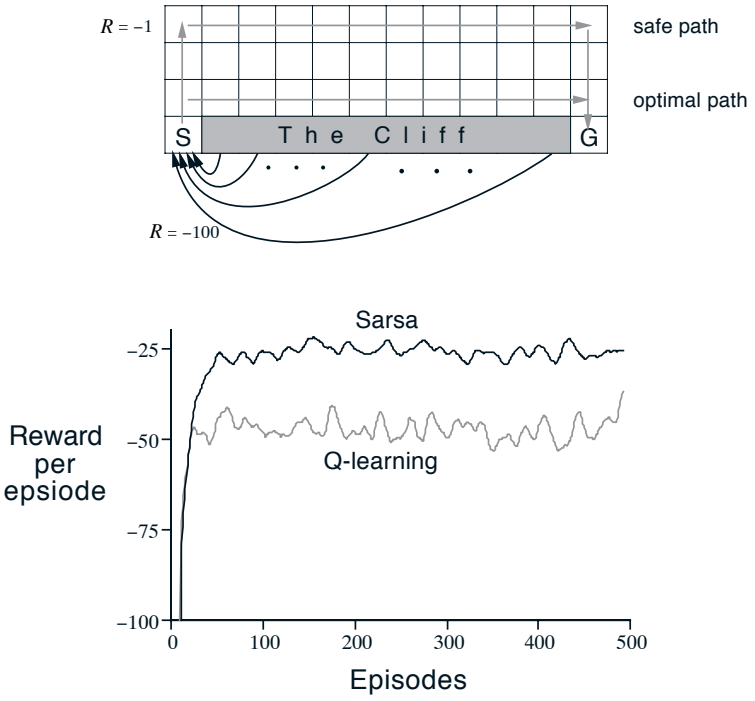


Figure 6.13: The cliff-walking task. The results are from a single run, but smoothed.

Example 6.6: Cliff Walking This gridworld example compares Sarsa and Q-learning, highlighting the difference between on-policy (Sarsa) and off-policy (Q-learning) methods. Consider the gridworld shown in the upper part of Figure 6.13. This is a standard undiscounted, episodic task, with start and goal states, and the usual actions causing movement up, down, right, and left.



Figure 6.14: The backup diagram for Q-learning.

Reward is -1 on all transitions except those into the the region marked "The Cliff." Stepping into this region incurs a reward of -100 and sends the agent instantly back to the start. The lower part of the figure shows the performance of the Sarsa and Q-learning methods with ε-greedy action selection, $\varepsilon = 0.1$. After an initial transient, Q-learning learns values for the optimal policy, that which travels right along the edge of the cliff. Unfortunately, this results in its occasionally falling off the cliff because of the ε-greedy action selection. Sarsa, on the other hand, takes the action selection into account and learns the longer but safer path through the upper part of the grid. Although Q-learning actually learns the values of the optimal policy, its on-line performance is worse than that of Sarsa, which learns the roundabout policy. Of course, if ε were gradually reduced, then both methods would asymptotically converge to the optimal policy. ■

Exercise 6.9 Why is Q-learning considered an *off-policy* control method?

Exercise 6.10 Consider the learning algorithm that is just like Q-learning except that instead of the maximum over next state–action pairs it uses the expected value, taking into account how likely each action is under the current policy. That is, consider the algorithm otherwise like Q-learning except with the update rule

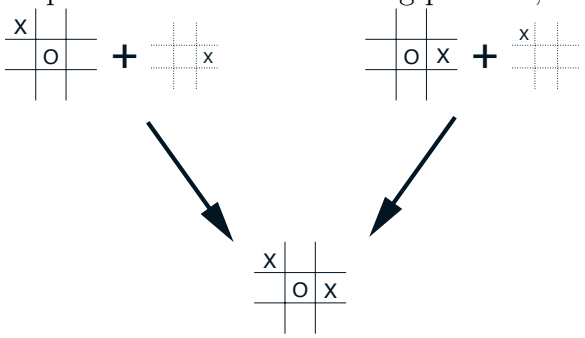
$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \mathbb{E}[Q(S_{t+1}, A_{t+1}) \mid S_{t+1}] - Q(S_t, A_t)\Big] \\ &\leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)\Big]. \end{aligned}$$

Is this new method an on-policy or off-policy method? What is the backup diagram for this algorithm? Given the same amount of experience, would you expect this method to work better or worse than Sarsa? What other considerations might impact the comparison of this method with Sarsa?

6.6 Games, Afterstates, and Other Special Cases

In this book we try to present a uniform approach to a wide class of tasks, but of course there are always exceptional tasks that are better treated in a specialized way. For example, our general approach involves learning an *action*-value function, but in Chapter 1 we presented a TD method for learning to play tic-tac-toe that learned something much more like a *state*-value function. If we look closely at that example, it becomes apparent that the function learned there is neither an action-value function nor a state-value function in the usual sense. A conventional state-value function evaluates states in which the agent has the option of selecting an action, but the state-value function used in tic-tac-toe evaluates board positions *after* the agent has made its move. Let us call these *afterstates*, and value functions over these, *afterstate value functions*. Afterstates are useful when we have knowledge of an initial part of the environment's dynamics but not necessarily of the full dynamics. For example, in games we typically know the immediate effects of our moves. We know for each possible chess move what the resulting position will be, but not how our opponent will reply. Afterstate value functions are a natural way to take advantage of this kind of knowledge and thereby produce a more efficient learning method.

The reason it is more efficient to design algorithms in terms of afterstates is apparent from the tic-tac-toe example. A conventional action-value function would map from positions *and* moves to an estimate of the value. But many position–move pairs produce the same resulting position, as in this example:



In such cases the position–move pairs are different but produce the same "afterposition," and thus must have the same value. A conventional action-value function would have to separately assess both pairs, whereas an afterstate value function would immediately assess both equally. Any learning about the position–move pair on the left would immediately transfer to the pair on the right.

Afterstates arise in many tasks, not just games. For example, in queuing tasks there are actions such as assigning customers to servers, rejecting cus-

tomers, or discarding information. In such cases the actions are in fact defined in terms of their immediate effects, which are completely known. For example, in the access-control queuing example described in the previous section, a more efficient learning method could be obtained by breaking the environment's dynamics into the immediate effect of the action, which is deterministic and completely known, and the unknown random processes having to do with the arrival and departure of customers. The afterstates would be the number of free servers after the action but before the random processes had produced the next conventional state. Learning an afterstate value function over the afterstates would enable all actions that produced the same number of free servers to share experience. This should result in a significant reduction in learning time.

It is impossible to describe all the possible kinds of specialized problems and corresponding specialized learning algorithms. However, the principles developed in this book should apply widely. For example, afterstate methods are still aptly described in terms of generalized policy iteration, with a policy and (afterstate) value function interacting in essentially the same way. In many cases one will still face the choice between on-policy and off-policy methods for managing the need for persistent exploration.

Exercise 6.11 Describe how the task of Jack's Car Rental (Example 4.2) could be reformulated in terms of afterstates. Why, in terms of this specific task, would such a reformulation be likely to speed convergence?

6.7 Summary

In this chapter we introduced a new kind of learning method, temporal-difference (TD) learning, and showed how it can be applied to the reinforcement learning problem. As usual, we divided the overall problem into a prediction problem and a control problem. TD methods are alternatives to Monte Carlo methods for solving the prediction problem. In both cases, the extension to the control problem is via the idea of generalized policy iteration (GPI) that we abstracted from dynamic programming. This is the idea that approximate policy and value functions should interact in such a way that they both move toward their optimal values.

One of the two processes making up GPI drives the value function to accurately predict returns for the current policy; this is the prediction problem. The other process drives the policy to improve locally (e.g., to be ε-greedy) with respect to the current value function. When the first process is based on experience, a complication arises concerning maintaining sufficient exploration. We

have grouped the TD control methods according to whether they deal with this complication by using an on-policy or off-policy approach. Sarsa and actor–critic methods are on-policy methods, and Q-learning and R-learning are off-policy methods.

The methods presented in this chapter are today the most widely used reinforcement learning methods. This is probably due to their great simplicity: they can be applied on-line, with a minimal amount of computation, to experience generated from interaction with an environment; they can be expressed nearly completely by single equations that can be implemented with small computer programs. In the next few chapters we extend these algorithms, making them slightly more complicated and significantly more powerful. All the new algorithms will retain the essence of those introduced here: they will be able to process experience on-line, with relatively little computation, and they will be driven by TD errors. The special cases of TD methods introduced in the present chapter should rightly be called *one-step, tabular, modelfree* TD methods. In the next three chapters we extend them to multistep forms (a link to Monte Carlo methods), forms using function approximation rather than tables (a link to artificial neural networks), and forms that include a model of the environment (a link to planning and dynamic programming).

Finally, in this chapter we have discussed TD methods entirely within the context of reinforcement learning problems, but TD methods are actually more general than this. They are general methods for learning to make long-term predictions about dynamical systems. For example, TD methods may be relevant to predicting financial data, life spans, election outcomes, weather patterns, animal behavior, demands on power stations, or customer purchases. It was only when TD methods were analyzed as pure prediction methods, independent of their use in reinforcement learning, that their theoretical properties first came to be well understood. Even so, these other potential applications of TD learning methods have not yet been extensively explored.

6.8 Bibliographical and Historical Remarks

As we outlined in Chapter 1, the idea of TD learning has its early roots in animal learning psychology and artificial intelligence, most notably the work of Samuel (1959) and Klopf (1972). Samuel's work is described as a case study in Section 15.2. Also related to TD learning are Holland's (1975, 1976) early ideas about consistency among value predictions. These influenced one of the authors (Barto), who was a graduate student from 1970 to 1975 at the University of Michigan, where Holland was teaching. Holland's ideas led to a number of TD-related systems, including the work of Booker (1982) and the

bucket brigade of Holland (1986), which is related to Sarsa as discussed below.

6.1–2 Most of the specific material from these sections is from Sutton (1988), including the TD(0) algorithm, the random walk example, and the term "temporal-difference learning." The characterization of the relationship to dynamic programming and Monte Carlo methods was influenced by Watkins (1989), Werbos (1987), and others. The use of backup diagrams here and in other chapters is new to this book. Example 6.4 is due to Sutton, but has not been published before.

Tabular TD(0) was proved to converge in the mean by Sutton (1988) and with probability 1 by Dayan (1992), based on the work of Watkins and Dayan (1992). These results were extended and strengthened by Jaakkola, Jordan, and Singh (1994) and Tsitsiklis (1994) by using extensions of the powerful existing theory of stochastic approximation. Other extensions and generalizations are covered in the next two chapters.

6.3 The optimality of the TD algorithm under batch training was established by Sutton (1988). The term *certainty equivalence* is from the adaptive control literature (e.g., Goodwin and Sin, 1984). Illuminating this result is Barnard's (1993) derivation of the TD algorithm as a combination of one step of an incremental method for learning a model of the Markov chain and one step of a method for computing predictions from the model.

6.4 The Sarsa algorithm was first explored by Rummery and Niranjan (1994), who called it *modified Q-learning*. The name "Sarsa" was introduced by Sutton (1996). The convergence of one-step tabular Sarsa (the form treated in this chapter) has been proved by Satinder Singh (personal communication). The "windy gridworld" example was suggested by Tom Kalt.

Holland's (1986) bucket brigade idea evolved into an algorithm closely related to Sarsa. The original idea of the bucket brigade involved chains of rules triggering each other; it focused on passing credit back from the current rule to the rules that triggered it. Over time, the bucket brigade came to be more like TD learning in passing credit back to any temporally preceding rule, not just to the ones that triggered the current rule. The modern form of the bucket brigade, when simplified in various natural ways, is nearly identical to one-step Sarsa, as detailed by Wilson (1994).

6.5 Q-learning was introduced by Watkins (1989), whose outline of a convergence proof was later made rigorous by Watkins and Dayan (1992). More general convergence results were proved by Jaakkola, Jordan, and Singh (1994) and Tsitsiklis (1994).

6.6 R-learning is due to Schwartz (1993). Mahadevan (1996), Tadepalli and Ok (1994), and Bertsekas and Tsitsiklis (1996) have studied reinforcement learning for undiscounted continuing tasks. In the literature, the undiscounted continuing case is often called the case of maximizing "average reward per time step" or the "average-reward case." The name R-learning was probably meant to be the alphabetic successor to Q-learning, but we prefer to think of it as a reference to the learning of *relative* values. The access-control queuing example was suggested by the work of Carlström and Nordström (1997).

Chapter 7

Eligibility Traces

Eligibility traces are one of the basic mechanisms of reinforcement learning. For example, in the popular TD(λ) algorithm, the λ refers to the use of an eligibility trace. Almost any temporal-difference (TD) method, such as Q-learning or Sarsa, can be combined with eligibility traces to obtain a more general method that may learn more efficiently.

There are two ways to view eligibility traces. The more theoretical view, which we emphasize here, is that they are a bridge from TD to Monte Carlo methods. When TD methods are augmented with eligibility traces, they produce a family of methods spanning a spectrum that has Monte Carlo methods at one end and one-step TD methods at the other. In between are intermediate methods that are often better than either extreme method. In this sense eligibility traces unify TD and Monte Carlo methods in a valuable and revealing way.

The other way to view eligibility traces is more mechanistic. From this perspective, an eligibility trace is a temporary record of the occurrence of an event, such as the visiting of a state or the taking of an action. The trace marks the memory parameters associated with the event as eligible for undergoing learning changes. When a TD error occurs, only the eligible states or actions are assigned credit or blame for the error. Thus, eligibility traces help bridge the gap between events and training information. Like TD methods themselves, eligibility traces are a basic mechanism for temporal credit assignment.

For reasons that will become apparent shortly, the more theoretical view of eligibility traces is called the *forward* view, and the more mechanistic view is called the *backward* view. The forward view is most useful for understanding *what* is computed by methods using eligibility traces, whereas the backward view is more appropriate for developing intuition about the algorithms them-

selves. In this chapter we present both views and then establish the senses in which they are equivalent, that is, in which they describe the same algorithms from two points of view. As usual, we first consider the prediction problem and then the control problem. That is, we first consider how eligibility traces are used to help in predicting returns as a function of state for a fixed policy (i.e., in estimating v_π). Only after exploring the two views of eligibility traces within this prediction setting do we extend the ideas to action values and control methods.

7.1 n-Step TD Prediction

What is the space of methods lying between Monte Carlo and TD methods? Consider estimating v_π from sample episodes generated using π. Monte Carlo methods perform a backup for each state based on the entire sequence of observed rewards from that state until the end of the episode. The backup of simple TD methods, on the other hand, is based on just the one next reward, using the value of the state one step later as a proxy for the remaining rewards. One kind of intermediate method, then, would perform a backup based on an intermediate number of rewards: more than one, but less than all of them until termination. For example, a two-step backup would be based on the first two rewards and the estimated value of the state two steps later. Similarly, we could have three-step backups, four-step backups, and so on. Figure 7.1 diagrams the spectrum of *n-step backups* for v_π, with one-step, simple TD backups on the left and up-until-termination Monte Carlo backups on the right.

The methods that use n-step backups are still TD methods because they still change an earlier estimate based on how it differs from a later estimate. Now the later estimate is not one step later, but n steps later. Methods in which the temporal difference extends over n steps are called *n-step TD methods*. The TD methods introduced in the previous chapter all use one-step backups, and henceforth we call them *one-step TD methods*.

More formally, consider the backup applied to state S_t as a result of the state–reward sequence, $S_t, R_{t+1}, S_{t+1}, R_{t+2}, \ldots, R_T, S_T$ (omitting the actions for simplicity). We know that in Monte Carlo backups the estimate $V(S_t)$ of $v_\pi(S_t)$ is updated in the direction of the complete return:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T,$$

where T is the last time step of the episode. Let us call this quantity the *target* of the backup. Whereas in Monte Carlo backups the target is the expected

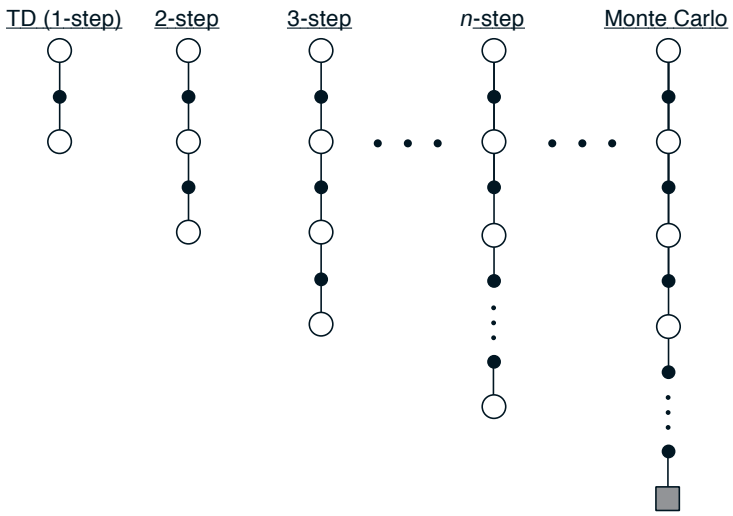


Figure 7.1: The spectrum ranging from the one-step backups of simple TD methods to the up-until-termination backups of Monte Carlo methods. In between are the n-step backups, based on n steps of real rewards and the estimated value of the nth next state, all appropriately discounted.

return, in one-step backups the target is the first reward plus the discounted estimated value of the next state:

$$G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1}).$$

This makes sense because $\gamma V(S_{t+1})$ takes the place of the remaining terms $\gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T$, as we discussed in the previous chapter. Our point now is that this idea makes just as much sense after two steps as it does after one. The two-step target is

$$G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}),$$

where now $\gamma^2 V(S_{t+2})$ takes the place of the terms $\gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots + \gamma^{T-t-1} R_T$. In general, the n-step target is

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}). \tag{7.1}$$

This quantity is sometimes called the "corrected n-step truncated return" because it is a return truncated after n steps and then approximately corrected for the truncation by adding the estimated value of the nth next state. That terminology is descriptive but a bit long. We instead refer to $G_t^{(n)}$ simply as the *n-step return* at time t.

Of course, if the episode ends in less than n steps, then the truncation in an n-step return occurs at the episode's end, resulting in the conventional complete return. In other words, if $T-t<n$, then $G_t^{(n)}=G_t^{(T-t)}=G_t$. Thus, the last n n-step returns of any episode are always complete returns, and an infinite-step return is always a complete return. This definition enables us to treat Monte Carlo methods as the special case of infinite-step returns. All of this is consistent with the tricks for treating episodic and continuing tasks equivalently that we introduced in Section 3.4. There we chose to treat the terminal state as a state that always transitions to itself with zero reward. Under this trick, all n-step returns that last up to or past termination have the same value as the complete return.

An *n-step backup* is defined to be a backup toward the n-step return. In the tabular, state-value case, the increment to $V_t(S_t)$ (the estimated value of $v_\pi(S_t)$ at time t), due to an n-step backup of S_t, is defined by

$$\Delta V_t(S_t)=\alpha\Big[G_t^{(n)}-V_t(S_t)\Big],$$

where α is a positive step-size parameter, as usual. Of course, the increments to the estimated values of the other states are $\Delta V_t(s)=0$, for all $s\neq S_t$. We define the n-step backup in terms of an increment, rather than as a direct update rule as we did in the previous chapter, in order to distinguish two different ways of making the updates. In *on-line updating*, the updates are done during the episode, as soon as the increment is computed. In this case we have $V_{t+1}(s)=V_t(s)+\Delta V_t(s)$ for all $s\in\mathcal{S}$. This is the case considered in the previous chapter. In *off-line updating*, on the other hand, the increments are accumulated "on the side" and are not used to change value estimates until the end of the episode. In this case, $V_t(s)$ is constant within an episode, for each s. If its value in this episode is $V(s)$, then its new value in the next episode will be $V(s)+\sum_{t=0}^{T-1}\Delta V_t(s)$.

The expected value of all n-step returns is guaranteed to improve in a certain way over the current value function as an approximation to the true value function. For any function $v:\mathcal{S}\to\mathbb{R}$, the expected value of the n-step return using v is guaranteed to be a better estimate of v_π than v is, in a worst-state sense. That is, the worst error under the new estimate is guaranteed to be less than or equal to γ^n times the worst error under v:

$$\max_s\left|\mathbb{E}_\pi\Big[G_t^{(n)}\;\Big|\;S_t{=}s\Big]-v_\pi(s)\right|\;\leq\;\gamma^n\max_s\left|v(s)-v_\pi(s)\right|. \tag{7.2}$$

This is called the *error reduction property* of n-step returns. Because of the error reduction property, one can show formally that on-line and off-line TD prediction methods using n-step backups converge to the correct predictions

under appropriate technical conditions. The n-step TD methods thus form a family of valid methods, with one-step TD methods and Monte Carlo methods as extreme members.

Nevertheless, n-step TD methods are rarely used because they are inconvenient to implement. Computing n-step returns requires waiting n steps to observe the resultant rewards and states. For large n, this can become problematic, particularly in control applications. The significance of n-step TD methods is primarily for theory and for understanding related methods that are more conveniently implemented. In the next few sections we use the idea of n-step TD methods to explain and justify eligibility trace methods.

Example 7.1: n-step TD Methods on the Random Walk Consider using n-step TD methods on the random walk task described in Example 6.2 and shown in Figure 6.5. Suppose the first episode progressed directly from the center state, C, to the right, through D and E, and then terminated on the right with a return of 1. Recall that the estimated values of all the states started at an intermediate value, $V_0(s) = 0.5$. As a result of this experience, a one-step method would change only the estimate for the last state, $V(\mathsf{E})$, which would be incremented toward 1, the observed return. A two-step method, on the other hand, would increment the values of the two states preceding termination: $V(\mathsf{D})$ and $V(\mathsf{E})$ would both be incremented toward 1. A three-step method, or any n-step method for $n > 2$, would increment the values of all three of the visited states toward 1, all by the same amount. Which n is better? Figure 7.2 shows the results of a simple empirical assessment for a larger random walk process, with 19 states (and with a -1 outcome on the left, all values initialized to 0). Shown is the root mean-squared error in the predictions at the end of an episode, averaged over states, the first 10 episodes, and 100 repetitions of the whole experiment (the same sets of walks were used for all methods). Results are shown for on-line and off-line n-step TD methods with a range of values for n and α. Empirically, on-line methods with an intermediate value of n seem to work best on this task. This illustrates how the generalization of TD and Monte Carlo methods to n-step methods can potentially perform better than either of the two extreme methods. ■

Exercise 7.1 Why do you think a larger random walk task (19 states instead of 5) was used in the examples of this chapter? Would a smaller walk have shifted the advantage to a different value of n? How about the change in left-side outcome from 0 to -1? Would that have made any difference in the best value of n?

Exercise 7.2 Why do you think on-line methods worked better than off-line methods on the example task?

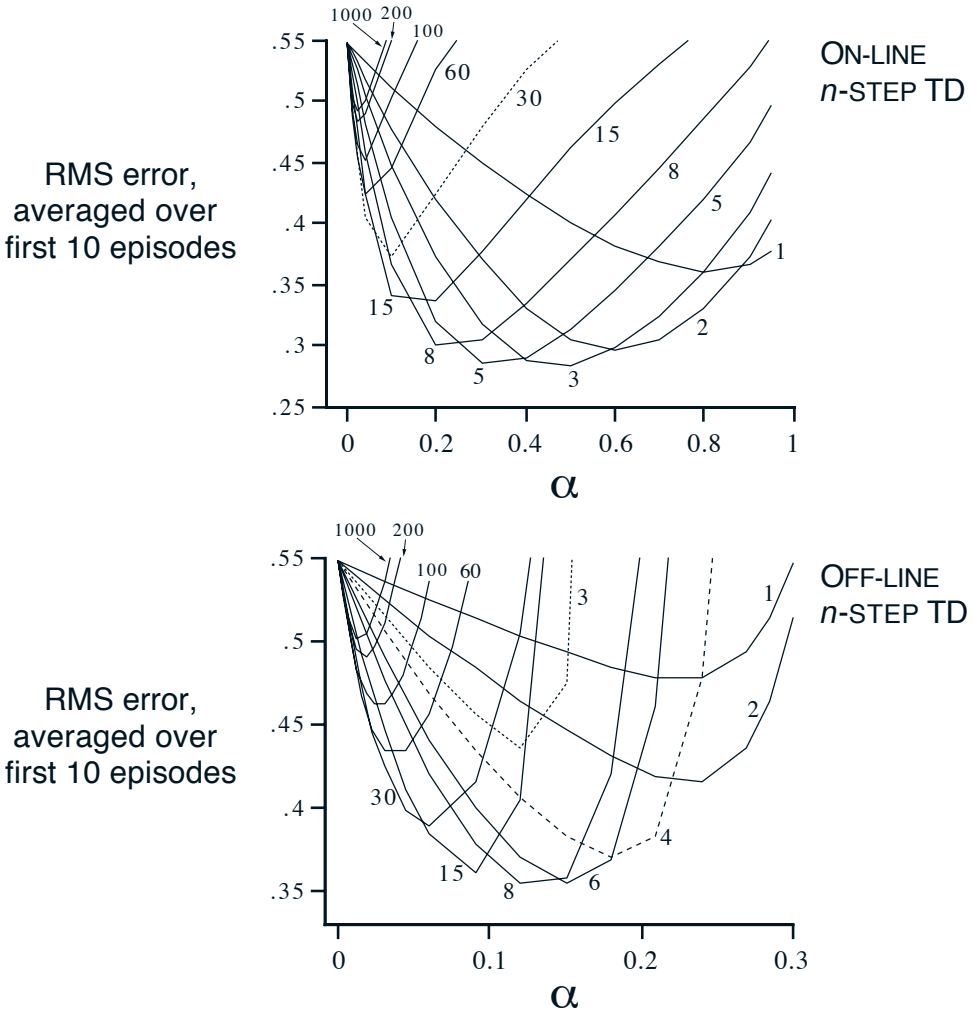


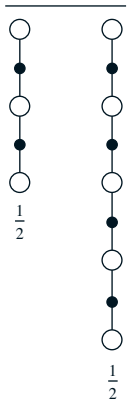
Figure 7.2: Performance of n-step TD methods as a function of α, for various values of n, on a 19-state random walk task. The performance measure shown is the root mean-squared (RMS) error between the true values of states and the values found by the learning methods, averaged over the 19 states, the first 10 trials, and 100 different sequences of walks.

*Exercise 7.3 In the lower part of Figure 7.2, notice that the plot for $n = 3$ is different from the others, dropping to low performance at a much lower value of α than similar methods. In fact, the same was observed for $n = 5$, $n = 7$, and $n = 9$. Can you explain why this might have been so? In fact, we are not sure ourselves. See `http://www.cs.utexas.edu/~ikarpov/Classes/RL/RandomWalk/` for an attempt at a thorough answer by Igor Karpov.

7.2 The Forward View of TD(λ)

Backups can be done not just toward any n-step return, but toward any *average* of n-step returns. For example, a backup can be done toward a return that is half of a two-step return and half of a four-step return: $G_t^{ave} = \frac{1}{2}G_t^{(2)} + \frac{1}{2}G_t^{(4)}$. Any set of returns can be averaged in this way, even an infinite set, as long as the weights on the component returns are positive and sum to 1. The overall return possesses an error reduction property similar to that of individual n-step returns (7.2) and thus can be used to construct backups with guaranteed convergence properties. Averaging produces a substantial new range of algorithms. For example, one could average one-step and infinite-step backups to obtain another way of interrelating TD and Monte Carlo methods. In principle, one could even average experience-based backups with DP backups to get a simple combination of experience-based and model-based methods (see Chapter 8).

A backup that averages simpler component backups in this way is called a *complex backup*. The backup diagram for a complex backup consists of the backup diagrams for each of the component backups with a horizontal line above them and the weighting fractions below. For example, the complex backup mentioned above, mixing half of a two-step backup and half of a four-step backup, has the diagram:



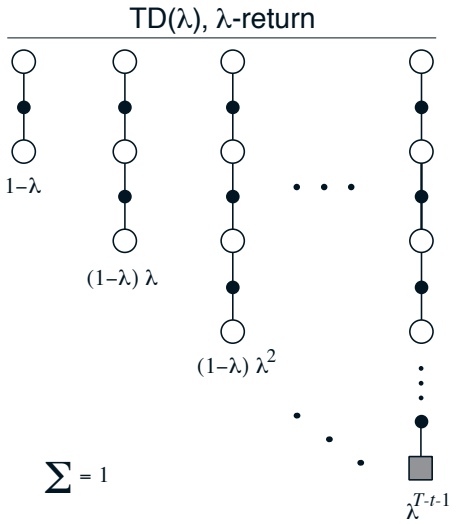


Figure 7.3: The backup digram for TD(λ). If $\lambda = 0$, then the overall backup reduces to its first component, the one-step TD backup, whereas if $\lambda = 1$, then the overall backup reduces to its last component, the Monte Carlo backup.

The TD(λ) algorithm can be understood as one particular way of averaging n-step backups. This average contains all the n-step backups, each weighted proportional to λ^{n-1}, where $0 \leq \lambda \leq 1$ (Figure 7.3). A normalization factor of $1 - \lambda$ ensures that the weights sum to 1. The resulting backup is toward a return, called the λ-*return*, defined by

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}.$$

Figure 7.4 illustrates this weighting sequence. The one-step return is given the largest weight, $1 - \lambda$; the two-step return is given the next largest weight, $(1 - \lambda)\lambda$; the three-step return is given the weight $(1 - \lambda)\lambda^2$; and so on. The weight fades by λ with each additional step. After a terminal state has been reached, all subsequent n-step returns are equal to G_t. If we want, we can separate these terms from the main sum, yielding

$$G_t^\lambda \quad = \quad (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_t^{(n)} \quad + \quad \lambda^{T-t-1} G_t. \tag{7.3}$$

This equation makes it clearer what happens when $\lambda = 1$. In this case the main sum goes to zero, and the remaining term reduces to the conventional return, G_t. Thus, for $\lambda = 1$, backing up according to the λ-return is the same as the Monte Carlo algorithm that we called constant-α MC (6.1) in

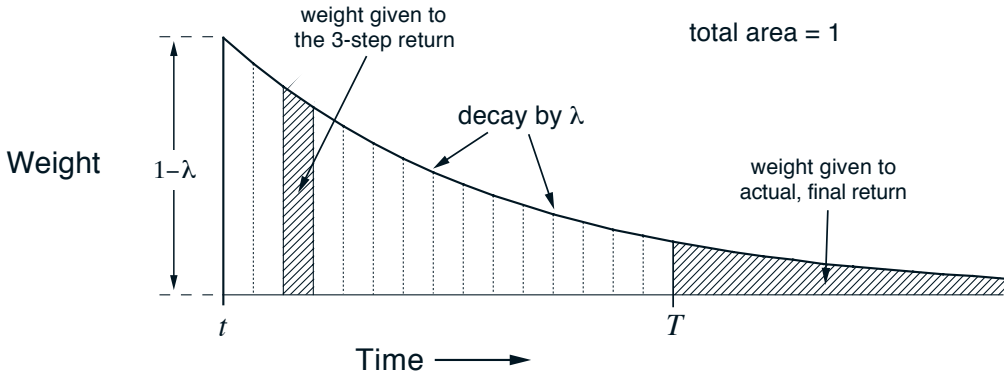


Figure 7.4: Weighting given in the λ-return to each of the n-step returns.

the previous chapter. On the other hand, if $\lambda = 0$, then the λ-return reduces to $G_t^{(1)}$, the one-step return. Thus, for $\lambda = 0$, backing up according to the λ-return is the same as the one-step TD method, TD(0).

We define the *λ-return algorithm* as the algorithm that performs backups using the λ-return. On each step, t, it computes an increment, $\Delta V_t(S_t)$, to the value of the state occurring on that step:

$$\Delta V_t(S_t) = \alpha\Big[G_t^\lambda - V_t(S_t)\Big]. \tag{7.4}$$

(The increments for other states are of course $\Delta V_t(s) = 0$, for all $s \neq S_t$.) As with the n-step TD methods, the updating can be either on-line or off-line.

The approach that we have been taking so far is what we call the theoretical, or *forward*, view of a learning algorithm. For each state visited, we look forward in time to all the future rewards and decide how best to combine them. We might imagine ourselves riding the stream of states, looking forward from each state to determine its update, as suggested by Figure 7.5. After looking forward from and updating one state, we move on to the next and never have to work with the preceding state again. Future states, on the other hand, are viewed and processed repeatedly, once from each vantage point preceding them.

The λ-return algorithm is the basis for the forward view of eligibility traces as used in the TD(λ) method. In fact, we show in a later section that, in the off-line case, the λ-return algorithm *is* the TD(λ) algorithm. The λ-return and TD(λ) methods use the λ parameter to shift from one-step TD methods to Monte Carlo methods. The specific way this shift is done is interesting, but not obviously better or worse than the way it is done with simple n-step methods by varying n. Ultimately, the most compelling motivation for the λ

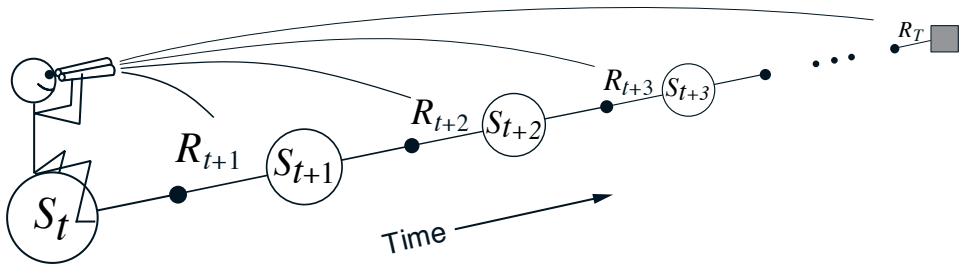


Figure 7.5: The forward or theoretical view. We decide how to update each state by looking forward to future rewards and states.

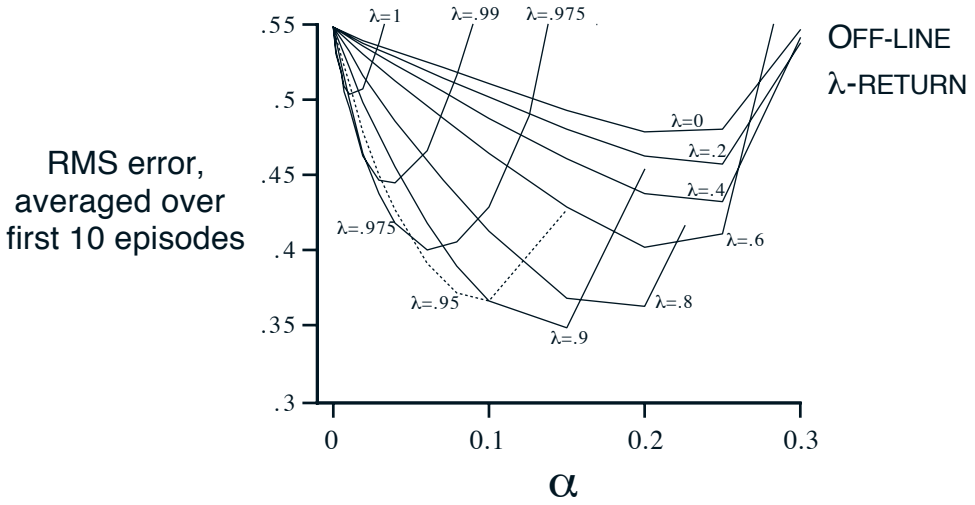


Figure 7.6: Performance of the off-line λ-return algorithm on a 19-state random walk task.

way of mixing n-step backups is that there is a simple algorithm—TD(λ)—for achieving it. This is a mechanism issue rather than a theoretical one. In the next few sections we develop the mechanistic, or backward, view of eligibility traces as used in TD(λ).

Example 7.2: λ-return on the Random Walk Task Figure 7.6 shows the performance of the off-line λ-return algorithm on the 19-state random walk task used with the n-step methods in Example 7.1. The experiment was just as in the n-step case except that here we varied λ instead of n. Note that we get best performance with an intermediate value of λ. ■

Exercise 7.4 The parameter λ characterizes how fast the exponential weighting in Figure 7.4 falls off, and thus how far into the future the λ-return algorithm looks in determining its backup. But a rate factor such as λ is sometimes an awkward way of characterizing the speed of the decay. For some purposes it

is better to specify a time constant, or half-life. What is the equation relating λ and the half-life, τ_λ, the time by which the weighting sequence will have fallen to half of its initial value?

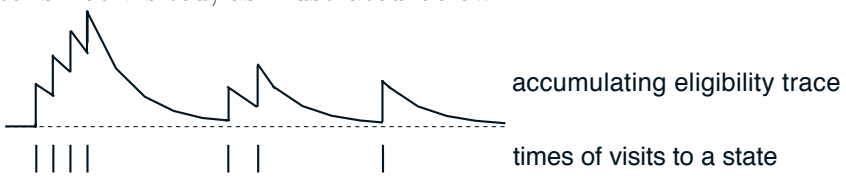
7.3 The Backward View of TD(λ)

In the previous section we presented the forward or theoretical view of the tabular TD(λ) algorithm as a way of mixing backups that parametrically shifts from a TD method to a Monte Carlo method. In this section we instead define TD(λ) mechanistically, and in the next section we show that this mechanism correctly implements the forward view. The mechanistic, or *backward*, view of TD(λ) is useful because it is simple conceptually and computationally. In particular, the forward view itself is not directly implementable because it is *acausal*, using at each step knowledge of what will happen many steps later. The backward view provides a causal, incremental mechanism for approximating the forward view and, in the off-line case, for achieving it exactly.

In the backward view of TD(λ), there is an additional memory variable associated with each state, its *eligibility trace*. The eligibility trace for state s at time t is a random variable denoted $Z_t(s) \in \mathbb{R}^+$. On each step, the eligibility traces for all states decay by $\gamma\lambda$, and the eligibility trace for the one state visited on the step is incremented by 1:

$$Z_t(s) = \begin{cases} \gamma\lambda Z_{t-1}(s) & \text{if } s \neq S_t; \\ \gamma\lambda Z_{t-1}(s) + 1 & \text{if } s = S_t, \end{cases} \tag{7.5}$$

for all nonterminal states s, where γ is the discount rate and λ is the parameter introduced in the previous section. Henceforth we refer to λ as the *trace-decay parameter*. This kind of eligibility trace is called an *accumulating* trace because it accumulates each time the state is visited, then fades away gradually when the state is not visited, as illustrated below:



At any time, the traces record which states have recently been visited, where "recently" is defined in terms of $\gamma\lambda$. The traces are said to indicate the degree to which each state is *eligible* for undergoing learning changes should a reinforcing event occur. The reinforcing events we are concerned with are the moment-by-moment one-step TD errors. For example, the TD error for

Initialize $V(s)$ arbitrarily
Repeat (for each episode):
  Initialize $Z(s) = 0$, for all $s \in \mathcal{S}$
  Initialize S
  Repeat (for each step of episode):
    $A \leftarrow$ action given by π for S
    Take action A, observe reward, R, and next state, S'
    $\delta \leftarrow R + \gamma V(S') - V(S)$
    $Z(S) \leftarrow Z(S) + 1$
    For all $s \in \mathcal{S}$:
      $V(s) \leftarrow V(s) + \alpha \delta Z(s)$
      $Z(s) \leftarrow \gamma \lambda Z(s)$
    $S \leftarrow S'$
  until S is terminal

Figure 7.7: On-line tabular TD(λ).

state-value prediction is

$$\delta_t \;=\; R_{t+1} + \gamma V_t(S_{t+1}) - V_t(S_t). \tag{7.6}$$

In the backward view of TD(λ), the global TD error signal triggers proportional updates to all recently visited states, as signaled by their nonzero traces:

$$\Delta V_t(s) = \alpha \delta_t Z_t(s), \qquad \text{for all } s \in \mathcal{S}. \tag{7.7}$$

As always, these increments could be done on each step to form an on-line algorithm, or saved until the end of the episode to produce an off-line algorithm. In either case, equations (7.5–7.7) provide the mechanistic definition of the TD(λ) algorithm. A complete algorithm for on-line TD(λ) is given in Figure 7.7.

The backward view of TD(λ) is oriented backward in time. At each moment we look at the current TD error and assign it backward to each prior state according to the state's eligibility trace at that time. We might imagine ourselves riding along the stream of states, computing TD errors, and shouting them back to the previously visited states, as suggested by Figure 7.8. Where the TD error and traces come together, we get the update given by (7.7).

To better understand the backward view, consider what happens at various values of λ. If $\lambda = 0$, then by (7.5) all traces are zero at t except for the trace corresponding to S_t. Thus the TD(λ) update (7.7) reduces to the simple TD rule (6.2), which we henceforth call TD(0). In terms of Figure 7.8, TD(0) is the case in which only the one state preceding the current one is changed by

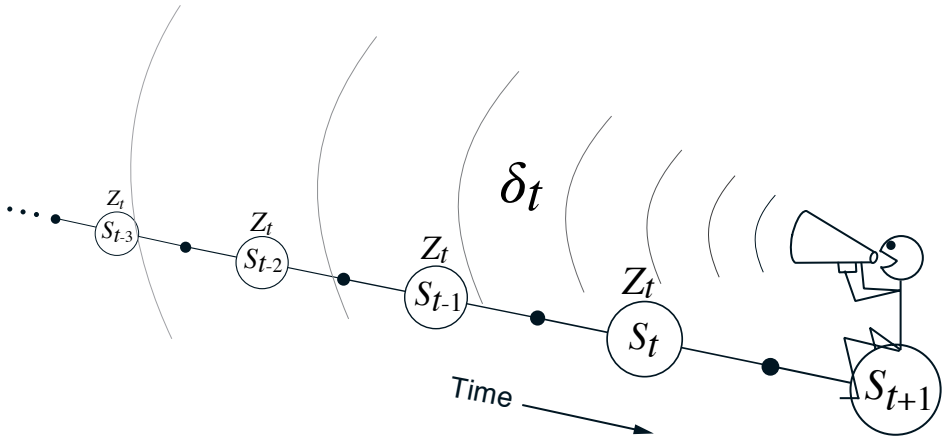


Figure 7.8: The backward or mechanistic view. Each update depends on the current TD error combined with traces of past events.

the TD error. For larger values of λ, but still $\lambda < 1$, more of the preceding states are changed, but each more temporally distant state is changed less because its eligibility trace is smaller, as suggested in the figure. We say that the earlier states are given less *credit* for the TD error.

If $\lambda = 1$, then the credit given to earlier states falls only by γ per step. This turns out to be just the right thing to do to achieve Monte Carlo behavior. For example, remember that the TD error, δ_t, includes an undiscounted term of R_{t+1}. In passing this back k steps it needs to be discounted, like any reward in a return, by γ^k, which is just what the falling eligibility trace achieves. If $\lambda = 1$ and $\gamma = 1$, then the eligibility traces do not decay at all with time. In this case the method behaves like a Monte Carlo method for an undiscounted, episodic task. If $\lambda = 1$, the algorithm is also known as TD(1).

TD(1) is a way of implementing Monte Carlo algorithms that is more general than those presented earlier and that significantly increases their range of applicability. Whereas the earlier Monte Carlo methods were limited to episodic tasks, TD(1) can be applied to discounted continuing tasks as well. Moreover, TD(1) can be performed incrementally and on-line. One disadvantage of Monte Carlo methods is that they learn nothing from an episode until it is over. For example, if a Monte Carlo control method does something that produces a very poor reward but does not end the episode, then the agent's tendency to do that will be undiminished during the episode. On-line TD(1), on the other hand, learns in an n-step TD way from the incomplete ongoing episode, where the n steps are all the way up to the current step. If something unusually good or bad happens during an episode, control methods based on TD(1) can learn immediately and alter their behavior on that same episode.

7.4 Equivalence of Forward and Backward Views

In this section we show that off-line TD(λ), as defined mechanistically above, achieves the same weight updates as the off-line λ-return algorithm. In this sense we align the forward (theoretical) and backward (mechanistic) views of TD(λ). Let $\Delta V_t^\lambda(S_t)$ denote the update at time t of $V(S_t)$ according to the λ-return algorithm (7.4), and let $\Delta V_t^{TD}(s)$ denote the update at time t of state s according to the mechanistic definition of TD(λ) as given by (7.7). Then our goal is to show that the sum of all the updates over an episode is the same for the two algorithms:

$$\sum_{t=0}^{T-1} \Delta V_t^{TD}(s) = \sum_{t=0}^{T-1} \Delta V_t^\lambda(S_t) I_{sS_t}, \qquad \text{for all } s \in \mathcal{S}, \tag{7.8}$$

where I_{sS_t} is an identity indicator function, equal to 1 if $s = S_t$ and equal to 0 otherwise.

First note that an accumulating eligibility trace can be written explicitly (nonrecursively) as

$$Z_t(s) = \sum_{k=0}^{t} (\gamma\lambda)^{t-k} I_{sS_k}.$$

Thus, the left-hand side of (7.8) can be written

$$\begin{aligned}
\sum_{t=0}^{T-1} \Delta V_t^{TD}(s) &= \sum_{t=0}^{T-1} \alpha \delta_t \sum_{k=0}^{t} (\gamma\lambda)^{t-k} I_{sS_k} \\
&= \sum_{k=0}^{T-1} \alpha \sum_{t=0}^{k} (\gamma\lambda)^{k-t} I_{sS_t} \delta_k \\
&= \sum_{t=0}^{T-1} \alpha \sum_{k=t}^{T-1} (\gamma\lambda)^{k-t} I_{sS_t} \delta_k \\
&= \sum_{t=0}^{T-1} \alpha I_{sS_t} \sum_{k=t}^{T-1} (\gamma\lambda)^{k-t} \delta_k.
\end{aligned} \tag{7.9}$$

Now we turn to the right-hand side of (7.8). Consider an individual update of the λ-return algorithm:

$$\frac{1}{\alpha} \Delta V_t^\lambda(S_t) = G_t^\lambda - V_t(S_t)$$

$$\begin{aligned}
&= -V_t(S_t) + (1-\lambda)\lambda^0[R_{t+1} + \gamma V_t(S_{t+1})] \\
&\qquad + (1-\lambda)\lambda^1[R_{t+1} + \gamma R_{t+2} + \gamma^2 V_t(S_{t+2})] \\
&\qquad + (1-\lambda)\lambda^2[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 V_t(S_{t+3})] \\
&\qquad\qquad \vdots \qquad \vdots \qquad \vdots \qquad \ddots
\end{aligned}$$

Examine the first column inside the brackets—all the R_{t+1}'s with their weighting factors of $1-\lambda$ times powers of λ. It turns out that all the weighting factors sum to 1. Thus we can pull out the first column and get an unweighted term of R_{t+1}. A similar trick pulls out the second column in brackets, starting from the second row, which sums to $\gamma\lambda R_{t+2}$. Repeating this for each column, we get

$$\begin{aligned}
\frac{1}{\alpha}\Delta V_t^\lambda(S_t) &= -V_t(S_t) \\
&\quad + (\gamma\lambda)^0 [R_{t+1} + \gamma V_t(S_{t+1}) - \gamma\lambda V_t(S_{t+1})] \\
&\quad + (\gamma\lambda)^1 [R_{t+2} + \gamma V_t(S_{t+2}) - \gamma\lambda V_t(S_{t+2})] \\
&\quad + (\gamma\lambda)^2 [R_{t+3} + \gamma V_t(S_{t+3}) - \gamma\lambda V_t(S_{t+3})] \\
&\quad \vdots \\
&= \quad (\gamma\lambda)^0 [R_{t+1} + \gamma V_t(S_{t+1}) - V_t(S_t)] \\
&\quad + (\gamma\lambda)^1 [R_{t+2} + \gamma V_t(S_{t+2}) - V_t(S_{t+1})] \\
&\quad + (\gamma\lambda)^2 [R_{t+3} + \gamma V_t(S_{t+3}) - V_t(S_{t+2})] \\
&\quad \vdots \\
&\approx \sum_{k=t}^{\infty} (\gamma\lambda)^{k-t}\delta_k \\
&\approx \sum_{k=t}^{T-1} (\gamma\lambda)^{k-t}\delta_k.
\end{aligned}$$

The approximation above is exact in the case of off-line updating, in which case V_t is the same for all t. The last step is exact (not an approximation) because all the δ_k terms omitted are due to fictitious steps "after" the terminal state has been entered. All these steps have zero rewards and zero values; thus all their δ's are zero as well. Thus, we have shown that in the off-line case the right-hand side of (7.8) can be written

$$\sum_{t=0}^{T-1} \Delta V_t^\lambda(S_t) I_{sS_t} = \sum_{t=0}^{T-1} \alpha I_{sS_t} \sum_{k=t}^{T-1} (\lambda\gamma)^{k-t}\delta_k,$$

which is the same as (7.9). This proves (7.8).

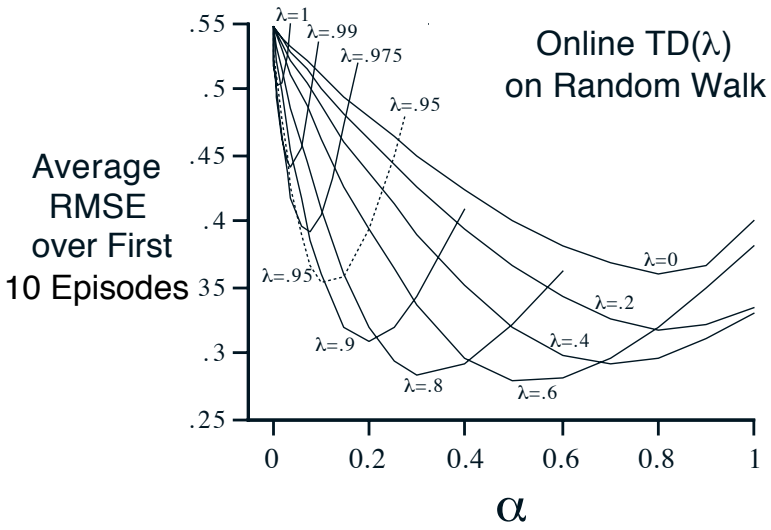


Figure 7.9: Performance of on-line TD(λ) on the 19-state random walk task.

In the case of on-line updating, the approximation made above will be close as long as α is small and thus V changes little during an episode. Even in the on-line case we can expect the updates of TD(λ) and of the λ-return algorithm to be similar.

For the moment let us assume that the increments are small enough during an episode that on-line TD(λ) gives essentially the same update over the course of an episode as does the λ-return algorithm. There still remain interesting questions about what happens *during* an episode. Consider the updating of the value of state S_t in midepisode, at time $t + k$. Under on-line TD(λ), the effect at $t + k$ is just as if we had done a λ-return update treating the last observed state as the terminal state of the episode with a nonzero terminal value equal to its current estimated value. This relationship is maintained from step to step as each new state is observed.

Example 7.3: Random Walk with TD(λ) Because off-line TD(λ) is equivalent to the λ-return algorithm, we already have the results for off-line TD(λ) on the 19-state random walk task; they are shown in Figure 7.6. The comparable results for on-line TD(λ) are shown in Figure 7.9. Note that the on-line algorithm works better over a broader range of parameters. This is often found to be the case for on-line methods. ■

*__Exercise 7.5__ Although TD(λ) only approximates the λ-return algorithm when done online, perhaps there's a slightly different TD method that would maintain the equivalence even in the on-line case. One idea is to define the TD error instead as $\delta_t = R_{t+1} + \gamma V_t(S_{t+1}) - V_{t-1}(S_t)$ and the n-step return as

$R_t^{(n)} = R_{t+1} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n})$. Show that in this case the modified TD(λ) algorithm would then achieve exactly

$$\Delta V_t(S_t) = \alpha \Big[G_t^\lambda - V_{t-1}(S_t) \Big],$$

even in the case of on-line updating with large α. In what ways might this modified TD(λ) be better or worse than the conventional one described in the text? Describe an experiment to assess the relative merits of the two algorithms.

7.5 Sarsa(λ)

How can eligibility traces be used not just for prediction, as in TD(λ), but for control? As usual, the main idea of one popular approach is simply to learn action values, $q_t(s,a)$, rather than state values, $V_t(s)$. In this section we show how eligibility traces can be combined with Sarsa in a straightforward way to produce an on-policy TD control method. The eligibility trace version of Sarsa we call *Sarsa*(λ), and the original version presented in the previous chapter we henceforth call *one-step Sarsa*.

The idea in Sarsa(λ) is to apply the TD(λ) prediction method to state–action pairs rather than to states. Obviously, then, we need a trace not just for each state, but for each state–action pair. Let $Z_t(s,a)$ denote the trace for state–action pair s, a. Otherwise the method is just like TD(λ), substituting state–action variables for state variables—$Q_t(s,a)$ for $V_t(s)$ and $Z_t(s,a)$ for $Z_t(s)$:

$$Q_{t+1}(s,a) = Q_t(s,a) + \alpha \delta_t Z_t(s,a), \qquad \text{for all } s, a$$

where

$$\delta_t = R_{t+1} + \gamma Q_t(S_{t+1}, A_{t+1}) - Q_t(S_t, A_t)$$

and

$$Z_t(s,a) = \begin{cases} \gamma\lambda Z_{t-1}(s,a) + 1 & \text{if } s = S_t \text{ and } a = A_t; \\ \gamma\lambda Z_{t-1}(s,a) & \text{otherwise.} \end{cases} \qquad \text{for all } s, a \quad (7.10)$$

Figure 7.10 shows the backup diagram for Sarsa(λ). Notice the similarity to the diagram of the TD(λ) algorithm (Figure 7.3). The first backup looks ahead one full step, to the next state–action pair, the second looks ahead two steps,

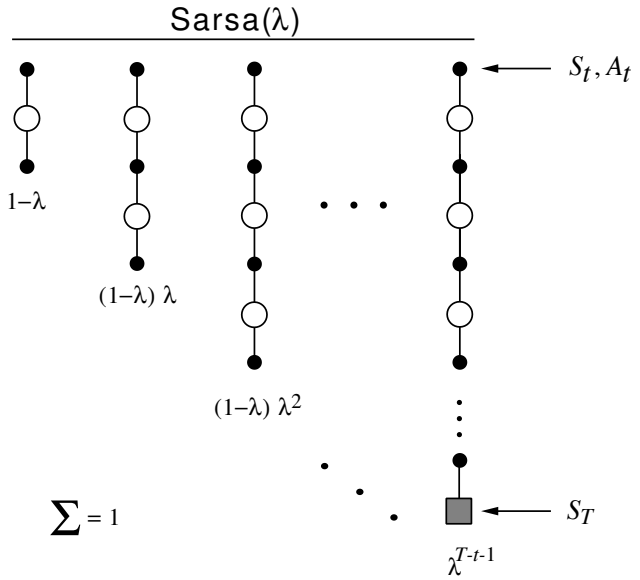


Figure 7.10: Sarsa(λ)'s backup diagram.

and so on. A final backup is based on the complete return. The weighting of each backup is just as in TD(λ) and the λ-return algorithm.

One-step Sarsa and Sarsa(λ) are on-policy algorithms, meaning that they approximate $q_\pi(s, a)$, the action values for the current policy, π, then improve the policy gradually based on the approximate values for the current policy. The policy improvement can be done in many different ways, as we have seen throughout this book. For example, the simplest approach is to use the ε-greedy policy with respect to the current action-value estimates. Figure 7.11 shows the complete Sarsa(λ) algorithm for this case.

Example 7.4: Traces in Gridworld The use of eligibility traces can substantially increase the efficiency of control algorithms. The reason for this is illustrated by the gridworld example in Figure 7.12. The first panel shows the path taken by an agent in a single episode, ending at a location of high reward, marked by the *. In this example the values were all initially 0, and all rewards were zero except for a positive reward at the * location. The arrows in the other two panels show which action values were strengthened as a result of this path by one-step Sarsa and Sarsa(λ) methods. The one-step method strengthens only the last action of the sequence of actions that led to the high reward, whereas the trace method strengthens many actions of the sequence. The degree of strengthening (indicated by the size of the arrows) falls off (according to $\gamma\lambda$) with steps from the reward. In this example, $\gamma = 1$ and $\lambda = 0.9$. ■

Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
Repeat (for each episode):
  $Z(s,a) = 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
  Initialize S, A
  Repeat (for each step of episode):
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $\delta \leftarrow R + \gamma Q(S', A') - Q(S, A)$
    $Z(S, A) \leftarrow Z(S, A) + 1$
    For all $s \in \mathcal{S}, a \in \mathcal{A}(s)$:
      $Q(s,a) \leftarrow Q(s,a) + \alpha \delta Z(s,a)$
      $Z(s,a) \leftarrow \gamma \lambda Z(s,a)$
    $S \leftarrow S'$; $A \leftarrow A'$
  until S is terminal

Figure 7.11: Tabular Sarsa(λ).

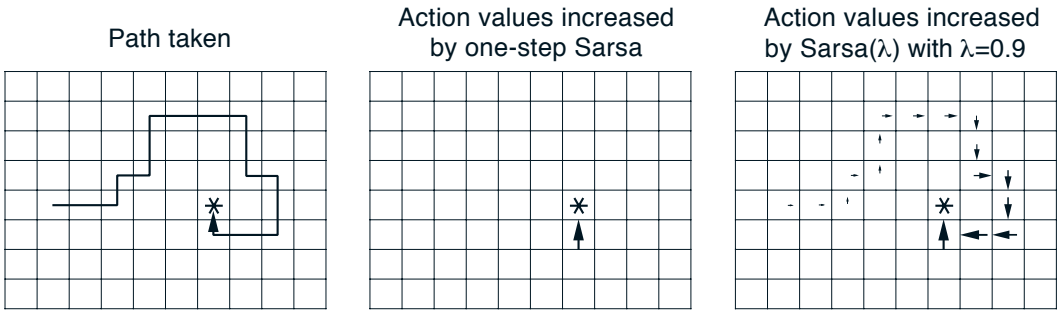


Figure 7.12: Gridworld example of the speedup of policy learning due to the use of eligibility traces.

7.6 Q(λ)

Two different methods have been proposed that combine eligibility traces and Q-learning; we call them *Watkins's* $Q(\lambda)$ and *Peng's* $Q(\lambda)$, after the researchers who first proposed them. First we describe Watkins's Q(λ).

Recall that Q-learning is an off-policy method, meaning that the policy learned about need not be the same as the one used to select actions. In particular, Q-learning learns about the greedy policy while it typically follows a policy involving exploratory actions—occasional selections of actions that are suboptimal according to Q. Because of this, special care is required when introducing eligibility traces.

Suppose we are backing up the state–action pair S_t, A_t at time t. Suppose that on the next two time steps the agent selects the greedy action, but on the third, at time $t+3$, the agent selects an exploratory, nongreedy action. In learning about the value of the greedy policy at S_t, A_t we can use subsequent experience only as long as the greedy policy is being followed. Thus, we can use the one-step and two-step returns, but not, in this case, the three-step return. The n-step returns for all $n \geq 3$ no longer have any necessary relationship to the greedy policy.

Thus, unlike TD(λ) or Sarsa(λ), Watkins's Q(λ) does not look ahead all the way to the end of the episode in its backup. It only looks ahead as far as the next exploratory action. Aside from this difference, however, Watkins's Q(λ) is much like TD(λ) and Sarsa(λ). Their lookahead stops at episode's end, whereas Q(λ)'s lookahead stops at the first exploratory action, or at episode's end if there are no exploratory actions before that. Actually, to be more precise, one-step Q-learning and Watkins's Q(λ) both look one action *past* the first exploration, using their knowledge of the action values. For example, suppose the first action, A_{t+1}, is exploratory. Watkins's Q(λ) would still do the one-step update of $Q_t(S_t, A_t)$ toward $R_{t+1} + \gamma \max_a Q_t(S_{t+1}, a)$. In general, if A_{t+n} is the first exploratory action, then the longest backup is toward

$$R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \max_a Q_t(S_{t+n}, a),$$

where we assume off-line updating. The backup diagram in Figure 7.13 illustrates the forward view of Watkins's Q(λ), showing all the component backups.

The mechanistic or backward view of Watkins's Q(λ) is also very simple. Eligibility traces are used just as in Sarsa(λ), except that they are set to zero whenever an exploratory (nongreedy) action is taken. The trace update is best thought of as occurring in two steps. First, the traces for all state–action pairs are either decayed by $\gamma\lambda$ or, if an exploratory action was taken, set to 0.

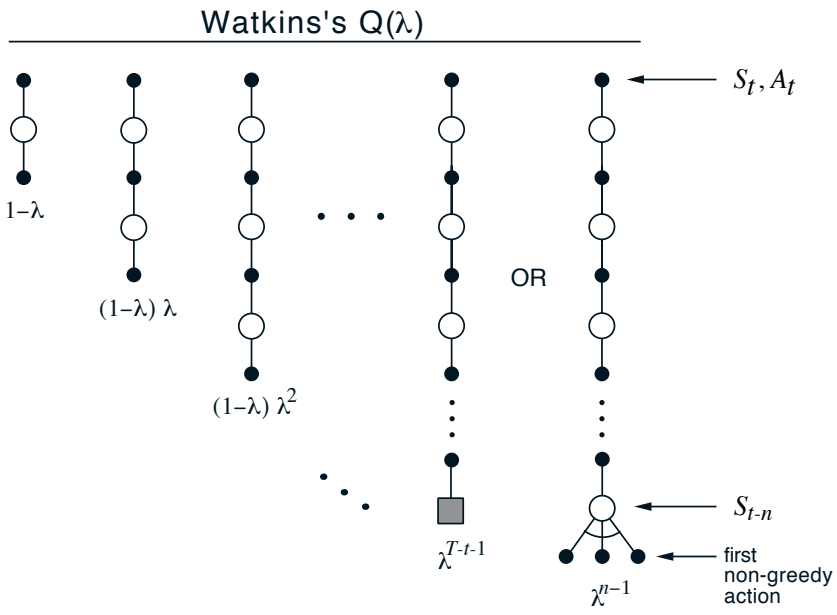


Figure 7.13: The backup diagram for Watkins's Q(λ). The series of component backups ends either with the end of the episode or with the first nongreedy action, whichever comes first.

Second, the trace corresponding to the current state and action is incremented by 1. The overall result is

$$Z_t(s,a) = I_{sS_t} \cdot I_{aA_t} + \begin{cases} \gamma\lambda Z_{t-1}(s,a) & \text{if } Q_{t-1}(S_t,A_t) = \max_a Q_{t-1}(S_t,a); \\ 0 & \text{otherwise,} \end{cases}$$

where, as before, I_{xy} is an identity indicator function, equal to 1 if $x = y$ and 0 otherwise. The rest of the algorithm is defined by

$$Q_{t+1}(s,a) = Q_t(s,a) + \alpha\delta_t Z_t(s,a), \quad \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$$

where

$$\delta_t = R_{t+1} + \gamma \max_{a'} Q_t(S_{t+1},a') - Q_t(S_t,A_t).$$

Figure 7.14 shows the complete algorithm in pseudocode.

Unfortunately, cutting off traces every time an exploratory action is taken loses much of the advantage of using eligibility traces. If exploratory actions are frequent, as they often are early in learning, then only rarely will backups of more than one or two steps be done, and learning may be little faster than

Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
Repeat (for each episode):
  $Z(s,a) = 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
  Initialize S, A
  Repeat (for each step of episode):
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $A^* \leftarrow \arg\max_a Q(S',a)$ (if A' ties for the max, then $A^* \leftarrow A'$)
    $\delta \leftarrow R + \gamma Q(S',A^*) - Q(S,A)$
    $Z(S,A) \leftarrow Z(S,A) + 1$
    For all $s \in \mathcal{S}, a \in \mathcal{A}(s)$:
      $Q(s,a) \leftarrow Q(s,a) + \alpha\delta Z(s,a)$
      If $A' = A^*$, then $Z(s,a) \leftarrow \gamma\lambda Z(s,a)$
                    else $Z(s,a) \leftarrow 0$
    $S \leftarrow S'$; $A \leftarrow A'$
  until S is terminal

Figure 7.14: Tabular version of Watkins's Q(λ) algorithm.

one-step Q-learning. Peng's Q(λ) is an alternate version of Q(λ) meant to remedy this. Peng's Q(λ) can be thought of as a hybrid of Sarsa(λ) and Watkins's Q(λ).

Conceptually, Peng's Q(λ) uses the mixture of backups shown in Figure 7.15. Unlike Q-learning, there is no distinction between exploratory and greedy actions. Each component backup is over many steps of actual experiences, and all but the last are capped by a final maximization over actions. The component backups, then, are neither on-policy nor off-policy. The earlier transitions of each are on-policy, whereas the last (fictitious) transition uses the greedy policy. As a consequence, for a fixed nongreedy policy, Q_t converges to neither q_π nor q_* under Peng's Q(λ), but to some hybrid of the two. However, if the policy is gradually made more greedy, then the method may still converge to q_*. As of this writing this has not yet been proved. Nevertheless, the method performs well empirically. Most studies have shown it performing significantly better than Watkins's Q(λ) and almost as well as Sarsa(λ).

On the other hand, Peng's Q(λ) cannot be implemented as simply as Watkins's Q(λ). For a complete description of the needed implementation, see Peng and Williams (1994, 1996). One could imagine yet a third version of Q(λ), let us call it *naive* $Q(\lambda)$, that is just like Watkins's Q(λ) except that the traces are not set to zero on exploratory actions. This method might have some of the advantages of Peng's Q(λ), but without the complex implementation.

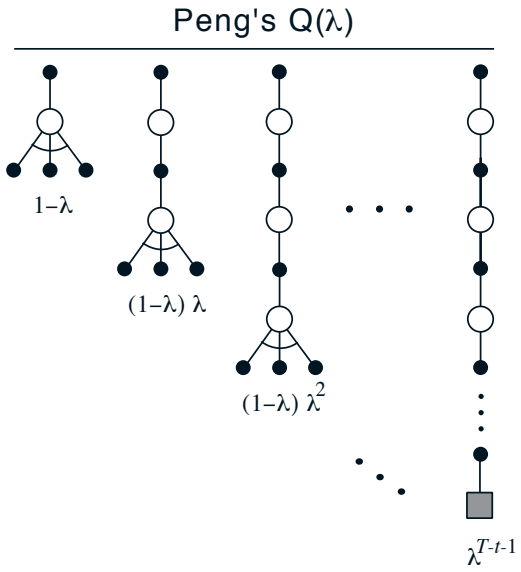


Figure 7.15: The backup diagram for Peng's Q(λ).

7.7 Replacing Traces

In some cases significantly better performance can be obtained by using a slightly modified kind of trace known as a *replacing trace*. Suppose a state is visited and then revisited before the trace due to the first visit has fully decayed to zero. With accumulating traces (7.5), the revisit causes a further increment in the trace, driving it greater than 1, whereas with replacing traces, the trace is reset to 1. Figure 7.16 contrasts these two kinds of traces. Formally, a replacing trace for a discrete state s is defined by

$$Z_t(s) = \begin{cases} \gamma\lambda Z_{t-1}(s) & \text{if } s \neq S_t; \\ 1 & \text{if } s = S_t. \end{cases} \tag{7.11}$$

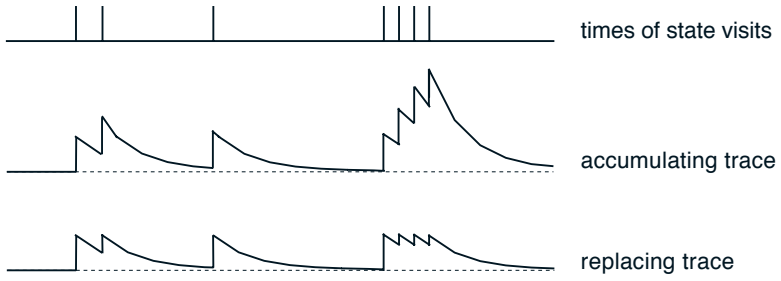


Figure 7.16: Accumulating and replacing traces.

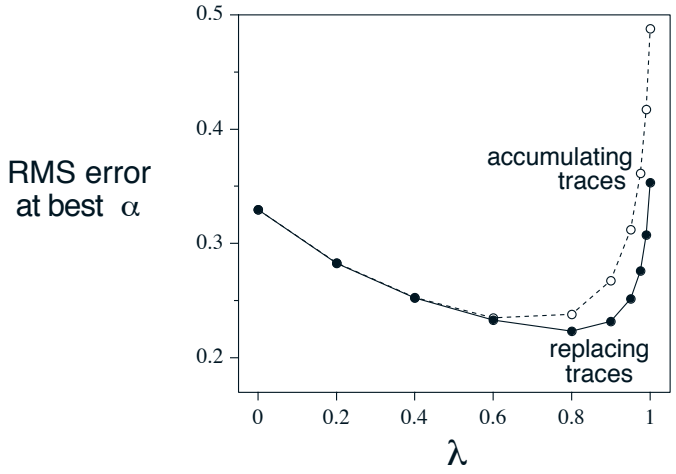


Figure 7.17: Error as a function of λ on a 19-state random walk task. These data are using the best value of α for each value of λ. The error is averaged over all 19 states and the first 20 trials of 100 different runs.

Prediction or control algorithms using replacing traces are often called *replace-trace* methods. Although replacing traces are only slightly different from accumulating traces, they can produce a significant improvement in learning rate. Figure 7.17 compares the performance of conventional and replace-trace versions of TD(λ) on the 19-state random walk prediction task. Other examples for a slightly more general case are given in Figure 9.10.

Example 7.5 Figure 7.18 shows an example of the kind of task that is difficult for control methods using accumulating eligibility traces. All rewards are zero except on entering the terminal state, which produces a reward of +1. From each state, selecting the `right` action brings the agent one step closer to the terminal reward, whereas the `wrong` (upper) action leaves it in the same state to try again. The full sequence of states is long enough that one would like to use long traces to get the fastest learning. However, problems occur if long accumulating traces are used. Suppose, on the first episode, at some state, s, the agent by chance takes the `wrong` action a few times before taking the `right` action. As the agent continues, the trace $Z(s, \texttt{wrong})$ is likely to be larger than the trace $Z(s, \texttt{right})$. The `right` action was more recent, but the `wrong` action was selected more times. When reward is finally received, then, the value for the wrong action is likely to go up more than the value for the right action. On the next episode the agent will be even more likely to go the `wrong` way many times before going `right`, making it even more likely that the `wrong` action will have the larger trace. Eventually, all of this will be corrected, but learning is significantly slowed. With replacing traces, on the other hand, this problem never occurs. No matter how many times the `wrong`

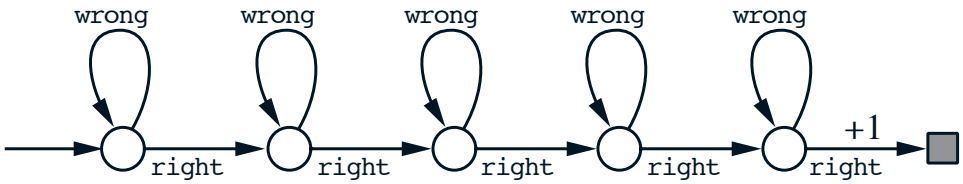


Figure 7.18: A simple task that causes problems for control methods using accumulating traces.

action is taken, its eligibility trace is always less than that for the `right` action after the `right` action has been taken. ■

There is an interesting relationship between replace-trace methods and Monte Carlo methods in the undiscounted case. Just as conventional TD(1) is related to the every-visit MC algorithm, so replace-trace TD(1) is related to the first-visit MC algorithm. In particular, the off-line version of replace-trace TD(1) is formally identical to first-visit MC (Singh and Sutton, 1996). How, or even whether, these methods and results extend to the discounted case is unknown.

There are several possible ways to generalize replacing eligibility traces for use in control methods. Obviously, when a state is revisited and a new action is selected, the trace for that action should be reset to 1. But what of the traces for the other actions for that state? The approach recommended by Singh and Sutton (1996) is to set the traces of all the other actions from the revisited state to 0. In this case, the state–action traces are updated by the following instead of (7.10):

$$Z_t(s,a) = \begin{cases} 1 & \text{if } s = S_t \text{ and } a = A_t; \\ 0 & \text{if } s = S_t \text{ and } a \neq A_t; \\ \gamma\lambda Z_{t-1}(s,a) & \text{if } s \neq S_t. \end{cases} \qquad \text{for all } s, a \qquad (7.12)$$

Note that this variant of replacing traces works out even better than the original replacing traces in the example task. Once the `right` action has been selected, the `wrong` action is left with no trace at all. The results shown in Figure 9.10 were obtained using this kind of replacing trace.

Exercise 7.6 In Example 7.5, suppose from state s the `wrong` action is taken twice before the `right` action is taken. If accumulating traces are used, then how big must the trace parameter λ be in order for the `wrong` action to end up with a larger eligibility trace than the `right` action?

Exercise 7.7 (programming) Program Example 7.5 and compare accumulate-trace and replace-trace versions of Sarsa(λ) on it, for $\lambda = 0.9$ and a range of α

values. Can you empirically demonstrate the claimed advantage of replacing traces on this example?

*__Exercise 7.8__ Draw a backup diagram for Sarsa(λ) with replacing traces.

7.8 Implementation Issues

It might at first appear that methods using eligibility traces are much more complex than one-step methods. A naive implementation would require every state (or state–action pair) to update both its value estimate and its eligibility trace on every time step. This would not be a problem for implementations on single-instruction, multiple-data parallel computers or in plausible neural implementations, but it is a problem for implementations on conventional serial computers. Fortunately, for typical values of λ and γ the eligibility traces of almost all states are almost always nearly zero; only those that have recently been visited will have traces significantly greater than zero. Only these few states really need to be updated because the updates at the others will have essentially no effect.

In practice, then, implementations on conventional computers keep track of and update only the few states with nonzero traces. Using this trick, the computational expense of using traces is typically a few times that of a one-step method. The exact multiple of course depends on λ and γ and on the expense of the other computations. Cichosz (1995) has demonstrated a further implementation technique that further reduces complexity to a constant independent of λ and γ. Finally, it should be noted that the tabular case is in some sense a worst case for the computational complexity of traces. When function approximation is used (Chapter 9), the computational advantages of not using traces generally decrease. For example, if artificial neural networks and backpropagation are used, then traces generally cause only a doubling of the required memory and computation per step.

__Exercise 7.9__ Write pseudocode for an implementation of TD(λ) that updates only value estimates for states whose traces are greater than some small positive constant.

*7.9 Variable λ

The λ-return can be significantly generalized beyond what we have described so far by allowing λ to vary from step to step, that is, by redefining the trace

update as

$$Z_t(s) = \begin{cases} \gamma\lambda_t Z_{t-1}(s) & \text{if } s \neq S_t; \\ \gamma\lambda_t Z_{t-1}(s) + 1 & \text{if } s = S_t, \end{cases}$$

where λ_t denotes the value of λ at time t. This is an advanced topic because the added generality has never been used in practical applications, but it is interesting theoretically and may yet prove useful. For example, one idea is to vary λ as a function of state: $\lambda_t = \lambda(S_t)$. If a state's value estimate is believed to be known with high certainty, then it makes sense to use that estimate fully, ignoring whatever states and rewards are received after it. This corresponds to cutting off all the traces once this state has been reached, that is, to choosing the λ for the certain state to be zero or very small. Similarly, states whose value estimates are highly uncertain, perhaps because even the state estimate is unreliable, can be given λs near 1. This causes their estimated values to have little effect on any updates. They are "skipped over" until a state that is known better is encountered. Some of these ideas were explored formally by Sutton and Singh (1994).

The eligibility trace equation above is the backward view of variable λs. The corresponding forward view is a more general definition of the λ-return:

$$\begin{aligned} G_t^\lambda &= \sum_{n=1}^{\infty} G_t^{(n)} (1 - \lambda_{t+n}) \prod_{i=t+1}^{t+n-1} \lambda_i \\ &= \sum_{k=t+1}^{T-1} G_t^{(k-t)} (1 - \lambda_k) \prod_{i=t+1}^{k-1} \lambda_i \quad + \quad G_t \prod_{i=t+1}^{T-1} \lambda_i. \end{aligned}$$

*__Exercise 7.10__ Prove that the forward and backward views of off-line TD(λ) remain equivalent under their new definitions with variable λ given in this section. Follow the example of the proof in Section 7.4.

7.10 Conclusions

Eligibility traces in conjunction with TD errors provide an efficient, incremental way of shifting and choosing between Monte Carlo and TD methods. Traces can be used without TD errors to achieve a similar effect, but only awkwardly. A method such as TD(λ) enables this to be done from partial experiences and with little memory and little nonmeaningful variation in predictions.

As we mentioned in Chapter 5, Monte Carlo methods may have advantages in non-Markov tasks because they do not bootstrap. Because eligibility

traces make TD methods more like Monte Carlo methods, they also can have advantages in these cases. If one wants to use TD methods because of their other advantages, but the task is at least partially non-Markov, then the use of an eligibility trace method is indicated. Eligibility traces are the first line of defense against both long-delayed rewards and non-Markov tasks.

By adjusting λ, we can place eligibility trace methods anywhere along a continuum from Monte Carlo to one-step TD methods. Where shall we place them? We do not yet have a good theoretical answer to this question, but a clear empirical answer appears to be emerging. On tasks with many steps per episode, or many steps within the half-life of discounting, it appears significantly better to use eligibility traces than not to (e.g., see Figure 9.10). On the other hand, if the traces are so long as to produce a pure Monte Carlo method, or nearly so, then performance degrades sharply. An intermediate mixture appears to be the best choice. Eligibility traces should be used to bring us toward Monte Carlo methods, but not all the way there. In the future it may be possible to vary the trade-off between TD and Monte Carlo methods more finely by using variable λ, but at present it is not clear how this can be done reliably and usefully.

Methods using eligibility traces require more computation than one-step methods, but in return they offer significantly faster learning, particularly when rewards are delayed by many steps. Thus it often makes sense to use eligibility traces when data are scarce and cannot be repeatedly processed, as is often the case in on-line applications. On the other hand, in off-line applications in which data can be generated cheaply, perhaps from an inexpensive simulation, then it often does not pay to use eligibility traces. In these cases the objective is not to get more out of a limited amount of data, but simply to process as much data as possible as quickly as possible. In these cases the speedup per datum due to traces is typically not worth their computational cost, and one-step methods are favored.

7.11 Bibliographical and Historical Remarks

7.1–2 The forward view of eligibility traces in terms of n-step returns and the λ-return is due to Watkins (1989), who also first discussed the error reduction property of n-step returns. Our presentation is based on the slightly modified treatment by Jaakkola, Jordan, and Singh (1994). The results in the random walk examples were made for this text based on work of Sutton (1988) and Singh and Sutton (1996). The use of backup diagrams to describe these and other algorithms in this chapter is new, as are the terms "forward view" and "backward view."

TD(λ) was proved to converge in the mean by Dayan (1992), and with probability 1 by many researchers, including Peng (1993), Dayan and Sejnowski (1994), and Tsitsiklis (1994). Jaakkola, Jordan, and Singh (1994), in addition, first proved convergence of TD(λ) under on-line updating. Gurvits, Lin, and Hanson (1994) proved convergence of a more general class of eligibility trace methods.

7.3 The idea that stimuli produce aftereffects in the nervous system that are important for learning is very old. Animal learning psychologists at least as far back as Pavlov (1927) and Hull (1943, 1952) included such ideas in their theories. However, stimulus traces in these theories are more like transient state representations than what we are calling eligibility traces: they could be associated with actions, whereas an eligibility trace is used only for credit assignment. The idea of a stimulus trace serving exclusively for credit assignment is apparently due to Klopf (1972), who hypothesized that under certain conditions a neuron's synapses would become "eligible" for subsequent modification should reinforcement later arrive at the neuron. Our use of eligibility traces was based on Klopf's work (Sutton, 1978a, 1978b, 1978c; Barto and Sutton, 1981a, 1981b; Sutton and Barto, 1981a; Barto, Sutton, and Anderson, 1983; Sutton, 1984). The TD(λ) algorithm is due to Sutton (1988).

7.4 The equivalence of forward and backward views, and the relationships to Monte Carlo methods, were proved by Sutton (1988) for undiscounted episodic tasks, then extended by Watkins (1989) to the general case. The idea in exercise 7.5 is new.

7.5 Sarsa(λ) was first explored as a control method by Rummery and Niranjan (1994) and Rummery (1995).

7.6 Watkins's Q(λ) is due to Watkins (1989). Peng's Q(λ) is due to Peng and Williams (Peng, 1993; Peng and Williams, 1994, 1996). Rummery (1995) made extensive comparative studies of these algorithms.

Convergence has not been proved for any control method for $0 < \lambda < 1$.

7.7 Replacing traces are due to Singh and Sutton (1996). The results in Figure 7.17 are from their paper. The task in Figure 7.18 was used to show the weakness of accumulating traces by Sutton (1984). The relationship of both kinds of traces to specific Monte Carlo methods was developed by Singh and Sutton (1996).

7.8-9 The ideas in these two sections were generally known for many years, but beyond what is in the sources cited in the sections themselves, this text may be the first place they have been described. Perhaps the first published discussion of variable λ was by Watkins (1989), who pointed out that the cutting off of the backup sequence (Figure 7.13) in his $Q(\lambda)$ when a nongreedy action was selected could be implemented by temporarily setting λ to 0.

Chapter 8

Planning and Learning with Tabular Methods

In this chapter we develop a unified view of methods that require a model of the environment, such as dynamic programming and heuristic search, and methods that can be used without a model, such as Monte Carlo and temporal-difference methods. We think of the former as *planning* methods and of the latter as *learning* methods. Although there are real differences between these two kinds of methods, there are also great similarities. In particular, the heart of both kinds of methods is the computation of value functions. Moreover, all the methods are based on looking ahead to future events, computing a backed-up value, and then using it to update an approximate value function. Earlier in this book we presented Monte Carlo and temporal-difference methods as distinct alternatives, then showed how they can be seamlessly integrated by using eligibility traces such as in TD(λ). Our goal in this chapter is a similar integration of planning and learning methods. Having established these as distinct in earlier chapters, we now explore the extent to which they can be intermixed.

8.1 Models and Planning

By a *model* of the environment we mean anything that an agent can use to predict how the environment will respond to its actions. Given a state and an action, a model produces a prediction of the resultant next state and next reward. If the model is stochastic, then there are several possible next states and next rewards, each with some probability of occurring. Some models produce a description of all possibilities and their probabilities; these we call *distribution models*. Other models produce just one of the possibilities, sampled

according to the probabilities; these we call *sample models*. For example, consider modeling the sum of a dozen dice. A distribution model would produce all possible sums and their probabilities of occurring, whereas a sample model would produce an individual sum drawn according to this probability distribution. The kind of model assumed in dynamic programming—estimates of the state transition probabilities and expected rewards, $p(s'|s,a)$ and $r(s,a,s')$—is a distribution model. The kind of model used in the blackjack example in Chapter 5 is a sample model. Distribution models are stronger than sample models in that they can always be used to produce samples. However, in surprisingly many applications it is much easier to obtain sample models than distribution models.

Models can be used to mimic or simulate experience. Given a starting state and action, a sample model produces a possible transition, and a distribution model generates all possible transitions weighted by their probabilities of occurring. Given a starting state and a policy, a sample model could produce an entire episode, and a distribution model could generate all possible episodes and their probabilities. In either case, we say the model is used to *simulate* the environment and produce *simulated experience*.

The word *planning* is used in several different ways in different fields. We use the term to refer to any computational process that takes a model as input and produces or improves a policy for interacting with the modeled environment:



Within artificial intelligence, there are two distinct approaches to planning according to our definition. In *state-space planning*, which includes the approach we take in this book, planning is viewed primarily as a search through the state space for an optimal policy or path to a goal. Actions cause transitions from state to state, and value functions are computed over states. In what we call *plan-space planning*, planning is instead viewed as a search through the space of plans. Operators transform one plan into another, and value functions, if any, are defined over the space of plans. Plan-space planning includes evolutionary methods and *partial-order planning*, a popular kind of planning in artificial intelligence in which the ordering of steps is not completely determined at all stages of planning. Plan-space methods are difficult to apply efficiently to the stochastic optimal control problems that are the focus in reinforcement learning, and we do not consider them further (but see Section 15.6 for one application of reinforcement learning within plan-space planning).

The unified view we present in this chapter is that all state-space planning methods share a common structure, a structure that is also present in the

learning methods presented in this book. It takes the rest of the chapter to develop this view, but there are two basic ideas: (1) all state-space planning methods involve computing value functions as a key intermediate step toward improving the policy, and (2) they compute their value functions by backup operations applied to simulated experience. This common structure can be diagrammed as follows:



Dynamic programming methods clearly fit this structure: they make sweeps through the space of states, generating for each state the distribution of possible transitions. Each distribution is then used to compute a backed-up value and update the state's estimated value. In this chapter we argue that various other state-space planning methods also fit this structure, with individual methods differing only in the kinds of backups they do, the order in which they do them, and in how long the backed-up information is retained.

Viewing planning methods in this way emphasizes their relationship to the learning methods that we have described in this book. The heart of both learning and planning methods is the estimation of value functions by backup operations. The difference is that whereas planning uses simulated experience generated by a model, learning methods use real experience generated by the environment. Of course this difference leads to a number of other differences, for example, in how performance is assessed and in how flexibly experience can be generated. But the common structure means that many ideas and algorithms can be transferred between planning and learning. In particular, in many cases a learning algorithm can be substituted for the key backup step of a planning method. Learning methods require only experience as input, and in many cases they can be applied to simulated experience just as well as to real experience. Figure 8.1 shows a simple example of a planning method based on one-step tabular Q-learning and on random samples from a sample model. This method, which we call *random-sample one-step tabular Q-planning*, converges to the optimal policy for the model under the same conditions that one-step tabular Q-learning converges to the optimal policy for the real environment (each state–action pair must be selected an infinite number of times in Step 1, and α must decrease appropriately over time).

In addition to the unified view of planning and learning methods, a second theme in this chapter is the benefits of planning in small, incremental steps. This enables planning to be interrupted or redirected at any time with little wasted computation, which appears to be a key requirement for efficiently intermixing planning with acting and with learning of the model. More surprisingly, later in this chapter we present evidence that planning in very small

Do forever:
1. Select a state, $S \in \mathcal{S}$, and an action, $A \in \mathcal{A}(s)$, at random
2. Send S, A to a sample model, and obtain
   a sample next reward, R, and a sample next state, S'
3. Apply one-step tabular Q-learning to S, A, R, S':
   $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$

Figure 8.1: Random-sample one-step tabular Q-planning

steps may be the most efficient approach even on pure planning problems if the problem is too large to be solved exactly.

8.2 Integrating Planning, Acting, and Learning

When planning is done on-line, while interacting with the environment, a number of interesting issues arise. New information gained from the interaction may change the model and thereby interact with planning. It may be desirable to customize the planning process in some way to the states or decisions currently under consideration, or expected in the near future. If decision-making and model-learning are both computation-intensive processes, then the available computational resources may need to be divided between them. To begin exploring these issues, in this section we present Dyna-Q, a simple architecture integrating the major functions needed in an on-line planning agent. Each function appears in Dyna-Q in a simple, almost trivial, form. In subsequent sections we elaborate some of the alternate ways of achieving each function and the trade-offs between them. For now, we seek merely to illustrate the ideas and stimulate your intuition.

Within a planning agent, there are at least two roles for real experience: it can be used to improve the model (to make it more accurately match the real environment) and it can be used to directly improve the value function and policy using the kinds of reinforcement learning methods we have discussed in previous chapters. The former we call *model-learning*, and the latter we call *direct reinforcement learning* (direct RL). The possible relationships between experience, model, values, and policy are summarized in Figure 8.2. Each arrow shows a relationship of influence and presumed improvement. Note how experience can improve value and policy functions either directly or indirectly via the model. It is the latter, which is sometimes called *indirect reinforcement learning*, that is involved in planning.

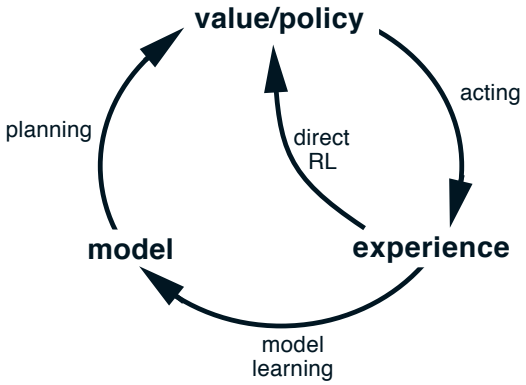


Figure 8.2: Relationships among learning, planning, and acting.

Both direct and indirect methods have advantages and disadvantages. Indirect methods often make fuller use of a limited amount of experience and thus achieve a better policy with fewer environmental interactions. On the other hand, direct methods are much simpler and are not affected by biases in the design of the model. Some have argued that indirect methods are always superior to direct ones, while others have argued that direct methods are responsible for most human and animal learning. Related debates in psychology and AI concern the relative importance of cognition as opposed to trial-and-error learning, and of deliberative planning as opposed to reactive decision-making. Our view is that the contrast between the alternatives in all these debates has been exaggerated, that more insight can be gained by recognizing the similarities between these two sides than by opposing them. For example, in this book we have emphasized the deep similarities between dynamic programming and temporal-difference methods, even though one was designed for planning and the other for modelfree learning.

Dyna-Q includes all of the processes shown in Figure 8.2—planning, acting, model-learning, and direct RL—all occurring continually. The planning method is the random-sample one-step tabular Q-planning method given in Figure 8.1. The direct RL method is one-step tabular Q-learning. The model-learning method is also table-based and assumes the world is deterministic. After each transition $S_t, A_t \rightsquigarrow R_{t+1}, S_{t+1}$, the model records in its table entry for S_t, A_t the prediction that R_{t+1}, S_{t+1} will deterministically follow. Thus, if the model is queried with a state–action pair that has been experienced before, it simply returns the last-observed next state and next reward as its prediction. During planning, the Q-planning algorithm randomly samples only from state–action pairs that have previously been experienced (in Step 1), so the model is never queried with a pair about which it has no information.

The overall architecture of Dyna agents, of which the Dyna-Q algorithm is

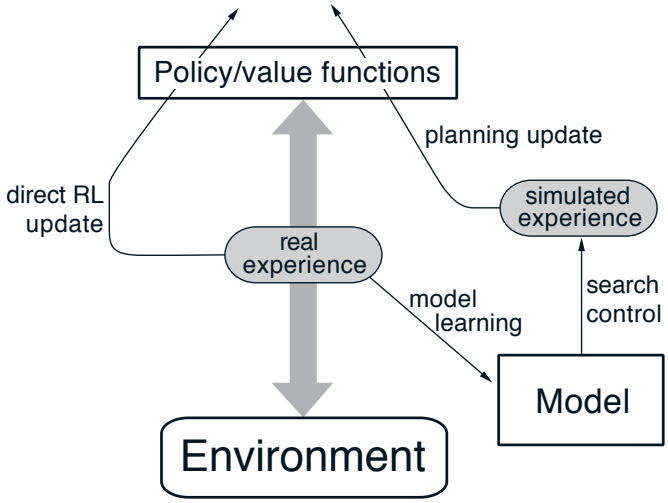


Figure 8.3: The general Dyna Architecture

one example, is shown in Figure 8.3. The central column represents the basic interaction between agent and environment, giving rise to a trajectory of real experience. The arrow on the left of the figure represents direct reinforcement learning operating on real experience to improve the value function and the policy. On the right are model-based processes. The model is learned from real experience and gives rise to simulated experience. We use the term *search control* to refer to the process that selects the starting states and actions for the simulated experiences generated by the model. Finally, planning is achieved by applying reinforcement learning methods to the simulated experiences just as if they had really happened. Typically, as in Dyna-Q, the same reinforcement learning method is used both for learning from real experience and for planning from simulated experience. The reinforcement learning method is thus the "final common path" for both learning and planning. Learning and planning are deeply integrated in the sense that they share almost all the same machinery, differing only in the source of their experience.

Conceptually, planning, acting, model-learning, and direct RL occur simultaneously and in parallel in Dyna agents. For concreteness and implementation on a serial computer, however, we fully specify the order in which they occur within a time step. In Dyna-Q, the acting, model-learning, and direct RL processes require little computation, and we assume they consume just a fraction of the time. The remaining time in each step can be devoted to the planning process, which is inherently computation-intensive. Let us assume that there is time in each step, after acting, model-learning, and direct RL, to complete n iterations (Steps 1–3) of the Q-planning algorithm. Figure 8.4 shows the complete algorithm for Dyna-Q.

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Do forever:
(a) $S \leftarrow$ current (nonterminal) state
(b) $A \leftarrow \varepsilon$-greedy(S,Q)
(c) Execute action A; observe resultant reward, R, and state, S'
(d) $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$
(e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
(f) Repeat n times:
$S \leftarrow$ random previously observed state
$A \leftarrow$ random action previously taken in S
$R, S' \leftarrow Model(S,A)$
$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$

Figure 8.4: Dyna-Q Algorithm. $Model(s,a)$ denotes the contents of the model (predicted next state and reward) for state–action pair s,a. Direct reinforcement learning, model-learning, and planning are implemented by steps (d), (e), and (f), respectively. If (e) and (f) were omitted, the remaining algorithm would be one-step tabular Q-learning.

Example 8.1: Dyna Maze Consider the simple maze shown inset in Figure 8.5. In each of the 47 states there are four actions, `up`, `down`, `right`, and `left`, which take the agent deterministically to the corresponding neighboring states, except when movement is blocked by an obstacle or the edge of the maze, in which case the agent remains where it is. Reward is zero on all transitions, except those into the goal state, on which it is +1. After reaching the goal state (G), the agent returns to the start state (S) to begin a new episode. This is a discounted, episodic task with $\gamma = 0.95$.

The main part of Figure 8.5 shows average learning curves from an experiment in which Dyna-Q agents were applied to the maze task. The initial action values were zero, the step-size parameter was $\alpha = 0.1$, and the exploration parameter was $\varepsilon = 0.1$. When selecting greedily among actions, ties were broken randomly. The agents varied in the number of planning steps, n, they performed per real step. For each n, the curves show the number of steps taken by the agent in each episode, averaged over 30 repetitions of the experiment. In each repetition, the initial seed for the random number generator was held constant across algorithms. Because of this, the first episode was exactly the same (about 1700 steps) for all values of n, and its data are not shown in the figure. After the first episode, performance improved for all values of n, but much more rapidly for larger values. Recall that the $n = 0$ agent is a nonplanning agent, utilizing only direct reinforcement learning (one-step tabular Q-learning). This was by far the slowest agent on this problem, despite the fact that the parameter values (α and ε) were optimized for it. The

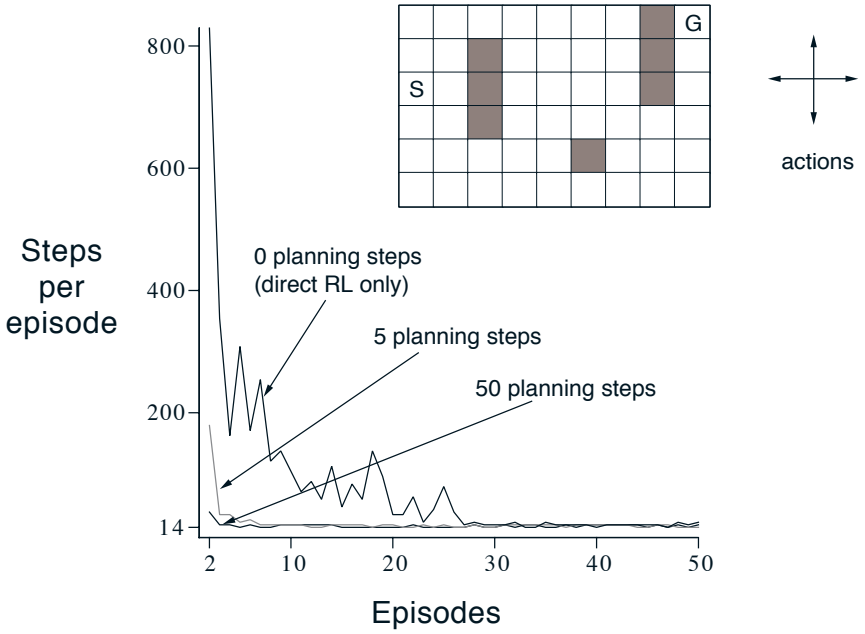


Figure 8.5: A simple maze (inset) and the average learning curves for Dyna-Q agents varying in their number of planning steps (n) per real step. The task is to travel from S to G as quickly as possible.

nonplanning agent took about 25 episodes to reach (ε-)optimal performance, whereas the $n = 5$ agent took about five episodes, and the $n = 50$ agent took only three episodes.

Figure 8.6 shows why the planning agents found the solution so much faster than the nonplanning agent. Shown are the policies found by the $n = 0$ and $n = 50$ agents halfway through the second episode. Without planning ($n = 0$), each episode adds only one additional step to the policy, and so only one step (the last) has been learned so far. With planning, again only one step is learned during the first episode, but here during the second episode an extensive policy has been developed that by the episode's end will reach almost back to the start state. This policy is built by the planning process while the agent is still wandering near the start state. By the end of the third episode a complete optimal policy will have been found and perfect performance attained. ■

In Dyna-Q, learning and planning are accomplished by exactly the same algorithm, operating on real experience for learning and on simulated experience for planning. Because planning proceeds incrementally, it is trivial to intermix planning and acting. Both proceed as fast as they can. The agent is always reactive and always deliberative, responding instantly to the latest sensory information and yet always planning in the background. Also ongoing in the

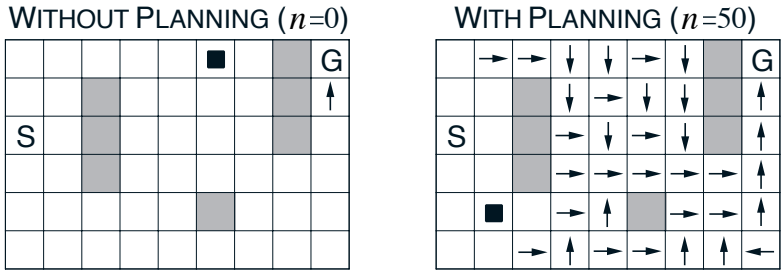


Figure 8.6: Policies found by planning and nonplanning Dyna-Q agents halfway through the second episode. The arrows indicate the greedy action in each state; no arrow is shown for a state if all of its action values are equal. The black square indicates the location of the agent.

background is the model-learning process. As new information is gained, the model is updated to better match reality. As the model changes, the ongoing planning process will gradually compute a different way of behaving to match the new model.

Exercise 8.1 The nonplanning method looks particularly poor in Figure 8.6 because it is a one-step method; a method using eligibility traces would do better. Do you think an eligibility trace method could do as well as the Dyna method? Explain why or why not.

8.3 When the Model Is Wrong

In the maze example presented in the previous section, the changes in the model were relatively modest. The model started out empty, and was then filled only with exactly correct information. In general, we cannot expect to be so fortunate. Models may be incorrect because the environment is stochastic and only a limited number of samples have been observed, because the model was learned using function approximation that has generalized imperfectly, or simply because the environment has changed and its new behavior has not yet been observed. When the model is incorrect, the planning process will compute a suboptimal policy.

In some cases, the suboptimal policy computed by planning quickly leads to the discovery and correction of the modeling error. This tends to happen when the model is optimistic in the sense of predicting greater reward or better state transitions than are actually possible. The planned policy attempts to exploit these opportunities and in doing so discovers that they do not exist.

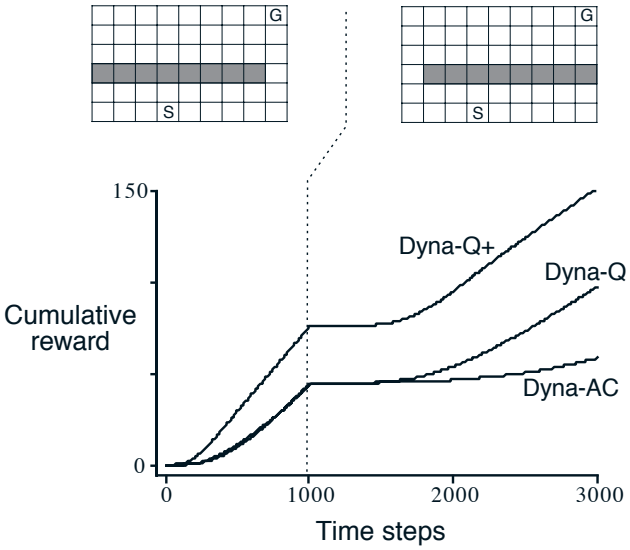


Figure 8.7: Average performance of Dyna agents on a blocking task. The left environment was used for the first 1000 steps, the right environment for the rest. Dyna-Q+ is Dyna-Q with an exploration bonus that encourages exploration. Dyna-AC is a Dyna agent that uses an actor–critic learning method instead of Q-learning.

Example 8.2: Blocking Maze A maze example illustrating this relatively minor kind of modeling error and recovery from it is shown in Figure 8.7. Initially, there is a short path from start to goal, to the right of the barrier, as shown in the upper left of the figure. After 1000 time steps, the short path is "blocked," and a longer path is opened up along the left-hand side of the barrier, as shown in upper right of the figure. The graph shows average cumulative reward for Dyna-Q and two other Dyna agents. The first part of the graph shows that all three Dyna agents found the short path within 1000 steps. When the environment changed, the graphs become flat, indicating a period during which the agents obtained no reward because they were wandering around behind the barrier. After a while, however, they were able to find the new opening and the new optimal behavior. ■

Greater difficulties arise when the environment changes to become *better* than it was before, and yet the formerly correct policy does not reveal the improvement. In these cases the modeling error may not be detected for a long time, if ever, as we see in the next example.

Example 8.3: Shortcut Maze The problem caused by this kind of environmental change is illustrated by the maze example shown in Figure 8.8. Initially, the optimal path is to go around the left side of the barrier (upper

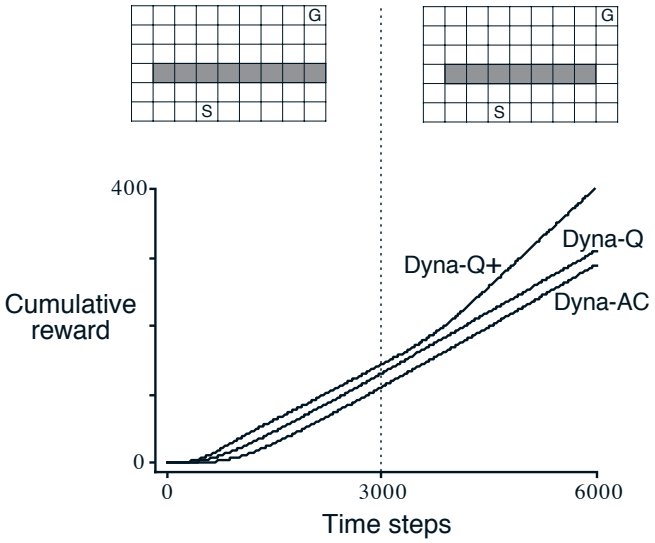


Figure 8.8: Average performance of Dyna agents on a shortcut task. The left environment was used for the first 3000 steps, the right environment for the rest.

left). After 3000 steps, however, a shorter path is opened up along the right side, without disturbing the longer path (upper right). The graph shows that two of the three Dyna agents never switched to the shortcut. In fact, they never realized that it existed. Their models said that there was no shortcut, so the more they planned, the less likely they were to step to the right and discover it. Even with an ε-greedy policy, it is very unlikely that an agent will take so many exploratory actions as to discover the shortcut. ■

The general problem here is another version of the conflict between exploration and exploitation. In a planning context, exploration means trying actions that improve the model, whereas exploitation means behaving in the optimal way given the current model. We want the agent to explore to find changes in the environment, but not so much that performance is greatly degraded. As in the earlier exploration/exploitation conflict, there probably is no solution that is both perfect and practical, but simple heuristics are often effective.

The Dyna-Q+ agent that did solve the shortcut maze uses one such heuristic. This agent keeps track for each state–action pair of how many time steps have elapsed since the pair was last tried in a real interaction with the environment. The more time that has elapsed, the greater (we might presume) the chance that the dynamics of this pair has changed and that the model of it is incorrect. To encourage behavior that tests long-untried actions, a spe-

cial "bonus reward" is given on simulated experiences involving these actions. In particular, if the modeled reward for a transition is R, and the transition has not been tried in τ time steps, then planning backups are done as if that transition produced a reward of $R + \kappa\sqrt{\tau}$, for some small κ. This encourages the agent to keep testing all accessible state transitions and even to plan long sequences of actions in order to carry out such tests. Of course all this testing has its cost, but in many cases, as in the shortcut maze, this kind of computational curiosity is well worth the extra exploration.

Exercise 8.2 Why did the Dyna agent with exploration bonus, Dyna-Q+, perform better in the first phase as well as in the second phase of the blocking and shortcut experiments?

Exercise 8.3 Careful inspection of Figure 8.8 reveals that the difference between Dyna-Q+ and Dyna-Q narrowed slightly over the first part of the experiment. What is the reason for this?

Exercise 8.4 (programming) The exploration bonus described above actually changes the estimated values of states and actions. Is this necessary? Suppose the bonus $\kappa\sqrt{\tau}$ was used not in backups, but solely in action selection. That is, suppose the action selected was always that for which $Q(S, a)+\kappa\sqrt{\tau_{Sa}}$ was maximal. Carry out a gridworld experiment that tests and illustrates the strengths and weaknesses of this alternate approach.

8.4 Prioritized Sweeping

In the Dyna agents presented in the preceding sections, simulated transitions are started in state–action pairs selected uniformly at random from all previously experienced pairs. But a uniform selection is usually not the best; planning can be much more efficient if simulated transitions and backups are focused on particular state–action pairs. For example, consider what happens during the second episode of the first maze task (Figure 8.6). At the beginning of the second episode, only the state–action pair leading directly into the goal has a positive value; the values of all other pairs are still zero. This means that it is pointless to back up along almost all transitions, because they take the agent from one zero-valued state to another, and thus the backups would have no effect. Only a backup along a transition into the state just prior to the goal, or from it into the goal, will change any values. If simulated transitions are generated uniformly, then many wasteful backups will be made before stumbling onto one of the two useful ones. As planning progresses, the region of useful backups grows, but planning is still far less efficient than it would be if focused where it would do the most good. In the much larger problems

that are our real objective, the number of states is so large that an unfocused search would be extremely inefficient.

This example suggests that search might be usefully focused by working *backward* from goal states. Of course, we do not really want to use any methods specific to the idea of "goal state." We want methods that work for general reward functions. Goal states are just a special case, convenient for stimulating intuition. In general, we want to work back not just from goal states but from any state whose value has changed. Assume that the values are initially correct given the model, as they were in the maze example prior to discovering the goal. Suppose now that the agent discovers a change in the environment and changes its estimated value of one state. Typically, this will imply that the values of many other states should also be changed, but the only useful one-step backups are those of actions that lead directly into the one state whose value has already been changed. If the values of these actions are updated, then the values of the predecessor states may change in turn. If so, then actions leading into them need to be backed up, and then *their* predecessor states may have changed. In this way one can work backward from arbitrary states that have changed in value, either performing useful backups or terminating the propagation.

As the frontier of useful backups propagates backward, it often grows rapidly, producing many state–action pairs that could usefully be backed up. But not all of these will be equally useful. The values of some states may have changed a lot, whereas others have changed little. The predecessor pairs of those that have changed a lot are more likely to also change a lot. In a stochastic environment, variations in estimated transition probabilities also contribute to variations in the sizes of changes and in the urgency with which pairs need to be backed up. It is natural to prioritize the backups according to a measure of their urgency, and perform them in order of priority. This is the idea behind *prioritized sweeping*. A queue is maintained of every state–action pair whose estimated value would change nontrivially if backed up, prioritized by the size of the change. When the top pair in the queue is backed up, the effect on each of its predecessor pairs is computed. If the effect is greater than some small threshold, then the pair is inserted in the queue with the new priority (if there is a previous entry of the pair in the queue, then insertion results in only the higher priority entry remaining in the queue). In this way the effects of changes are efficiently propagated backward until quiescence. The full algorithm for the case of deterministic environments is given in Figure 8.9.

Example 8.4: Prioritized Sweeping on Mazes Prioritized sweeping has been found to dramatically increase the speed at which optimal solutions are found in maze tasks, often by a factor of 5 to 10. A typical example is shown in Figure 8.10. These data are for a sequence of maze tasks of exactly

Initialize $Q(s,a)$, $Model(s,a)$, for all s,a, and $PQueue$ to empty
Do forever:
(a) $S \leftarrow$ current (nonterminal) state
(b) $A \leftarrow policy(S,Q)$
(c) Execute action A; observe resultant reward, R, and state, S'
(d) $Model(S,A) \leftarrow R, S'$
(e) $P \leftarrow |R + \gamma \max_a Q(S',a) - Q(S,A)|$.
(f) if $P > \theta$, then insert S,A into $PQueue$ with priority P
(g) Repeat n times, while $PQueue$ is not empty:
$S,A \leftarrow first(PQueue)$
$R, S' \leftarrow Model(S,A)$
$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$
Repeat, for all $\bar{S},\bar{A}$ predicted to lead to S:
$\bar{R} \leftarrow$ predicted reward for $\bar{S},\bar{A},S$
$P \leftarrow |\bar{R} + \gamma \max_a Q(S,a) - Q(\bar{S},\bar{A})|$.
if $P > \theta$ then insert $\bar{S},\bar{A}$ into $PQueue$ with priority P

Figure 8.9: The prioritized sweeping algorithm for a deterministic environment.

the same structure as the one shown in Figure 8.5, except that they vary in the grid resolution. Prioritized sweeping maintained a decisive advantage over unprioritized Dyna-Q. Both systems made at most $n = 5$ backups per environmental interaction. ■

Example 8.5: Rod Maneuvering The objective in this task is to maneuver a rod around some awkwardly placed obstacles to a goal position in the fewest number of steps (Figure 8.11). The rod can be translated along its long axis or perpendicular to that axis, or it can be rotated in either direction around its center. The distance of each movement is approximately 1/20 of the work space, and the rotation increment is 10 degrees. Translations are deterministic and quantized to one of 20×20 positions. The figure shows the obstacles and the shortest solution from start to goal, found by prioritized sweeping. This problem is still deterministic, but has four actions and 14,400 potential states (some of these are unreachable because of the obstacles). This problem is probably too large to be solved with unprioritized methods. ■

Prioritized sweeping is clearly a powerful idea, but the algorithms that have been developed so far appear not to extend easily to more interesting cases. The greatest problem is that the algorithms appear to rely on the assumption of discrete states. When a change occurs at one state, these methods perform a computation on all the predecessor states that may have been affected. If function approximation is used to learn the model or the value function, then a single backup could influence a great many other states. It is not apparent

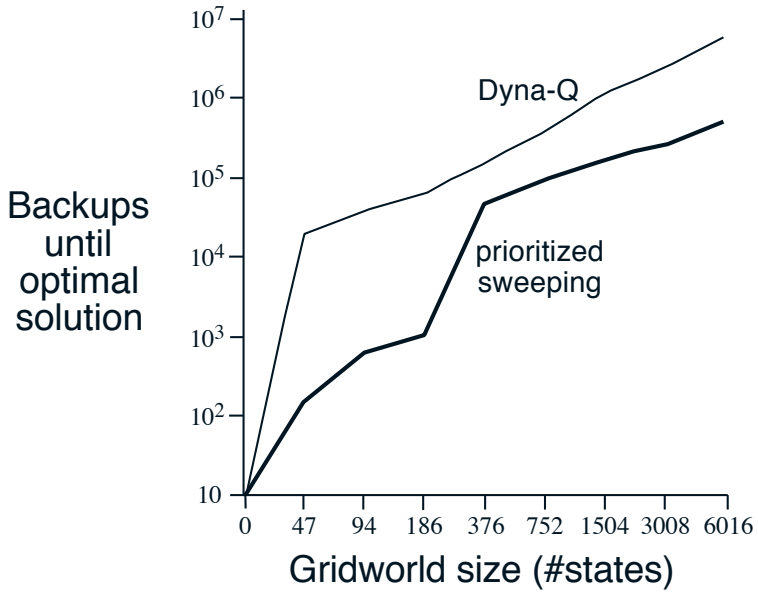


Figure 8.10: Prioritized sweeping significantly shortens learning time on the Dyna maze task for a wide range of grid resolutions. Reprinted from Peng and Williams (1993).

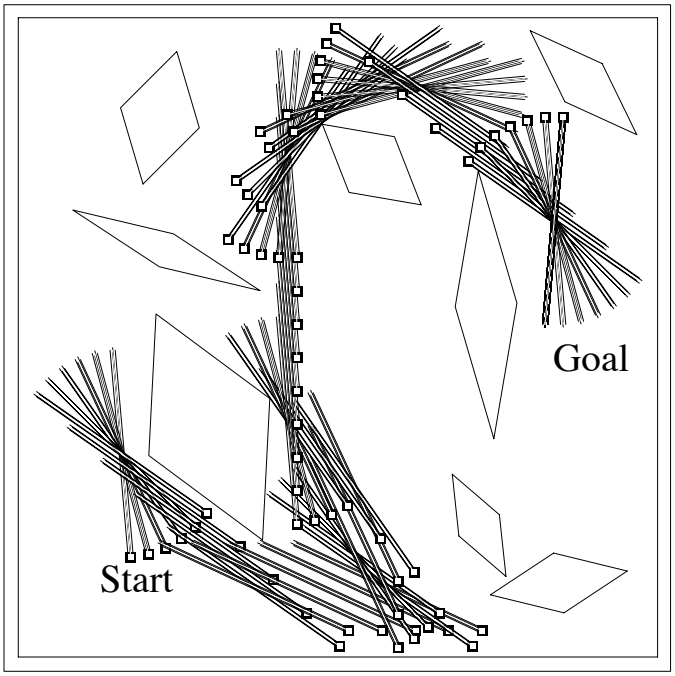


Figure 8.11: A rod-maneuvering task and its solution by prioritized sweeping. Reprinted from Moore and Atkeson (1993).

how these states could be identified or processed efficiently. On the other hand, the general idea of focusing search on the states believed to have changed in value, and then on their predecessors, seems intuitively to be valid in general. Additional research may produce more general versions of prioritized sweeping.

Extensions of prioritized sweeping to stochastic environments are relatively straightforward. The model is maintained by keeping counts of the number of times each state–action pair has been experienced and of what the next states were. It is natural then to backup each pair not with a sample backup, as we have been using so far, but with a full backup, taking into account all possible next states and their probabilities of occurring.

8.5 Full vs. Sample Backups

The examples in the previous sections give some idea of the range of possibilities for combining methods of learning and planning. In the rest of this chapter, we analyze some of the component ideas involved, starting with the relative advantages of full and sample backups.

Much of this book has been about different kinds of backups, and we have considered a great many varieties. Focusing for the moment on one-step backups, they vary primarily along three binary dimensions. The first two dimensions are whether they back up state values or action values and whether they estimate the value for the optimal policy or for an arbitrary given policy. These two dimensions give rise to four classes of backups for approximating the four value functions, q_*, v_*, q_π, and v_π. The other binary dimension is whether the backups are *full* backups, considering all possible events that might happen, or *sample* backups, considering a single sample of what might happen. These three binary dimensions give rise to eight cases, seven of which correspond to specific algorithms, as shown in Figure 8.12. (The eighth case does not seem to correspond to any useful backup.) Any of these one-step backups can be used in planning methods. The Dyna-Q agents discussed earlier use q_* sample backups, but they could just as well use q_* full backups, or either full or sample q_π backups. The Dyna-AC system uses v_π sample backups together with a learning policy structure. For stochastic problems, prioritized sweeping is always done using one of the full backups.

When we introduced one-step sample backups in Chapter 6, we presented them as substitutes for full backups. In the absence of a distribution model, full backups are not possible, but sample backups can be done using sample transitions from the environment or a sample model. Implicit in that point of view is that full backups, if possible, are preferable to sample backups.

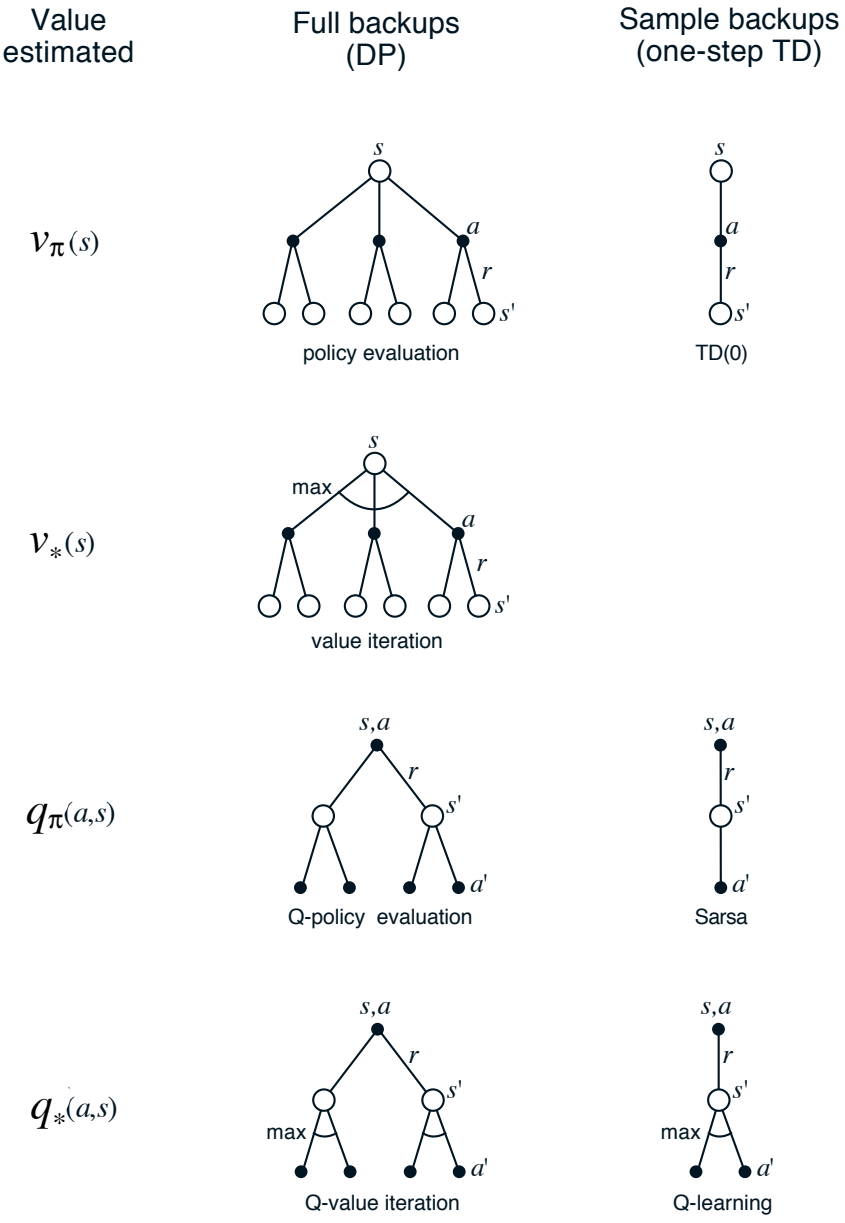


Figure 8.12: The one-step backups.

But are they? Full backups certainly yield a better estimate because they are uncorrupted by sampling error, but they also require more computation, and computation is often the limiting resource in planning. To properly assess the relative merits of full and sample backups for planning we must control for their different computational requirements.

For concreteness, consider the full and sample backups for approximating q_*, and the special case of discrete states and actions, a table-lookup representation of the approximate value function, Q, and a model in the form of estimated state-transition probabilities, $\hat{p}(s'|s,a)$, and expected rewards, $\hat{r}(s,a,s')$. The full backup for a state–action pair, s,a, is:

$$Q(s,a) \leftarrow \sum_{s'} \hat{p}(s'|s,a)\Big[\hat{r}(s,a,s') + \gamma \max_{a'} Q(s',a')\Big]. \tag{8.1}$$

The corresponding sample backup for s,a, given a sample next state, S', is the Q-learning-like update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[\hat{r}(s,a,S') + \gamma \max_{a'} Q(S',a') - Q(s,a)\Big], \tag{8.2}$$

where α is the usual positive step-size parameter and the model's expected value of the reward, $\hat{r}(s,a,S')$ is used in place of the sample reward that is used in applying Q-learning without a model.

The difference between these full and sample backups is significant to the extent that the environment is stochastic, specifically, to the extent that, given a state and action, many possible next states may occur with various probabilities. If only one next state is possible, then the full and sample backups given above are identical (taking $\alpha = 1$). If there are many possible next states, then there may be significant differences. In favor of the full backup is that it is an exact computation, resulting in a new $Q(s,a)$ whose correctness is limited only by the correctness of the $Q(s',a')$ at successor states. The sample backup is in addition affected by sampling error. On the other hand, the sample backup is cheaper computationally because it considers only one next state, not all possible next states. In practice, the computation required by backup operations is usually dominated by the number of state–action pairs at which Q is evaluated. For a particular starting pair, s,a, let b be the *branching factor* (i.e., the number of possible next states, s', for which $\hat{p}(s'|s,a) > 0$). Then a full backup of this pair requires roughly b times as much computation as a sample backup.

If there is enough time to complete a full backup, then the resulting estimate is generally better than that of b sample backups because of the absence of sampling error. But if there is insufficient time to complete a full backup,

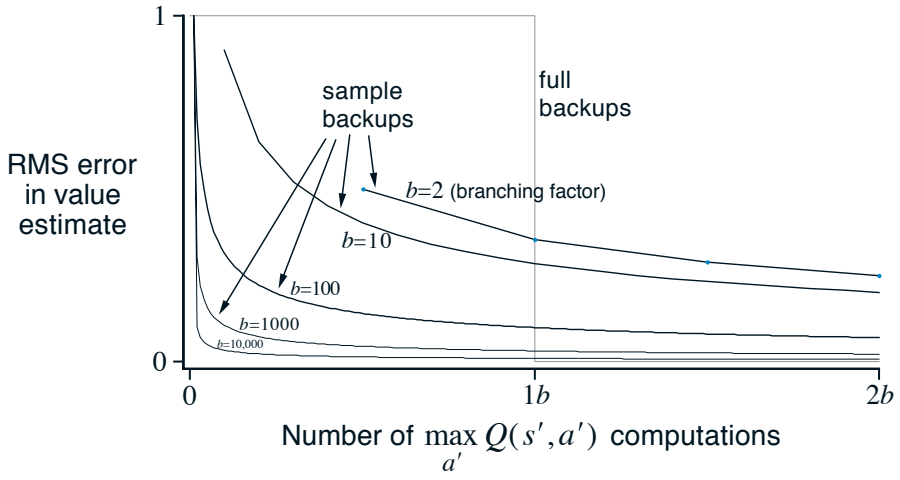


Figure 8.13: Comparison of efficiency of full and sample backups.

then sample backups are always preferable because they at least make some improvement in the value estimate with fewer than b backups. In a large problem with many state–action pairs, we are often in the latter situation. With so many state–action pairs, full backups of all of them would take a very long time. Before that we may be much better off with a few sample backups at many state–action pairs than with full backups at a few pairs. Given a unit of computational effort, is it better devoted to a few full backups or to b times as many sample backups?

Figure 8.13 shows the results of an analysis that suggests an answer to this question. It shows the estimation error as a function of computation time for full and sample backups for a variety of branching factors, b. The case considered is that in which all b successor states are equally likely and in which the error in the initial estimate is 1. The values at the next states are assumed correct, so the full backup reduces the error to zero upon its completion. In this case, sample backups reduce the error according to $\frac{1}{\sqrt{t}}\frac{b-1}{b}$ where t is the number of sample backups that have been performed (assuming sample averages, i.e., $\alpha = 1/t$). The key observation is that for moderately large b the error falls dramatically with a tiny fraction of b backups. For these cases, many state–action pairs could have their values improved dramatically, to within a few percent of the effect of a full backup, in the same time that one state–action pair could be backed up fully.

The advantage of sample backups shown in Figure 8.13 is probably an underestimate of the real effect. In a real problem, the values of the successor states would themselves be estimates updated by backups. By causing estimates to be more accurate sooner, sample backups will have a second ad-

vantage in that the values backed up from the successor states will be more accurate. These results suggest that sample backups are likely to be superior to full backups on problems with large stochastic branching factors and too many states to be solved exactly.

Exercise 8.5 The analysis above assumed that all of the b possible next states were equally likely to occur. Suppose instead that the distribution was highly skewed, that some of the b states were much more likely to occur than most. Would this strengthen or weaken the case for sample backups over full backups? Support your answer.

8.6 Trajectory Sampling

In this section we compare two ways of distributing backups. The classical approach, from dynamic programming, is to perform sweeps through the entire state (or state–action) space, backing up each state (or state–action pair) once per sweep. This is problematic on large tasks because there may not be time to complete even one sweep. In many tasks the vast majority of the states are irrelevant because they are visited only under very poor policies or with very low probability. Exhaustive sweeps implicitly devote equal time to all parts of the state space rather than focusing where it is needed. As we discussed in Chapter 4, exhaustive sweeps and the equal treatment of all states that they imply are not necessary properties of dynamic programming. In principle, backups can be distributed any way one likes (to assure convergence, all states or state–action pairs must be visited in the limit an infinite number of times), but in practice exhaustive sweeps are often used.

The second approach is to sample from the state or state–action space according to some distribution. One could sample uniformly, as in the Dyna-Q agent, but this would suffer from some of the same problems as exhaustive sweeps. More appealing is to distribute backups according to the on-policy distribution, that is, according to the distribution observed when following the current policy. One advantage of this distribution is that it is easily generated; one simply interacts with the model, following the current policy. In an episodic task, one starts in the start state (or according to the starting-state distribution) and simulates until the terminal state. In a continuing task, one starts anywhere and just keeps simulating. In either case, sample state transitions and rewards are given by the model, and sample actions are given by the current policy. In other words, one simulates explicit individual trajectories and performs backups at the state or state–action pairs encountered along the way. We call this way of generating experience and backups *trajectory sampling*.

It is hard to imagine any efficient way of distributing backups according to the on-policy distribution other than by trajectory sampling. If one had an explicit representation of the on-policy distribution, then one could sweep through all states, weighting the backup of each according to the on-policy distribution, but this leaves us again with all the computational costs of exhaustive sweeps. Possibly one could sample and update individual state–action pairs from the distribution, but even if this could be done efficiently, what benefit would this provide over simulating trajectories? Even knowing the on-policy distribution in an explicit form is unlikely. The distribution changes whenever the policy changes, and computing the distribution requires computation comparable to a complete policy evaluation. Consideration of such other possibilities makes trajectory sampling seem both efficient and elegant.

Is the on-policy distribution of backups a good one? Intuitively it seems like a good choice, at least better than the uniform distribution. For example, if you are learning to play chess, you study positions that might arise in real games, not random positions of chess pieces. The latter may be valid states, but to be able to accurately value them is a different skill from evaluating positions in real games. We will also see in Chapter 9 that the on-policy distribution has significant advantages when function approximation is used. Whether or not function approximation is used, one might expect on-policy focusing to significantly improve the speed of planning.

Focusing on the on-policy distribution could be beneficial because it causes vast, uninteresting parts of the space to be ignored, or it could be detrimental because it causes the same old parts of the space to be backed up over and over. We conducted a small experiment to assess the effect empirically. To isolate the effect of the backup distribution, we used entirely one-step full tabular backups, as defined by (8.1). In the *uniform* case, we cycled through all state–action pairs, backing up each in place, and in the *on-policy* case we simulated episodes, backing up each state–action pair that occurred under the current ε-greedy policy ($\varepsilon = 0.1$). The tasks were undiscounted episodic tasks, generated randomly as follows. From each of the $|\mathcal{S}|$ states, two actions were possible, each of which resulted in one of b next states, all equally likely, with a different random selection of b states for each state–action pair. The branching factor, b, was the same for all state–action pairs. In addition, on all transitions there was a 0.1 probability of transition to the terminal state, ending the episode. We used episodic tasks to get a clear measure of the quality of the current policy. At any point in the planning process one can stop and exhaustively compute $v_{\tilde{\pi}}(s_0)$, the true value of the start state under the greedy policy, $\tilde{\pi}$, given the current action-value function Q, as an indication of how well the agent would do on a new episode on which it acted greedily (all the while assuming the model is correct).

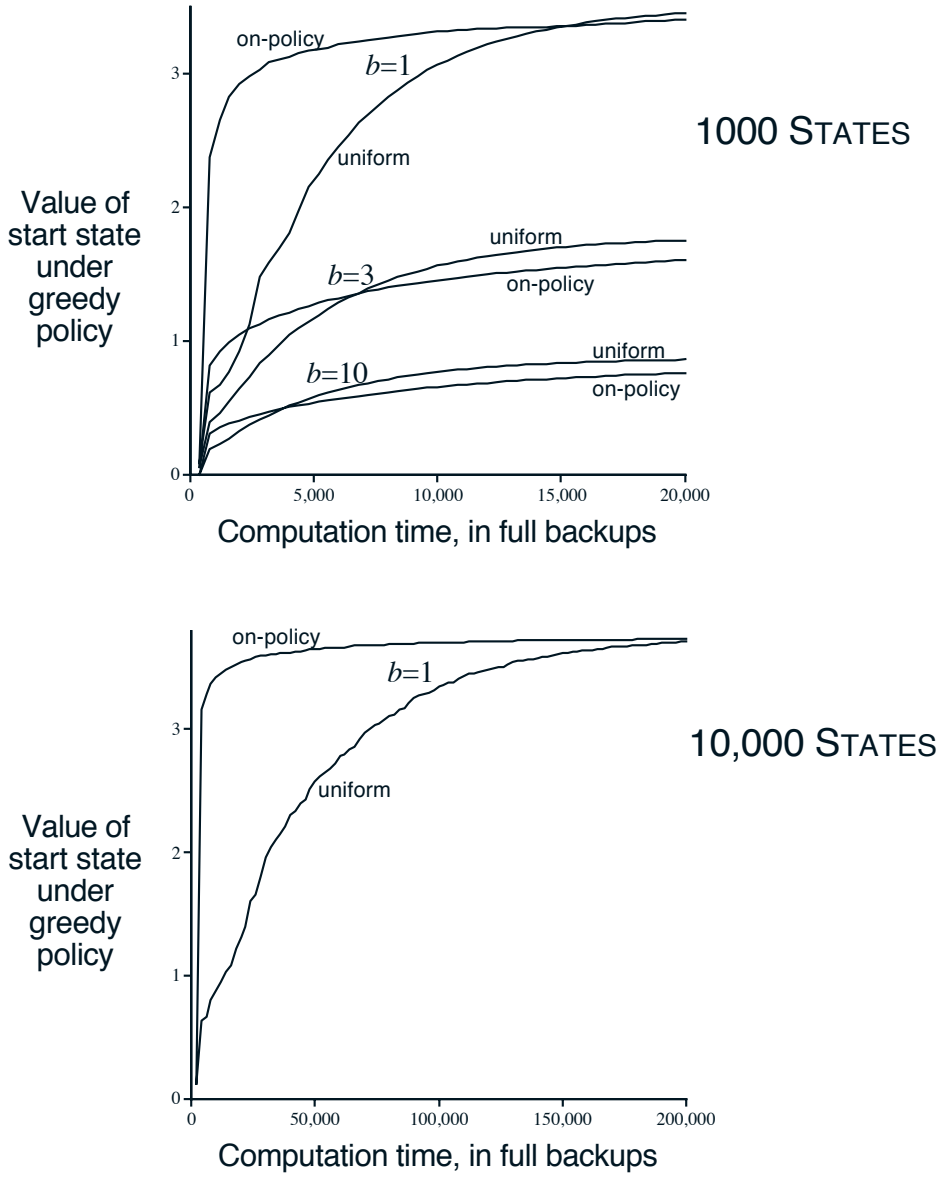


Figure 8.14: Relative efficiency of backups distributed uniformly across the state space versus focused on simulated on-policy trajectories. Results are for randomly generated tasks of two sizes and various branching factors, b.

The upper part of Figure 8.14 shows results averaged over 200 sample tasks with 1000 states and branching factors of 1, 3, and 10. The quality of the policies found is plotted as a function of the number of full backups completed. In all cases, sampling according to the on-policy distribution resulted in faster planning initially and retarded planning in the long run. The effect was stronger, and the initial period of faster planning was longer, at smaller branching factors. In other experiments, we found that these effects also became stronger as the number of states increased. For example, the lower part of Figure 8.14 shows results for a branching factor of 1 for tasks with 10,000 states. In this case the advantage of on-policy focusing is large and long-lasting.

All of these results make sense. In the short term, sampling according to the on-policy distribution helps by focusing on states that are near descendants of the start state. If there are many states and a small branching factor, this effect will be large and long-lasting. In the long run, focusing on the on-policy distribution may hurt because the commonly occurring states all already have their correct values. Sampling them is useless, whereas sampling other states may actually perform some useful work. This presumably is why the exhaustive, unfocused approach does better in the long run, at least for small problems. These results are not conclusive because they are only for problems generated in a particular, random way, but they do suggest that sampling according to the on-policy distribution can be a great advantage for large problems, in particular for problems in which a small subset of the state–action space is visited under the on-policy distribution.

Exercise 8.6 Some of the graphs in Figure 8.14 seem to be scalloped in their early portions, particularly the upper graph for $b = 1$ and the uniform distribution. Why do you think this is? What aspects of the data shown support your hypothesis?

Exercise 8.7 (programming) If you have access to a moderately large computer, try replicating the experiment whose results are shown in the lower part of Figure 8.14. Then try the same experiment but with $b = 3$. Discuss the meaning of your results.

8.7 Heuristic Search

The predominant state-space planning methods in artificial intelligence are collectively known as *heuristic search*. Although superficially different from the planning methods we have discussed so far in this chapter, heuristic search and some of its component ideas can be combined with these methods in

useful ways. Unlike these methods, heuristic search is not concerned with changing the approximate, or "heuristic," value function, but only with making improved action selections given the current value function. In other words, heuristic search is planning as part of a policy computation.

In heuristic search, for each state encountered, a large tree of possible continuations is considered. The approximate value function is applied to the leaf nodes and then backed up toward the current state at the root. The backing up within the search tree is just the same as in the max-backups (those for v_* and q_*) discussed throughout this book. The backing up stops at the state–action nodes for the current state. Once the backed-up values of these nodes are computed, the best of them is chosen as the current action, and then all backed-up values are discarded.

In conventional heuristic search no effort is made to save the backed-up values by changing the approximate value function. In fact, the value function is generally designed by people and never changed as a result of search. However, it is natural to consider allowing the value function to be improved over time, using either the backed-up values computed during heuristic search or any of the other methods presented throughout this book. In a sense we have taken this approach all along. Our greedy and ε-greedy action-selection methods are not unlike heuristic search, albeit on a smaller scale. For example, to compute the greedy action given a model and a state-value function, we must look ahead from each possible action to each possible next state, backup the rewards and estimated values, and then pick the best action. Just as in conventional heuristic search, this process computes backed-up values of the possible actions, but does not attempt to save them. Thus, heuristic search can be viewed as an extension of the idea of a greedy policy beyond a single step.

The point of searching deeper than one step is to obtain better action selections. If one has a perfect model and an imperfect action-value function, then in fact deeper search will usually yield better policies.[1] Certainly, if the search is all the way to the end of the episode, then the effect of the imperfect value function is eliminated, and the action determined in this way must be optimal. If the search is of sufficient depth k such that γ^k is very small, then the actions will be correspondingly near optimal. On the other hand, the deeper the search, the more computation is required, usually resulting in a slower response time. A good example is provided by Tesauro's grandmaster-level backgammon player, TD-Gammon (Section 15.1). This system used TD(λ) to learn an afterstate value function through many games of self-play, using a form of heuristic search to make its moves. As a model, TD-Gammon used a

[1] There are interesting exceptions to this. See, e.g., Pearl (1984).

priori knowledge of the probabilities of dice rolls and the assumption that the opponent always selected the actions that TD-Gammon rated as best for it. Tesauro found that the deeper the heuristic search, the better the moves made by TD-Gammon, but the longer it took to make each move. Backgammon has a large branching factor, yet moves must be made within a few seconds. It was only feasible to search ahead selectively a few steps, but even so the search resulted in significantly better action selections.

So far we have emphasized heuristic search as an action-selection technique, but this may not be its most important aspect. Heuristic search also suggests ways of selectively distributing backups that may lead to better and faster approximation of the optimal value function. A great deal of research on heuristic search has been devoted to making the search as efficient as possible. The search tree is grown selectively, deeper along some lines and shallower along others. For example, the search tree is often deeper for the actions that seem most likely to be best, and shallower for those that the agent will probably not want to take anyway. Can we use a similar idea to improve the distribution of backups? Perhaps it can be done by preferentially updating state–action pairs whose values appear to be close to the maximum available from the state. To our knowledge, this and other possibilities for distributing backups based on ideas borrowed from heuristic search have not yet been explored.

We should not overlook the most obvious way in which heuristic search focuses backups: on the current state. Much of the effectiveness of heuristic search is due to its search tree being tightly focused on the states and actions that might immediately follow the current state. You may spend more of your life playing chess than checkers, but when you play checkers, it pays to think about checkers and about your particular checkers position, your likely next moves, and successor positions. However you select actions, it is these states and actions that are of highest priority for backups and where you most urgently want your approximate value function to be accurate. Not only should your computation be preferentially devoted to imminent events, but so should your limited memory resources. In chess, for example, there are far too many possible positions to store distinct value estimates for each of them, but chess programs based on heuristic search can easily store distinct estimates for the millions of positions they encounter looking ahead from a single position. This great focusing of memory and computational resources on the current decision is presumably the reason why heuristic search can be so effective.

The distribution of backups can be altered in similar ways to focus on the current state and its likely successors. As a limiting case we might use exactly the methods of heuristic search to construct a search tree, and then perform the individual, one-step backups from bottom up, as suggested by Figure 8.15. If

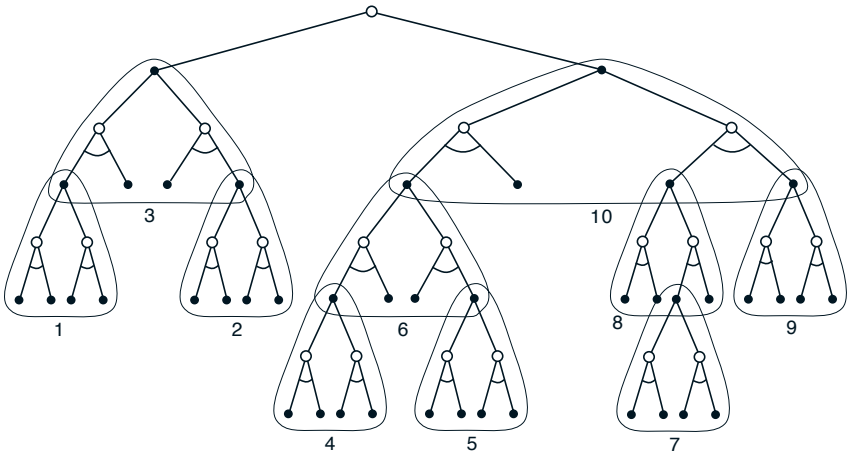


Figure 8.15: The deep backups of heuristic search can be implemented as a sequence of one-step backups (shown here outlined). The ordering shown is for a selective depth-first search.

the backups are ordered in this way and a table-lookup representation is used, then exactly the same backup would be achieved as in heuristic search. Any state-space search can be viewed in this way as the piecing together of a large number of individual one-step backups. Thus, the performance improvement observed with deeper searches is not due to the use of multistep backups as such. Instead, it is due to the focus and concentration of backups on states and actions immediately downstream from the current state. By devoting a large amount of computation specifically relevant to the candidate actions, a much better decision can be made than by relying on unfocused backups.

8.8 Summary

We have presented a perspective emphasizing the surprisingly close relationships between planning optimal behavior and learning optimal behavior. Both involve estimating the same value functions, and in both cases it is natural to update the estimates incrementally, in a long series of small backup operations. This makes it straightforward to integrate learning and planning processes simply by allowing both to update the same estimated value function. In addition, any of the learning methods can be converted into planning methods simply by applying them to simulated (model-generated) experience rather than to real experience. In this case learning and planning become even more similar; they are possibly identical algorithms operating on two different sources of experience.

It is straightforward to integrate incremental planning methods with acting and model-learning. Planning, acting, and model-learning interact in a circular fashion (Figure 8.2), each producing what the other needs to improve; no other interaction among them is either required or prohibited. The most natural approach is for all processes to proceed asynchronously and in parallel. If the processes must share computational resources, then the division can be handled almost arbitrarily—by whatever organization is most convenient and efficient for the task at hand.

In this chapter we have touched upon a number of dimensions of variation among state-space planning methods. One of the most important of these is the distribution of backups, that is, of the focus of search. Prioritized sweeping focuses on the predecessors of states whose values have recently changed. Heuristic search applied to reinforcement learning focuses, inter alia, on the successors of the current state. Trajectory sampling is a convenient way of focusing on the on-policy distribution. All of these approaches can significantly speed planning and are current topics of research.

Another interesting dimension of variation is the size of backups. The smaller the backups, the more incremental the planning methods can be. Among the smallest backups are one-step sample backups. We presented one study suggesting that one-step sample backups may be preferable on very large problems. A related issue is the depth of backups. In many cases deep backups can be implemented as sequences of shallow backups.

8.9 Bibliographical and Historical Remarks

9.1 The overall view of planning and learning presented here has developed gradually over a number of years, in part by the authors (Sutton, 1990, 1991a, 1991b; Barto, Bradtke, and Singh, 1991, 1995; Sutton and Pinette, 1985; Sutton and Barto, 1981b); it has been strongly influenced by Agre and Chapman (1990; Agre 1988), Bertsekas and Tsitsiklis (1989), Singh (1993), and others. The authors were also strongly influenced by psychological studies of latent learning (Tolman, 1932) and by psychological views of the nature of thought (e.g., Galanter and Gerstenhaber, 1956; Craik, 1943; Campbell, 1960; Dennett, 1978).

9.2–3 The terms *direct* and *indirect*, which we use to describe different kinds of reinforcement learning, are from the adaptive control literature (e.g., Goodwin and Sin, 1984), where they are used to make the same kind of distinction. The term *system identification* is used in adaptive control for what we call *model-learning* (e.g., Goodwin and Sin, 1984; Ljung

and Söderstrom, 1983; Young, 1984). The Dyna architecture is due to Sutton (1990), and the results in these sections are based on results reported there.

9.4 Prioritized sweeping was developed simultaneously and independently by Moore and Atkeson (1993) and Peng and Williams (1993). The results in Figure 8.10 are due to Peng and Williams (1993). The results in Figure 8.11 are due to Moore and Atkeson.

9.5 This section was strongly influenced by the experiments of Singh (1993).

9.7 For further reading on heuristic search, the reader is encouraged to consult texts and surveys such as those by Russell and Norvig (1995) and Korf (1988). Peng and Williams (1993) explored a forward focusing of backups much as is suggested in this section.

Part III

Approximate Solution Methods

Chapter 9

On-policy Approximation of Action Values

We have so far assumed that our estimates of value functions are represented as a table with one entry for each state or for each state–action pair. This is a particularly clear and instructive case, but of course it is limited to tasks with small numbers of states and actions. The problem is not just the memory needed for large tables, but the time and data needed to fill them accurately. In other words, the key issue is that of *generalization*. How can experience with a limited subset of the state space be usefully generalized to produce a good approximation over a much larger subset?

This is a severe problem. In many tasks to which we would like to apply reinforcement learning, most states encountered will never have been experienced exactly before. This will almost always be the case when the state or action spaces include continuous variables or complex sensations, such as a visual image. The only way to learn anything at all on these tasks is to generalize from previously experienced states to ones that have never been seen.

Fortunately, generalization from examples has already been extensively studied, and we do not need to invent totally new methods for use in reinforcement learning. To a large extent we need only combine reinforcement learning methods with existing generalization methods. The kind of generalization we require is often called *function approximation* because it takes examples from a desired function (e.g., a value function) and attempts to generalize from them to construct an approximation of the entire function. Function approximation is an instance of *supervised learning*, the primary topic studied in machine learning, artificial neural networks, pattern recognition, and statistical curve fitting. In principle, any of the methods studied in these fields can be used in

reinforcement learning as described in this chapter.

9.1 Value Prediction with Function Approximation

As usual, we begin with the prediction problem of estimating the state-value function v_π from experience generated using policy π. The novelty in this chapter is that the approximate value function is represented not as a table but as a parameterized functional form with parameter vector $\mathbf{w} \in \mathbb{R}^n$. We will write , $\hat{v}(s, \mathbf{w})$ for the approximated value of state s given weight vector $\mathbf{w}$. For example, $\hat{v}$ might be the function computed by an artificial neural network, with $\mathbf{w}$ the vector of connection weights. By adjusting the weights, any of a wide range of different functions $\hat{v}$ can be implemented by the network. Or $\hat{v}$ might be the function computed by a decision tree, where $\mathbf{w}$ is all the parameters defining the split points and leaf values of the tree. Typically, the number of parameters n (the number of components of $\mathbf{w}$) is much less than the number of states, and changing one parameter changes the estimated value of many states. Consequently, when a single state is backed up, the change generalizes from that state to affect the values of many other states.

All of the prediction methods covered in this book have been described as backups, that is, as updates to an estimated value function that shift its value at particular states toward a "backed-up value" for that state. Let us refer to an individual backup by the notation $s \mapsto v$, where s is the state backed up and v is the backed-up value, or target, that s's estimated value is shifted toward. For example, the Monte Carlo backup for value prediction is $S_t \mapsto G_t$, the TD(0) backup is $S_t \mapsto R_{t+1} + \gamma\hat{v}(S_{t+1}, \mathbf{w}_t)$, and the general TD($\lambda$) backup is $S_t \mapsto G_t^\lambda$. In the DP policy evaluation backup $s \mapsto \mathbb{E}_\pi[R_{t+1} + \gamma\hat{v}(S_{t+1}, \mathbf{w}_t) \mid S_t = s]$, an arbitrary state s is backed up, whereas in the the other cases the state, S_t, encountered in (possibly simulated) experience is backed up.

It is natural to interpret each backup as specifying an example of the desired input–output behavior of the estimated value function. In a sense, the backup $s \mapsto v$ means that the estimated value for state s should be more like v. Up to now, the actual update implementing the backup has been trivial: the table entry for s's estimated value has simply been shifted a fraction of the way toward v. Now we permit arbitrarily complex and sophisticated function approximation methods to implement the backup. The normal inputs to these methods are examples of the desired input–output behavior of the function they are trying to approximate. We use these methods for value prediction

simply by passing to them the $s \mapsto v$ of each backup as a training example. We then interpret the approximate function they produce as an estimated value function.

Viewing each backup as a conventional training example in this way enables us to use any of a wide range of existing function approximation methods for value prediction. In principle, we can use any method for supervised learning from examples, including artificial neural networks, decision trees, and various kinds of multivariate regression. However, not all function approximation methods are equally well suited for use in reinforcement learning. The most sophisticated neural network and statistical methods all assume a static training set over which multiple passes are made. In reinforcement learning, however, it is important that learning be able to occur on-line, while interacting with the environment or with a model of the environment. To do this requires methods that are able to learn efficiently from incrementally acquired data. In addition, reinforcement learning generally requires function approximation methods able to handle nonstationary target functions (target functions that change over time). For example, in GPI control methods we often seek to learn q_π while π changes. Even if the policy remains the same, the target values of training examples are nonstationary if they are generated by bootstrapping methods (DP and TD). Methods that cannot easily handle such nonstationarity are less suitable for reinforcement learning.

What performance measures are appropriate for evaluating function approximation methods? Most supervised learning methods seek to minimize the root-mean-squared error (RMSE) over some distribution over the inputs. In our value prediction problem, the inputs are states and the target function is the true value function v_π, so RMSE for an approximation $\hat{v}$, using parameter $\mathbf{w}$, is

$$RMSE(\mathbf{w}) = \sqrt{\sum_{s \in \mathcal{S}} d(s) \Big[v_\pi(s) - \hat{v}(s, \mathbf{w}) \Big]^2}, \tag{9.1}$$

where $d : \mathcal{S} \to [0, 1]$, such that $\sum_s d(s) = 1$, is a distribution over the states specifying the relative importance of errors in different states. This distribution is important because it is usually not possible to reduce the error to zero at all states. After all, there are generally far more states than there are components to $\mathbf{w}$. The flexibility of the function approximator is thus a scarce resource. Better approximation at some states can be gained, generally, only at the expense of worse approximation at other states. The distribution specifies how these trade-offs should be made.

The distribution d is also usually the distribution from which the states in the training examples are drawn, and thus the distribution of states at which

backups are done. If we wish to minimize error over a certain distribution of states, then it makes sense to train the function approximator with examples from that same distribution. For example, if you want a uniform level of error over the entire state set, then it makes sense to train with backups distributed uniformly over the entire state set, such as in the exhaustive sweeps of some DP methods. Henceforth, let us assume that the distribution of states at which backups are done and the distribution that weights errors, d, are the same.

A distribution of particular interest is the one describing the frequency with which states are encountered while the agent is interacting with the environment and selecting actions according to π, the policy whose value function we are approximating. We call this the *on-policy distribution*, in part because it is the distribution of backups in on-policy control methods. Minimizing error over the on-policy distribution focuses function approximation resources on the states that actually occur while following the policy, ignoring those that never occur. The on-policy distribution is also the one for which it is easiest to get training examples using Monte Carlo or TD methods. These methods generate backups from sample experience using the policy π. Because a backup is generated for each state encountered in the experience, the training examples available are naturally distributed according to the on-policy distribution. Stronger convergence results are available for the on-policy distribution than for other distributions, as we discuss later.

It is not completely clear that we should care about minimizing the RMSE. Our goal in value prediction is potentially different because our ultimate purpose is to use the predictions to aid in finding a better policy. The best predictions for that purpose are not necessarily the best for minimizing RMSE. However, it is not yet clear what a more useful alternative goal for value prediction might be. For now, we continue to focus on RMSE.

An ideal goal in terms of RMSE would be to find a *global optimum*, a parameter vector $\mathbf{w}^*$ for which $RMSE(\mathbf{w}^*) \leq RMSE(\mathbf{w})$ for all possible $\mathbf{w}$. Reaching this goal is sometimes possible for simple function approximators such as linear ones, but is rarely possible for complex function approximators such as artificial neural networks and decision trees. Short of this, complex function approximators may seek to converge instead to a *local optimum*, a parameter vector $\mathbf{w}^*$ for which $RMSE(\mathbf{w}^*) \leq RMSE(\mathbf{w})$ for all $\mathbf{w}$ in some neighborhood of $\mathbf{w}^*$. Although this guarantee is only slightly reassuring, it is typically the best that can be said for nonlinear function approximators. For many cases of interest in reinforcement learning, convergence to an optimum, or even all bound of an optimum may still be achieved with some methods. Other methods may in fact diverge, with their RMSE approaching infinity in the limit.

In this section we have outlined a framework for combining a wide range of reinforcement learning methods for value prediction with a wide range of function approximation methods, using the backups of the former to generate training examples for the latter. We have also outlined a range of RMSE performance measures to which these methods may aspire. The range of possible methods is far too large to cover all, and anyway too little is known about most of them to make a reliable evaluation or recommendation. Of necessity, we consider only a few possibilities. In the rest of this chapter we focus on function approximation methods based on gradient principles, and on linear gradient-descent methods in particular. We focus on these methods in part because we consider them to be particularly promising and because they reveal key theoretical issues, but also because they are simple and our space is limited. If we had another chapter devoted to function approximation, we would also cover at least memory-based and decision-tree methods.

9.2 Gradient-Descent Methods

We now develop in detail one class of learning methods for function approximation in value prediction, those based on gradient descent. Gradient-descent methods are among the most widely used of all function approximation methods and are particularly well suited to online reinforcement learning.

In gradient-descent methods, the parameter vector is a column vector with a fixed number of real valued components, $\mathbf{w} = (w_1, w_2, \ldots, w_n)^\top$ (the $^\top$ here denotes transpose), and the approximate value function $\hat{v}(s, \mathbf{w})$ is a smooth differentiable function of $\mathbf{w}$ for all $s \in \mathcal{S}$. We will be updating $\mathbf{w}$ at each of a series of discrete time steps, $t = 1, 2, 3, \ldots$, so we will need a notation $\mathbf{w}_t$ for the weight vector at each step. For now, let us assume that, on each step, we observe a new example $S_t \mapsto v_\pi(S_t)$ consisting of a (possibly randomly selected) state S_t and its true value under the policy. These states might be successive states from an interaction with the environment, but for now we do not assume so. Even though we are given the exact, correct values, $v_\pi(S_t)$ for each S_t, there is still a difficult problem because our function approximator has limited resources and thus limited resolution. In particular, there is generally no $\mathbf{w}$ that gets all the states, or even all the examples, exactly correct. In addition, we must generalize to all the other states that have not appeared in examples.

We assume that states appear in examples with the same distribution, d, over which we are trying to minimize the RMSE as given by (9.1). A good strategy in this case is to try to minimize error on the observed examples. Gradient-descent methods do this by adjusting the parameter vector after

each example by a small amount in the direction that would most reduce the error on that example:

$$\begin{aligned}\mathbf{w}_{t+1} &= \mathbf{w}_t - \frac{1}{2}\alpha\nabla_{\mathbf{w}_t}\Big[v_\pi(S_t) - \hat{v}(S_t,\mathbf{w}_t)\Big]^2 \\ &= \mathbf{w}_t + \alpha\Big[v_\pi(S_t) - \hat{v}(S_t,\mathbf{w}_t)\Big]\nabla_{\mathbf{w}_t}\hat{v}(S_t,\mathbf{w}_t), \end{aligned} \tag{9.2}$$

where α is a positive step-size parameter, and $\nabla_{\mathbf{w}_t} f(\mathbf{w}_t)$, for any function f, denotes the vector of partial derivatives,

$$\left(\frac{\partial f(\mathbf{w}_t)}{\partial w_{t,1}}, \frac{\partial f(\mathbf{w}_t)}{\partial w_{t,2}}, \ldots, \frac{\partial f(\mathbf{w}_t)}{\partial w_{t,n}}\right)^\top.$$

This derivative vector is the *gradient* of f with respect to $\mathbf{w}_t$. This kind of method is called *gradient descent* because the overall step in $\mathbf{w}_t$ is proportional to the negative gradient of the example's squared error. This is the direction in which the error falls most rapidly.

It may not be immediately apparent why only a small step is taken in the direction of the gradient. Could we not move all the way in this direction and completely eliminate the error on the example? In many cases this could be done, but usually it is not desirable. Remember that we do not seek or expect to find a value function that has zero error on all states, but only an approximation that balances the errors in different states. If we completely corrected each example in one step, then we would not find such a balance. In fact, the convergence results for gradient methods assume that the step-size parameter decreases over time. If it decreases in such a way as to satisfy the standard stochastic approximation conditions (2.7), then the gradient-descent method (9.2) is guaranteed to converge to a local optimum.

We turn now to the case in which the target output, V_t, of the tth training example, $S_t \mapsto V_t$, is not the true value, $v_\pi(S_t)$, but some, possibly random, approximation of it. For example, V_t might be a noise-corrupted version of $v_\pi(S_t)$, or it might be one of the backed-up values using $\hat{v}$ mentioned in the previous section. In such cases we cannot perform the exact update (9.2) because $v_\pi(S_t)$ is unknown, but we can approximate it by substituting V_t in place of $v_\pi(S_t)$. This yields the general gradient-descent method for state-value prediction:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\Big[V_t - \hat{v}(S_t,\mathbf{w}_t)\Big]\nabla_{\mathbf{w}_t}\hat{v}(S_t,\mathbf{w}_t). \tag{9.3}$$

If V_t is an *unbiased* estimate, that is, if $\mathbb{E}[V_t] = v_\pi(S_t)$, for each t, then $\mathbf{w}_t$ is guaranteed to converge to a local optimum under the usual stochastic approximation conditions (2.7) for decreasing the step-size parameter α.

For example, suppose the states in the examples are the states generated by interaction (or simulated interaction) with the environment using policy π. Let G_t denote the return following each state, S_t. Because the true value of a state is the expected value of the return following it, the Monte Carlo target $V_t = G_t$ is by definition an unbiased estimate of $v_\pi(S_t)$. With this choice, the general gradient-descent method (9.3) converges to a locally optimal approximation to $v_\pi(S_t)$. Thus, the gradient-descent version of Monte Carlo state-value prediction is guaranteed to find a locally optimal solution.

Similarly, we can use n-step TD returns and their averages for V_t. For example, the gradient-descent form of TD(λ) uses the λ-return, $V_t = G_t^\lambda$, as its approximation to $v_\pi(S_t)$, yielding the forward-view update:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\Big[G_t^\lambda - \hat{v}(S_t, \mathbf{w}_t)\Big]\nabla_{\mathbf{w}_t}\hat{v}(S_t, \mathbf{w}_t). \tag{9.4}$$

Unfortunately, for $\lambda < 1$, G_t^λ is not an unbiased estimate of $v_\pi(S_t)$, and thus this method does not converge to a local optimum. The situation is the same when DP targets are used such as $V_t = \mathbb{E}_\pi[R_{t+1} + \gamma\hat{v}(S_{t+1}, \mathbf{w}_t) \mid S_t]$. Nevertheless, such bootstrapping methods can be quite effective, and other performance guarantees are available for important special cases, as we discuss later in this chapter. For now we emphasize the relationship of these methods to the general gradient-descent form (9.3). Although increments as in (9.4) are not themselves gradients, it is useful to view this method as a gradient-descent method (9.3) with a bootstrapping approximation in place of the desired output, $v_\pi(S_t)$.

As (9.4) provides the forward view of gradient-descent TD(λ), so the backward view is provided by

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\delta_t\mathbf{z}_t, \tag{9.5}$$

where δ_t is the usual TD error, now using $\hat{v}$,

$$\delta_t = R_{t+1} + \gamma\hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t), \tag{9.6}$$

and $\mathbf{z}_t = (z_{t,1}, z_{t,2}, \ldots, z_{t,n})^\top$ is a column vector of eligibility traces, one for each component of $\mathbf{w}_t$, updated by

$$\mathbf{z}_t = \gamma\lambda\mathbf{z}_{t-1} + \nabla_{\mathbf{w}_t}\hat{v}(S_t, \mathbf{w}_t), \tag{9.7}$$

with $\mathbf{z}_0 = \mathbf{0}$. A complete algorithm for on-line gradient-descent TD(λ) is given in Figure 9.1.

Initialize $\mathbf{w}$ as appropriate for the problem, e.g., $\mathbf{w} = \mathbf{0}$
Repeat (for each episode):
  $\mathbf{z} = 0$
  $S \leftarrow$ initial state of episode
  Repeat (for each step of episode):
    $A \leftarrow$ action given by π for S
    Take action A, observe reward, R, and next state, S'
    $\delta \leftarrow R + \gamma\hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$
    $\mathbf{z} \leftarrow \gamma\lambda\mathbf{z} + \nabla_{\mathbf{w}}\hat{v}(S, \mathbf{w})$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\delta\mathbf{z}$
    $S \leftarrow S'$
  until S' is terminal

Figure 9.1: On-line gradient-descent TD(λ) for estimating v_π.

Two methods for gradient-based function approximation have been used widely in reinforcement learning. One is multilayer artificial neural networks using the error backpropagation algorithm. This maps immediately onto the equations and algorithms just given, where the backpropagation process is the way of computing the gradients. The second popular form is the linear form, which we discuss extensively in the next section.

Exercise 9.1 Show that table-lookup TD(λ) is a special case of general TD(λ) as given by equations (9.5–9.7).

Exercise 9.2 *State aggregation* is a simple form of generalizing function approximation in which states are grouped together, with one table entry (value estimate) used for each group. Whenever a state in a group is encountered, the group's entry is used to determine the state's value, and when the state is updated, the group's entry is updated. Show that this kind of state aggregation is a special case of a gradient method such as (9.4).

Exercise 9.3 The equations given in this section are for the on-line version of gradient-descent TD(λ). What are the equations for the *off-line* version? Give a complete description specifying the new weight vector at the end of an episode, $\mathbf{w}'$, in terms of the weight vector used during the episode, $\mathbf{w}$. Start by modifying a forward-view equation for TD(λ), such as (9.4).

Exercise 9.4 For off-line updating, show that equations (9.5–9.7) produce updates identical to (9.4).

9.3 Linear Methods

One of the most important special cases of gradient-descent function approximation is that in which the approximate function, $\hat{v}$, is a linear function of the parameter vector, $\mathbf{w}$. Corresponding to every state s, there is a vector of features $\mathbf{x}(s) = (x_1(s), x_2(s), \ldots, x_n(s))^\top$, with the same number of components as $\mathbf{w}$. The features may be constructed from the states in many different ways; we cover a few possibilities below. However the features are constructed, the approximate state-value function is given by

$$\hat{v}(s, \mathbf{w}) = \mathbf{w}^\top \mathbf{x}(s) = \sum_{i=1}^{n} w_i x_i(s). \tag{9.8}$$

In this case the approximate value function is said to be *linear in the parameters*, or simply *linear*.

It is natural to use gradient-descent updates with linear function approximation. The gradient of the approximate value function with respect to $\mathbf{w}$ in this case is

$$\nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w}) = \mathbf{x}(s).$$

Thus, the general gradient-descent update (9.3) reduces to a particularly simple form in the linear case. In addition, in the linear case there is only one optimum $\mathbf{w}^*$ (or, in degenerate cases, one set of equally good optima). Thus, any method guaranteed to converge to or near a local optimum is automatically guaranteed to converge to or near the global optimum. Because it is simple in these ways, the linear, gradient-descent case is one of the most favorable for mathematical analysis. Almost all useful convergence results for learning systems of all kinds are for linear (or simpler) function approximation methods.

In particular, the gradient-descent TD(λ) algorithm discussed in the previous section (Figure 9.1) has been proved to converge in the linear case if the step-size parameter is reduced over time according to the usual conditions (2.7). Convergence is not to the minimum-error parameter vector, $\mathbf{w}^*$, but to a nearby parameter vector, $\mathbf{w}_\infty$, whose error is bounded according to

$$RMSE(\mathbf{w}_\infty) \quad \leq \quad \frac{1-\gamma\lambda}{1-\gamma} RMSE(\mathbf{w}^*). \tag{9.9}$$

That is, the asymptotic error is no more than $\frac{1-\gamma\lambda}{1-\gamma}$ times the smallest possible error. As λ approaches 1, the bound approaches the minimum error. An analogous bound applies to other on-policy bootstrapping methods. For example,

linear gradient-descent DP backups (9.3), with the on-policy distribution, will converge to the same result as TD(0). Technically, this bound applies only to discounted continuing tasks, but a related result presumably holds for episodic tasks. There are also a few technical conditions on the rewards, features, and decrease in the step-size parameter, which we are omitting here. The full details can be found in the original paper (Tsitsiklis and Van Roy, 1997).

Critical to the above result is that states are backed up according to the on-policy distribution. For other backup distributions, bootstrapping methods using function approximation may actually diverge to infinity. Examples of this and a discussion of possible solution methods are given in Section 8.5

Beyond these theoretical results, linear learning methods are also of interest because in practice they can be very efficient in terms of both data and computation. Whether or not this is so depends critically on how the states are represented in terms of the features. Choosing features appropriate to the task is an important way of adding prior domain knowledge to reinforcement learning systems. Intuitively, the features should correspond to the natural features of the task, those along which generalization is most appropriate. If we are valuing geometric objects, for example, we might want to have features for each possible shape, color, size, or function. If we are valuing states of a mobile robot, then we might want to have features for locations, degrees of remaining battery power, recent sonar readings, and so on.

In general, we also need features for combinations of these natural qualities. This is because the linear form prohibits the representation of interactions between features, such as the presence of feature i being good only in the absence of feature j. For example, in the pole-balancing task (Example 3.4), a high angular velocity may be either good or bad depending on the angular position. If the angle is high, then high angular velocity means an imminent danger of falling, a bad state, whereas if the angle is low, then high angular velocity means the pole is righting itself, a good state. In cases with such interactions one needs to introduce features for conjunctions of feature values when using linear function approximation methods. We next consider some general ways of doing this.

Exercise 9.5 How could we reproduce the tabular case within the linear framework?

Exercise 9.6 How could we reproduce the state aggregation case (see Exercise 8.4) within the linear framework?

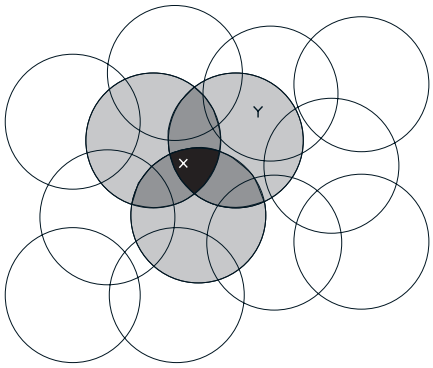


Figure 9.2: Coarse coding. Generalization from state X to state Y depends on the number of their features whose receptive fields (in this case, circles) overlap. These states have one feature in common, so there will be slight generalization between them.

Coarse Coding

Consider a task in which the state set is continuous and two-dimensional. A state in this case is a point in 2-space, a vector with two real components. One kind of feature for this case is those corresponding to *circles* in state space, as shown in Figure 9.2. If the state is inside a circle, then the corresponding feature has the value 1 and is said to be *present*; otherwise the feature is 0 and is said to be *absent*. This kind of 1–0-valued feature is called a *binary feature*. Given a state, which binary features are present indicate within which circles the state lies, and thus coarsely code for its location. Representing a state with features that overlap in this way (although they need not be circles or binary) is known as *coarse coding*.

Assuming linear gradient-descent function approximation, consider the effect of the size and density of the circles. Corresponding to each circle is a single parameter (a component of $\mathbf{w}$) that is affected by learning. If we train at one point (state) X, then the parameters of all circles intersecting X will be affected. Thus, by (9.8), the approximate value function will be affected at all points within the union of the circles, with a greater effect the more circles a point has "in common" with X, as shown in Figure 9.2. If the circles are small, then the generalization will be over a short distance, as in Figure 9.3a, whereas if they are large, it will be over a large distance, as in Figure 9.3b. Moreover, the shape of the features will determine the nature of the generalization. For example, if they are not strictly circular, but are elongated in one direction, then generalization will be similarly affected, as in Figure 9.3c.

Features with large receptive fields give broad generalization, but might also seem to limit the learned function to a coarse approximation, unable

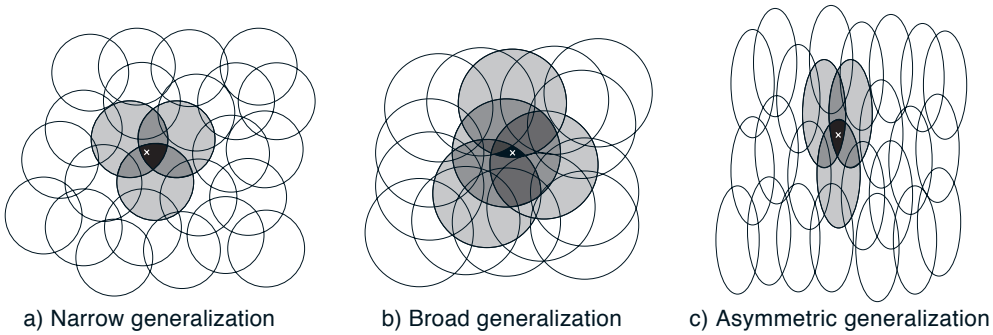


Figure 9.3: Generalization in linear function approximation methods is determined by the sizes and shapes of the features' receptive fields. All three of these cases have roughly the same number and density of features.

to make discriminations much finer than the width of the receptive fields. Happily, this is not the case. Initial generalization from one point to another is indeed controlled by the size and shape of the receptive fields, but acuity, the finest discrimination ultimately possible, is controlled more by the total number of features.

Example 9.1: Coarseness of Coarse Coding This example illustrates the effect on learning of the size of the receptive fields in coarse coding. Linear function approximation based on coarse coding and (9.3) was used to learn a one-dimensional square-wave function (shown at the top of Figure 9.4). The values of this function were used as the targets, V_t. With just one dimension, the receptive fields were intervals rather than circles. Learning was repeated with three different sizes of the intervals: narrow, medium, and broad, as shown at the bottom of the figure. All three cases had the same density of features, about 50 over the extent of the function being learned. Training examples were generated uniformly at random over this extent. The step-size parameter was $\alpha = \frac{0.2}{m}$, where m is the number of features that were present at one time. Figure 9.4 shows the functions learned in all three cases over the course of learning. Note that the width of the features had a strong effect early in learning. With broad features, the generalization tended to be broad; with narrow features, only the close neighbors of each trained point were changed, causing the function learned to be more bumpy. However, the final function learned was affected only slightly by the width of the features. Receptive field shape tends to have a strong effect on generalization but little effect on asymptotic solution quality. ■

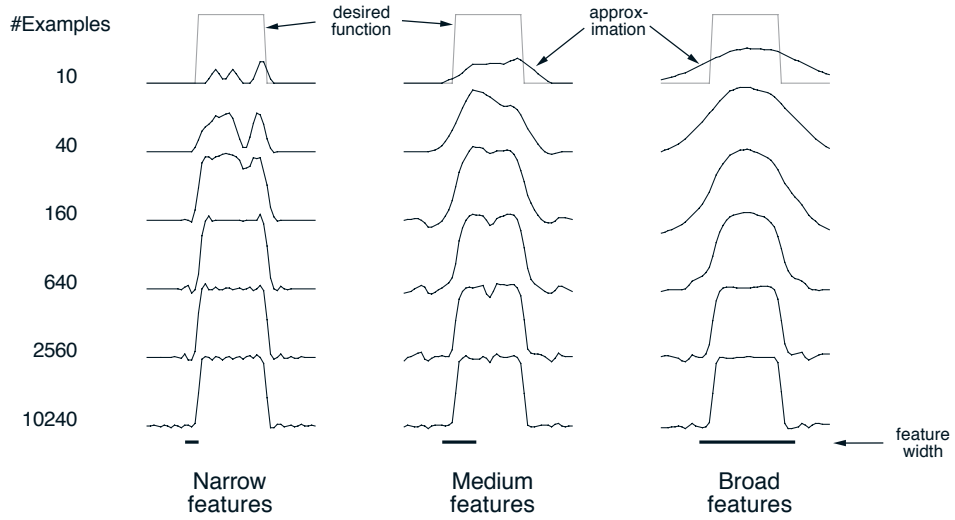


Figure 9.4: Example of feature width's strong effect on initial generalization (first row) and weak effect on asymptotic accuracy (last row).

Tile Coding

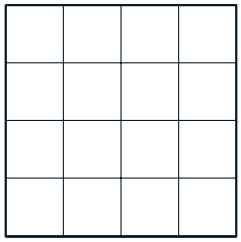
Tile coding is a form of coarse coding that is particularly well suited for use on sequential digital computers and for efficient on-line learning. In tile coding the receptive fields of the features are grouped into exhaustive partitions of the input space. Each such partition is called a *tiling*, and each element of the partition is called a *tile*. Each tile is the receptive field for one binary feature.

An immediate advantage of tile coding is that the overall number of features that are present at one time is strictly controlled and independent of the input state. Exactly one feature is present in each tiling, so the total number of features present is always the same as the number of tilings. This allows the step-size parameter, α, to be set in an easy, intuitive way. For example, choosing $\alpha = \frac{1}{m}$, where m is the number of tilings, results in exact one-trial learning. If the example $s \mapsto v$ is received, then whatever the prior value, $\hat{v}(s)$, the new value will be $\hat{v}(s) = v$. Usually one wishes to change more slowly than this, to allow for generalization and stochastic variation in target outputs. For example, one might choose $\alpha = \frac{1}{10m}$, in which case one would move one-tenth of the way to the target in one update.

Because tile coding uses exclusively binary (0–1-valued) features, the weighted sum making up the approximate value function (9.8) is almost trivial to compute. Rather than performing n multiplications and additions, one simply computes the indices of the $m \ll n$ present features and then adds up the m corresponding components of the parameter vector. The eligibility trace

computation (9.7) is also simplified because the components of the gradient, $\nabla_{\mathbf{w}} \hat{v}(s)$, are also usually 0, and otherwise 1.

The computation of the indices of the present features is particularly easy if gridlike tilings are used. The ideas and techniques here are best illustrated by examples. Suppose we address a task with two continuous state variables. Then the simplest way to tile the space is with a uniform two-dimensional grid:



Given the x and y coordinates of a point in the space, it is computationally easy to determine the index of the tile it is in. When multiple tilings are used, each is offset by a different amount, so that each cuts the space in a different way. In the example shown in Figure 9.5, an extra row and an extra column of tiles have been added to the grid so that no points are left uncovered. The two tiles highlighted are those that are present in the state indicated by the X. The different tilings may be offset by random amounts, or by cleverly designed deterministic strategies (simply offsetting each dimension by the same increment is known not to be a good idea). The effects on generalization and asymptotic accuracy illustrated in Figures 9.3 and 9.4 apply here as well. The width and shape of the tiles should be chosen to match the width of generalization that one expects to be appropriate. The number of tilings should be chosen to influence the density of tiles. The denser the tiling, the finer and more accurately the desired function can be approximated, but the greater the computational costs.

It is important to note that the tilings can be arbitrary and need not be

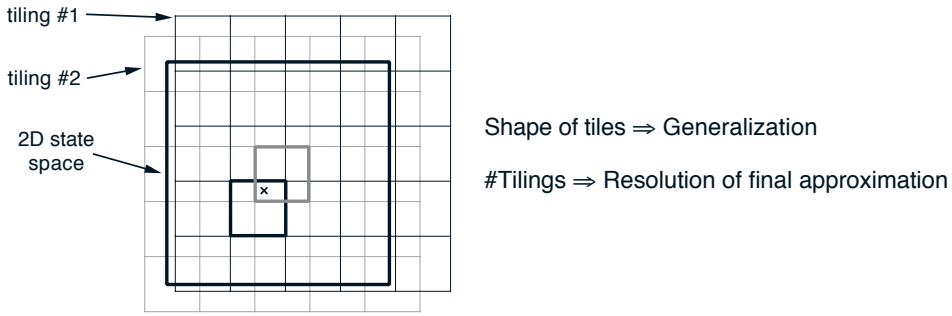


Figure 9.5: Multiple, overlapping gridtilings.

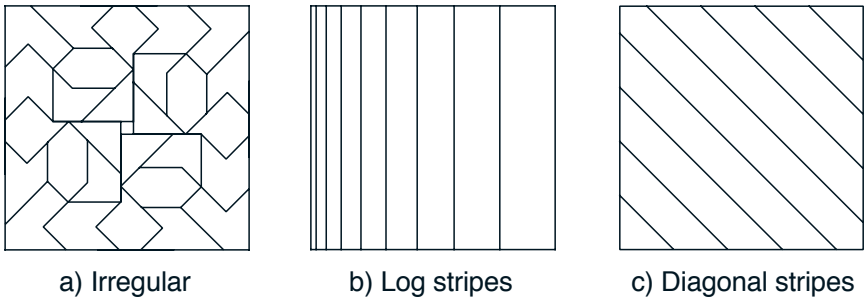
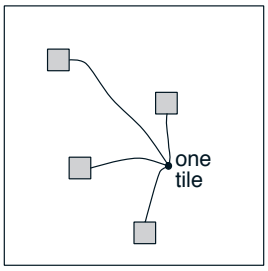


Figure 9.6: Tilings.

uniform grids. Not only can the tiles be strangely shaped, as in Figure 9.6a, but they can be shaped and distributed to give particular kinds of generalization. For example, the stripe tiling in Figure 9.6b will promote generalization along the vertical dimension and discrimination along the horizontal dimension, particularly on the left. The diagonal stripe tiling in Figure 9.6c will promote generalization along one diagonal. In higher dimensions, axis-aligned stripes correspond to ignoring some of the dimensions in some of the tilings, that is, to hyperplanar slices.

Another important trick for reducing memory requirements is *hashing*—a consistent pseudo-random collapsing of a large tiling into a much smaller set of tiles. Hashing produces tiles consisting of noncontiguous, disjoint regions randomly spread throughout the state space, but that still form an exhaustive tiling. For example, one tile might consist of the four subtiles shown below:



Through hashing, memory requirements are often reduced by large factors with little loss of performance. This is possible because high resolution is needed in only a small fraction of the state space. Hashing frees us from the curse of dimensionality in the sense that memory requirements need not be exponential in the number of dimensions, but need merely match the real demands of the task. Good public-domain implementations of tile coding, including hashing, are widely available.

Exercise 9.7 Suppose we believe that one of two state dimensions is more likely to have an effect on the value function than is the other, that general-

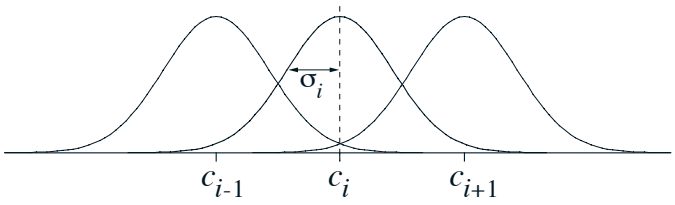


Figure 9.7: One-dimensional radial basis functions.

ization should be primarily across this dimension rather than along it. What kind of tilings could be used to take advantage of this prior knowledge?

Radial Basis Functions

Radial basis functions (RBFs) are the natural generalization of coarse coding to continuous-valued features. Rather than each feature being either 0 or 1, it can be anything in the interval $[0, 1]$, reflecting various *degrees* to which the feature is present. A typical RBF feature, i, has a Gaussian (bell-shaped) response $x_i(s)$ dependent only on the distance between the state, s, and the feature's prototypical or center state, c_i, and relative to the feature's width, σ_i:

$$x_i(s) = \exp\left(-\frac{||s - c_i||^2}{2\sigma_i^2}\right).$$

The norm or distance metric of course can be chosen in whatever way seems most appropriate to the states and task at hand. Figure 9.7 shows a one-dimensional example with a Euclidean distance metric.

An *RBF network* is a linear function approximator using RBFs for its features. Learning is defined by equations (9.3) and (9.8), exactly as in other linear function approximators. The primary advantage of RBFs over binary features is that they produce approximate functions that vary smoothly and are differentiable. In addition, some learning methods for RBF networks change the centers and widths of the features as well. Such nonlinear methods may be able to fit the target function much more precisely. The downside to RBF networks, and to nonlinear RBF networks especially, is greater computational complexity and, often, more manual tuning before learning is robust and efficient.

Kanerva Coding

On tasks with very high dimensionality, say *hundreds* of dimensions, tile coding and RBF networks become impractical. If we take either method at face value, its computational complexity increases exponentially with the number of dimensions. There are a number of tricks that can reduce this growth (such as hashing), but even these become impractical after a few tens of dimensions.

On the other hand, some of the general ideas underlying these methods can be practical for high-dimensional tasks. In particular, the idea of representing states by a list of the features present and then mapping those features linearly to an approximation may scale well to large tasks. The key is to keep the number of features from scaling explosively. Is there any reason to think this might be possible?

First we need to establish some realistic expectations. Roughly speaking, a function approximator of a given complexity can only accurately approximate target functions of comparable complexity. But as dimensionality increases, the size of the state space inherently increases exponentially. It is reasonable to assume that in the worst case the complexity of the target function scales like the size of the state space. Thus, if we focus the worst case, then there is no solution, no way to get good approximations for high-dimensional tasks without using resources exponential in the dimension.

A more useful way to think about the problem is to focus on the complexity of the target function as separate and distinct from the size and dimensionality of the state space. The size of the state space may give an upper bound on complexity, but short of that high bound, complexity and dimension can be unrelated. For example, one might have a 1000-dimensional task where only one of the dimensions happens to matter. Given a certain level of complexity, we then seek to be able to accurately approximate any target function of that complexity or less. As the target level of complexity increases, we would like to get by with a proportionate increase in computational resources.

From this point of view, the real source of the problem is the complexity of the target function, or of a reasonable approximation of it, not the dimensionality of the state space. Thus, adding dimensions, such as new sensors or new features, to a task should be almost without consequence if the complexity of the needed approximations remains the same. The new dimensions may even make things easier if the target function can be simply expressed in terms of them. Unfortunately, methods like tile coding and RBF coding do not work this way. Their complexity increases exponentially with dimensionality even if the complexity of the target function does not. For these methods, dimensionality itself is still a problem. We need methods whose complexity is unaffected

by dimensionality per se, methods that are limited only by, and scale well with, the complexity of what they approximate.

One simple approach that meets these criteria, which we call *Kanerva coding*, is to choose binary features that correspond to particular *prototype states*. For definiteness, let us say that the prototypes are randomly selected from the entire state space. The receptive field of such a feature is all states sufficiently close to the prototype. Kanerva coding uses a different kind of distance metric than in is used in tile coding and RBFs. For definiteness, consider a *binary* state space and the *hamming distance*, the number of bits at which two states differ. States are considered similar if they agree on enough dimensions, even if they are totally different on others.

The strength of Kanerva coding is that the complexity of the functions that can be learned depends entirely on the number of features, which bears no necessary relationship to the dimensionality of the task. The number of features can be more or less than the number of dimensions. Only in the worst case must it be exponential in the number of dimensions. Dimensionality itself is thus no longer a problem. Complex functions are still a problem, as they have to be. To handle more complex tasks, a Kanerva coding approach simply needs more features. There is not a great deal of experience with such systems, but what there is suggests that their abilities increase in proportion to their computational resources. This is an area of current research, and significant improvements in existing methods can still easily be found.

9.4 Control with Function Approximation

We now extend value prediction methods using function approximation to control methods, following the pattern of GPI. First we extend the state-value prediction methods to action-value prediction methods, then we combine them with policy improvement and action selection techniques. As usual, the problem of ensuring exploration is solved by pursuing either an on-policy or an off-policy approach.

The extension to action-value prediction is straightforward. In this case it is the approximate action-value function, $\hat{q} \approx q_\pi$, that is represented as a parameterized functional form with parameter vector $\mathbf{w}$. Whereas before we considered random training examples of the form $S_t \mapsto V_t$, now we consider examples of the form $S_t, A_t \mapsto Q_t$. The target output, Q_t, can be any approximation of $q_\pi(S_t, A_t)$, including the usual backed-up values such as the full Monte Carlo return, G_t, or the one-step Sarsa-style return, $G_{t+1} + \gamma\hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t)$.

The general gradient-descent update for action-value prediction is

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\Big[Q_t - \hat{q}(S_t, A_t, \mathbf{w}_t)\Big]\nabla_{\mathbf{w}_t}\hat{q}(S_t, A_t, \mathbf{w}_t).$$

For example, the backward view of the action-value method analogous to TD(λ) is

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\delta_t\mathbf{z}_t,$$

where

$$\delta_t = R_{t+1} + \gamma\hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t),$$

and

$$\mathbf{z}_t = \gamma\lambda\mathbf{z}_{t-1} + \nabla_{\mathbf{w}_t}\hat{q}(S_t, A_t, \mathbf{w}_t),$$

with $\mathbf{z}_0 = \mathbf{0}$. We call this method *gradient-descent Sarsa(λ)*, particularly when it is elaborated to form a full control method. For a constant policy, this method converges in the same way that TD(λ) does, with the same kind of error bound (9.9).

To form control methods, we need to couple such action-value prediction methods with techniques for policy improvement and action selection. Suitable techniques applicable to continuous actions, or to actions from large discrete sets, are a topic of ongoing research with as yet no clear resolution. On the other hand, if the action set is discrete and not too large, then we can use the techniques already developed in previous chapters. That is, for each possible action, a, available in the current state, S_t, we can compute $\hat{q}(S_t, a, \mathbf{w}_t)$ and then find the greedy action $a^*_t = \arg\max_a \hat{q}(S_t, a, \mathbf{w}_t)$. Policy improvement is done by changing the estimation policy to the greedy policy (in off-policy methods) or to a soft approximation of the greedy policy such as the ε-greedy policy (in on-policy methods). Actions are selected according to this same policy in on-policy methods, or by an arbitrary policy in off-policy methods.

Figures 9.8 and 9.9 show examples of on-policy (Sarsa(λ)) and off-policy (Watkins's Q(λ)) control methods using function approximation. Both methods use linear, gradient-descent function approximation with binary features, such as in tile coding and Kanerva coding. Both methods use an ε-greedy policy for action selection, and the Sarsa method uses it for GPI as well. Both compute the sets of present features, $\mathcal{F}_a$, corresponding to the current state and all possible actions, a. If the value function for each action is a separate linear function of the same features (a common case), then the indices of

Let $\mathbf{w}$ and $\mathbf{z}$ be vectors with one component for each possible feature
Let $\mathcal{F}_a$, for every possible action a, be a set of feature indices, initially empty
Initialize $\mathbf{w}$ as appropriate for the problem, e.g., $\mathbf{w} = \mathbf{0}$
Repeat (for each episode):
    $\mathbf{z} = \mathbf{0}$
    $S, A \leftarrow$ initial state and action of episode
    $\mathcal{F}_A \leftarrow$ set of features present in S, A
    Repeat (for each step of episode):
        For all $i \in \mathcal{F}_A$:
            $z_i \leftarrow z_i + 1$ (accumulating traces)
            or $z_i \leftarrow 1$ (replacing traces)
        Take action A, observe reward, R, and next state, S
        $\delta \leftarrow R - \sum_{i \in \mathcal{F}_A} w_i$
        (Note that from here and below, S and A denote the new state and action)
        If S is not terminal, then:
            With probability $1 - \varepsilon$:
                For all $a \in \mathcal{A}(S)$:
                    $\mathcal{F}_a \leftarrow$ set of features present in S, a
                    $\hat{q}_a \leftarrow \sum_{i \in \mathcal{F}_a} w_i$
                $A \leftarrow \arg\max_{a \in \mathcal{A}(S)} \hat{q}_a$
            else
                $A \leftarrow$ a random action $\in \mathcal{A}(S)$
                $\mathcal{F}_A \leftarrow$ set of features present in S, A
                $\hat{q}_A \leftarrow \sum_{i \in \mathcal{F}_A} w_i$
            $\delta \leftarrow \delta + \gamma \hat{q}_A$
        $\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta \mathbf{z}$
        $\mathbf{z} \leftarrow \gamma \lambda \mathbf{z}$

Figure 9.8: Linear, gradient-descent Sarsa(λ) with binary features and ε-greedy policy. Updates for both accumulating and replacing traces are specified, including the option (when using replacing traces) of clearing the traces of nonselected actions.

Let $\mathbf{w}$ and $\mathbf{z}$ be vectors with one component for each possible feature
Let $\mathcal{F}_a$, for every possible action a, be a set of feature indices, initially empty
Initialize $\mathbf{w}$ as appropriate for the problem, e.g., $\mathbf{w} = \mathbf{0}$
Repeat (for each episode):
    $\mathbf{z} = \mathbf{0}$
    $S \leftarrow$ initial state of episode
    Repeat (for each step of episode):
        For all $a \in \mathcal{A}(s)$:
            $\mathcal{F}_a \leftarrow$ set of features present in s, a
            $\hat{q}_a \leftarrow \sum_{i \in \mathcal{F}_a} w_i$
        $A \leftarrow \arg\max_a \hat{q}_a$ with prob. $1 - \varepsilon$, else a random action $\in \mathcal{A}(s)$
        Take action A, observe reward, R, and next state, S'
        $\delta \leftarrow R - \hat{q}_A$
        For all $i \in \mathcal{F}_a$: $z_i = 1$
        If S' is terminal, then $\mathbf{w} \leftarrow \mathbf{w} + \alpha\delta\mathbf{z}$; go to next episode
        For all $a \in \mathcal{A}(s)$:
            $\mathcal{F}_a \leftarrow$ set of features present in s, a
            $\hat{q}_a \leftarrow \sum_{i \in \mathcal{F}_a} w_i$
        $\delta \leftarrow \delta + \gamma \max_{a \in \mathcal{A}(s)} \hat{q}_a$
        $\mathbf{w} \leftarrow \mathbf{w} + \alpha\delta\mathbf{z}$
        $\mathbf{z} \leftarrow \gamma\lambda\mathbf{z}$
        $S \leftarrow S'$

Figure 9.9: A linear, gradient-descent version of Watkins's Q(λ) with binary features, ε-greedy policy, and accumulating traces.

the $\mathcal{F}_a$ for each action are essentially the same, simplifying the computation significantly.

All the methods we have discussed above have used *accumulating* eligibility traces. Although replacing traces (Section 7.8) are known to have advantages in tabular methods, replacing traces do not directly extend to the use of function approximation. Recall that the idea of replacing traces is to reset a state's trace to 1 each time it is visited instead of incrementing it by 1. But with function approximation there is no single trace corresponding to a state, just a trace for each component of $\mathbf{w}$, which corresponds to many states. One approach that seems to work well for linear, gradient-descent function approximation methods with binary features is to treat the features as if they were states for the purposes of replacing traces. That is, each time a state is encountered that has feature i, the trace for feature i is set to 1 rather than being incremented by 1, as it would be with accumulating traces.

When working with state–action traces, it may also be useful to clear (set to zero) the traces of all nonselected actions in the states encountered (see Section 7.8). This idea can also be extended to the case of linear function approximation with binary features. For each state encountered, we first clear the traces of all features for the state and the actions not selected, then we set to 1 the traces of the features for the state and the action that was selected. As we noted for the tabular case, this may or may not be the best way to proceed when using replacing traces. A procedural specification of both kinds of traces, including the optional clearing for nonselected actions, is given for the Sarsa algorithm in Figure 9.8.

Example 9.2: Mountain–Car Task Consider the task of driving an underpowered car up a steep mountain road, as suggested by the diagram in the upper left of Figure 9.10. The difficulty is that gravity is stronger than the car's engine, and even at full throttle the car cannot accelerate up the steep slope. The only solution is to first move away from the goal and up the opposite slope on the left. Then, by applying full throttle the car can build up enough inertia to carry it up the steep slope even though it is slowing down the whole way. This is a simple example of a continuous control task where things have to get worse in a sense (farther from the goal) before they can get better. Many control methodologies have great difficulties with tasks of this kind unless explicitly aided by a human designer.

The reward in this problem is -1 on all time steps until the car moves past its goal position at the top of the mountain, which ends the episode. There are three possible actions: full throttle forward ($+1$), full throttle reverse (-1), and zero throttle (0). The car moves according to a simplified physics. Its

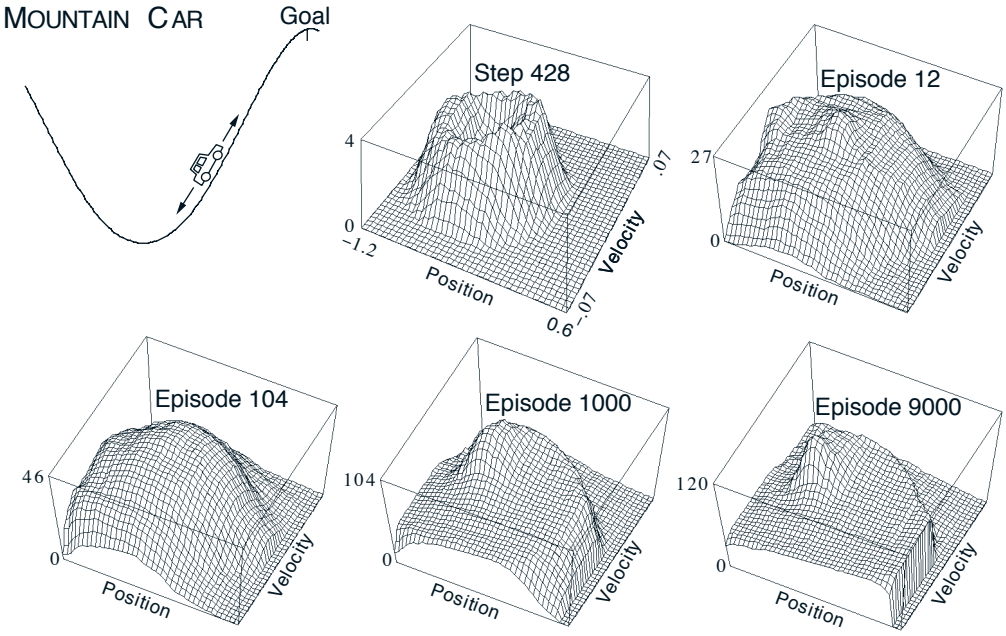


Figure 9.10: The mountain–car task (upper left panel) and the cost-to-go function ($-\max_a \hat{q}(s,a)$) learned during one run.

position, p_t, and velocity, $\dot{p}_t$, are updated by

$$p_{t+1} = \mathit{bound}[p_t + \dot{p}_{t+1}]$$

$$\dot{p}_{t+1} = \mathit{bound}[\dot{p}_t + 0.001A_t - 0.0025\cos(3p_t)],$$

where the *bound* operation enforces $-1.2 \leq p_{t+1} \leq 0.5$ and $-0.07 \leq \dot{p}_{t+1} \leq 0.07$. When p_{t+1} reached the left bound, $\dot{p}_{t+1}$ was reset to zero. When it reached the right bound, the goal was reached and the episode was terminated. Each episode started from a random position and velocity uniformly chosen from these ranges. To convert the two continuous state variables to binary features, we used gridtilings as in Figure 9.5. We used ten 9×9 tilings, each offset by a random fraction of a tile width.

The Sarsa algorithm in Figure 9.8 (using replace traces and the optional clearing) readily solved this task, learning a near optimal policy within 100 episodes. Figure 9.10 shows the negative of the value function (the *cost-to-go* function) learned on one run, using the parameters $\lambda = 0.9$, $\varepsilon = 0$, and $\alpha = 0.05$ (e.g., $\frac{0.5}{m}$). The initial action values were all zero, which was optimistic (all true values are negative in this task), causing extensive exploration to occur even though the exploration parameter, ε, was 0. This can be seen in the middle-top panel of the figure, labeled "Step 428." At this time not even

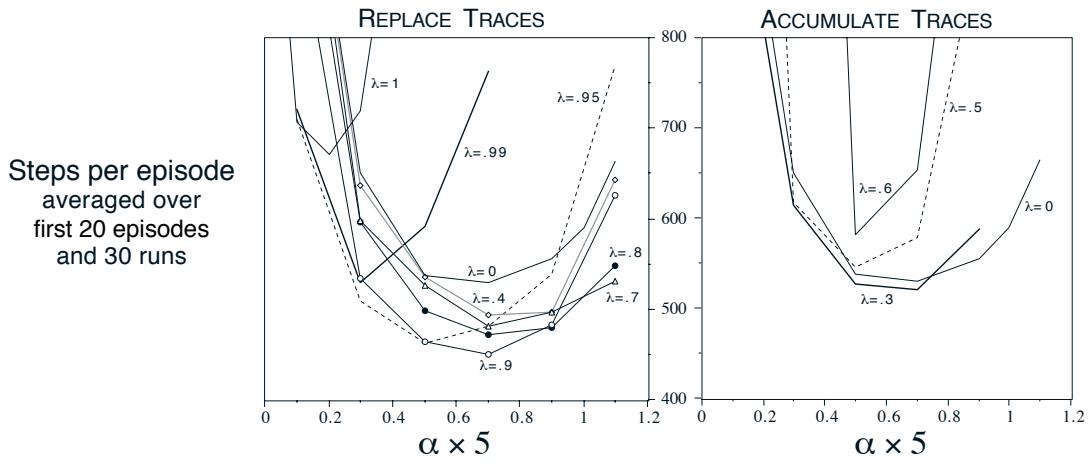


Figure 9.11: The effect of α, λ, and the kind of traces on early performance on the mountain–car task. This study used five 9×9 tilings.

one episode had been completed, but the car has oscillated back and forth in the valley, following circular trajectories in state space. All the states visited frequently are valued worse than unexplored states, because the actual rewards have been worse than what was (unrealistically) expected. This continually drives the agent away from wherever it has been, to explore new states, until a solution is found. Figure 9.11 shows the results of a detailed study of the effect of the parameters α and λ, and of the kind of traces, on the rate of learning on this task. ■

9.5 Should We Bootstrap?

At this point you may be wondering why we bother with bootstrapping methods at all. Nonbootstrapping methods can be used with function approximation more reliably and over a broader range of conditions than bootstrapping methods. Nonbootstrapping methods achieve a lower asymptotic error than bootstrapping methods, even when backups are done according to the on-policy distribution. By using eligibility traces and $\lambda = 1$, it is even possible to implement nonbootstrapping methods on-line, in a step-by-step incremental manner. Despite all this, in practice bootstrapping methods are usually the methods of choice.

In empirical comparisons, bootstrapping methods usually perform much better than nonbootstrapping methods. A convenient way to make such comparisons is to use a TD method with eligibility traces and vary λ from 0 (pure bootstrapping) to 1 (pure nonbootstrapping). Figure 9.12 summarizes a col-

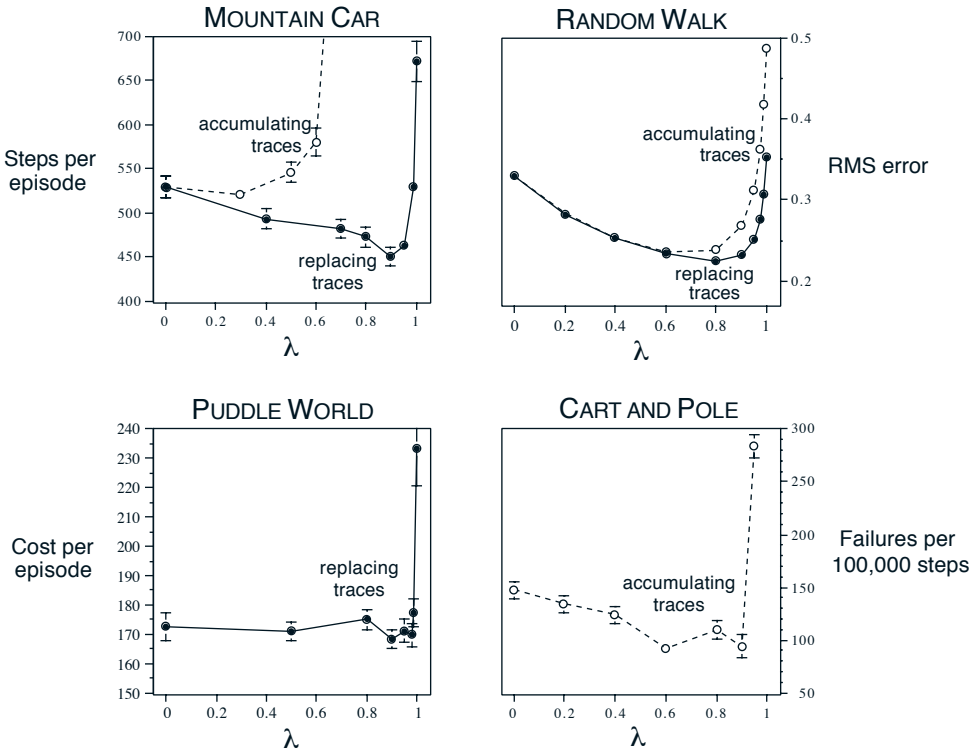


Figure 9.12: The effect of λ on reinforcement learning performance. In all cases, the better the performance, the *lower* the curve. The two left panels are applications to simple continuous-state control tasks using the Sarsa(λ) algorithm and tile coding, with either replacing or accumulating traces (Sutton, 1996). The upper-right panel is for policy evaluation on a random walk task using TD(λ) (Singh and Sutton, 1996). The lower right panel is unpublished data for the pole-balancing task (Example 3.4) from an earlier study (Sutton, 1984).

lection of such results. In all cases, performance became much worse as λ approached 1, the nonbootstrapping case. The example in the upper right of the figure is particularly significant in this regard. This is a policy evaluation (prediction) task and the performance measure used is RMSE (at the end of each episode, averaged over the first 20 episodes). Asymptotically, the $\lambda = 1$ case must be best according to this measure, but here, short of the asymptote, we see it performing much worse.

At this time it is unclear why methods that involve some bootstrapping perform so much better than pure nonbootstrapping methods. It could be that bootstrapping methods learn faster, or it could be that they actually learn something better than nonbootstrapping methods. The available results indicate that nonbootstrapping methods are better than bootstrapping methods at reducing RMSE from the true value function, but reducing RMSE is not necessarily the most important goal. For example, if you add 1000 to the true action-value function at all state–action pairs, then it will have very poor RMSE, but you will still get the optimal policy. Nothing quite that simple is going on with bootstrapping methods, but they do seem to do something right. We expect the understanding of these issues to improve as research continues.

9.6 Summary

Reinforcement learning systems must be capable of *generalization* if they are to be applicable to artificial intelligence or to large engineering applications. To achieve this, any of a broad range of existing methods for *supervised-learning function approximation* can be used simply by treating each backup as a training example. *Gradient-descent methods*, in particular, allow a natural extension to function approximation of all the techniques developed in previous chapters, including eligibility traces. *Linear* gradient-descent methods are particularly appealing theoretically and work well in practice when provided with appropriate features. Choosing the features is one of the most important ways of adding prior domain knowledge to reinforcement learning systems. Linear methods include radial basis functions, tile coding, and Kanerva coding. Backpropagation methods for multilayer neural networks are methods for *nonlinear* gradient-descent function approximation.

For the most part, the extension of reinforcement learning prediction and control methods to gradient-descent forms is straightforward for the on-policy case. On-policy bootstrapping methods converge reliably with linear gradient-descent function approximation to a solution with mean-squared error bounded by $\frac{1-\gamma\lambda}{1-\gamma}$ times the minimum possible error. Bootstrapping methods are of persistent interest in reinforcement learning, despite their limited theoretical

guarantees, because in practice they usually work significantly better than nonbootstrapping methods. The off-policy case involves considerably greater subtlety and is postponed to a later (future) chapter.

9.7 Bibliographical and Historical Remarks

Despite our treatment of generalization and function approximation late in the book, they have always been an integral part of reinforcement learning. It is only in the last decade or less that the field has focused on the tabular case, as we have here for the first seven chapters. Bertsekas and Tsitsiklis (1996) present the state of the art in function approximation in reinforcement learning, and the collection of papers by Boyan, Moore, and Sutton (1995) is also useful. Some of the early work with function approximation in reinforcement learning is discussed at the end of this section.

8.2 Gradient-descent methods for the minimizing mean-squared error in supervised learning are well known. Widrow and Hoff (1960) introduced the least-mean-square (LMS) algorithm, which is the prototypical incremental gradient-descent algorithm. Details of this and related algorithms are provided in many texts (e.g., Widrow and Stearns, 1985; Bishop, 1995; Duda and Hart, 1973).

Gradient-descent analyses of TD learning date back at least to Sutton (1988). Methods more sophisticated than the simple gradient-descent methods covered in this section have also been studied in the context of reinforcement learning, such as quasi-Newton methods (Werbos, 1990) and recursive-least-squares methods (Bradtke, 1993, 1994; Bradtke and Barto, 1996; Bradtke, Ydstie, and Barto, 1994). Bertsekas and Tsitsiklis (1996) provide a good discussion of these methods.

The earliest use of state aggregation in reinforcement learning may have been Michie and Chambers's BOXES system (1968). The theory of state aggregation in reinforcement learning has been developed by Singh, Jaakkola, and Jordan (1995) and Tsitsiklis and Van Roy (1996).

8.3 TD(λ) with linear gradient-descent function approximation was first explored by Sutton (1984, 1988), who proved convergence of TD(0) in the mean to the minimal RMSE solution for the case in which the feature vectors, $\{\mathbf{x}(s) : s \in \mathcal{S}\}$, are linearly independent. Convergence with probability 1 for general λ was proved by several researchers at about the same time (Peng, 1993; Dayan and Sejnowski, 1994; Tsitsiklis, 1994; Gurvits, Lin, and Hanson, 1994). In addition, Jaakkola, Jordan, and

Singh (1994) proved convergence under on-line updating. All of these results assumed linearly independent feature vectors, which implies at least as many components to $\mathbf{w}_t$ as there are states. Convergence of linear TD(λ) for the more interesting case of general (dependent) feature vectors was first shown by Dayan (1992). A significant generalization and strengthening of Dayan's result was proved by Tsitsiklis and Van Roy (1997). They proved the main result presented in Section 8.2, the bound on the asymptotic error of TD(λ) and other bootstrapping methods. Recently they extended their analysis to the undiscounted continuing case (Tsitsiklis and Van Roy, 1999).

Our presentation of the range of possibilities for linear function approximation is based on that by Barto (1990). The term *coarse coding* is due to Hinton (1984), and our Figure 9.2 is based on one of his figures. Waltz and Fu (1965) provide an early example of this type of function approximation in a reinforcement learning system.

Tile coding, including hashing, was introduced by Albus (1971, 1981). He described it in terms of his "cerebellar model articulator controller," or CMAC, as tile coding is known in the literature. The term "tile coding" is new to this book, though the idea of describing CMAC in these terms is taken from Watkins (1989). Tile coding has been used in many reinforcement learning systems (e.g., Shewchuk and Dean, 1990; Lin and Kim, 1991; Miller, Scalera, and Kim, 1994; Sofge and White, 1992; Tham, 1994; Sutton, 1996; Watkins, 1989) as well as in other types of learning control systems (e.g., Kraft and Campagna, 1990; Kraft, Miller, and Dietz, 1992).

Function approximation using radial basis functions (RBFs) has received wide attention ever since being related to neural networks by Broomhead and Lowe (1988). Powell (1987) reviewed earlier uses of RBFs, and Poggio and Girosi (1989, 1990) extensively developed and applied this approach.

What we call "Kanerva coding" was introduced by Kanerva (1988) as part of his more general idea of *sparse distributed memory*. A good review of this and related memory models is provided by Kanerva (1993). This approach has been pursued by Gallant (1993) and by Sutton and Whitehead (1993), among others.

8.4 Q(λ) with function approximation was first explored by Watkins (1989). Sarsa(λ) with function approximation was first explored by Rummery and Niranjan (1994). The mountain–car example is based on a similar task studied by Moore (1990). The results on it presented here are from Sutton (1996) and Singh and Sutton (1996).

Convergence of the Sarsa control method presented in this section has not been proved. The Q-learning control method is now known not to be sound and will diverge for some problems. Convergence results for control methods with state aggregation and other special kinds of function approximation are proved by Tsitsiklis and Van Roy (1996), Singh, Jaakkola, and Jordan (1995), and Gordon (1995).

The use of function approximation in reinforcement learning goes back to the early neural networks of Farley and Clark (1954; Clark and Farley, 1955), who used reinforcement learning to adjust the parameters of linear threshold functions representing policies. The earliest example we know of in which function approximation methods were used for learning value functions was Samuel's checkers player (1959, 1967). Samuel followed Shannon's (1950) suggestion that a value function did not have to be exact to be a useful guide to selecting moves in a game and that it might be approximated by linear combination of features. In addition to linear function approximation, Samuel experimented with lookup tables and hierarchical lookup tables called signature tables (Griffith, 1966, 1974; Page, 1977; Biermann, Fairfield, and Beres, 1982).

At about the same time as Samuel's work, Bellman and Dreyfus (1959) proposed using function approximation methods with DP. (It is tempting to think that Bellman and Samuel had some influence on one another, but we know of no reference to the other in the work of either.) There is now a fairly extensive literature on function approximation methods and DP, such as multigrid methods and methods using splines and orthogonal polynomials (e.g., Bellman and Dreyfus, 1959; Bellman, Kalaba, and Kotkin, 1973; Daniel, 1976; Whitt, 1978; Reetz, 1977; Schweitzer and Seidmann, 1985; Chow and Tsitsiklis, 1991; Kushner and Dupuis, 1992; Rust, 1996).

Holland's (1986) classifier system used a selective feature-match technique to generalize evaluation information across state–action pairs. Each classifier matched a subset of states having specified values for a subset of features, with the remaining features having arbitrary values ("wild cards"). These subsets were then used in a conventional state-aggregation approach to function approximation. Holland's idea was to use a genetic algorithm to evolve a set of classifiers that collectively would implement a useful action-value function. Holland's ideas influenced the early research of the authors on reinforcement learning, but we focused on different approaches to function approximation. As function approximators, classifiers are limited in several ways. First, they are state-aggregation methods, with concomitant limitations in scaling and in representing smooth functions efficiently. In addition, the matching rules of classifiers can implement only aggregation boundaries that are parallel to the

feature axes. Perhaps the most important limitation of conventional classifier systems is that the classifiers are learned via the genetic algorithm, an evolutionary method. As we discussed in Chapter 1, there is available during learning much more detailed information about how to learn than can be used by evolutionary methods. This perspective led us to instead adapt supervised learning methods for use in reinforcement learning, specifically gradient-descent and neural network methods. These differences between Holland's approach and ours are not surprising because Holland's ideas were developed during a period when neural networks were generally regarded as being too weak in computational power to be useful, whereas our work was at the beginning of the period that saw widespread questioning of that conventional wisdom. There remain many opportunities for combining aspects of these different approaches.

A number of reinforcement learning studies using function approximation methods that we have not covered previously should be mentioned. Barto, Sutton, and Brouwer (1981) and Barto and Sutton (1981b) extended the idea of an associative memory network (e.g., Kohonen, 1977; Anderson, Silverstein, Ritz, and Jones, 1977) to reinforcement learning. Hampson (1983, 1989) was an early proponent of multilayer neural networks for learning value functions. Anderson (1986, 1987) coupled a TD algorithm with the error backpropagation algorithm to learn a value function. Barto and Anandan (1985) introduced a stochastic version of Widrow, Gupta, and Maitra's (1973) *selective bootstrap algorithm*, which they called the *associative reward-penalty* (A_{R-P}) *algorithm*. Williams (1986, 1987, 1988, 1992) extended this type of algorithm to a general class of REINFORCE algorithms, showing that they perform stochastic gradient ascent on the expected reinforcement. Gullapalli (1990) and Williams devised algorithms for learning generalizing policies for the case of continuous actions. Phansalkar and Thathachar (1995) proved both local and global convergence theorems for modified versions of REINFORCE algorithms. Christensen and Korf (1986) experimented with regression methods for modifying coefficients of linear value function approximations in the game of chess. Chapman and Kaelbling (1991) and Tan (1991) adapted decision-tree methods for learning value functions. Explanation-based learning methods have also been adapted for learning value functions, yielding compact representations (Yee, Saxena, Utgoff, and Barto, 1990; Dietterich and Flann, 1995).

Chapter 10

Off-policy Approximation of Action Values

Chapter 11

Policy Approximation

All of the methods we have considered so far in this book have learned the values of states or state–action pairs. To use them for control, we learned the values of state–action pairs, and then used those action values directly to implement the policy (e.g., ε-greedy) and select actions. All methods of this form can be called *action-value methods*.

In this chapter we explore methods that are not action-value methods. They may still compute action (or state) values, but they do not use them directly to select actions. Instead, the policy is represented directly, with its own weights independent of any value function.

11.1 Actor–Critic Methods

Actor–critic methods are TD methods that have a separate memory structure to explicitly represent the policy independent of the value function. The policy structure is known as the *actor*, because it is used to select actions, and the estimated value function is known as the *critic*, because it criticizes the actions made by the actor. Learning is always on-policy: the critic must learn about and critique whatever policy is currently being followed by the actor. The critique takes the form of a TD error. This scalar signal is the sole output of the critic and drives all learning in both actor and critic, as suggested by Figure 11.1.

Actor–critic methods are the natural extension of the idea of reinforcement comparison methods (Section 2.8) to TD learning and to the full reinforcement learning problem. Typically, the critic is a state-value function. After each action selection, the critic evaluates the new state to determine whether things

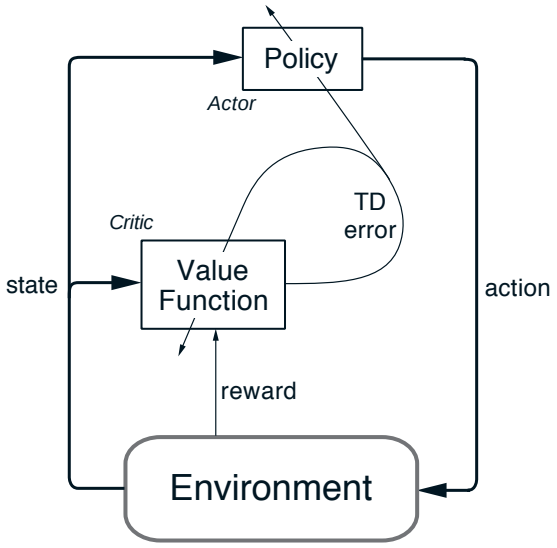


Figure 11.1: The actor–critic architecture.

have gone better or worse than expected. That evaluation is the TD error:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t),$$

where V is the current value function implemented by the critic. This TD error can be used to evaluate the action just selected, the action A_t taken in state S_t. If the TD error is positive, it suggests that the tendency to select A_t should be strengthened for the future, whereas if the TD error is negative, it suggests the tendency should be weakened. Suppose actions are generated by the Gibbs softmax method:

$$\pi_t(a|s) = \mathbf{Pr}\{A_t{=}a \mid S_t{=}s\} = \frac{e^{h(a|s)}}{\sum_b e^{h(b|s)}},$$

where the $h(a|s)$ are the values at time t of the modifiable policy parameters of the actor, indicating the tendency to select (*preference* for) each action a when in each state s. Then the strengthening or weakening described above can be implemented by increasing or decreasing $h(A_t|S_t)$, for instance, by

$$h(A_t|S_t) \leftarrow h(A_t|S_t) + \beta\delta_t,$$

where β is another positive step-size parameter.

This is just one example of an actor–critic method. Other variations select the actions in different ways, or use eligibility traces like those described in the

next chapter. Another common dimension of variation, as in reinforcement comparison methods, is to include additional factors varying the amount of credit assigned to the action taken, A_t. For example, one of the most common such factors is inversely related to the probability of selecting A_t, resulting in the update rule:

$$h(A_t|S_t) \leftarrow h(A_t|S_t) + \beta\delta_t\Big[1 - \pi_t(S_t, A_t)\Big].$$

These issues were explored early on, primarily for the immediate reward case (Sutton, 1984; Williams, 1992) and have not been brought fully up to date.

Many of the earliest reinforcement learning systems that used TD methods were actor–critic methods (Witten, 1977; Barto, Sutton, and Anderson, 1983). Since then, more attention has been devoted to methods that learn action-value functions and determine a policy exclusively from the estimated values (such as Sarsa and Q-learning). This divergence may be just historical accident. For example, one could imagine intermediate architectures in which both an action-value function and an independent policy would be learned. In any event, actor–critic methods are likely to remain of current interest because of two significant apparent advantages:

- They require minimal computation in order to select actions. Consider a case where there are an infinite number of possible actions—for example, a continuous-valued action. Any method learning just action values must search through this infinite set in order to pick an action. If the policy is explicitly stored, then this extensive computation may not be needed for each action selection.

- They can learn an explicitly stochastic policy; that is, they can learn the optimal probabilities of selecting various actions. This ability turns out to be useful in competitive and non-Markov cases (e.g., see Singh, Jaakkola, and Jordan, 1994).

In addition, the separate actor in actor–critic methods makes them more appealing in some respects as psychological and biological models. In some cases it may also make it easier to impose domain-specific constraints on the set of allowed policies.

11.2 R-Learning and the Average-Reward Setting

When the policy is approximated, we generally have to abandon the discounted-reward setting that we have relied on up to now. We replace it with the *average-reward setting*, which we discuss in this section.

R-learning is an off-policy control method for the advanced version of the reinforcement learning problem in which one neither discounts nor divides experience into distinct episodes with finite returns. In this *average-reward setting*, one seeks to maximize the average reward per time step. The value functions for a policy, π, are defined relative to the average expected reward per step under the policy, $\bar{r}(\pi)$:

$$\bar{r}(\pi) = \lim_{n\to\infty} \frac{1}{n} \sum_{t=1}^{n} \mathbb{E}_\pi[R_t] \,.$$

This average reward is well defined if we assume that the process is ergodic (nonzero probability of reaching any state from any other under any policy), and thus that $\bar{r}(\pi)$ does not depend on the starting state. From any state, in the long run the average reward is the same, but there is a transient. From some states better-than-average rewards are received for a while, and from others worse-than-average rewards are received. It is this transient that defines the value of a state:

$$v_\pi(s) = \sum_{k=1}^{\infty} \mathbb{E}_\pi[R_{t+k} - \bar{r}(\pi) \mid S_t = s] \,,$$

and the value of a state–action pair is similarly the transient difference in reward when starting in that state and taking that action:

$$q_\pi(s,a) = \sum_{k=1}^{\infty} \mathbb{E}_\pi[R_{t+k} - \bar{r}(\pi) \mid S_t = s, A_t = a] \,.$$

We call these *relative values* because they are relative to the average reward under the current policy.

There are subtle distinctions that need to be drawn between different kinds of optimality in the undiscounted continuing case. Nevertheless, for most practical purposes it may be adequate simply to order policies according to their average reward per time step, in other words, according to their $\bar{r}(\pi)$.

Initialize $\bar{R}$ and $Q(s,a)$, for all s, a, arbitrarily
Repeat forever:
    $S \leftarrow$ current state
    Choose action A in S using behavior policy (e.g., ε-greedy)
    Take action A, observe R, S'
    $\delta \leftarrow R - \bar{R} + \max_a Q(S', a) - Q(S, A)$
    $Q(S, A) \leftarrow Q(S, A) + \alpha\delta$
    If $Q(S, A) = \max_a Q(S, b)$, then:
        $\bar{R} \leftarrow \bar{R} + \beta\delta$

Figure 11.2: R-learning: An off-policy TD control algorithm for undiscounted, continuing tasks. The scalars α and β are step-size parameters.

For now let us consider all policies that attain the maximal value of $\bar{r}(\pi)$ to be optimal.

Other than its use of relative values, R-learning is a standard TD control method based on off-policy GPI, much like Q-learning. It maintains two policies, a behavior policy and an estimation policy, plus an action-value function and an estimated average reward. The behavior policy is used to generate experience; it might be the ε-greedy policy with respect to the action-value function. The estimation policy is the one involved in GPI. It is typically the greedy policy with respect to the action-value function. If π is the estimation policy, then the action-value function, Q, is an approximation of q_π and the average reward, $\bar{R}$, is an approximation of $\bar{r}(\pi)$. The complete algorithm is given in Figure 11.2.

Example 11.1: An Access-Control Queuing Task This is a decision task involving access control to a set of n servers. Customers of four different priorities arrive at a single queue. If given access to a server, the customers pay a reward of 1, 2, 4, or 8, depending on their priority, with higher priority customers paying more. In each time step, the customer at the head of the queue is either accepted (assigned to one of the servers) or rejected (removed from the queue). In either case, on the next time step the next customer in the queue is considered. The queue never empties, and the proportion of (randomly distributed) high priority customers in the queue is h. Of course a customer can be served only if there is a free server. Each busy server becomes free with probability p on each time step. Although we have just described them for definiteness, let us assume the statistics of arrivals and departures are unknown. The task is to decide on each step whether to accept or reject the next customer, on the basis of his priority and the number of free servers, so as to maximize long-term reward without discounting. Figure 11.3 shows the solution found by R-learning for this task with $n = 10$, $h = 0.5$, and $p = 0.06$.

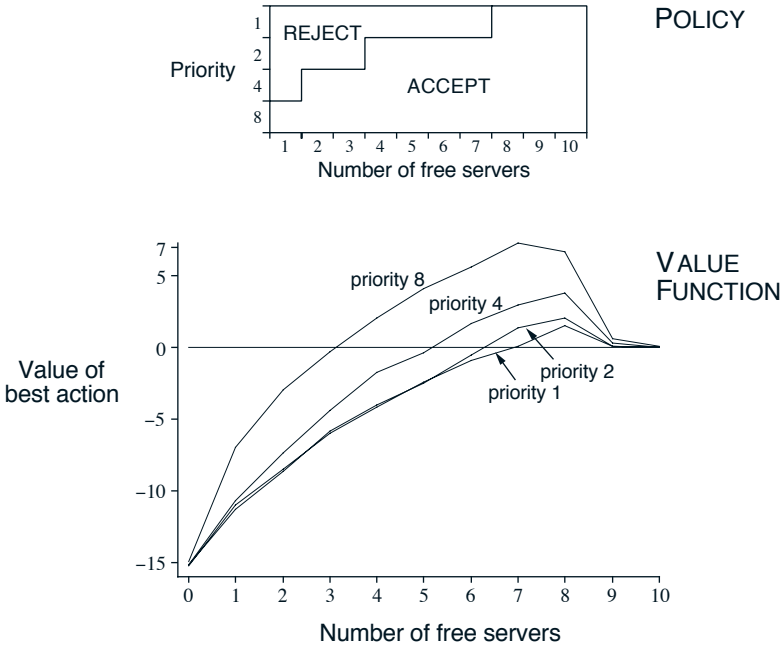


Figure 11.3: The policy and value function found by R-learning on the access-control queuing task after 2 million steps. The drop on the right of the graph is probably due to insufficient data; many of these states were never experienced. The value learned for $\bar{R}$ was about 2.73.

The R-learning parameters were $\alpha = 0.01$, $\beta = 0.01$, and $\varepsilon = 0.1$. The initial action values and $\bar{R}$ were zero. ■

*__Exercise 11.1__ Design an on-policy method for undiscounted, continuing tasks.

Chapter 12

State Estimation

Chapter 13

Temporal Abstraction

Part IV

Frontiers

Chapter 14

Biological Reinforcement Learning

Chapter 15

Applications and Case Studies

In this final chapter we present a few case studies of reinforcement learning. Several of these are substantial applications of potential economic significance. One, Samuel's checkers player, is primarily of historical interest. Our presentations are intended to illustrate some of the trade-offs and issues that arise in real applications. For example, we emphasize how domain knowledge is incorporated into the formulation and solution of the problem. We also highlight the representation issues that are so often critical to successful applications. The algorithms used in some of these case studies are substantially more complex than those we have presented in the rest of the book. Applications of reinforcement learning are still far from routine and typically require as much art as science. Making applications easier and more straightforward is one of the goals of current research in reinforcement learning.

15.1 TD-Gammon

One of the most impressive applications of reinforcement learning to date is that by Gerry Tesauro to the game of backgammon (Tesauro, 1992, 1994, 1995). Tesauro's program, *TD-Gammon*, required little backgammon knowledge, yet learned to play extremely well, near the level of the world's strongest grandmasters. The learning algorithm in TD-Gammon was a straightforward combination of the TD(λ) algorithm and nonlinear function approximation using a multilayer neural network trained by backpropagating TD errors.

Backgammon is a major game in the sense that it is played throughout the world, with numerous tournaments and regular world championship matches. It is in part a game of chance, and it is a popular vehicle for waging significant sums of money. There are probably more professional backgammon players

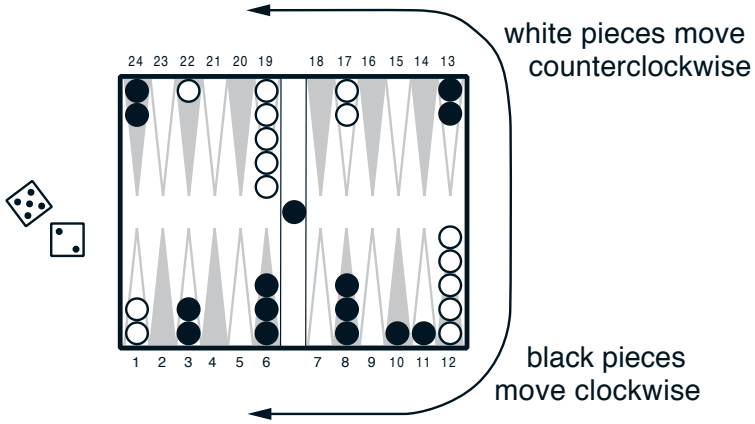


Figure 15.1: A backgammon position

than there are professional chess players. The game is played with 15 white and 15 black pieces on a board of 24 locations, called *points*. Figure 15.1 shows a typical position early in the game, seen from the perspective of the white player.

In this figure, white has just rolled the dice and obtained a 5 and a 2. This means that he can move one of his pieces 5 steps and one (possibly the same piece) 2 steps. For example, he could move two pieces from the 12 point, one to the 17 point, and one to the 14 point. White's objective is to advance all of his pieces into the last quadrant (points 19–24) and then off the board. The first player to remove all his pieces wins. One complication is that the pieces interact as they pass each other going in different directions. For example, if it were black's move in Figure 15.1, he could use the dice roll of 2 to move a piece from the 24 point to the 22 point, "hitting" the white piece there. Pieces that have been hit are placed on the "bar" in the middle of the board (where we already see one previously hit black piece), from whence they reenter the race from the start. However, if there are two pieces on a point, then the opponent cannot move to that point; the pieces are protected from being hit. Thus, white cannot use his 5–2 dice roll to move either of his pieces on the 1 point, because their possible resulting points are occupied by groups of black pieces. Forming contiguous blocks of occupied points to block the opponent is one of the elementary strategies of the game.

Backgammon involves several further complications, but the above description gives the basic idea. With 30 pieces and 24 possible locations (26, counting the bar and off-the-board) it should be clear that the number of possible backgammon positions is enormous, far more than the number of memory elements one could have in any physically realizable computer. The number of moves possible from each position is also large. For a typical dice roll there might be 20 different ways of playing. In considering future moves, such as

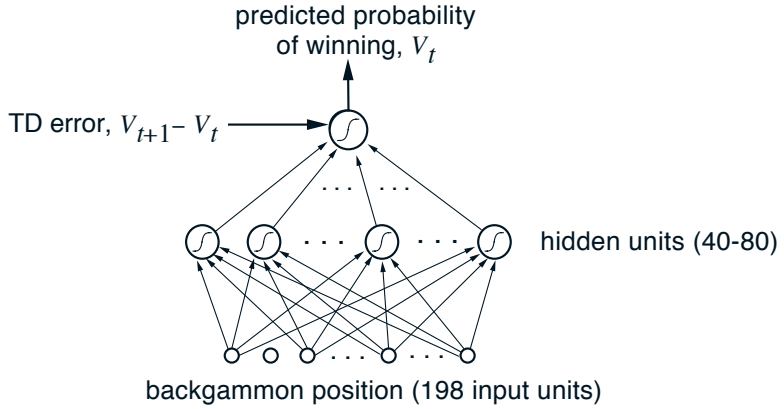


Figure 15.2: The neural network used in TD-Gammon

the response of the opponent, one must consider the possible dice rolls as well. The result is that the game tree has an effective branching factor of about 400. This is far too large to permit effective use of the conventional heuristic search methods that have proved so effective in games like chess and checkers.

On the other hand, the game is a good match to the capabilities of TD learning methods. Although the game is highly stochastic, a complete description of the game's state is available at all times. The game evolves over a sequence of moves and positions until finally ending in a win for one player or the other, ending the game. The outcome can be interpreted as a final reward to be predicted. On the other hand, the theoretical results we have described so far cannot be usefully applied to this task. The number of states is so large that a lookup table cannot be used, and the opponent is a source of uncertainty and time variation.

TD-Gammon used a nonlinear form of TD(λ). The estimated value, $\hat{v}(s)$, of any state (board position) s was meant to estimate the probability of winning starting from state s. To achieve this, rewards were defined as zero for all time steps except those on which the game is won. To implement the value function, TD-Gammon used a standard multilayer neural network, much as shown in Figure 15.2. (The real network had two additional units in its final layer to estimate the probability of each player's winning in a special way called a "gammon" or "backgammon.") The network consisted of a layer of input units, a layer of hidden units, and a final output unit. The input to the network was a representation of a backgammon position, and the output was an estimate of the value of that position.

In the first version of TD-Gammon, TD-Gammon 0.0, backgammon positions were represented to the network in a relatively direct way that involved little backgammon knowledge. It did, however, involve substantial knowledge

of how neural networks work and how information is best presented to them. It is instructive to note the exact representation Tesauro chose. There were a total of 198 input units to the network. For each point on the backgammon board, four units indicated the number of white pieces on the point. If there were no white pieces, then all four units took on the value zero. If there was one piece, then the first unit took on the value 1. If there were two pieces, then both the first and the second unit were 1. If there were three or more pieces on the point, then all of the first three units were 1. If there were more than three pieces, the fourth unit also came on, to a degree indicating the number of additional pieces beyond three. Letting n denote the total number of pieces on the point, if $n > 3$, then the fourth unit took on the value $(n-3)/2$. With four units for white and four for black at each of the 24 points, that made a total of 192 units. Two additional units encoded the number of white and black pieces on the bar (each took the value $n/2$, where n is the number of pieces on the bar), and two more encoded the number of black and white pieces already successfully removed from the board (these took the value $n/15$, where n is the number of pieces already borne off). Finally, two units indicated in a binary fashion whether it was white's or black's turn to move. The general logic behind these choices should be clear. Basically, Tesauro tried to represent the position in a straightforward way, making little attempt to minimize the number of units. He provided one unit for each conceptually distinct possibility that seemed likely to be relevant, and he scaled them to roughly the same range, in this case between 0 and 1.

Given a representation of a backgammon position, the network computed its estimated value in the standard way. Corresponding to each connection from an input unit to a hidden unit was a real-valued weight. Signals from each input unit were multiplied by their corresponding weights and summed at the hidden unit. The output, $h(j)$, of hidden unit j was a nonlinear sigmoid function of the weighted sum:

$$h(j) = \sigma\left(\sum_i w_{ij}x_i\right) = \frac{1}{1 + e^{-\sum_i w_{ij}x_i}},$$

where x_i is the value of the ith input unit and w_{ij} is the weight of its connection to the jth hidden unit. The output of the sigmoid is always between 0 and 1, and has a natural interpretation as a probability based on a summation of evidence. The computation from hidden units to the output unit was entirely analogous. Each connection from a hidden unit to the output unit had a separate weight. The output unit formed the weighted sum and then passed it through the same sigmoid nonlinearity.

TD-Gammon used the gradient-descent form of the TD(λ) algorithm de-

scribed in Section 8.2, with the gradients computed by the error backpropagation algorithm (Rumelhart, Hinton, and Williams, 1986). Recall that the general update rule for this case is

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\Big[R_{t+1} + \gamma\hat{v}(S_{t+1}) - \hat{v}(S_t)\Big]\mathbf{z}_t, \tag{15.1}$$

where $\mathbf{w}_t$ is the vector of all modifiable parameters (in this case, the weights of the network) and $\mathbf{z}_t$ is a vector of eligibility traces, one for each component of $\mathbf{w}_t$, updated by

$$\mathbf{z}_t = \gamma\lambda\mathbf{z}_{t-1} + \nabla_{\mathbf{w}_t}\hat{v}(S_t),$$

with $\mathbf{z}_0 = \mathbf{0}$. The gradient in this equation can be computed efficiently by the backpropagation procedure. For the backgammon application, in which $\gamma = 1$ and the reward is always zero except upon winning, the TD error portion of the learning rule is usually just $\hat{v}(S_{t+1}) - \hat{v}(S_t)$, as suggested in Figure 15.2.

To apply the learning rule we need a source of backgammon games. Tesauro obtained an unending sequence of games by playing his learning backgammon player against itself. To choose its moves, TD-Gammon considered each of the 20 or so ways it could play its dice roll and the corresponding positions that would result. The resulting positions are *afterstates* as discussed in Section 6.8. The network was consulted to estimate each of their values. The move was then selected that would lead to the position with the highest estimated value. Continuing in this way, with TD-Gammon making the moves for both sides, it was possible to easily generate large numbers of backgammon games. Each game was treated as an episode, with the sequence of positions acting as the states, $S_0, S_1, S_2, \ldots$. Tesauro applied the nonlinear TD rule (15.1) fully incrementally, that is, after each individual move.

The weights of the network were set initially to small random values. The initial evaluations were thus entirely arbitrary. Since the moves were selected on the basis of these evaluations, the initial moves were inevitably poor, and the initial games often lasted hundreds or thousands of moves before one side or the other won, almost by accident. After a few dozen games however, performance improved rapidly.

After playing about 300,000 games against itself, TD-Gammon 0.0 as described above learned to play approximately as well as the best previous backgammon computer programs. This was a striking result because all the previous high-performance computer programs had used extensive backgammon knowledge. For example, the reigning champion program at the time was, arguably, *Neurogammon*, another program written by Tesauro that used a neural network but not TD learning. Neurogammon's network was trained

Program	Hidden Units	Training Games	Opponents	Results
TD-Gam 0.0	40	300,000	other programs	tied for best
TD-Gam 1.0	80	300,000	Robertie, Magriel, ...	−13 pts / 51 games
TD-Gam 2.0	40	800,000	various Grandmasters	−7 pts / 38 games
TD-Gam 2.1	80	1,500,000	Robertie	−1 pt / 40 games
TD-Gam 3.0	80	1,500,000	Kazaros	+6 pts / 20 games

Table 15.1: Summary of TD-Gammon Results

on a large training corpus of exemplary moves provided by backgammon experts, and, in addition, started with a set of features specially crafted for backgammon. Neurogammon was a highly tuned, highly effective backgammon program that decisively won the World Backgammon Olympiad in 1989. TD-Gammon 0.0, on the other hand, was constructed with essentially zero backgammon knowledge. That it was able to do as well as Neurogammon and all other approaches is striking testimony to the potential of self-play learning methods.

The tournament success of TD-Gammon 0.0 with zero backgammon knowledge suggested an obvious modification: add the specialized backgammon features but keep the self-play TD learning method. This produced TD-Gammon 1.0. TD-Gammon 1.0 was clearly substantially better than all previous backgammon programs and found serious competition only among human experts. Later versions of the program, TD-Gammon 2.0 (40 hidden units) and TD-Gammon 2.1 (80 hidden units), were augmented with a selective two-ply search procedure. To select moves, these programs looked ahead not just to the positions that would immediately result, but also to the opponent's possible dice rolls and moves. Assuming the opponent always took the move that appeared immediately best for him, the expected value of each candidate move was computed and the best was selected. To save computer time, the second ply of search was conducted only for candidate moves that were ranked highly after the first ply, about four or five moves on average. Two-ply search affected only the moves selected; the learning process proceeded exactly as before. The most recent version of the program, TD-Gammon 3.0, uses 160 hidden units and a selective three-ply search. TD-Gammon illustrates the combination of learned value functions and decide-time search as in heuristic search methods. In more recent work, Tesauro and Galperin (1997) have begun exploring trajectory sampling methods as an alternative to search.

Tesauro was able to play his programs in a significant number of games against world-class human players. A summary of the results is given in Table 15.1. Based on these results and analyses by backgammon grandmasters (Robertie, 1992; see Tesauro, 1995), TD-Gammon 3.0 appears to be at, or

very near, the playing strength of the best human players in the world. It may already be the world champion. These programs have already changed the way the best human players play the game. For example, TD-Gammon learned to play certain opening positions differently than was the convention among the best human players. Based on TD-Gammon's success and further analysis, the best human players now play these positions as TD-Gammon does (Tesauro, 1995).

15.2 Samuel's Checkers Player

An important precursor to Tesauro's TD-Gammon was the seminal work of Arthur Samuel (1959, 1967) in constructing programs for learning to play checkers. Samuel was one of the first to make effective use of heuristic search methods and of what we would now call temporal-difference learning. His checkers players are instructive case studies in addition to being of historical interest. We emphasize the relationship of Samuel's methods to modern reinforcement learning methods and try to convey some of Samuel's motivation for using them.

Samuel first wrote a checkers-playing program for the IBM 701 in 1952. His first *learning* program was completed in 1955 and was demonstrated on television in 1956. Later versions of the program achieved good, though not expert, playing skill. Samuel was attracted to game-playing as a domain for studying machine learning because games are less complicated than problems "taken from life" while still allowing fruitful study of how heuristic procedures and learning can be used together. He chose to study checkers instead of chess because its relative simplicity made it possible to focus more strongly on learning.

Samuel's programs played by performing a lookahead search from each current position. They used what we now call heuristic search methods to determine how to expand the search tree and when to stop searching. The terminal board positions of each search were evaluated, or "scored," by a value function, or "scoring polynomial," using linear function approximation. In this and other respects Samuel's work seems to have been inspired by the suggestions of Shannon (1950). In particular, Samuel's program was based on Shannon's minimax procedure to find the best move from the current position. Working backward through the search tree from the scored terminal positions, each position was given the score of the position that would result from the best move, assuming that the machine would always try to maximize the score, while the opponent would always try to minimize it. Samuel called this the *backed-up score* of the position. When the minimax procedure reached the

search tree's root—the current position—it yielded the best move under the assumption that the opponent would be using the same evaluation criterion, shifted to its point of view. Some versions of Samuel's programs used sophisticated search control methods analogous to what are known as "alpha-beta" cutoffs (e.g., see Pearl, 1984).

Samuel used two main learning methods, the simplest of which he called *rote learning*. It consisted simply of saving a description of each board position encountered during play together with its backed-up value determined by the minimax procedure. The result was that if a position that had already been encountered were to occur again as a terminal position of a search tree, the depth of the search was effectively amplified since this position's stored value cached the results of one or more searches conducted earlier. One initial problem was that the program was not encouraged to move along the most direct path to a win. Samuel gave it a "a sense of direction" by decreasing a position's value a small amount each time it was backed up a level (called a ply) during the minimax analysis. "If the program is now faced with a choice of board positions whose scores differ only by the ply number, it will automatically make the most advantageous choice, choosing a low-ply alternative if winning and a high-ply alternative if losing" (Samuel, 1959, p. 80). Samuel found this discounting-like technique essential to successful learning. Rote learning produced slow but continuous improvement that was most effective for opening and endgame play. His program became a "better-than-average novice" after learning from many games against itself, a variety of human opponents, and from book games in a supervised learning mode.

Rote learning and other aspects of Samuel's work strongly suggest the essential idea of temporal-difference learning—that the value of a state should equal the value of likely following states. Samuel came closest to this idea in his second learning method, his "learning by generalization" procedure for modifying the parameters of the value function. Samuel's method was the same in concept as that used much later by Tesauro in TD-Gammon. He played his program many games against another version of itself and performed a backup operation after each move. The idea of Samuel's backup is suggested by the diagram in Figure 15.3. Each open circle represents a position where the program moves next, an *on-move* position, and each solid circle represents a position where the opponent moves next. A backup was made to the value of each on-move position after a move by each side, resulting in a second on-move position. The backup was toward the minimax value of a search launched from the second on-move position. Thus, the overall effect was that of a backup consisting of one full move of real events and then a search over possible events, as suggested by Figure 15.3. Samuel's actual algorithm was significantly more complex than this for computational reasons, but this was the basic idea.

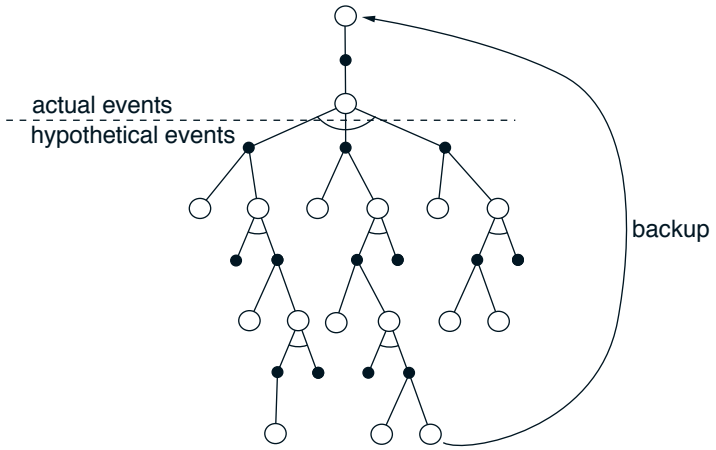


Figure 15.3: The backup diagram for Samuel's checkers player.

Samuel did not include explicit rewards. Instead, he fixed the weight of the most important feature, the *piece advantage* feature, which measured the number of pieces the program had relative to how many its opponent had, giving higher weight to kings, and including refinements so that it was better to trade pieces when winning than when losing. Thus, the goal of Samuel's program was to improve its piece advantage, which in checkers is highly correlated with winning.

However, Samuel's learning method may have been missing an essential part of a sound temporal-difference algorithm. Temporal-difference learning can be viewed as a way of making a value function consistent with itself, and this we can clearly see in Samuel's method. But also needed is a way of tying the value function to the true value of the states. We have enforced this via rewards and by discounting or giving a fixed value to the terminal state. But Samuel's method included no rewards and no special treatment of the terminal positions of games. As Samuel himself pointed out, his value function could have become consistent merely by giving a constant value to all positions. He hoped to discourage such solutions by giving his piece-advantage term a large, nonmodifiable weight. But although this may decrease the likelihood of finding useless evaluation functions, it does not prohibit them. For example, a constant function could still be attained by setting the modifiable weights so as to cancel the effect of the nonmodifiable one.

Since Samuel's learning procedure was not constrained to find useful evaluation functions, it should have been possible for it to become worse with experience. In fact, Samuel reported observing this during extensive self-play training sessions. To get the program improving again, Samuel had to intervene and set the weight with the largest absolute value back to zero. His

interpretation was that this drastic intervention jarred the program out of local optima, but another possibility is that it jarred the program out of evaluation functions that were consistent but had little to do with winning or losing the game.

Despite these potential problems, Samuel's checkers player using the generalization learning method approached "better-than-average" play. Fairly good amateur opponents characterized it as "tricky but beatable" (Samuel, 1959). In contrast to the rote-learning version, this version was able to develop a good middle game but remained weak in opening and endgame play. This program also included an ability to search through sets of features to find those that were most useful in forming the value function. A later version (Samuel, 1967) included refinements in its search procedure, such as alpha-beta pruning, extensive use of a supervised learning mode called "book learning," and hierarchical lookup tables called signature tables (Griffith, 1966) to represent the value function instead of linear function approximation. This version learned to play much better than the 1959 program, though still not at a master level. Samuel's checkers-playing program was widely recognized as a significant achievement in artificial intelligence and machine learning.

15.3 The Acrobot

Reinforcement learning has been applied to a wide variety of physical control tasks (e.g., for a collection of robotics applications, see Connell and Mahadevan, 1993). One such task is the *acrobot*, a two-link, underactuated robot roughly analogous to a gymnast swinging on a high bar (Figure 15.4). The first joint (corresponding to the gymnast's hands on the bar) cannot exert torque, but the second joint (corresponding to the gymnast bending at the waist) can. The system has four continuous state variables: two joint positions and two joint velocities. The equations of motion are given in Figure 15.5. This system has been widely studied by control engineers (e.g., Spong, 1994) and machine-learning researchers (e.g., Dejong and Spong, 1994; Boone, 1997).

One objective for controlling the acrobot is to swing the tip (the "feet") above the first joint by an amount equal to one of the links in minimum time. In this task, the torque applied at the second joint is limited to three choices: positive torque of a fixed magnitude, negative torque of the same magnitude, or no torque. A reward of -1 is given on all time steps until the goal is reached, which ends the episode. No discounting is used ($\gamma = 1$). Thus, the optimal value, $v_*(s)$, of any state, s, is the minimum time to reach the goal (an integer number of steps) starting from s.

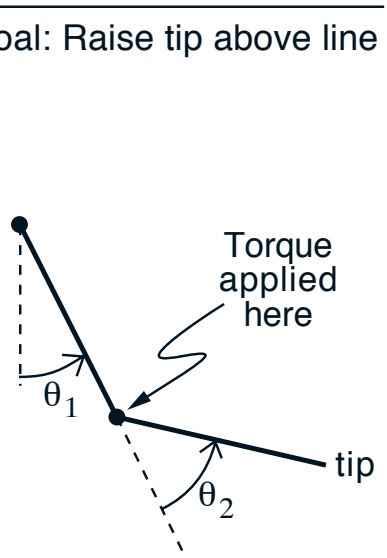


Figure 15.4: The acrobot.

$$
\begin{aligned}
\ddot{\theta}_1 &= -d_1^{-1}(d_2\ddot{\theta}_2 + \phi_1) \\
\ddot{\theta}_2 &= \left(m_2 l_{c2}^2 + I_2 - \frac{d_2^2}{d_1}\right)^{-1} \left(\tau + \frac{d_2}{d_1}\phi_1 - m_2 l_1 l_{c2}\dot{\theta}_1^2 \sin\theta_2 - \phi_2\right) \\
d_1 &= m_1 l_{c1}^2 + m_2(l_1^2 + l_{c2}^2 + 2 l_1 l_{c2}\cos\theta_2) + I_1 + I_2 \\
d_2 &= m_2(l_{c2}^2 + l_1 l_{c2}\cos\theta_2) + I_2 \\
\phi_1 &= -m_2 l_1 l_{c2}\dot{\theta}_2^2 \sin\theta_2 - 2 m_2 l_1 l_{c2}\dot{\theta}_2\dot{\theta}_1 \sin\theta_2 \\
&\quad + (m_1 l_{c1} + m_2 l_1) g \cos(\theta_1 - \pi/2) + \phi_2 \\
\phi_2 &= m_2 l_{c2} g \cos(\theta_1 + \theta_2 - \pi/2)
\end{aligned}
$$

Figure 15.5: The equations of motions of the simulated acrobot. A time step of 0.05 seconds was used in the simulation, with actions chosen after every four time steps. The torque applied at the second joint is denoted by $\tau \in \{+1, -1, 0\}$. There were no constraints on the joint positions, but the angular velocities were limited to $\dot{\theta}_1 \in [-4\pi, 4\pi]$ and $\dot{\theta}_2 \in [-9\pi, 9\pi]$. The constants were $m_1 = m_2 = 1$ (masses of the links), $l_1 = l_2 = 1$ (lengths of links), $l_{c1} = l_{c2} = 0.5$ (lengths to center of mass of links), $I_1 = I_2 = 1$ (moments of inertia of links), and $g = 9.8$ (gravity).

Sutton (1996) addressed the acrobot swing-up task in an on-line, modelfree context. Although the acrobot was simulated, the simulator was not available for use by the agent/controller in any way. The training and interaction were just as if a real, physical acrobot had been used. Each episode began with both links of the acrobot hanging straight down and at rest. Torques were applied by the reinforcement learning agent until the goal was reached, which always happened eventually. Then the acrobot was restored to its initial rest position and a new episode was begun.

The learning algorithm used was Sarsa(λ) with linear function approximation, tile coding, and replacing traces as in Figure 9.8. With a small, discrete action set, it is natural to use a separate set of tilings for each action. The next choice is of the continuous variables with which to represent the state. A clever designer would probably represent the state in terms of the angular position and velocity of the center of mass and of the second link, which might make the solution simpler and consistent with broad generalization. But since this was just a test problem, a more naive, direct representation was used in terms of the positions and velocities of the links: $\theta_1, \dot{\theta}_1, \theta_2$, and $\dot{\theta}_2$. The two angles are restricted to a limited range by the physics of the acrobot (see Figure 15.5) and the two angles are naturally restricted to $[0, 2\pi]$. Thus, the state space in this task is a bounded rectangular region in four dimensions.

This leaves the question of what tilings to use. There are many possibilities, as discussed in Chapter 9. One is to use a complete grid, slicing the four-dimensional space along all dimensions, and thus into many small four-dimensional tiles. Alternatively, one could slice along only one of the dimensions, making hyperplanar stripes. In this case one has to pick which dimension to slice along. And of course in all cases one has to pick the width of the slices, the number of tilings of each kind, and, if there are multiple tilings, how to offset them. One could also slice along pairs or triplets of dimensions to get other tilings. For example, if one expected the velocities of the two links to interact strongly in their effect on value, then one might make many tilings that sliced along both of these dimensions. If one thought the region around zero velocity was particularly critical, then the slices could be more closely spaced there.

Sutton used tilings that sliced in a variety of simple ways. Each of the four dimensions was divided into six equal intervals. A seventh interval was added to the angular velocities so that tilings could be offset by a random fraction of an interval in all dimensions (see Chapter 9, subsection "Tile Coding"). Of the total of 48 tilings, 12 sliced along all four dimensions as discussed above, dividing the space into $6 \times 7 \times 6 \times 7 = 1764$ tiles each. Another 12 tilings sliced along three dimensions (3 randomly offset tilings each for each of the 4 sets of three dimensions), and another 12 sliced along two dimensions (2 tilings for

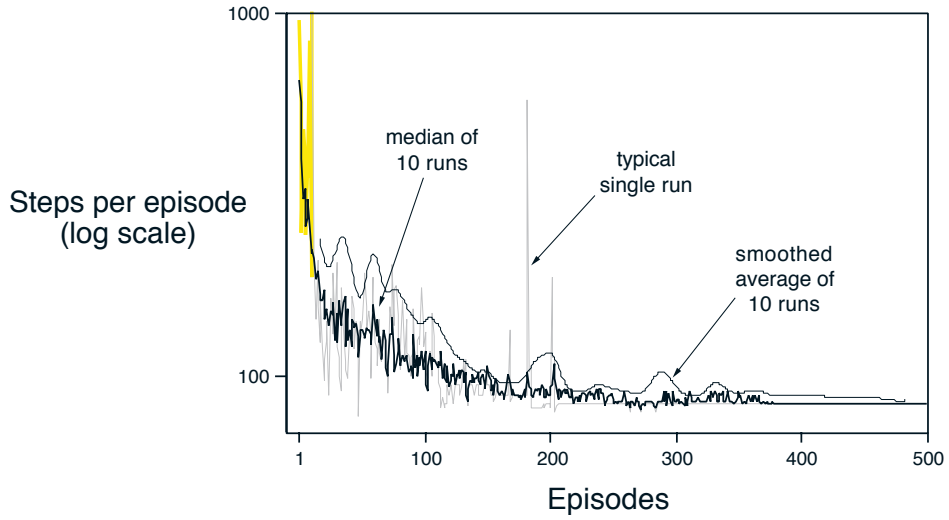


Figure 15.6: Learning curves for Sarsa(λ) on the acrobot task.

each of the 6 sets of two dimensions. Finally, a set of 12 tilings depended each on only one dimension (3 tilings for each of the 4 dimensions). This resulted in a total of approximately $25,000$ tiles for each action. This number is small enough that hashing was not necessary. All tilings were offset by a random fraction of an interval in all relevant dimensions.

The remaining parameters of the learning algorithm were $\alpha = 0.2/48$, $\lambda = 0.9$, $\varepsilon = 0$, and $\hat{q}_1 = 0$. The use of a greedy policy ($\varepsilon = 0$) seemed preferable on this task because long sequences of correct actions are needed to do well. One exploratory action could spoil a whole sequence of good actions. Exploration was ensured instead by starting the action values optimistically, at the low value of 0. As discussed in Section 2.7 and Example 8.2, this makes the agent continually disappointed with whatever rewards it initially experiences, driving it to keep trying new things.

Figure 15.6 shows learning curves for the acrobot task and the learning algorithm described above. Note from the single-run curve that single episodes were sometimes extremely long. On these episodes, the acrobot was usually spinning repeatedly at the second joint while the first joint changed only slightly from vertical down. Although this often happened for many time steps, it always eventually ended as the action values were driven lower. All runs ended with an efficient policy for solving the problem, usually lasting about 75 steps. A typical final solution is shown in Figure 15.7. First the acrobot pumps back and forth several times symmetrically, with the second link always down. Then, once enough energy has been added to the system, the second link is swung upright and stabbed to the goal height.

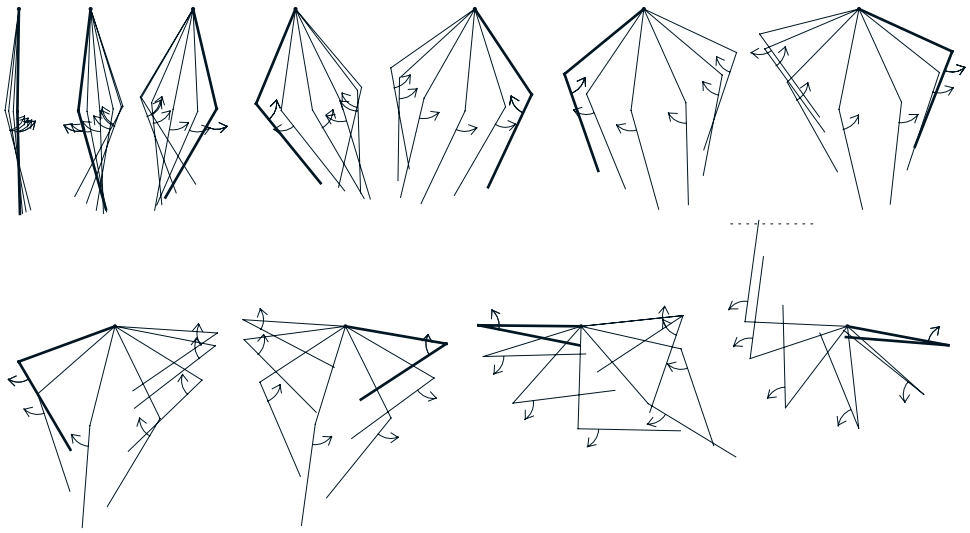


Figure 15.7: A typical learned behavior of the acrobot. Each group is a series of consecutive positions, the thicker line being the first. The arrow indicates the torque applied at the second joint.

15.4 Elevator Dispatching

Waiting for an elevator is a situation with which we are all familiar. We press a button and then wait for an elevator to arrive traveling in the right direction. We may have to wait a long time if there are too many passengers or not enough elevators. Just how long we wait depends on the dispatching strategy the elevators use to decide where to go. For example, if passengers on several floors have requested pickups, which should be served first? If there are no pickup requests, how should the elevators distribute themselves to await the next request? Elevator dispatching is a good example of a stochastic optimal control problem of economic importance that is too large to solve by classical techniques such as dynamic programming.

Crites and Barto (1996; Crites, 1996) studied the application of reinforcement learning techniques to the four-elevator, ten-floor system shown in Figure 15.8. Along the right-hand side are pickup requests and an indication of how long each has been waiting. Each elevator has a position, direction, and speed, plus a set of buttons to indicate where passengers want to get off. Roughly quantizing the continuous variables, Crites and Barto estimated that the system has over 10^{22} states. This large state set rules out classical dynamic programming methods such as value iteration. Even if one state could be backed up every microsecond it would still require over 1000 years to complete just one sweep through the state space.

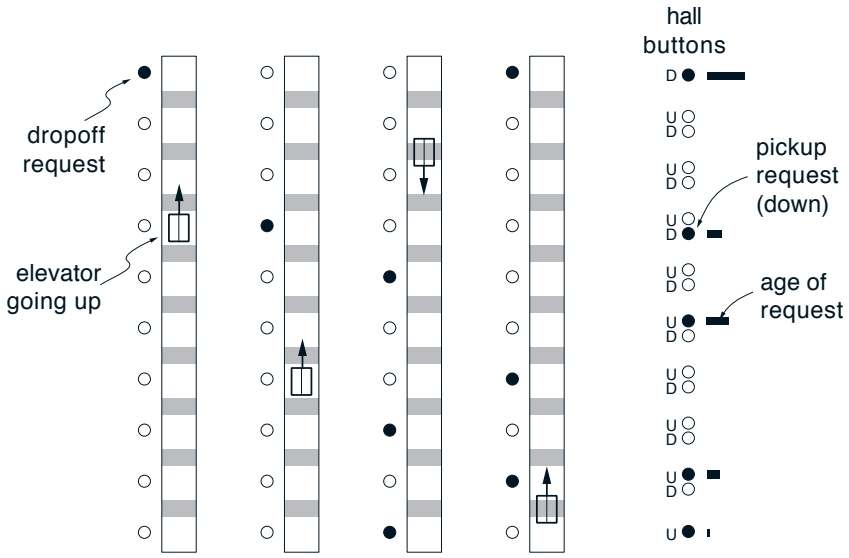


Figure 15.8: Four elevators in a ten-story building.

In practice, modern elevator dispatchers are designed heuristically and evaluated on simulated buildings. The simulators are quite sophisticated and detailed. The physics of each elevator car is modeled in continuous time with continuous state variables. Passenger arrivals are modeled as discrete, stochastic events, with arrival rates varying frequently over the course of a simulated day. Not surprisingly, the times of greatest traffic and greatest challenge to the dispatching algorithm are the morning and evening rush hours. Dispatchers are generally designed primarily for these difficult periods.

The performance of elevator dispatchers is measured in several different ways, all with respect to an average passenger entering the system. The average *waiting time* is how long the passenger waits before getting on an elevator, and the average *system time* is how long the passenger waits before being dropped off at the destination floor. Another frequently encountered statistic is the percentage of passengers whose waiting time exceeds 60 seconds. The objective that Crites and Barto focused on is the average *squared waiting time*. This objective is commonly used because it tends to keep the waiting times low while also encouraging fairness in serving all the passengers.

Crites and Barto applied a version of one-step Q-learning augmented in several ways to take advantage of special features of the problem. The most important of these concerned the formulation of the actions. First, each elevator made its own decisions independently of the others. Second, a number of constraints were placed on the decisions. An elevator carrying passengers could not pass by a floor if any of its passengers wanted to get off there, nor

could it reverse direction until all of its passengers wanting to go in its current direction had reached their floors. In addition, a car was not allowed to stop at a floor unless someone wanted to get on or off there, and it could not stop to pick up passengers at a floor if another elevator was already stopped there. Finally, given a choice between moving up or down, the elevator was constrained always to move up (otherwise evening rush hour traffic would tend to push all the elevators down to the lobby). These last three constraints were explicitly included to provide some prior knowledge and make the problem easier. The net result of all these constraints was that each elevator had to make few and simple decisions. The only decision that had to be made was whether or not to stop at a floor that was being approached and that had passengers waiting to be picked up. At all other times, no choices needed to be made.

That each elevator made choices only infrequently permitted a second simplification of the problem. As far as the learning agent was concerned, the system made discrete jumps from one time at which it had to make a decision to the next. When a continuous-time decision problem is treated as a discrete-time system in this way it is known as a *semi-Markov* decision process. To a large extent, such processes can be treated just like any other Markov decision process by taking the reward on each discrete transition as the integral of the reward over the corresponding continuous-time interval. The notion of return generalizes naturally from a discounted sum of future rewards to a discounted *integral* of future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \qquad \text{becomes} \qquad G_t = \int_0^{\infty} e^{-\beta\tau} R_{t+\tau}\, d\tau,$$

where R_t on the left is the usual immediate reward in discrete time and $R_{t+\tau}$ on the right is the instantaneous reward at continuous time $t + \tau$. In the elevator problem the continuous-time reward is the negative of the sum of the squared waiting times of all waiting passengers. The parameter $\beta > 0$ plays a role similar to that of the discount-rate parameter $\gamma \in [0, 1)$.

The basic idea of the extension of Q-learning to semi-Markov decision problems can now be explained. Suppose the system is in state S and takes action A at time t_1, and then the next decision is required at time t_2 in state S'. After this discrete-event transition, the semi-Markov Q-learning backup for a tabular action-value function, Q, would be:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[\int_{t_1}^{t_2} e^{-\beta(\tau - t_1)} R_\tau\, d\tau + e^{-\beta(t_2 - t_1)} \min_a Q(S', a) - Q(S, A) \right].$$

Note how $e^{-\beta(t_2 - t_1)}$ acts as a variable discount factor that depends on the

amount of time between events. This method is due to Bradtke and Duff (1995).

One complication is that the reward as defined—the negative sum of the squared waiting times—is not something that would normally be known while an actual elevator was running. This is because in a real elevator system one does not know how many people are waiting at a floor, only how long it has been since the button requesting a pickup on that floor was pressed. Of course this information is known in a simulator, and Crites and Barto used it to obtain their best results. They also experimented with another technique that used only information that would be known in an on-line learning situation with a real set of elevators. In this case one can use how long since each button has been pushed together with an estimate of the arrival rate to compute an *expected* summed squared waiting time for each floor. Using this in the reward measure proved nearly as effective as using the actual summed squared waiting time.

For function approximation, a nonlinear neural network trained by back-propagation was used to represent the action-value function. Crites and Barto experimented with a wide variety of ways of representing states to the network. After much exploration, their best results were obtained using networks with 47 input units, 20 hidden units, and two output units, one for each action. The way the state was encoded by the input units was found to be critical to the effectiveness of the learning. The 47 input units were as follows:

- 18 units: Two units encoded information about each of the nine hall buttons for down pickup requests. A real-valued unit encoded the elapsed time if the button had been pushed, and a binary unit was on if the button had not been pushed.

- 16 units: A unit for each possible location and direction for the car whose decision was required. Exactly one of these units was on at any given time.

- 10 units: The location of the other elevators superimposed over the 10 floors. Each elevator had a "footprint" that depended on its direction and speed. For example, a stopped elevator caused activation only on the unit corresponding to its current floor, but a moving elevator caused activation on several units corresponding to the floors it was approaching, with the highest activations on the closest floors. No information was provided about which one of the other cars was at a particular location.

- 1 unit: This unit was on if the elevator whose decision was required was at the highest floor with a passenger waiting.

- 1 unit: This unit was on if the elevator whose decision was required was at the floor with the passenger who had been waiting for the longest amount of time.

- 1 unit: Bias unit was always on.

Two architectures were used. In RL1, each elevator was given its own action-value function and its own neural network. In RL2, there was only one network and one action-value function, with the experiences of all four elevators contributing to learning in the one network. In both cases, each elevator made its decisions independently of the other elevators, but shared a single reward signal with them. This introduced additional stochasticity as far as each elevator was concerned because its reward depended in part on the actions of the other elevators, which it could not control. In the architecture in which each elevator had its own action-value function, it was possible for different elevators to learn different specialized strategies (although in fact they tended to learn the same strategy). On the other hand, the architecture with a common action-value function could learn faster because it learned simultaneously from the experiences of all elevators. Training time was an issue here, even though the system was trained in simulation. The reinforcement learning methods were trained for about four days of computer time on a 100 mips processor (corresponding to about 60,000 hours of simulated time). While this is a considerable amount of computation, it is negligible compared with what would be required by any conventional dynamic programming algorithm.

The networks were trained by simulating a great many evening rush hours while making dispatching decisions using the developing, learned action-value functions. Crites and Barto used the Gibbs softmax procedure to select actions as described in Section 2.3, reducing the "temperature" gradually over training. A temperature of zero was used during test runs on which the performance of the learned dispatchers was assessed.

Figure 15.9 shows the performance of several dispatchers during a simulated evening rush hour, what researchers call *down-peak* traffic. The dispatchers include methods similar to those commonly used in the industry, a variety of heuristic methods, sophisticated research algorithms that repeatedly run complex optimization algorithms on-line (Bao et al., 1994), and dispatchers learned by using the two reinforcement learning architectures. By all of the performance measures, the reinforcement learning dispatchers compare favorably with the others. Although the optimal policy for this problem is unknown, and the state of the art is difficult to pin down because details of commercial dispatching strategies are proprietary, these learned dispatchers appeared to perform very well.

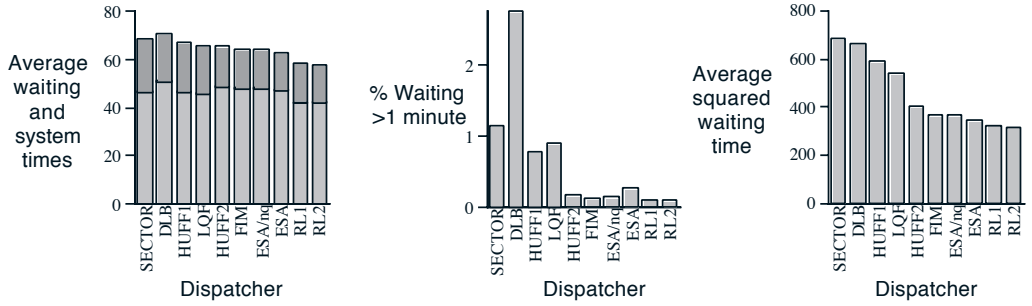


Figure 15.9: Comparison of elevator dispatchers. The SECTOR dispatcher is similar to what is used in many actual elevator systems. The RL1 and RL2 dispatchers were constructed through reinforcement learning.

15.5 Dynamic Channel Allocation

An important problem in the operation of a cellular telephone system is how to efficiently use the available bandwidth to provide good service to as many customers as possible. This problem is becoming critical with the rapid growth in the use of cellular telephones. Here we describe a study due to Singh and Bertsekas (1997) in which they applied reinforcement learning to this problem.

Mobile telephone systems take advantage of the fact that a communication channel—a band of frequencies—can be used simultaneously by many callers if these callers are spaced physically far enough apart that their calls do not interfere with each another. The minimum distance at which there is no interference is called the *channel reuse constraint*. In a cellular telephone system, the service area is divided into a number of regions called cells. In each cell is a base station that handles all the calls made within the cell. The total available bandwidth is divided permanently into a number of channels. Channels must then be allocated to cells and to calls made within cells without violating the channel reuse constraint. There are a great many ways to do this, some of which are better than others in terms of how reliably they make channels available to new calls, or to calls that are "handed off" from one cell to another as the caller crosses a cell boundary. If no channel is available for a new or a handed-off call, the call is lost, or *blocked*. Singh and Bertsekas considered the problem of allocating channels so that the number of blocked calls is minimized.

A simple example provides some intuition about the nature of the problem. Imagine a situation with three cells sharing two channels. The three cells are arranged in a line where no two adjacent cells can use the same channel without violating the channel reuse constraint. If the left cell is serving a call on channel 1 while the right cell is serving another call on channel 2, as in the left diagram

below, then any new call arriving in the middle cell must be blocked.



Obviously, it would be better for both the left and the right cells to use channel 1 for their calls. Then a new call in the middle cell could be assigned channel 2, as in the right diagram, without violating the channel reuse constraint. Such interactions and possible optimizations are typical of the channel assignment problem. In larger and more realistic cases with many cells, channels, and calls, and uncertainty about when and where new calls will arrive or existing calls will have to be handed off, the problem of allocating channels to minimize blocking can become extremely complex.

The simplest approach is to permanently assign channels to cells in such a way that the channel reuse constraint can never be violated even if all channels of all cells are used simultaneously. This is called a *fixed assignment* method. In a *dynamic assignment* method, in contrast, all channels are potentially available to all cells and are assigned to cells dynamically as calls arrive. If this is done right, it can take advantage of temporary changes in the spatial and temporal distribution of calls in order to serve more users. For example, when calls are concentrated in a few cells, these cells can be assigned more channels without increasing the blocking rate in the lightly used cells.

The channel assignment problem can be formulated as a semi-Markov decision process much as the elevator dispatching problem was in the previous section. A state in the semi-MDP formulation has two components. The first is the configuration of the entire cellular system that gives for each cell the usage state (occupied or unoccupied) of each channel for that cell. A typical cellular system with 49 cells and 70 channels has a staggering 70^{49} configurations, ruling out the use of conventional dynamic programming methods. The other state component is an indicator of what kind of event caused a state transition: arrival, departure, or handoff. This state component determines what kinds of actions are possible. When a call arrives, the possible actions are to assign it a free channel or to block it if no channels are available. When a call departs, that is, when a caller hangs up, the system is allowed to reassign the channels in use in that cell in an attempt to create a better configuration. At time t the immediate reward, R_t, is the number of calls taking place at that time, and the return is

$$G_t = \int_0^\infty e^{-\beta\tau} R_{t+\tau}\, d\tau,$$

where $\beta > 0$ plays a role similar to that of the discount-rate parameter γ. Max-

imizing the expectation of this return is the same as minimizing the expected (discounted) number of calls blocked over an infinite horizon.

This is another problem greatly simplified if treated in terms of afterstates (Section 6.8). For each state and action, the immediate result is a new configuration, an afterstate. A value function is learned over just these configurations. To select among the possible actions, the resulting configuration was determined and evaluated. The action was then selected that would lead to the configuration of highest estimated value. For example, when a new call arrived at a cell, it could be assigned to any of the free channels, if there were any; otherwise, it had to be blocked. The new configuration that would result from each assignment was easy to compute because it was always a simple deterministic consequence of the assignment. When a call terminated, the newly released channel became available for reassigning to any of the ongoing calls. In this case, the actions of reassigning each ongoing call in the cell to the newly released channel were considered. An action was then selected leading to the configuration with the highest estimated value.

Linear function approximation was used for the value function: the estimated value of a configuration was a weighted sum of features. Configurations were represented by two sets of features: an availability feature for each cell and a packing feature for each cell–channel pair. For any configuration, the availability feature for a cell gave the number of additional calls it could accept without conflict if the rest of the cells were frozen in the current configuration. For any given configuration, the packing feature for a cell–channel pair gave the number of times that channel was being used in that configuration within a four-cell radius of that cell. All of these features were normalized to lie between -1 and 1. A semi-Markov version of linear TD(0) was used to update the weights.

Singh and Bertsekas compared three channel allocation methods using a simulation of a 7×7 cellular array with 70 channels. The channel reuse constraint was that calls had to be 3 cells apart to be allowed to use the same channel. Calls arrived at cells randomly according to Poisson distributions possibly having different means for different cells, and call durations were determined randomly by an exponential distribution with a mean of three minutes. The methods compared were a fixed assignment method (FA), a dynamic allocation method called "borrowing with directional channel locking" (BDCL), and the reinforcement learning method (RL). BDCL (Zhang and Yum, 1989) was the best dynamic channel allocation method they found in the literature. It is a heuristic method that assigns channels to cells as in FA, but channels can be borrowed from neighboring cells when needed. It orders the channels in each cell and uses this ordering to determine which channels to borrow and how calls are dynamically reassigned channels within a cell.

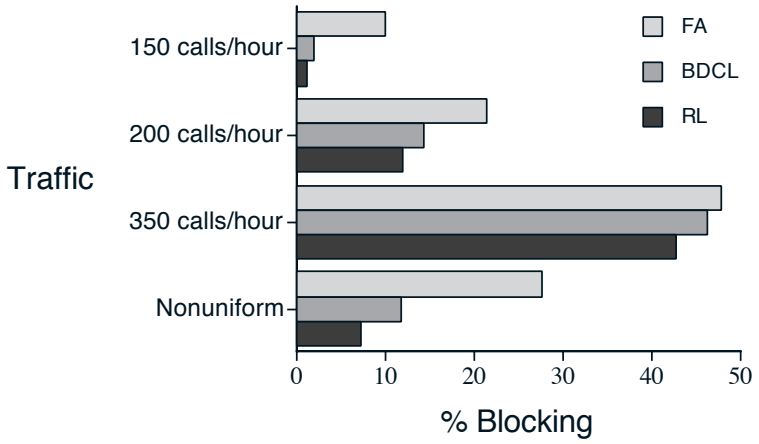


Figure 15.10: Performance of FA, BDCL, and RL channel allocation methods for different mean call arrival rates.

Figure 15.10 shows the blocking probabilities of these methods for mean arrival rates of 150, 200, and 350 calls/hour as well as for a case in which different cells had different mean arrival rates. The reinforcement learning method learned on-line. The data shown are for its asymptotic performance, but in fact learning was rapid. The RL method blocked calls less frequently than did the other methods for all arrival rates and soon after starting to learn. Note that the differences between the methods decreased as the call arrival rate increased. This is to be expected because as the system gets saturated with calls there are fewer opportunities for a dynamic allocation method to set up favorable usage patterns. In practice, however, it is the performance of the unsaturated system that is most important. For marketing reasons, cellular telephone systems are built with enough capacity that more than 10% blocking is rare.

Nie and Haykin (1996) also studied the application of reinforcement learning to dynamic channel allocation. They formulated the problem somewhat differently than Singh and Bertsekas did. Instead of trying to minimize the probability of blocking a call directly, their system tried to minimize a more indirect measure of system performance. Cost was assigned to patterns of channel use depending on the distances between calls using the same channels. Patterns in which channels were being used by multiple calls that were close to each other were favored over patterns in which channel-sharing calls were far apart. Nie and Haykin compared their system with a method called MAXAVAIL (Sivarajan, McEliece, and Ketchum, 1990), considered to be one of the best dynamic channel allocation methods. For each new call, it selects the channel that maximizes the total number of channels available in the entire system. Nie and Haykin showed that the blocking probability achieved by

their reinforcement learning system was closely comparable to that of MAXAVAIL under a variety of conditions in a 49-cell, 70-channel simulation. A key point, however, is that the allocation policy produced by reinforcement learning can be implemented on-line much more efficiently than MAXAVAIL, which requires so much on-line computation that it is not feasible for large systems.

The studies we described in this section are so recent that the many questions they raise have not yet been answered. We can see, though, that there can be different ways to apply reinforcement learning to the same real-world problem. In the near future, we expect to see many refinements of these applications, as well as many new applications of reinforcement learning to problems arising in communication systems.

15.6 Job-Shop Scheduling

Many jobs in industry and elsewhere require completing a collection of tasks while satisfying temporal and resource constraints. Temporal constraints say that some tasks have to be finished before others can be started; resource constraints say that two tasks requiring the same resource cannot be done simultaneously (e.g., the same machine cannot do two tasks at once). The objective is to create a schedule specifying when each task is to begin and what resources it will use that satisfies all the constraints while taking as little overall time as possible. This is the job-shop scheduling problem. In its general form, it is NP-complete, meaning that there is probably no efficient procedure for exactly finding shortest schedules for arbitrary instances of the problem. Job-shop scheduling is usually done using heuristic algorithms that take advantage of special properties of each specific instance.

Zhang and Dietterich (1995, 1996; Zhang, 1996) were motivated to apply reinforcement learning to job-shop scheduling because the design of domain-specific, heuristic algorithms can be expensive and time-consuming. Their goal was to show how reinforcement learning can be used to learn how to quickly find constraint-satisfying schedules of short duration in specific domains, thereby reducing the amount of hand engineering required. They addressed the NASA space shuttle payload processing problem (SSPP), which requires scheduling the tasks required for installation and testing of shuttle cargo bay payloads. An SSPP typically requires scheduling for two to six shuttle missions, each requiring between 34 and 164 tasks. An example of a task is MISSION-SEQUENCE-TEST, which has a duration of 7200 time units and requires the following resources: two quality control officers, two technicians, one ATE, one SPCDS, and one HITS. Some resources are divided

into pools, and if a task needs more than one resource of a specific type, the resources must belong to the same pool, and the pool has to be the right one. For example, if a task needs two quality control officers, they both have to be in the pool of quality control officers working on the same shift at the right site. It is not too hard to find a conflict-free schedule for a job, one that meets all the temporal and resource constraints, but the objective is to find a conflict-free schedule with the shortest possible total duration, which is much more difficult.

How can you do this using reinforcement learning? Job-shop scheduling is usually formulated as a search in the space of schedules, what is called a discrete, or combinatorial, optimization problem. A typical solution method would sequentially generate schedules, attempting to improve each over its predecessor in terms of constraint violations and duration (a hill-climbing, or local search, method). You could think of this as a nonassociative reinforcement learning problem of the type we discussed in Chapter 2 with a very large number of possible actions: all the possible schedules! But aside from the problem of having so many actions, any solution obtained this way would just be a *single* schedule for a *single* job instance. In contrast, what Zhang and Dietterich wanted their learning system to end up with was a *policy* that could quickly find good schedules for *any* SSPP. They wanted it to learn a skill for job-shop scheduling in this specific domain.

For clues about how to do this, they looked to an existing optimization approach to SSPP, in fact, the one actually in use by NASA at the time of their research: the iterative repair method developed by Zweben and Daun (1994). The starting point for the search is a *critical path schedule*, a schedule that meets the temporal constraints but ignores the resource constraints. This schedule can be constructed efficiently by scheduling each task prior to launch as late as the temporal constraints permit, and each task after landing as early as these constraints permit. Resource pools are assigned randomly. Two types of operators are used to modify schedules. They can be applied to any task that violates a resource constraint. A Reassign-Pool operator changes the pool assigned to one of the task's resources. This type of operator applies only if it can reassign a pool so that the resource requirement is satisfied. A Move operator moves a task to the first earlier or later time at which its resource needs can be satisfied and uses the critical path method to reschedule all of the task's temporal dependents.

At each step of the iterative repair search, one operator is applied to the current schedule, selected according to the following rules. The earliest task with a resource constraint violation is found, and a Reassign-Pool operator is applied to this task if possible. If more than one applies, that is, if several different pool reassignments are possible, one is selected at random.

If no REASSIGN-POOL operator applies, then a MOVE operator is selected at random based on a heuristic that prefers short-distance moves of tasks having few temporal dependents and whose resource requirements are close to the task's overallocation. After an operator is applied, the number of constraint violations of the resulting schedule is determined. A simulated annealing procedure is used decide whether to accept or reject this new schedule. If ΔV denotes the number of constraint violations removed by the repair, then the new schedule is accepted with probability $\exp(-\Delta V/T)$, where T is the current computational temperature that is gradually decreased throughout the search. If accepted, the new schedule becomes the current schedule for the next iteration; otherwise, the algorithm attempts to repair the old schedule again, which will usually produce different results due to the random decisions involved. Search stops when all constraints are satisfied. Short schedules are obtained by running the algorithm several times and selecting the shortest of the resulting conflict-free schedules.

Zhang and Dietterich treated entire schedules as states in the sense of reinforcement learning. The actions were the applicable REASSIGN-POOL and MOVE operators, typically numbering about 20. The problem was treated as episodic, each episode starting with the same critical path schedule that the iterative repair algorithm would start with and ending when a schedule was found that did not violate any constraint. The initial state—a critical path schedule—is denoted S_0. The rewards were designed to promote the quick construction of conflict-free schedules of short duration. The system received a small negative reward (-0.001) on each step that resulted in a schedule that still violated a constraint. This encouraged the agent to find conflict-free schedules quickly, that is, with a small number of repairs to S_0. Encouraging the system to find short schedules is more difficult because what it means for a schedule to be short depends on the specific SSPP instance. The shortest schedule for a difficult instance, one with a lot of tasks and constraints, will be longer than the shortest schedule for a simpler instance. Zhang and Dietterich devised a formula for a *resource dilation factor* (RDF), intended to be an instance-independent measure of a schedule's duration. To account for an instance's intrinsic difficulty, the formula makes use of a measure of the resource overallocation of S_0. Since longer schedules tend to produce larger RDFs, the negative of the RDF of the final conflict-free schedule was used as a reward at the end of each episode. With this reward function, if it takes N repairs starting from a schedule s to obtain a final conflict-free schedule, S_f, the return from s is $-RDF(S_f) - 0.001(N-1)$.

This reward function was designed to try to make a system learn to satisfy the two goals of finding conflict-free schedules of short duration and finding conflict-free schedules quickly. But the reinforcement learning system really

has only one goal—maximizing expected return—so the particular reward values determine how a learning system will tend to trade off these two goals. Setting the immediate reward to the small value of -0.001 means that the learning system will regard one repair, one step in the scheduling process, as being worth 0.001 units of RDF. So, for example, if from some schedule it is possible to produce a conflict-free schedule with one repair or with two, an optimal policy will take extra repair only if it promises a reduction in final RDF of more than 0.001.

Zhang and Dietterich used TD(λ) to learn the value function. Function approximation was by a multilayer neural network trained by backpropagating TD errors. Actions were selected by an ε-greedy policy, with ε decreasing during learning. One-step lookahead search was used to find the greedy action. Their knowledge of the problem made it easy to predict the schedules that would result from each repair operation. They experimented with a number of modifications to this basic procedure to improve its performance. One was to use the TD(λ) algorithm *backward* after each episode, with the eligibility trace extending to future rather than to past states. Their results suggested that this was more accurate and efficient than forward learning. In updating the weights of the network, they also sometimes performed multiple weight updates when the TD error was large. This is apparently equivalent to dynamically varying the step-size parameter in an error-dependent way during learning.

They also tried an *experience replay* technique due to Lin (1992). At any point in learning, the agent remembered the best episode up to that point. After every four episodes, it replayed this remembered episode, learning from it as if it were a new episode. At the start of training, they similarly allowed the system to learn from episodes generated by a good scheduler, and these could also be replayed later in learning. To make the lookahead search faster for large-scale problems, which typically had a branching factor of about 20, they used a variant they called *random sample greedy search* that estimated the greedy action by considering only random samples of actions, increasing the sample size until a preset confidence was reached that the greedy action of the sample was the true greedy action. Finally, having discovered that learning could be slowed considerably by excessive looping in the scheduling process, they made their system explicitly check for loops and alter action selections when a loop was detected. Although all of these techniques could improve the efficiency of learning, it is not clear how crucial all of them were for the success of the system.

Zhang and Dietterich experimented with two different network architectures. In the first version of their system, each schedule was represented using a set of 20 handcrafted features. To define these features, they studied small scheduling problems to find features that had some ability to predict RDF. For

example, experience with small problems showed that only four of the resource pools tended to cause allocation problems. The mean and standard deviation of each of these pools' unused portions over the entire schedule were computed, resulting in 10 real-valued features. Two other features were the RDF of the current schedule and the percentage of its duration during which it violated resource constraints. The network had 20 input units, one for each feature, a hidden layer of 40 sigmoidal units, and an output layer of 8 sigmoidal units. The output units coded the value of a schedule using a code in which, roughly, the location of the activity peak over the 8 units represented the value. Using the appropriate TD error, the network weights were updated using error backpropagation, with the multiple weight-update technique mentioned above.

The second version of the system (Zhang and Dietterich, 1996) used a more complicated time-delay neural network (TDNN) borrowed from the field of speech recognition (Lang, Waibel, and Hinton, 1990). This version divided each schedule into a sequence of blocks (maximal time intervals during which tasks and resource assignments did not change) and represented each block by a set of features similar to those used in the first program. It then scanned a set of "kernel" networks across the blocks to create a set of more abstract features. Since different schedules had different numbers of blocks, another layer averaged these abstract features over each third of the blocks. Then a final layer of 8 sigmoidal output units represented the schedule's value using the same code as in the first version of the system. In all, this network had 1123 adjustable weights.

A set of 100 artificial scheduling problems was constructed and divided into subsets used for training, determining when to stop training (a validation set), and final testing. During training they tested the system on the validation set after every 100 episodes and stopped training when performance on the validation set stopped changing, which generally took about 10,000 episodes. They trained networks with different values of λ (0.2 and 0.7), with three different training sets, and they saved both the final set of weights and the set of weights producing the best performance on the validation set. Counting each set of weights as a different network, this produced 12 networks, each of which corresponded to a different scheduling algorithm.

Figure 15.11 shows how the mean performance of the 12 TDNN networks (labeled G12TDN) compared with the performances of two versions of Zweben and Daun's iterative repair algorithm, one using the number of constraint violations as the function to be minimized by simulated annealing (IR-V) and the other using the RDF measure (IR-RDF). The figure also shows the performance of the first version of their system that did not use a TDNN (G12N). The mean RDF of the best schedule found by repeatedly running an algorithm is plotted against the total number of schedule repairs (using a

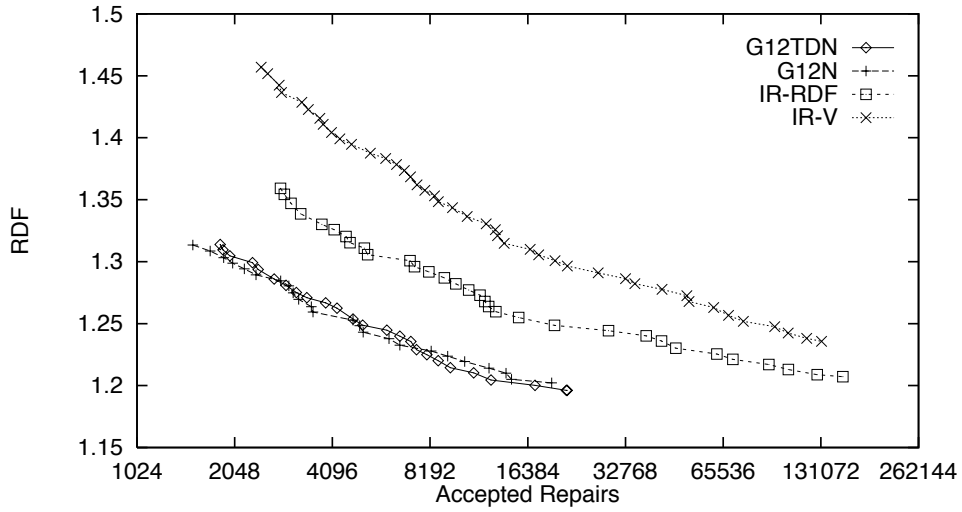


Figure 15.11: Comparison of accepted schedule repairs. Reprinted with permission from Zhang and Dietterich, 1996.

log scale). These results show that the learning system produced scheduling algorithms that needed many fewer repairs to find conflict-free schedules of the same quality as those found by the iterative repair algorithms. Figure 15.12 compares the computer time required by each scheduling algorithm to find schedules of various RDFs. According to this measure of performance, the best trade-off between computer time and schedule quality is produced by the non-TDNN algorithm (G12N). The TDNN algorithm (G12TDN) suffered due to the time it took to apply the kernel-scanning process, but Zhang and Dietterich point out that there are many ways to make it run faster.

These results do not unequivocally establish the utility of reinforcement learning for job-shop scheduling or for other difficult search problems. But they do suggest that it is possible to use reinforcement learning methods to learn how to improve the efficiency of search. Zhang and Dietterich's job-shop scheduling system is the first successful instance of which we are aware in which reinforcement learning was applied in *plan-space*, that is, in which states are complete plans (job-shop schedules in this case), and actions are plan modifications. This is a more abstract application of reinforcement learning than we are used to thinking about. Note that in this application the system learned not just to efficiently create *one* good schedule, a skill that would not be particularly useful; it learned how to quickly find good schedules for a class of related scheduling problems. It is clear that Zhang and Dietterich went through a lot of trial-and-error learning of their own in developing this example. But remember that this was a groundbreaking exploration of a new aspect of reinforcement learning. We expect that future applications of this

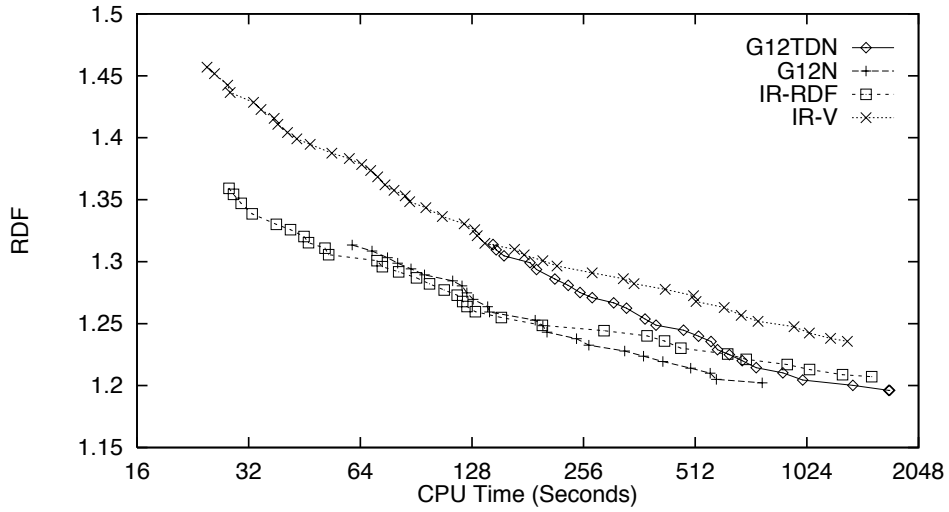


Figure 15.12: Comparison of CPU time. Reprinted with permission from Zhang and Dietterich, 1996.

kind and complexity will become more routine as experience accumulates.

Chapter 16

Prospects

In this book we have tried to present reinforcement learning not as a collection of individual methods, but as a coherent set of ideas cutting across methods. Each idea can be viewed as a dimension along which methods vary. The set of such dimensions spans a large space of possible methods. By exploring this space at the level of dimensions we hope to obtain the broadest and most lasting understanding. In this chapter we use the concept of dimensions in method space to recapitulate the view of reinforcement learning we have developed in this book and to identify some of the more important gaps in our coverage of the field.

16.1 The Unified View

All of the reinforcement learning methods we have explored in this book have three key ideas in common. First, the objective of all of them is the estimation of value functions. Second, all operate by backing up values along actual or possible state trajectories. Third, all follow the general strategy of generalized policy iteration (GPI), meaning that they maintain an approximate value function and an approximate policy, and they continually try to improve each on the basis of the other. These three ideas that the methods have in common circumscribe the subject covered in this book. We suggest that value functions, backups, and GPI are powerful organizing principles potentially relevant to any model of intelligence.

Two of the most important dimensions along which the methods vary are shown in Figure 16.1. These dimensions have to do with the kind of backup used to improve the value function. The vertical dimension is whether they are sample backups (based on a sample trajectory) or full backups (based

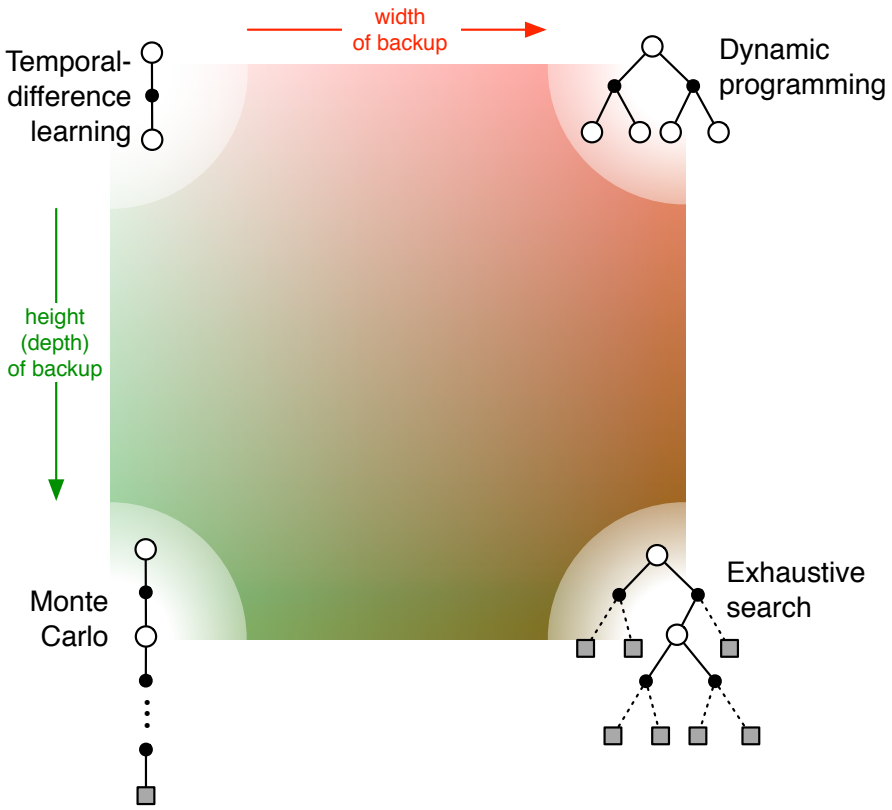


Figure 16.1: A slice of the space of reinforcement learning methods.

on a distribution of possible trajectories). Full backups of course require a model, whereas sample backups can be done either with or without a model (another dimension of variation). The horizontal dimension corresponds to the depth of backups, that is, to the degree of bootstrapping. At three of the four corners of the space are the three primary methods for estimating values: DP, TD, and Monte Carlo. Along the lower edge of the space are the sample-backup methods, ranging from one-step TD backups to full-return Monte Carlo backups. Between these is a spectrum including methods based on n-step backups and mixtures of n-step backups such as the λ-backups implemented by eligibility traces.

DP methods are shown in the extreme upper-left corner of the space because they involve one-step full backups. The upper-right corner is the extreme case of full backups so deep that they run all the way to terminal states (or, in a continuing task, until discounting has reduced the contribution of any further rewards to a negligible level). This is the case of exhaustive search. Intermedi-

ate methods along this dimension include heuristic search and related methods that search and backup up to a limited depth, perhaps selectively. There are also methods that are intermediate along the vertical dimension. These include methods that mix full and sample backups, as well as the possibility of methods that mix samples and distributions within a single backup. The interior of the square is filled in to represent the space of all such intermediate methods.

A third important dimension is that of function approximation. Function approximation can be viewed as an orthogonal spectrum of possibilities ranging from tabular methods at one extreme through state aggregation, a variety of linear methods, and then a diverse set of nonlinear methods. This third dimension might be visualized as perpendicular to the plane of the page in Figure 16.1.

Another dimension that we heavily emphasized in this book is the binary distinction between on-policy and off-policy methods. In the former case, the agent learns the value function for the policy it is currently following, whereas in the latter case it learns the value function for the policy that it currently thinks is best. These two policies are often different because of the need to explore. The interaction between this dimension and the bootstrapping and function approximation dimension discussed in Chapter 9 illustrates the advantages of analyzing the space of methods in terms of dimensions. Even though this did involve an interaction between three dimensions, many other dimensions were found to be irrelevant, greatly simplifying the analysis and increasing its significance.

In addition to the four dimensions just discussed, we have identified a number of others throughout the book:

Definition of return Is the task episodic or continuing, discounted or undiscounted?

Action values vs. state values vs. afterstate values What kind of values should be estimated? If only state values are estimated, then either a model or a separate policy (as in actor–critic methods) is required for action selection.

Action selection/exploration How are actions selected to ensure a suitable trade-off between exploration and exploitation? We have considered only the simplest ways to do this: ε-greedy and softmax action selection, and optimistic initialization of values.

Synchronous vs. asynchronous Are the backups for all states performed simultaneously or one by one in some order?

Replacing vs. accumulating traces If eligibility traces are used, which kind is most appropriate?

Real vs. simulated Should one backup real experience or simulated experience? If both, how much of each?

Location of backups What states or state–action pairs should be backed up? Modelfree methods can choose only among the states and state–action pairs actually encountered, but model-based methods can choose arbitrarily. There are many potent possibilities here.

Timing of backups Should backups be done as part of selecting actions, or only afterward?

Memory for backups How long should backed-up values be retained? Should they be retained permanently, or only while computing an action selection, as in heuristic search?

Of course, these dimensions are neither exhaustive nor mutually exclusive. Individual algorithms differ in many other ways as well, and many algorithms lie in several places along several dimensions. For example, Dyna methods use both real and simulated experience to affect the same value function. It is also perfectly sensible to maintain multiple value functions computed in different ways or over different state and action representations. These dimensions do, however, constitute a coherent set of ideas for describing and exploring a wide space of possible methods.

16.2 Other Frontier Dimensions

Much research remains to be done within this space of reinforcement learning methods. For example, even for the tabular case no control method using multistep backups has been proved to converge to an optimal policy. Among planning methods, basic ideas such as trajectory sampling and focusing sample backups are almost completely unexplored. On closer inspection, parts of the space will undoubtedly turn out to have far greater complexity and greater internal structure than is now apparent. There are also other dimensions along which reinforcement learning can be extended, we have not yet mentioned, that lead to a much larger space of methods. Here we identify some of these dimensions and note some of the open questions and frontiers that have been left out of the preceding chapters.

One of the most important extensions of reinforcement learning beyond what we have treated in this book is to eliminate the requirement that the

state representation have the Markov property. There are a number of interesting approaches to the non-Markov case. Most strive to construct from the given state signal and its past values a new signal that is Markov, or more nearly Markov. For example, one approach is based on the theory of partially observable MDPs (POMDPs). POMDPs are finite MDPs in which the state is not observable, but another "sensation" signal stochastically related to the state is observable. The theory of POMDPs has been extensively studied for the case of complete knowledge of the dynamics of the POMDP. In this case, Bayesian methods can be used to compute at each time step the probability of the environment's being in each state of the underlying MDP. This probability distribution can then be used as a new state signal for the original problem. The downside for the Bayesian POMDP approach is its computational expense and its strong reliance on complete environment models. Some of the recent work pursuing this approach is by Littman, Cassandra, and Kaelbling (1995), Parr and Russell (1995), and Chrisman (1992). If we are not willing to assume a complete model of a POMDP's dynamics, then existing theory seems to offer little guidance. Nevertheless, one can still attempt to construct a Markov state signal from the sequence of sensations. Various statistical and ad hoc methods along these lines have been explored (e.g., Chrisman, 1992; McCallum, 1993, 1995; Lin and Mitchell, 1992; Chapman and Kaelbling, 1991; Moore, 1994; Rivest and Schapire, 1987; Colombetti and Dorigo, 1994; Whitehead and Ballard, 1991; Hochreiter and Schmidhuber, 1997).

All of the above methods involve constructing an improved state representation from the non-Markov one provided by the environment. Another approach is to leave the state representation unchanged and use methods that are not too adversely affected by its being non-Markov (e.g., Singh, Jaakkola, and Jordan, 1994, 1995; Jaakkola, Singh and Jordan, 1995). In fact, most function approximation methods can be viewed in this way. For example, state aggregation methods for function approximation are in effect equivalent to a non-Markov representation in which all members of a set of states are mapped into a common sensation. There are other parallels between the issues of function approximation and non-Markov representations. In both cases the overall problem divides into two parts: constructing an improved representation, and making do with the current representation. In both cases the "making do" part is relatively well understood, whereas the constructive part is unclear and wide open. At this point we can only guess as to whether or not these parallels point to any common solution methods for the two problems.

Another important direction for extending reinforcement learning beyond what we have covered in this book is to incorporate ideas of modularity and hierarchy. Introductory reinforcement learning is about learning value functions and one-step models of the dynamics of the environment. But much of

what people learn does not seem to fall exactly into either of these categories. For example, consider what we know about tying our shoes, making a phone call, or traveling to London. Having learned how to do such things, we are then able to choose among them and plan as if they were primitive actions. What we have learned in order to do this are not conventional value functions or one-step models. We are able to plan and learn at a variety of levels and flexibly interrelate them. Much of our learning appears not to be about learning values directly, but about preparing us to quickly estimate values later in response to new situations or new information. Considerable reinforcement learning research has been directed at capturing such abilities (e.g., Watkins, 1989; Dayan and Hinton, 1993; Singh, 1992a, 1992b; Ring, 1994, Kaelbling, 1993b; Sutton, 1995).

Researchers have also explored ways of using the structure of particular tasks to advantage. For example, many problems have state representations that are naturally lists of variables, like the readings of multiple sensors or actions that are lists of component actions. The independence or near independence of some variables from others can sometimes be exploited to obtain more efficient special forms of reinforcement learning algorithms. It is sometimes even possible to decompose a problem into several independent subproblems that can be solved by separate learning agents. A reinforcement learning problem can usually be structured in many different ways, some reflecting natural aspects of the problem, such as the existence of physical sensors, and others being the result of explicit attempts to decompose the problem into simpler subproblems. Possibilities for exploiting structure in reinforcement learning and related planning problems have been studied by many researchers (e.g., Boutilier, Dearden, and Goldszmidt, 1995; Dean and Lin, 1995). There are also related studies of multiagent or distributed reinforcement learning (e.g., Littman, 1994; Markey, 1994; Crites and Barto, 1996; Tan, 1993).

Finally, we want to emphasize that reinforcement learning is meant to be a *general* approach to learning from interaction. It is general enough not to require special-purpose teachers and domain knowledge, but also general enough to utilize such things if they are available. For example, it is often possible to accelerate reinforcement learning by giving advice or hints to the agent (Clouse and Utgoff, 1992; Maclin and Shavlik, 1994) or by demonstrating instructive behavioral trajectories (Lin, 1992). Another way to make learning easier, related to "shaping" in psychology, is to give the learning agent a series of relatively easy problems building up to the harder problem of ultimate interest (e.g., Selfridge, Sutton, and Barto, 1985). These methods, and others not yet developed, have the potential to give the machine-learning terms *training* and *teaching* new meanings that are closer to their meanings for animal and human learning.

References

Agre, P. E. (1988). *The Dynamic Structure of Everyday Life*. Ph.D. thesis, Massachusetts Institute of Technology. AI-TR 1085, MIT Artificial Intelligence Laboratory.

Agre, P. E., Chapman, D. (1990). What are plans for? *Robotics and Autonomous Systems*, 6:17–34.

Albus, J. S. (1971). A theory of cerebellar function. *Mathematical Biosciences*, 10:25–61.

Albus, J. S. (1981). *Brain, Behavior, and Robotics*. Byte Books, Peterborough, NH.

Anderson, C. W. (1986). *Learning and Problem Solving with Multilayer Connectionist Systems*. Ph.D. thesis, University of Massachusetts, Amherst.

Anderson, C. W. (1987). Strategy learning with multilayer connectionist representations. *Proceedings of the Fourth International Workshop on Machine Learning*, pp. 103–114. Morgan Kaufmann, San Mateo, CA.

Anderson, J. A., Silverstein, J. W., Ritz, S. A., Jones, R. S. (1977). Distinctive features, categorical perception, and probability learning: Some applications of a neural model. *Psychological Review*, 84:413–451.

Andreae, J. H. (1963). STELLA: A scheme for a learning machine. In *Proceedings of the 2nd IFAC Congress, Basle*, pp. 497–502. Butterworths, London.

Andreae, J. H. (1969a). A learning machine with monologue. *International Journal of Man–Machine Studies*, 1:1–20.

Andreae, J. H. (1969b). Learning machines—a unified view. In A. R. Meetham and R. A. Hudson (eds.), *Encyclopedia of Information, Linguistics, and Control*, pp. 261–270. Pergamon, Oxford.

Andreae, J. H. (1977). *Thinking with the Teachable Machine*. Academic Press, London.

Auer, P. (2002). Using confidence bounds for exploitation-exploration tradeoffs. *Journal of Machine Learning Research 3*:397–422.

Baird, L. C. (1995). Residual algorithms: Reinforcement learning with function approximation. In *Proceedings of the Twelfth International Conference on Machine Learning*, pp. 30–37. Morgan Kaufmann, San Francisco.

Bao, G., Cassandras, C. G., Djaferis, T. E., Gandhi, A. D., Looze, D. P. (1994). Elevator dispatchers for down peak traffic. Technical report. ECE Department, University of Massachusetts, Amherst.

Barnard, E. (1993). Temporal-difference methods and Markov models. *IEEE Transactions on Systems, Man, and Cybernetics*, 23:357–365.

Barto, A. G. (1985). Learning by statistical cooperation of self-interested neuron-like computing elements. *Human Neurobiology*, 4:229–256.

Barto, A. G. (1986). Game-theoretic cooperativity in networks of self-interested units. In J. S. Denker (ed.), *Neural Networks for Computing*, pp. 41–46. American Institute of Physics, New York.

Barto, A. G. (1990). Connectionist learning for control: An overview. In T. Miller, R. S. Sutton, and P. J. Werbos (eds.), *Neural Networks for Control*, pp. 5–58. MIT Press, Cambridge, MA.

Barto, A. G. (1991). Some learning tasks from a control perspective. In L. Nadel and D. L. Stein (eds.), *1990 Lectures in Complex Systems*, pp. 195–223. Addison-Wesley, Redwood City, CA.

Barto, A. G. (1992). Reinforcement learning and adaptive critic methods. In D. A. White and D. A. Sofge (eds.), *Handbook of Intelligent Control: Neural, Fuzzy, and Adaptive Approaches*, pp. 469–491. Van Nostrand Reinhold, New York.

Barto, A. G. (1995a). Adaptive critics and the basal ganglia. In J. C. Houk, J. L. Davis, and D. G. Beiser (eds.), *Models of Information Processing in the Basal Ganglia*, pp. 215–232. MIT Press, Cambridge, MA.

Barto, A. G. (1995b). Reinforcement learning. In M. A. Arbib (ed.), *Handbook of Brain Theory and Neural Networks*, pp. 804–809. MIT Press, Cambridge, MA.

Barto, A. G., Anandan, P. (1985). Pattern recognizing stochastic learning automata. *IEEE Transactions on Systems, Man, and Cybernetics*, 15:360–375.

Barto, A. G., Anderson, C. W. (1985). Structural learning in connectionist systems. In *Program of the Seventh Annual Conference of the Cognitive Science Society*, pp. 43–54.

Barto, A. G., Anderson, C. W., Sutton, R. S. (1982). Synthesis of nonlinear control surfaces by a layered associative search network. *Biological Cybernetics*, 43:175–185.

Barto, A. G., Bradtke, S. J., Singh, S. P. (1991). Real-time learning and control using asynchronous dynamic programming. Technical Report 91-57. Department of Computer and Information Science, University of Massachusetts, Amherst.

Barto, A. G., Bradtke, S. J., Singh, S. P. (1995). Learning to act using real-time dynamic programming. *Artificial Intelligence*, 72:81–138.

Barto, A. G., Duff, M. (1994). Monte Carlo matrix inversion and reinforcement learning. In J. D. Cohen, G. Tesauro, and J. Alspector (eds.), *Advances in Neural Information Processing Systems: Proceedings of the 1993 Conference*, pp. 687–694. Morgan Kaufmann, San Francisco.

Barto, A. G., Jordan, M. I. (1987). Gradient following without back-propagation in layered networks. In M. Caudill and C. Butler (eds.), *Proceedings of the IEEE First Annual Conference on Neural Networks*, pp. II629–II636. SOS Printing, San Diego, CA.

Barto, A. G., Sutton, R. S. (1981a). Goal seeking components for adaptive intelligence: An initial assessment. Technical Report AFWAL-TR-81-1070. Air Force Wright Aeronautical Laboratories/Avionics Laboratory, Wright-Patterson AFB, OH.

Barto, A. G., Sutton, R. S. (1981b). Landmark learning: An illustration of associative search. *Biological Cybernetics*, 42:1–8.

Barto, A. G., Sutton, R. S. (1982). Simulation of anticipatory responses in classical conditioning by a neuron-like adaptive element. *Behavioural Brain Research*, 4:221–235.

Barto, A. G., Sutton, R. S., Anderson, C. W. (1983). Neuronlike elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, 13:835–846. Reprinted in J. A. Anderson and E. Rosenfeld (eds.), *Neurocomputing: Foundations of Research*, pp. 535–549. MIT Press, Cambridge, MA, 1988.

Barto, A. G., Sutton, R. S., Brouwer, P. S. (1981). Associative search network: A reinforcement learning associative memory. *Biological Cybernetics*, 40:201–211.

Bellman, R. E. (1956). A problem in the sequential design of experiments. *Sankhya*, 16:221–229.

Bellman, R. E. (1957a). *Dynamic Programming*. Princeton University Press,

Princeton.

Bellman, R. E. (1957b). A Markov decision process. *Journal of Mathematical Mechanics*, 6:679–684.

Bellman, R. E., Dreyfus, S. E. (1959). Functional approximations and dynamic programming. *Mathematical Tables and Other Aids to Computation*, 13:247–251.

Bellman, R. E., Kalaba, R., Kotkin, B. (1973). Polynomial approximation—A new computational technique in dynamic programming: Allocation processes. *Mathematical Computation*, 17:155–161.

Berry, D. A., Fristedt, B. (1985). *Bandit Problems*. Chapman and Hall, London.

Bertsekas, D. P. (1982). Distributed dynamic programming. *IEEE Transactions on Automatic Control*, 27:610–616.

Bertsekas, D. P. (1983). Distributed asynchronous computation of fixed points. *Mathematical Programming*, 27:107–120.

Bertsekas, D. P. (1987). *Dynamic Programming: Deterministic and Stochastic Models*. Prentice-Hall, Englewood Cliffs, NJ.

Bertsekas, D. P. (1995). *Dynamic Programming and Optimal Control*. Athena Scientific, Belmont, MA.

Bertsekas, D. P., Tsitsiklis, J. N. (1989). *Parallel and Distributed Computation: Numerical Methods*. Prentice-Hall, Englewood Cliffs, NJ.

Bertsekas, D. P., Tsitsiklis, J. N. (1996). *Neural Dynamic Programming*. Athena Scientific, Belmont, MA.

Biermann, A. W., Fairfield, J. R. C., Beres, T. R. (1982). Signature table systems and learning. *IEEE Transactions on Systems, Man, and Cybernetics*, 12:635–648.

Bishop, C. M. (1995). *Neural Networks for Pattern Recognition*. Clarendon, Oxford.

Booker, L. B. (1982). *Intelligent Behavior as an Adaptation to the Task Environment*. Ph.D. thesis, University of Michigan, Ann Arbor.

Boone, G. (1997). Minimum-time control of the acrobot. In *1997 International Conference on Robotics and Automation*, pp. 3281–3287. IEEE Robotics and Automation Society.

Boutilier, C., Dearden, R., Goldszmidt, M. (1995). Exploiting structure in policy construction. In *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence*, pp. 1104–1111. Morgan Kaufmann.

Boyan, J. A., Moore, A. W. (1995). Generalization in reinforcement learning: Safely approximating the value function. In G. Tesauro, D. S. Touretzky, and T. Leen (eds.), *Advances in Neural Information Processing Systems: Proceedings of the 1994 Conference*, pp. 369–376. MIT Press, Cambridge, MA.

Boyan, J. A., Moore, A. W., Sutton, R. S. (eds.). (1995). *Proceedings of the Workshop on Value Function Approximation. Machine Learning Conference 1995*. Technical Report CMU-CS-95-206. School of Computer Science, Carnegie Mellon University, Pittsburgh, PA.

Bradtke, S. J. (1993). Reinforcement learning applied to linear quadratic regulation. In S. J. Hanson, J. D. Cowan, and C. L. Giles (eds.), *Advances in Neural Information Processing Systems: Proceedings of the 1992 Conference*, pp. 295–302. Morgan Kaufmann, San Mateo, CA.

Bradtke, S. J. (1994). *Incremental Dynamic Programming for On-Line Adaptive Optimal Control*. Ph.D. thesis, University of Massachusetts, Amherst. Appeared as CMPSCI Technical Report 94-62.

Bradtke, S. J., Barto, A. G. (1996). Linear least–squares algorithms for temporal difference learning. *Machine Learning*, 22:33–57.

S. J. Bradtke, B. E. Ydstie, A. G. Barto (1994). Adaptive linear quadratic control using policy iteration. In *Proceedings of the American Control Conference*, pp. 3475–3479. American Automatic Control Council, Evanston, IL.

Bradtke, S. J., Duff, M. O. (1995). Reinforcement learning methods for continuous-time Markov decision problems. In G. Tesauro, D. Touretzky, and T. Leen (eds.), *Advances in Neural Information Processing Systems: Proceedings of the 1994 Conference*, pp. 393–400. MIT Press, Cambridge, MA.

Bridle, J. S. (1990). Training stochastic model recognition algorithms as networks can lead to maximum mutual information estimates of parameters. In D. S. Touretzky (ed.), *Advances in Neural Information Processing Systems: Proceedings of the 1989 Conference*, pp. 211–217. Morgan Kaufmann, San Mateo, CA.

Broomhead, D. S., Lowe, D. (1988). Multivariable functional interpolation and adaptive networks. *Complex Systems*, 2:321–355.

Bryson, A. E., Jr. (1996). Optimal control—1950 to 1985. *IEEE Control Systems*, 13(3):26–33.

Bush, R. R., Mosteller, F. (1955). *Stochastic Models for Learning*. Wiley, New York.

Byrne, J. H., Gingrich, K. J., Baxter, D. A. (1990). Computational capabilities of single neurons: Relationship to simple forms of associative and nonassociative learning in *aplysia*. In R. D. Hawkins and G. H. Bower (eds.), *Computational Models of Learning*, pp. 31–63. Academic Press, New York.

Campbell, D. T. (1960). Blind variation and selective survival as a general strategy in knowledge-processes. In M. C. Yovits and S. Cameron (eds.), *Self-Organizing Systems*, pp. 205–231. Pergamon, New York.

Carlström, J., Nordström, E. (1997). Control of self-similar ATM call traffic by reinforcement learning. In *Proceedings of the International Workshop on Applications of Neural Networks to Telecommunications 3*, pp. 54–62. Erlbaum, Hillsdale, NJ.

Chapman, D., Kaelbling, L. P. (1991). Input generalization in delayed reinforcement learning: An algorithm and performance comparisons. In *Proceedings of the Twelfth International Conference on Artificial Intelligence*, pp. 726–731. Morgan Kaufmann, San Mateo, CA.

Chow, C.-S., Tsitsiklis, J. N. (1991). An optimal one-way multigrid algorithm for discrete-time stochastic control. *IEEE Transactions on Automatic Control*, 36:898–914.

Chrisman, L. (1992). Reinforcement learning with perceptual aliasing: The perceptual distinctions approach. In *Proceedings of the Tenth National Conference on Artificial Intelligence*, pp. 183–188. AAAI/MIT Press, Menlo Park, CA.

Christensen, J., Korf, R. E. (1986). A unified theory of heuristic evaluation functions and its application to learning. In *Proceedings of the Fifth National Conference on Artificial Intelligence*, pp. 148–152. Morgan Kaufmann, San Mateo, CA.

Cichosz, P. (1995). Truncating temporal differences: On the efficient implementation of TD(λ) for reinforcement learning. *Journal of Artificial Intelligence Research*, 2:287–318.

Clark, W. A., Farley, B. G. (1955). Generalization of pattern recognition in a self-organizing system. In *Proceedings of the 1955 Western Joint Computer Conference*, pp. 86–91.

Clouse, J. (1996). *On Integrating Apprentice Learning and Reinforcement Learning TITLE2*. Ph.D. thesis, University of Massachusetts, Amherst. Appeared as CMPSCI Technical Report 96-026.

Clouse, J., Utgoff, P. (1992). A teaching method for reinforcement learning systems. In *Proceedings of the Ninth International Machine Learning*

Conference, pp. 92–101. Morgan Kaufmann, San Mateo, CA.

Colombetti, M., Dorigo, M. (1994). Training agent to perform sequential behavior. *Adaptive Behavior*, 2(3):247–275.

Connell, J. (1989). A colony architecture for an artificial creature. Technical Report AI-TR-1151. MIT Artificial Intelligence Laboratory, Cambridge, MA.

Connell, J., Mahadevan, S. (1993). *Robot Learning*. Kluwer Academic, Boston.

Craik, K. J. W. (1943). *The Nature of Explanation*. Cambridge University Press, Cambridge.

Crites, R. H. (1996). *Large-Scale Dynamic Optimization Using Teams of Reinforcement Learning Agents*. Ph.D. thesis, University of Massachusetts, Amherst.

Crites, R. H., Barto, A. G. (1996). Improving elevator performance using reinforcement learning. In D. S. Touretzky, M. C. Mozer, and M. E. Hasselmo (eds.), *Advances in Neural Information Processing Systems: Proceedings of the 1995 Conference*, pp. 1017–1023. MIT Press, Cambridge, MA.

Curtiss, J. H. (1954). A theoretical comparison of the efficiencies of two classical methods and a Monte Carlo method for computing one component of the solution of a set of linear algebraic equations. In H. A. Meyer (ed.), *Symposium on Monte Carlo Methods*, pp. 191–233. Wiley, New York.

Cziko, G. (1995). *Without Miracles: Universal Selection Theory and the Second Darvinian Revolution*. MIT Press, Cambridge, MA.

Daniel, J. W. (1976). Splines and efficiency in dynamic programming. *Journal of Mathematical Analysis and Applications*, 54:402–407.

Dayan, P. (1991). Reinforcement comparison. In D. S. Touretzky, J. L. Elman, T. J. Sejnowski, and G. E. Hinton (eds.), *Connectionist Models: Proceedings of the 1990 Summer School*, pp. 45–51. Morgan Kaufmann, San Mateo, CA.

Dayan, P. (1992). The convergence of TD(λ) for general λ. *Machine Learning*, 8:341–362.

Dayan, P., Hinton, G. E. (1993). Feudal reinforcement learning. In S. J. Hanson, J. D. Cohen, and C. L. Giles (eds.), *Advances in Neural Information Processing Systems: Proceedings of the 1992 Conference*, pp. 271–278. Morgan Kaufmann, San Mateo, CA.

Dayan, P., Sejnowski, T. (1994). TD(λ) converges with probability 1. *Machine Learning*, 14:295–301.

Dean, T., Lin, S.-H. (1995). Decomposition techniques for planning in stochastic domains. In *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence*, pp. 1121–1127. Morgan Kaufmann. See also Technical Report CS-95-10, Brown University, Department of Computer Science, 1995.

DeJong, G., Spong, M. W. (1994). Swinging up the acrobot: An example of intelligent control. In *Proceedings of the American Control Conference*, pp. 2158–2162. American Automatic Control Council, Evanston, IL.

Denardo, E. V. (1967). Contraction mappings in the theory underlying dynamic programming. *SIAM Review*, 9:165–177.

Dennett, D. C. (1978). *Brainstorms*, pp. 71–89. Bradford/MIT Press, Cambridge, MA.

Dietterich, T. G., Flann, N. S. (1995). Explanation-based learning and reinforcement learning: A unified view. In A. Prieditis and S. Russell (eds.), *Proceedings of the Twelfth International Conference on Machine Learning*, pp. 176–184. Morgan Kaufmann, San Francisco.

Doya, K. (1996). Temporal difference learning in continuous time and space. In D. S. Touretzky, M. C. Mozer, and M. E. Hasselmo (eds.), *Advances in Neural Information Processing Systems: Proceedings of the 1995 Conference*, pp. 1073–1079. MIT Press, Cambridge, MA.

Doyle, P. G., Snell, J. L. (1984). *Random Walks and Electric Networks*. The Mathematical Association of America. Carus Mathematical Monograph 22.

Dreyfus, S. E., Law, A. M. (1977). *The Art and Theory of Dynamic Programming*. Academic Press, New York.

Duda, R. O., Hart, P. E. (1973). *Pattern Classification and Scene Analysis*. Wiley, New York.

Duff, M. O. (1995). Q-learning for bandit problems. In A. Prieditis and S. Russell (eds.), *Proceedings of the Twelfth International Conference on Machine Learning*, pp. 209–217. Morgan Kaufmann, San Francisco.

Estes, W. K. (1950). Toward a statistical theory of learning. *Psycholollgical Review*, 57:94–107.

Farley, B. G., Clark, W. A. (1954). Simulation of self-organizing systems by digital computer. *IRE Transactions on Information Theory*, 4:76–84.

Feldbaum, A. A. (1965). *Optimal Control Systems*. Academic Press, New York.

Friston, K. J., Tononi, G., Reeke, G. N., Sporns, O., Edelman, G. M. (1994).

Value-dependent selection in the brain: Simulation in a synthetic neural model. *Neuroscience*, 59:229–243.

Fu, K. S. (1970). Learning control systems—Review and outlook. *IEEE Transactions on Automatic Control*, 15:210–221.

Galanter, E., Gerstenhaber, M. (1956). On thought: The extrinsic theory. *Psychological Review*, 63:218–227.

Gallant, S. I. (1993). *Neural Network Learning and Expert Systems*. MIT Press, Cambridge, MA.

Gällmo, O., Asplund, L. (1995). Reinforcement learning by construction of hypothetical targets. In J. Alspector, R. Goodman, and T. X. Brown (eds.), *Proceedings of the International Workshop on Applications of Neural Networks to Telecommunications 2*, pp. 300–307. Erlbaum, Hillsdale, NJ.

Gardner, M. (1973). Mathematical games. *Scientific American*, 228(1):108–115.

Gelperin, A., Hopfield, J. J., Tank, D. W. (1985). The logic of *limax* learning. In A. Selverston (ed.), *Model Neural Networks and Behavior*, pp. 247–261. Plenum Press, New York.

Gittins, J. C., Jones, D. M. (1974). A dynamic allocation index for the sequential design of experiments. In J. Gani, K. Sarkadi, and I. Vincze (eds.), *Progress in Statistics*, pp. 241–266. North-Holland, Amsterdam–London.

Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading, MA.

Goldstein, H. (1957). *Classical Mechanics*. Addison-Wesley, Reading, MA.

Goodwin, G. C., Sin, K. S. (1984). *Adaptive Filtering Prediction and Control*. Prentice-Hall, Englewood Cliffs, NJ.

Gordon, G. J. (1995). Stable function approximation in dynamic programming. In A. Prieditis and S. Russell (eds.), *Proceedings of the Twelfth International Conference on Machine Learning*, pp. 261–268. Morgan Kaufmann, San Francisco. An expanded version was published as Technical Report CMU-CS-95-103. Carnegie Mellon University, Pittsburgh, PA, 1995.

Gordon, G. J. (1996). Stable fitted reinforcement learning. In D. S. Touretzky, M. C. Mozer, M. E. Hasselmo (eds.), *Advances in Neural Information Processing Systems: Proceedings of the 1995 Conference*, pp. 1052–1058. MIT Press, Cambridge, MA.

Griffith, A. K. (1966). A new machine learning technique applied to the game of checkers. Technical Report Project MAC, Artificial Intelligence Memo

94. Massachusetts Institute of Technology, Cambridge, MA.

Griffith, A. K. (1974). A comparison and evaluation of three machine learning procedures as applied to the game of checkers. *Artificial Intelligence*, 5:137–148.

Gullapalli, V. (1990). A stochastic reinforcement algorithm for learning real-valued functions. *Neural Networks*, 3:671–692.

Gurvits, L., Lin, L.-J., Hanson, S. J. (1994). Incremental learning of evaluation functions for absorbing Markov chains: New methods and theorems. Preprint.

Hampson, S. E. (1983). *A Neural Model of Adaptive Behavior*. Ph.D. thesis, University of California, Irvine.

Hampson, S. E. (1989). *Connectionist Problem Solving: Computational Aspects of Biological Learning*. Birkhauser, Boston.

Hawkins, R. D., Kandel, E. R. (1984). Is there a cell-biological alphabet for simple forms of learning? *Psychological Review*, 91:375–391.

Hersh, R., Griego, R. J. (1969). Brownian motion and potential theory. *Scientific American*, 220:66–74.

Hilgard, E. R., Bower, G. H. (1975). *Theories of Learning*. Prentice-Hall, Englewood Cliffs, NJ.

Hinton, G. E. (1984). Distributed representations. Technical Report CMU-CS-84-157. Department of Computer Science, Carnegie-Mellon University, Pittsburgh, PA.

Hochreiter, S., Schmidhuber, J. (1997). LTSM can solve hard time lag problems. In *Advances in Neural Information Processing Systems: Proceedings of the 1996 Conference*, pp. 473–479. MIT Press, Cambridge, MA.

Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor.

Holland, J. H. (1976). Adaptation. In R. Rosen and F. M. Snell (eds.), *Progress in Theoretical Biology*, vol. 4, pp. 263–293. Academic Press, New York.

Holland, J. H. (1986). Escaping brittleness: The possibility of general-purpose learning algorithms applied to rule-based systems. In R. S. Michalski, J. G. Carbonell, and T. M. Mitchell (eds.), *Machine Learning: An Artificial Intelligence Approach*, vol. 2, pp. 593–623. Morgan Kaufmann, San Mateo, CA.

Houk, J. C., Adams, J. L., Barto, A. G. (1995). A model of how the basal

ganglia generates and uses neural signals that predict reinforcement. In J. C. Houk, J. L. Davis, and D. G. Beiser (eds.), *Models of Information Processing in the Basal Ganglia*, pp. 249–270. MIT Press, Cambridge, MA.

Howard, R. (1960). *Dynamic Programming and Markov Processes.* MIT Press, Cambridge, MA.

Hull, C. L. (1943). *Principles of Behavior.* Appleton-Century, New York.

Hull, C. L. (1952). *A Behavior System.* Wiley, New York.

Jaakkola, T., Jordan, M. I., Singh, S. P. (1994). On the convergence of stochastic iterative dynamic programming algorithms. *Neural Computation*, 6:1185–1201.

Jaakkola, T., Singh, S. P., Jordan, M. I. (1995). Reinforcement learning algorithm for partially observable Markov decision problems. In G. Tesauro, D. S. Touretzky, T. Leen (eds.), *Advances in Neural Information Processing Systems: Proceedings of the 1994 Conference*, pp. 345–352. MIT Press, Cambridge, MA.

Kaelbling, L. P. (1993a). Hierarchical learning in stochastic domains: Preliminary results. In *Proceedings of the Tenth International Conference on Machine Learning*, pp. 167–173. Morgan Kaufmann, San Mateo, CA.

Kaelbling, L. P. (1993b). *Learning in Embedded Systems.* MIT Press, Cambridge, MA.

Kaelbling, L. P. (ed.). (1996). Special issue of *Machine Learning* on reinforcement learning, 22.

Kaelbling, L. P., Littman, M. L., Moore, A. W. (1996). Reinforcement learning: A survey. *Journal of Artificial Intelligence Research*, 4:237–285.

Kakutani, S. (1945). Markov processes and the Dirichlet problem. *Proceedings of the Japan Academy*, 21:227–233.

Kalos, M. H., Whitlock, P. A. (1986). *Monte Carlo Methods.* Wiley, New York.

Kanerva, P. (1988). *Sparse Distributed Memory.* MIT Press, Cambridge, MA.

Kanerva, P. (1993). Sparse distributed memory and related models. In M. H. Hassoun (ed.), *Associative Neural Memories: Theory and Implementation*, pp. 50–76. Oxford University Press, New York.

Kashyap, R. L., Blaydon, C. C., Fu, K. S. (1970). Stochastic approximation. In J. M. Mendel and K. S. Fu (eds.), *Adaptive, Learning, and Pattern Recognition Systems: Theory and Applications*, pp. 329–355. Academic Press, New York.

Keerthi, S. S., Ravindran, B. (1997). Reinforcement learning. In E. Fiesler and R. Beale (eds.), *Handbook of Neural Computation*, C3. Oxford University Press, New York.

Kimble, G. A. (1961). *Hilgard and Marquis' Conditioning and Learning*. Appleton-Century-Crofts, New York.

Kimble, G. A. (1967). *Foundations of Conditioning and Learning*. Appleton-Century-Crofts, New York.

Kirkpatrick, S., Gelatt, C. D., Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220:671–680.

Klopf, A. H. (1972). Brain function and adaptive systems—A heterostatic theory. Technical Report AFCRL-72-0164, Air Force Cambridge Research Laboratories, Bedford, MA. A summary appears in *Proceedings of the International Conference on Systems, Man, and Cybernetics*. IEEE Systems, Man, and Cybernetics Society, Dallas, TX, 1974.

Klopf, A. H. (1975). A comparison of natural and artificial intelligence. *SIGART Newsletter*, 53:11–13.

Klopf, A. H. (1982). *The Hedonistic Neuron: A Theory of Memory, Learning, and Intelligence*. Hemisphere, Washington, DC.

Klopf, A. H. (1988). A neuronal model of classical conditioning. *Psychobiology*, 16:85–125.

Kohonen, T. (1977). *Associative Memory: A System Theoretic Approach*. Springer-Verlag, Berlin.

Korf, R. E. (1988). Optimal path finding algorithms. In L. N. Kanal and V. Kumar (eds.), *Search in Artificial Intelligence*, pp. 223–267. Springer Verlag, Berlin.

Kraft, L. G., Campagna, D. P. (1990). A summary comparison of CMAC neural network and traditional adaptive control systems. In T. Miller, R. S. Sutton, and P. J. Werbos (eds.), *Neural Networks for Control*, pp. 143–169. MIT Press, Cambridge, MA.

Kraft, L. G., Miller, W. T., Dietz, D. (1992). Development and application of CMAC neural network-based control. In D. A. White and D. A. Sofge (eds.), *Handbook of Intelligent Control: Neural, Fuzzy, and Adaptive Approaches*, pp. 215–232. Van Nostrand Reinhold, New York.

Kumar, P. R., Varaiya, P. (1986). *Stochastic Systems: Estimation, Identification, and Adaptive Control*. Prentice-Hall, Englewood Cliffs, NJ.

Kumar, P. R. (1985). A survey of some results in stochastic adaptive control. *SIAM Journal of Control and Optimization*, 23:329–380.

Kumar, V., Kanal, L. N. (1988). The CDP: A unifying formulation for heuristic search, dynamic programming, and branch-and-bound. In L. N. Kanal and V. Kumar (eds.), *Search in Artificial Intelligence*, pp. 1–37. Springer-Verlag, Berlin.

Kushner, H. J., Dupuis, P. (1992). *Numerical Methods for Stochastic Control Problems in Continuous Time*. Springer-Verlag, New York.

Lai, T. L. (1987). Adaptive treatment allocation and the multi-armed bandit problem. *The Annals of Statistics*, 15(3):1091–1114.

Lang, K. J., Waibel, A. H., Hinton, G. E. (1990). A time-delay neural network architecture for isolated word recognition. *Neural Networks*, 3:33–43.

Lin, C.-S., Kim, H. (1991). CMAC-based adaptive critic self-learning control. *IEEE Transactions on Neural Networks*, 2:530–533.

Lin, L.-J. (1992). Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8:293–321.

Lin, L.-J., Mitchell, T. (1992). Reinforcement learning with hidden states. In *Proceedings of the Second International Conference on Simulation of Adaptive Behavior: From Animals to Animats*, pp. 271–280. MIT Press, Cambridge, MA.

Littman, M. L. (1994). Markov games as a framework for multi-agent reinforcement learning. In *Proceedings of the Eleventh International Conference on Machine Learning*, pp. 157–163. Morgan Kaufmann, San Francisco.

Littman, M. L., Cassandra, A. R., Kaelbling, L. P. (1995). Learning policies for partially observable environments: Scaling up. In A. Prieditis and S. Russell (eds.), *Proceedings of the Twelfth International Conference on Machine Learning*, pp. 362–370. Morgan Kaufmann, San Francisco.

Littman, M. L., Dean, T. L., Kaelbling, L. P. (1995). On the complexity of solving Markov decision problems. In *Proceedings of the Eleventh Annual Conference on Uncertainty in Artificial Intelligence*, pp. 394–402.

Ljung, L., Söderstrom, T. (1983). *Theory and Practice of Recursive Identification*. MIT Press, Cambridge, MA.

Lovejoy, W. S. (1991). A survey of algorithmic methods for partially observed Markov decision processes. *Annals of Operations Research*, 28:47–66.

Luce, D. (1959). *Individual Choice Behavior*. Wiley, New York.

Maclin, R., Shavlik, J. W. (1994). Incorporating advice into agents that learn from reinforcements. In *Proceedings of the Twelfth National Conference on Artificial Intelligence*, pp. 694–699. AAAI Press, Menlo Park, CA.

Mahadevan, S. (1996). Average reward reinforcement learning: Foundations, algorithms, and empirical results. *Machine Learning*, 22:159–196.

Markey, K. L. (1994). Efficient learning of multiple degree-of-freedom control problems with quasi-independent Q-agents. In M. C. Mozer, P. Smolensky, D. S. Touretzky, J. L. Elman, and A. S. Weigend (eds.), *Proceedings of the 1990 Connectionist Models Summer School*. Erlbaum, Hillsdale, NJ.

Mazur, J. E. (1994). *Learning and Behavior*, 3rd ed. Prentice-Hall, Englewood Cliffs, NJ.

McCallum, A. K. (1993). Overcoming incomplete perception with utile distinction memory. In *Proceedings of the Tenth International Conference on Machine Learning*, pp. 190–196. Morgan Kaufmann, San Mateo, CA.

McCallum, A. K. (1995). *Reinforcement Learning with Selective Perception and Hidden State*. Ph.D. thesis, University of Rochester, Rochester, NY.

Mendel, J. M. (1966). A survey of learning control systems. *ISA Transactions*, 5:297–303.

Mendel, J. M., McLaren, R. W. (1970). Reinforcement learning control and pattern recognition systems. In J. M. Mendel and K. S. Fu (eds.), *Adaptive, Learning and Pattern Recognition Systems: Theory and Applications*, pp. 287–318. Academic Press, New York.

Michie, D. (1961). Trial and error. In S. A. Barnett and A. McLaren (eds.), *Science Survey, Part 2*, pp. 129–145. Penguin, Harmondsworth.

Michie, D. (1963). Experiments on the mechanisation of game learning. 1. characterization of the model and its parameters. *Computer Journal*, 1:232–263.

Michie, D. (1974). *On Machine Intelligence*. Edinburgh University Press, Edinburgh.

Michie, D., Chambers, R. A. (1968). BOXES: An experiment in adaptive control. In E. Dale and D. Michie (eds.), *Machine Intelligence 2*, pp. 137–152. Oliver and Boyd, Edinburgh.

Miller, S., Williams, R. J. (1992). Learning to control a bioreactor using a neural net Dyna-Q system. In *Proceedings of the Seventh Yale Workshop on Adaptive and Learning Systems*, pp. 167–172. Center for Systems Science, Dunham Laboratory, Yale University, New Haven.

Miller, W. T., Scalera, S. M., Kim, A. (1994). Neural network control of dynamic balance for a biped walking robot. In *Proceedings of the Eighth Yale Workshop on Adaptive and Learning Systems*, pp. 156–161. Center for Systems Science, Dunham Laboratory, Yale University, New Haven.

Minsky, M. L. (1954). *Theory of Neural-Analog Reinforcement Systems and Its Application to the Brain-Model Problem*. Ph.D. thesis, Princeton University.

Minsky, M. L. (1961). Steps toward artificial intelligence. *Proceedings of the Institute of Radio Engineers*, 49:8–30. Reprinted in E. A. Feigenbaum and J. Feldman (eds.), *Computers and Thought*, pp. 406–450. McGraw-Hill, New York, 1963.

Minsky, M. L. (1967). *Computation: Finite and Infinite Machines*. Prentice-Hall, Englewood Cliffs, NJ.

Montague, P. R., Dayan, P., Sejnowski, T. J. (1996). A framework for mesencephalic dopamine systems based on predictive Hebbian learning. *Journal of Neuroscience*, 16:1936–1947.

Moore, A. W. (1990). *Efficient Memory-Based Learning for Robot Control*. Ph.D. thesis, University of Cambridge.

Moore, A. W. (1994). The parti-game algorithm for variable resolution reinforcement learning in multidimensional spaces. In J. D. Cohen, G. Tesauro and J. Alspector (eds.), *Advances in Neural Information Processing Systems: Proceedings of the 1993 Conference*, pp. 711–718. Morgan Kaufmann, San Francisco.

Moore, A. W., Atkeson, C. G. (1993). Prioritized sweeping: Reinforcement learning with less data and less real time. *Machine Learning*, 13:103–130.

Moore, J. W., Desmond, J. E., Berthier, N. E., Blazis, E. J., Sutton, R. S., and Barto, A. G. (1986). Simulation of the classically conditioned nictitating membrane response by a neuron-like adaptive element: I. Response topography, neuronal firing, and interstimulus intervals. *Behavioural Brain Research*, 21:143–154.

Narendra, K. S., Thathachar, M. A. L. (1974). Learning automata—A survey. *IEEE Transactions on Systems, Man, and Cybernetics*, 4:323–334.

Narendra, K. S., Thathachar, M. A. L. (1989). *Learning Automata: An Introduction*. Prentice-Hall, Englewood Cliffs, NJ.

Narendra, K. S., Wheeler, R. M. (1986). Decentralized learning in finite Markov chains. *IEEE Transactions on Automatic Control*, AC31(6):519–526.

Nie, J., Haykin, S. (1996). A dynamic channel assignment policy through Q-learning. CRL Report 334. Communications Research Laboratory, McMaster University, Hamilton, Ontario.

Page, C. V. (1977). Heuristics for signature table analysis as a pattern recog-

nition technique. *IEEE Transactions on Systems, Man, and Cybernetics*, 7:77–86.

Parr, R., Russell, S. (1995). Approximating optimal policies for partially observable stochastic domains. In *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence*, pp. 1088–1094. Morgan Kaufmann.

Pavlov, P. I. (1927). *Conditioned Reflexes*. Oxford University Press, London.

Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, Reading, MA.

Peng, J. (1993). *Efficient Dynamic Programming-Based Learning for Control*. Ph.D. thesis, Northeastern University, Boston.

Peng, J. and Williams, R. J. (1993). Efficient learning and planning within the Dyna framework. *Adaptive Behavior*, 1(4):437–454.

Peng, J., Williams, R. J. (1994). Incremental multi-step Q-learning. In W. W. Cohen and H. Hirsh (eds.), *Proceedings of the Eleventh International Conference on Machine Learning*, pp. 226–232. Morgan Kaufmann, San Francisco.

Peng, J., Williams, R. J. (1996). Incremental multi-step Q-learning. *Machine Learning*, 22:283–290.

Phansalkar, V. V., Thathachar, M. A. L. (1995). Local and global optimization algorithms for generalized learning automata. *Neural Computation*, 7:950–973.

Poggio, T., Girosi, F. (1989). A theory of networks for approximation and learning. A.I. Memo 1140. Artificial Intelligence Laboratory, Massachusetts Institute of Technology, Cambridge, MA.

Poggio, T., Girosi, F. (1990). Regularization algorithms for learning that are equivalent to multilayer networks. *Science*, 247:978–982.

Powell, M. J. D. (1987). Radial basis functions for multivariate interpolation: A review. In J. C. Mason and M. G. Cox (eds.), *Algorithms for Approximation*, pp. 143–167. Clarendon Press, Oxford.

Puterman, M. L. (1994). *Markov Decision Problems*. Wiley, New York.

Puterman, M. L., Shin, M. C. (1978). Modified policy iteration algorithms for discounted Markov decision problems. *Management Science*, 24:1127–1137.

Reetz, D. (1977). Approximate solutions of a discounted Markovian decision process. *Bonner Mathematische Schriften*, 98:77–92.

Ring, M. B. (1994). *Continual Learning in Reinforcement Environments*. Ph.D. thesis, University of Texas, Austin.

Rivest, R. L., Schapire, R. E. (1987). Diversity-based inference of finite automata. In *Proceedings of the Twenty-Eighth Annual Symposium on Foundations of Computer Science*, pp. 78–87. Computer Society Press of the IEEE, Washington, DC.

Robbins, H. (1952). Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*, 58:527–535.

Robertie, B. (1992). Carbon versus silicon: Matching wits with TD-Gammon. *Inside Backgammon*, 2:14–22.

Rosenblatt, F. (1962). *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Spartan Books, Washington, DC.

Ross, S. (1983). *Introduction to Stochastic Dynamic Programming*. Academic Press, New York.

Rubinstein, R. Y. (1981). *Simulation and the Monte Carlo Method*. Wiley, New York.

Rumelhart, D. E., Hinton, G. E., Williams, R. J. (1986). Learning internal representations by error propagation. In D. E. Rumelhart and J. L. McClelland (eds.), *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, vol. I, *Foundations*. Bradford/MIT Press, Cambridge, MA.

Rummery, G. A. (1995). *Problem Solving with Reinforcement Learning*. Ph.D. thesis, Cambridge University.

Rummery, G. A., Niranjan, M. (1994). On-line Q-learning using connectionist systems. Technical Report CUED/F-INFENG/TR 166. Engineering Department, Cambridge University.

Russell, S., Norvig, P. (1995). *Artificial Intelligence: A Modern Approach*. Prentice-Hall, Englewood Cliffs, NJ.

Rust, J. (1996). Numerical dynamic programming in economics. In H. Amman, D. Kendrick, and J. Rust (eds.), *Handbook of Computational Economics*, pp. 614–722. Elsevier, Amsterdam.

Samuel, A. L. (1959). Some studies in machine learning using the game of checkers. *IBM Journal on Research and Development*, 3:211–229. Reprinted in E. A. Feigenbaum and J. Feldman (eds.), *Computers and Thought*, pp. 71–105. McGraw-Hill, New York, 1963.

Samuel, A. L. (1967). Some studies in machine learning using the game of checkers. II—Recent progress. *IBM Journal on Research and Develop-*

ment, 11:601–617.

Schultz, D. G., Melsa, J. L. (1967). *State Functions and Linear Control Systems*. McGraw-Hill, New York.

Schultz, W., Dayan, P., Montague, P. R. (1997). A neural substrate of prediction and reward. *Science*, 275:1593–1598.

Schwartz, A. (1993). A reinforcement learning method for maximizing undiscounted rewards. In *Proceedings of the Tenth International Conference on Machine Learning*, pp. 298–305. Morgan Kaufmann, San Mateo, CA.

Schweitzer, P. J., Seidmann, A. (1985). Generalized polynomial approximations in Markovian decision processes. *Journal of Mathematical Analysis and Applications*, 110:568–582.

Selfridge, O. J., Sutton, R. S., Barto, A. G. (1985). Training and tracking in robotics. In A. Joshi (ed.), *Proceedings of the Ninth International Joint Conference on Artificial Intelligence*, pp. 670–672. Morgan Kaufmann, San Mateo, CA.

Shannon, C. E. (1950). Programming a computer for playing chess. *Philosophical Magazine*, 41:256–275.

Shewchuk, J., Dean, T. (1990). Towards learning time-varying functions with high input dimensionality. In *Proceedings of the Fifth IEEE International Symposium on Intelligent Control*, pp. 383–388. IEEE Computer Society Press, Los Alamitos, CA.

Singh, S. P. (1992a). Reinforcement learning with a hierarchy of abstract models. In *Proceedings of the Tenth National Conference on Artificial Intelligence*, pp. 202–207. AAAI/MIT Press, Menlo Park, CA.

Singh, S. P. (1992b). Scaling reinforcement learning algorithms by learning variable temporal resolution models. In *Proceedings of the Ninth International Machine Learning Conference*, pp. 406–415. Morgan Kaufmann, San Mateo, CA.

Singh, S. P. (1993). *Learning to Solve Markovian Decision Processes*. Ph.D. thesis, University of Massachusetts, Amherst. Appeared as CMPSCI Technical Report 93-77.

Singh, S. P., Bertsekas, D. (1997). Reinforcement learning for dynamic channel allocation in cellular telephone systems. In *Advances in Neural Information Processing Systems: Proceedings of the 1996 Conference*, pp. 974–980. MIT Press, Cambridge, MA.

Singh, S. P., Jaakkola, T., Jordan, M. I. (1994). Learning without state-estimation in partially observable Markovian decision problems. In W. W. Co-

hen and H. Hirsch (eds.), *Proceedings of the Eleventh International Conference on Machine Learning*, pp. 284–292. Morgan Kaufmann, San Francisco.

Singh, S. P., Jaakkola, T., Jordan, M. I. (1995). Reinforcement learing with soft state aggregation. In G. Tesauro, D. S. Touretzky, T. Leen (eds.), *Advances in Neural Information Processing Systems: Proceedings of the 1994 Conference*, pp. 359–368. MIT Press, Cambridge, MA.

Singh, S. P., Sutton, R. S. (1996). Reinforcement learning with replacing eligibility traces. *Machine Learning*, 22:123–158.

Sivarajan, K. N., McEliece, R. J., Ketchum, J. W. (1990). Dynamic channel assignment in cellular radio. In *Proceedings of the 40th Vehicular Technology Conference*, pp. 631–637.

Skinner, B. F. (1938). *The Behavior of Organisms*. Appleton-Century, New York.

Sofge, D. A., White, D. A. (1992). Applied learning: Optimal control for manufacturing. In D. A. White and D. A. Sofge (eds.), *Handbook of Intelligent Control: Neural, Fuzzy, and Adaptive Approaches*, pp. 259–281. Van Nostrand Reinhold, New York.

Spong, M. W. (1994). Swing up control of the acrobot. In *Proceedings of the 1994 IEEE Conference on Robotics and Automation*, pp. 2356-2361. IEEE Computer Society Press, Los Alamitos, CA.

Staddon, J. E. R. (1983). *Adaptive Behavior and Learning*. Cambridge University Press, Cambridge.

Sutton, R. S. (1978a). Learning theory support for a single channel theory of the brain. Unpublished report.

Sutton, R. S. (1978b). Single channel theory: A neuronal theory of learning. *Brain Theory Newsletter*, 4:72–75. Center for Systems Neuroscience, University of Massachusetts, Amherst, MA.

Sutton, R. S. (1978c). A unified theory of expectation in classical and instrumental conditioning. Bachelors thesis, Stanford University.

Sutton, R. S. (1984). *Temporal Credit Assignment in Reinforcement Learning*. Ph.D. thesis, University of Massachusetts, Amherst.

Sutton, R. S. (1988). Learning to predict by the method of temporal differences. *Machine Learning*, 3:9–44.

Sutton, R. S. (1990). Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the Seventh International Conference on Machine Learning*, pp. 216–

224. Morgan Kaufmann, San Mateo, CA.

Sutton, R. S. (1991a). Dyna, an integrated architecture for learning, planning, and reacting. *SIGART Bulletin*, 2:160–163. ACM Press.

Sutton, R. S. (1991b). Planning by incremental dynamic programming. In L. A. Birnbaum and G. C. Collins (eds.), *Proceedings of the Eighth International Workshop on Machine Learning*, pp. 353–357. Morgan Kaufmann, San Mateo, CA.

Sutton, R. S. (1995). TD models: Modeling the world at a mixture of time scales. In A. Prieditis and S. Russell (eds.), *Proceedings of the Twelfth International Conference on Machine Learning*, pp. 531–539. Morgan Kaufmann, San Francisco.

Sutton, R. S. (1996). Generalization in reinforcement learning: Successful examples using sparse coarse coding. In D. S. Touretzky, M. C. Mozer and M. E. Hasselmo (eds.), *Advances in Neural Information Processing Systems: Proceedings of the 1995 Conference*, pp. 1038–1044. MIT Press, Cambridge, MA.

Sutton, R. S. (ed.). (1992). Special issue of *Machine Learning* on reinforcement learning, 8. Also published as *Reinforcement Learning*. Kluwer Academic, Boston, 1992.

Sutton, R. S., Barto, A. G. (1981a). Toward a modern theory of adaptive networks: Expectation and prediction. *Psychological Review*, 88:135–170.

Sutton, R. S., Barto, A. G. (1981b). An adaptive network that constructs and uses an internal model of its world. *Cognition and Brain Theory*, 3:217–246.

Sutton, R. S., Barto, A. G. (1987). A temporal-difference model of classical conditioning. In *Proceedings of the Ninth Annual Conference of the Cognitive Science Society*, pp. 355-378. Erlbaum, Hillsdale, NJ.

Sutton, R. S., Barto, A. G. (1990). Time-derivative models of Pavlovian reinforcement. In M. Gabriel and J. Moore (eds.), *Learning and Computational Neuroscience: Foundations of Adaptive Networks*, pp. 497–537. MIT Press, Cambridge, MA.

Sutton, R. S., Pinette, B. (1985). The learning of world models by connectionist networks. In *Proceedings of the Seventh Annual Conference of the Cognitive Science Society*, pp. 54-64.

Sutton, R. S., Singh, S. (1994). On bias and step size in temporal-difference learning. In *Proceedings of the Eighth Yale Workshop on Adaptive and Learning Systems*, pp. 91–96. Center for Systems Science, Dunham Labo-

ratory, Yale University, New Haven.

Sutton, R. S., Whitehead, D. S. (1993). Online learning with random representations. In *Proceedings of the Tenth International Machine Learning Conference*, pp. 314-321. Morgan Kaufmann, San Mateo, CA.

Tadepalli, P., Ok, D. (1994). H-learning: A reinforcement learning method to optimize undiscounted average reward. Technical Report 94-30-01. Oregon State University, Computer Science Department, Corvallis.

Tan, M. (1991). Learning a cost-sensitive internal representation for reinforcement learning. In L. A. Birnbaum and G. C. Collins (eds.), *Proceedings of the Eighth International Workshop on Machine Learning*, pp. 358–362. Morgan Kaufmann, San Mateo, CA.

Tan, M. (1993). Multi-agent reinforcement learning: Independent vs. cooperative agents. In *Proceedings of the Tenth International Conference on Machine Learning*, pp. 330–337. Morgan Kaufmann, San Mateo, CA.

Tesauro, G. J. (1986). Simple neural models of classical conditioning. *Biological Cybernetics*, 55:187–200.

Tesauro, G. J. (1992). Practical issues in temporal difference learning. *Machine Learning*, 8:257–277.

Tesauro, G. J. (1994). TD-Gammon, a self-teaching backgammon program, achieves master-level play. *Neural Computation*, 6(2):215–219.

Tesauro, G. J. (1995). Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38:58–68.

Tesauro, G. J., Galperin, G. R. (1997). On-line policy improvement using Monte-Carlo search. In *Advances in Neural Information Processing Systems: Proceedings of the 1996 Conference*, pp. 1068–1074. MIT Press, Cambridge, MA.

Tham, C. K. (1994). *Modular On-Line Function Approximation for Scaling up Reinforcement Learning*. PhD thesis, Cambridge University.

Thathachar, M. A. L. and Sastry, P. S. (1985). A new approach to the design of reinforcement schemes for learning automata. *IEEE Transactions on Systems, Man, and Cybernetics*, 15:168–175.

Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25:285–294.

Thompson, W. R. (1934). On the theory of apportionment. *American Journal of Mathematics*, 57:450–457.

Thorndike, E. L. (1911). *Animal Intelligence*. Hafner, Darien, CT.

Thorp, E. O. (1966). *Beat the Dealer: A Winning Strategy for the Game of Twenty-One*. Random House, New York.

Tolman, E. C. (1932). *Purposive Behavior in Animals and Men*. Century, New York.

Tsetlin, M. L. (1973). *Automaton Theory and Modeling of Biological Systems*. Academic Press, New York.

Tsitsiklis, J. N. (1994). Asynchronous stochastic approximation and Q-learning. *Machine Learning*, 16:185–202.

Tsitsiklis, J. N. (2002). On the convergence of optimistic policy iteration. *Journal of Machine Learning Research*, 3:59–72.

Tsitsiklis, J. N. and Van Roy, B. (1996). Feature-based methods for large scale dynamic programming. *Machine Learning*, 22:59–94.

Tsitsiklis, J. N., Van Roy, B. (1997). An analysis of temporal-difference learning with function approximation. *IEEE Transactions on Automatic Control*, 42:674–690.

Tsitsiklis, J. N., Van Roy, B. (1999). Average cost temporal-difference learning. *Automatica*, 35:1799–1808. Also: Technical Report LIDS-P-2390. Laboratory for Information and Decision Systems, Massachusetts Institute of Technology, Cambridge, MA, 1997.

Ungar, L. H. (1990). A bioreactor benchmark for adaptive network-based process control. In W. T. Miller, R. S. Sutton, and P. J. Werbos (eds.), *Neural Networks for Control*, pp. 387–402. MIT Press, Cambridge, MA.

Waltz, M. D., Fu, K. S. (1965). A heuristic approach to reinforcement learning control systems. *IEEE Transactions on Automatic Control*, 10:390–398.

Watkins, C. J. C. H. (1989). *Learning from Delayed Rewards*. Ph.D. thesis, Cambridge University.

Watkins, C. J. C. H., Dayan, P. (1992). Q-learning. *Machine Learning*, 8:279–292.

Werbos, P. J. (1977). Advanced forecasting methods for global crisis warning and models of intelligence. *General Systems Yearbook*, 22:25–38.

Werbos, P. J. (1982). Applications of advances in nonlinear sensitivity analysis. In R. F. Drenick and F. Kozin (eds.), *System Modeling and Optimization*, pp. 762–770. Springer-Verlag, Berlin.

Werbos, P. J. (1987). Building and understanding adaptive systems: A statistical/numerical approach to factory automation and brain research. *IEEE*

Transactions on Systems, Man, and Cybernetics, 17:7–20.

Werbos, P. J. (1988). Generalization of back propagation with applications to a recurrent gas market model. *Neural Networks*, 1:339–356.

Werbos, P. J. (1989). Neural networks for control and system identification. In *Proceedings of the 28th Conference on Decision and Control*, pp. 260–265. IEEE Control Systems Society.

Werbos, P. J. (1990). Consistency of HDP applied to a simple reinforcement learning problem. *Neural Networks*, 3:179–189.

Werbos, P. J. (1992). Approximate dynamic programming for real-time control and neural modeling. In D. A. White and D. A. Sofge (eds.), *Handbook of Intelligent Control: Neural, Fuzzy, and Adaptive Approaches*, pp. 493–525. Van Nostrand Reinhold, New York.

White, D. J. (1969). *Dynamic Programming*. Holden-Day, San Francisco.

White, D. J. (1985). Real applications of Markov decision processes. *Interfaces*, 15:73–83.

White, D. J. (1988). Further real applications of Markov decision processes. *Interfaces*, 18:55–61.

White, D. J. (1993). A survey of applications of Markov decision processes. *Journal of the Operational Research Society*, 44:1073–1096.

Whitehead, S. D., Ballard, D. H. (1991). Learning to perceive and act by trial and error. *Machine Learning*, 7:45–83.

Whitt, W. (1978). Approximations of dynamic programs I. *Mathematics of Operations Research*, 3:231–243.

Whittle, P. (1982). *Optimization over Time*, vol. 1. Wiley, New York.

Whittle, P. (1983). *Optimization over Time*, vol. 2. Wiley, New York.

Widrow, B., Gupta, N. K., Maitra, S. (1973). Punish/reward: Learning with a critic in adaptive threshold systems. *IEEE Transactions on Systems, Man, and Cybernetics*, 3:455–465.

Widrow, B., Hoff, M. E. (1960). Adaptive switching circuits. In *1960 WESCON Convention Record Part IV*, pp. 96–104. Institute of Radio Engineers, New York. Reprinted in J. A. Anderson and E. Rosenfeld, *Neurocomputing: Foundations of Research*, pp. 126–134. MIT Press, Cambridge, MA, 1988.

Widrow, B., Smith, F. W. (1964). Pattern-recognizing control systems. In J. T. Tou and R. H. Wilcox (eds.), *Computer and Information Sciences*, pp. 288–317. Spartan, Washington, DC.

Widrow, B., Stearns, S. D. (1985). *Adaptive Signal Processing*. Prentice-Hall, Englewood Cliffs, NJ.

Williams, R. J. (1986). Reinforcement learning in connectionist networks: A mathematical analysis. Technical Report ICS 8605. Institute for Cognitive Science, University of California at San Diego, La Jolla.

Williams, R. J. (1987). Reinforcement-learning connectionist systems. Technical Report NU-CCS-87-3. College of Computer Science, Northeastern University, Boston.

Williams, R. J. (1988). On the use of backpropagation in associative reinforcement learning. In *Proceedings of the IEEE International Conference on Neural Networks*, pp. I263–I270. IEEE San Diego section and IEEE TAB Neural Network Committee.

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256.

Williams, R. J., Baird, L. C. (1990). A mathematical analysis of actor-critic architectures for learning optimal controls through incremental dynamic programming. In *Proceedings of the Sixth Yale Workshop on Adaptive and Learning Systems*, pp. 96–101. Center for Systems Science, Dunham Laboratory, Yale University, New Haven.

Wilson, S. W. (1994). ZCS: A zeroth order classifier system. *Evolutionary Computation*, 2:1–18.

Witten, I. H. (1976). The apparent conflict between estimation and control—A survey of the two-armed problem. *Journal of the Franklin Institute*, 301:161–189.

Witten, I. H. (1977). An adaptive optimal controller for discrete-time Markov environments. *Information and Control*, 34:286–295.

Witten, I. H., Corbin, M. J. (1973). Human operators and automatic adaptive controllers: A comparative study on a particular control task. *International Journal of Man–Machine Studies*, 5:75–104.

Yee, R. C., Saxena, S., Utgoff, P. E., Barto, A. G. (1990). Explaining temporal differences to create useful concepts for evaluating states. In *Proceedings of the Eighth National Conference on Artificial Intelligence*, pp. 882–888. AAAI Press, Menlo Park, CA.

Young, P. (1984). *Recursive Estimation and Time-Series Analysis*. Springer-Verlag, Berlin.

Zhang, M., Yum, T. P. (1989). Comparisons of channel-assignment strategies in cellular mobile telephone systems. *IEEE Transactions on Vehicular*

Technology, 38:211-215.

Zhang, W. (1996). *Reinforcement Learning for Job-shop Scheduling*. Ph.D. thesis, Oregon State University. Technical Report CS-96-30-1.

Zhang, W., Dietterich, T. G. (1995). A reinforcement learning approach to job-shop scheduling. In *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence*, pp. 1114–1120. Morgan Kaufmann.

Zhang, W., Dietterich, T. G. (1996). High-performance job-shop scheduling with a time–delay TD(λ) network. In D. S. Touretzky, M. C. Mozer, M. E. Hasselmo (eds.), *Advances in Neural Information Processing Systems: Proceedings of the 1995 Conference*, pp. 1024–1030. MIT Press, Cambridge, MA.

Zweben, M., Daun, B., Deale, M. (1994). Scheduling and rescheduling with iterative repair. In M. Zweben and M. S. Fox (eds.), *Intelligent Scheduling*, pp. 241–255. Morgan Kaufmann, San Francisco.

Index